

VCS Native Low Power (NLP) User Guide

M-2017.03, March 2017



Copyright Notice and Proprietary Information

© 2017 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Third-Party Software Notices

VCS®/VCSi™ and VCS® MX/VCS® MXi™ includes or is bundled with software licensed to Synopsys under free or open-source licenses. For additional information regarding Synopsys's use of free and open-source software, refer to the `third_party_notices.txt` file included within the `<install_path>/doc` directory of the installed VCS/VCS MX software.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <http://www.synopsys.com/Company/Pages/Trademarks.aspx>.

All other product or company names may be trademarks of their respective owners.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
690 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Contents

1. Getting Started

Introduction to Low Power Simulation	1-2
Low Power Design Strategies	1-3
Increasing Challenges of Power	1-3
Types of Power Consumption	1-5
Dynamic Power	1-5
Static (Leakage) Power	1-7
Power Reduction Methods.	1-9
Supply Voltage Reduction	1-9
Clock Gating.	1-10
Multiple-Vt Library Cells	1-11
Multi-Voltage Design	1-12
Power Gating	1-14
Dynamic Voltage and Frequency Scaling.	1-18
Specifying Power Intent of the Design Using UPF	1-19

2. Low Power Design Concepts

Power Architectural Elements	2-1
--	-----

Library Requirements for Low Power Designs	2-2
Level-Shifter Cells	2-2
Isolation Cells	2-3
Power Switch Cells	2-4
Always-On Logic Cells	2-5
Retention Register Cells	2-6
Power Domains	2-7
Power Distribution Elements	2-10
Supply Ports	2-10
Supply Nets	2-12
Power Switch	2-13
Supply Sets	2-14
Supply Set Handles	2-15
Power State Tables	2-17
 3. Using UPF Commands to Specify Power Intent	
UPF Commands Supported by VCS NLP	3-3
Basic Power Network Commands	3-4
associate_supply_set	3-4
create_power_domain	3-5
create_power_switch	3-5
create_supply_port	3-6
create_supply_net	3-7
connect_supply_net	3-7
connect_supply_set	3-8
create_supply_set	3-8
map_power_switch	3-9

set_domain_supply_net	3-9
Level Shifter Commands	3-10
set_level_shifter	3-10
map_level_shifter_cell	3-11
name_format	3-11
Isolation Commands	3-12
set_isolation	3-12
set_isolation_control.	3-13
map_isolation_cell	3-14
Retention Commands	3-14
set_retention.	3-15
set_retention_control	3-16
set_retention_elements	3-17
map_retention_cell.	3-17
Power State Table Commands.	3-18
create_pst.	3-18
add_port_state	3-19
add_power_state	3-19
create_power_state_group.	3-20
add_pst_state.	3-20
Logic Editing Commands.	3-21
create_logic_net.	3-21
create_logic_port	3-21
connect_logic_net	3-22
Query Commands	3-22
find_objects	3-23
query_cell_instances	3-24
query_cell_mapped	3-24
query_net_ports	3-24

query_port_net	3-25
query_pst	3-25
query_pst_state	3-25
query_power_switch.	3-26
query_upf	3-26
query_design_attributes	3-26
query_hdl2upf_vct	3-27
query_isolation	3-27
query_power_domain.	3-27
query_retention	3-28
query_retention_control	3-28
query_supply_net.	3-28
Simulation/Verification Extension.	3-29
set_design_top.	3-29
set_partial_on_translation	3-29
Utility Commands.	3-30
load_upf	3-30
set_scope	3-30
set_design_attributes	3-31
set_port_attributes	3-31
Extending Switch	3-32
set_power_switch.	3-32
Merging Power Domains	3-33
merge_power_domains	3-33
Other Supported Power Intent Commands	3-34
add_domain_elements	3-34
bind_checker	3-35
create_composite_domain	3-36
create_hdl2upf_vct.	3-36

create_upf2hdl_vct	3-36
describe_state_transition	3-37
load_simstate_behavior	3-37
load_upf_protected	3-38
set_design_top	3-38
set_equivalent	3-38
set_simstate_behavior	3-39
upf_version	3-39
Using add_power_state Command	3-40
Use Model	3-40
Supply Expressions and Logic Expressions	3-41
Power State Evaluation of Supply Set in Presence of -supply_expr/-logic_expr	3-42
Defining Power State Tables Using the add_power_state Command 3-42	
Use Model	3-43
Defining PSTs Using add_power_state Options	3-45
Composing States Hierarchically	3-47
Examples	3-48
Interaction Between create_pst/add_pst_state and add_power_state Syntax	3-51
Limitations	3-54
Limitations of add_power_state Support	3-55
Using associate_supply_set Command	3-55
Using the bind_checker Command	3-56
Using create_power_switch Command	3-60
Handling create_power_switch -ack_port Connections	3-60

Using Supply Set of the Power Switch for Acknowledge Port Corruption.	3-61
Driver Checks on Power Switch Acknowledge Port Signal . .	3-61
Limitation	3-64
Using create_supply_net Command	3-64
Support for -resolve parallel Option	3-64
Limitation	3-65
Specifying User-Defined Supply Net Resolution As a Verilog Function or Task in the Package	3-65
Limitations	3-66
Using map_isolation_cell Command	3-67
Key Points to Note	3-67
Isolation Cell Modeling Example	3-68
Using Value Conversion Table (VCT) for Model Supply Connection	3-69
Limitations	3-71
Using query_cell_mapped Command	3-71
Limitations	3-72
Using set_design_top Command	3-72
Using set_equivalent Command	3-73
Example	3-74
Limitations	3-74
Using set_isolation Command	3-74
Use Model	3-75
Examples	3-77
Precedence Details	3-81

Defining Isolation Strategies	3-82
Using set_related_supply_net Command	3-83
Example	3-84
Using set_port_attributes Command	3-85
Use Model	3-85
Attributes	3-86
Comparison with set_related_supply_net (SRSN)	3-87
set_port_attributes on Non-Boundary Ports	3-88
Using the -repeater_supply Option	3-88
Support for Unconnected Ports Inside Macro Blocks	3-90
Specifying Clamp Value for Ports	3-92
Inserting SPA and SRSN Buffers on HighConn	3-93
Defining Retention Strategies	3-94
Support for Voltage Triplets in the add_port_state and add_power_state Commands	3-95
Using upf_version Command	3-96
Using upf_tool Variable	3-97
Usage Example	3-97
Using synopsys_program_name Variable	3-98
Usage Example	3-99
Setting Supply Port Default Value	3-100
Use Model	3-100
Example	3-100
Support for Auto-Completion of UPF Commands	3-101
Support for Feedthrough Paths Inside Macro Blocks	3-102

Key Points to Note	3-103
Limitations	3-104
Generic Handles Support	3-104
Generic Handles Support in Retention Strategy	3-105
Limitations	3-107
Generic Handles Support in the bind_checker Command. . .	3-107
Generic Handles Support for Isolation in the bind_checker Command	3-107
Generic Handles Support for Retention in the bind_checker Command	3-111
Bias Support for Low Power Designs	3-114
Enabling Bias Feature	3-115
Establishing Connections to Bias Pins/Supplies	3-117
UPF Structural Components	3-118
UPF Script Examples	3-120
Simple Multi-Voltage Design	3-120
Bottom-Up Power Intent Specification	3-121
Top-Down Power Intent Specification	3-122
Supply Set Example	3-123

4. Wildcard Support in UPF Commands

Wildcard Usage with Isolation	4-4
Wildcard Usage with Retention	4-4
Wildcard Usage with set_design_attributes	4-5
Wildcard Usage with set_related_supply_net	4-5
Wildcard Usage with connect_supply_net	4-5

Treating Generate Instance as a Single Token for Wildcard Matching	4-6
Limitations of Wildcard Support.	4-7
5. Supply-Set-Based Power Intent Specification	
Explicit Supply Sets with Explicit Supply Nets.	5-2
Explicit Supply Sets with Implicit Supply Nets.	5-4
Implicit Supply Sets.	5-4
Supply Network Initialization	5-5
Supply Set Association	5-6
Disabling the Propagation of Supply Set State Names to Functional Nets	5-7
Backward Compatibility	5-9
Valid Usage Examples.	5-11
Error Scenarios	5-12
Supply Sets in Strategies	5-13
Limitation	5-13
Setting Power State Using set_supply_set_state Function from	
Testbench	5-14
Simstates.	5-15
Setting Default Power State for a Supply Set	5-16
6. Low Power Simulation	
Running Low Power Designs	6-2
VCS NLP Low Power Simulation Flow	6-2

Using Low Power in VCS MX	6-3
Power Techniques Supported in VCS MX	6-4
Limitations	6-4
HDL Constructs and Techniques in VCS.	6-5
Support for UPF Supply Functions	6-6
UCLI Support for Supply Functions	6-8
Voltage-Based supply_on	6-9
Modeling Delay Specification for a Power Switch	6-10
Monitoring the Simstate of a Power-Managed Instance	6-11
Reporting Write Attempt on OFF Domain Element	6-13
Supported Scenarios	6-13
Use Model	6-14
Limitations	6-15
Initial Block Retriggering	6-15
Enabling Initial Block Retriggering in VCS NLP	6-16
Initial Block Retriggering Supported Attributes	6-17
Reinit Property	6-17
Transitive Reinit	6-18
Examples for Different Reinit Scenarios	6-18
Transitive Reinit Limitations	6-20
 7. Compile-Time Static Reports	
Redirecting/Renaming mvsim_native_reports Directory	7-2
Key Points to Note	7-2
Example	7-2

Compile-Time Static Report for set_design_attributes	7-3
Default Reports Dumped Under mvsim_native_reports Directory	7-4
8. Handling Corruption in Low Power Designs	
Simulation Semantics for Corruption	8-2
Overriding Corruption with HDL Forces	8-3
Corruption of XMR Based on its Domain Simstate	8-4
Example	8-4
Limitations	8-5
Random Corruption	8-5
Usage	8-5
Key Points to Note	8-7
Examples	8-8
Limitations	8-9
Preventing Corruption of Memories During Power Shutdown	8-10
Limitations	8-10
Corruption of Bound Modules	8-10
Controlling the Corruption and Wake-up Semantics of the Initial and Always Blocks	8-11
Treating Continuous Assignments as Pass Through	8-13
Limitations	8-14
Treating Continuous Assignment Statements as Feedthrough	8-14
Configuring Semantics for Runtime Forces During Power Shutdown and Wake-up	8-15
Force Priority	8-16

Force and Corrupt	8-16
Flush Forces	8-17
Defer Forces	8-17
Use Model	8-19
Limitations	8-19
CORRUPT_ON_ACTIVITY Simstate (Low-Vdd Standby).	8-20
Examples	8-20
Simulation Behavior	8-20
Limitations	8-21
Excluding Parts of Design from Being Power Managed	8-22
Use Model	8-22
Key Points to Note	8-22
Module Support	8-23
Module Named Block Support	8-24
Module Signal Support	8-24
Support for a Module Inside a Library	8-24
Usage Examples	8-25
Limitations	8-25
Skipping Corruption	8-25
 9. Isolation Support	
Isolation Support for MX Designs	9-2
Limitations	9-2
Isolation Support for SystemVerilog Design Constructs	9-3
Limitations	9-3

Isolating Interface Modports.	9-3
Limitations	9-4
Skipping Isolation on the Undriven/Floating Ports.	9-4
NOR-Style Isolation Support	9-5
Key points to note.	9-6
Corruption Semantics for NOR-Style Isolation.	9-7
Limitations	9-7
Support for Conservative diff_supply_only Isolation	9-7
Name-Based Isolation Cell Association in Netlist	9-9
Support for Isolation Terminal Points at the Hierarchical Boundaries	9-11
Introduction	9-11
Use Model	9-12
Enhancement to the -driver_supply and	
-receiver_supply Port Attributes	9-13
Examples	9-13
Example-1: Isolation with -source	9-13
Example-2: Isolation with -sink.	9-15
Example-3: Isolation with -driver_supply	9-16
Example-4: Isolation with -reciever_supply	9-18
Limitations	9-19
Path-Based Isolation Support for Heterogeneous Loads	9-19
Key Points to Note	9-20
Heterogeneous Load Scenarios	9-21
Precedence Rule Scenarios	9-25
Case-1	9-25
Case-2	9-27

Support for -exclude_elements and -update Options	9-29
Use Model	9-29
Using -update with set_isolation Command	9-30
Support for -applies_to with -elements for Isolation	9-31
Key Points to Note	9-31
Support for Specifying Instances with set_isolation -elements	9-33
Generating Virtual Isolation Reports	9-34
Virtual Isolation Insertion Report	9-34
Reporting Format	9-35
Virtual Isolation Inference Report.	9-36
Isolation Inference Report Format	9-36
Isolation/Corruption on Ports with Logical Expressions in Port Map	9-38
Limitations	9-38
Supporting Lower Domain Boundary for Isolation.	9-39
Use Model	9-40
Key Points to Note	9-40
Examples	9-41
Block-Level Control of Lower Domain Boundary	9-42
Use Model	9-42
Examples	9-44
Limitation	9-45
Behavior of the Isolation Commands.	9-45
Commands Supported for Delayed Error Check	9-46
Impact on Isolation Engine.	9-46
General isolation strategies (without –elements)	9-47

Specific Isolation Strategies (with –elements)	9-48
Inserting Back-to-Back Isolation Cells in the Lower Domain Boundary	9-49
Order of Precedence of Isolation Strategies.	9-50
Limitations	9-50
 10. Handling of Constants for Corruption and Isolation	
Default Behavior	10-2
Supply Based Corruption of Constants	10-2
Skip Corruption on Constants	10-2
 11. State Retention Support	
Introduction	11-2
Using set_retention command.	11-3
Support for the -update Option	11-4
Use Model	11-4
Using -no_retention Option	11-5
Using -use_retention_as_primary Option	11-5
Supported Retention Schemes	11-6
Two-pin Retention	11-7
Semantics.	11-7
Example: outp1 is the primary register, save_outp1 is the shadow register	11-8
Single-pin Retention	11-9
Semantics.	11-10
Example: outp1 is the primary register, save_outp1 is the shadow register	11-10

Zero-pin Retention	11-11
Semantics.	11-12
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-12
Clock-Gated Zero-Pin Retention	11-13
Save Signal Only	11-13
Semantics.	11-14
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-14
Restore Signal Only	11-14
Semantics.	11-15
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-15
Conditions	11-16
<code>-save_condition/-restore_condition</code>	11-16
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-17
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-18
<code>-retention_condition</code>	11-18
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-19
Mutexes.	11-19
<code>assert_s_mutex</code>	11-20
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-20
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-21
<code>assert_r_mutex</code>	11-22

Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-22
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-23
<code>assert_rs_mutex</code>	11-24
Example: <code>outp1</code> is the primary register, <code>save_outp1</code> is the shadow register	11-25
Parameters	11-26
Supported Parameters with <code>set_retention_control</code>	11-26
Supported Parameters with <code>set_retention</code>	11-31
Creating Retention List	11-33
Key Points to Note	11-34
Usage Examples	11-35
Invalid Usage Scenarios	11-36
Limitations	11-37
Ignoring Latches From Retention	11-38
Automated Assertions	11-38
Generic Handles	11-38
Debug Hooks for Retention	11-39
Hierarchical Flop Inference Report	11-39
Shadow Register in DVE	11-40
Retention Strategy in Power Pane	11-40
Limitations of Retention Support	11-41

12. Unified Low Power Messaging

Message Format	12-2
Message Control	12-2
Runtime Configuration File	12-3
Logging Mode.	12-3
Turn ON/OFF Messages	12-4
Override the Default Severity of Messages	12-5
Filter Messages Based on Severity	12-5
Function Calls from Testbench.	12-6
Override the Default Severity of Messages	12-6
Turn ON/OFF Messages	12-6
Filter Messages Based on Severity	12-7
Wildcard Support.	12-7
Example	12-7
User-Defined Messages in LP_MSG Format	12-8
Register Custom Message.	12-8
Printing the Registered Messages	12-8
Checking if the Registered User-Defined Message is ON/OFF	
12-9	
Assertions Related Clock/Reset Wiggle During Shutdown/Standby	12-9
Limitations	12-10
Checking for Assertion of Isolation Enable When Source Supply is OFF	
12-10	
Key Points to Note	12-11
Limitations	12-11
Power-on Reset Assertions Support.	12-11

Example	12-12
Use Model	12-13
Limitations	12-15
Power Aware Verification Environment	12-15
Custom Assertions	12-16
Use Model	12-16
Specifying File List with Multiple Bind Checker Tcl Files During Compilation	12-17
Writing Custom Low Power Assertions	12-18
Example	12-20
Compile-Time UPF Parser Checks	12-23
Lint Check for Explicit Primary Supplies for a Domain.	12-24
Error Checks for Non-Existing Design Objects Referred in UPF	12-25
Lint Check for Mismatch Between Verilog and Liberty Definitions of a Cell	12-27
13. Debugging Low Power Designs Using DVE	
Disabling the Dumping of Low Power Data to VPD.	13-2
Low Power Debug Features	13-2
Power Pane	13-3
Filtering the Objects in the Power Pane	13-8
Viewing Supply Sets in the Power Pane	13-9
Low Power Objects in Hierarchy Pane	13-13
Isolated Signals	13-16
Shadow Registers	13-17
Marking for Low Power Signals in the Data Pane	13-18

Viewing mvsim_flag Variable	13-19
Low Power Objects in the Data Pane.	13-21
UPF Source View.	13-25
Using UPF Source Code in the Source View	13-27
Waveform Shading	13-30
Isolation Shading Support	13-32
Shading Support for the isolation_power_off Shading	13-33
Support for Toggle Shading on Input Ports	13-34
Shading Support for set_related_supply_net (SRSN) on Ports 13-35	
Primary Power Off Shading	13-36
Power-Off Shading for DB pins	13-37
Configuring the Waveform Shading Pattern Color	13-37
Limitations	13-39
Power State Table	13-40
Viewing a Power State Table (PST) in the Power Pane and Wave View	13-41
Shading in the Schematic View	13-45
Symbol for Isolation Cell in Schematic.	13-46
Drivers/Loads Pane	13-48
Power Domain Information for Drivers/Loads	13-48
Driver/load Tracing for the Supply Network and Power Pane Data Elements.	13-48
Viewing Logic Nets in DVE	13-51
Limitations	13-52
Support for Active Drivers on Low Power Design Elements	13-52
Power Domain	13-53
Power Switch	13-55
Acknowledge Port of the Power Switch	13-55

Isolation Strategy	13-56
Retention Strategy	13-59
UPF Supply Functions	13-62
set_related_supply_net (SRSN).....	13-63
set_port_attributes (SPA).....	13-65
Limitations	13-66
Viewing the DB Information of Cells	13-66
Low Power Query Commands.....	13-70
Limitations	13-74
14. Coverage Support for Low Power Objects	
Creating Covergroups for the Low Power Objects	14-2
Enabling Coverage on Low Power Objects.....	14-3
Runtime Control for Low Power Coverage	14-6
Excluding UPF Objects from Low Power Coverage	14-6
Use Model	14-6
Example.....	14-7
Limitations	14-8
Constraining Low Power Coverage.....	14-8
Key points to note.....	14-11
Usage Example of describe_state_transition.....	14-12
Post-Processing Options for Low Power Functional Coverage. .	14-15
Filtering Covergroups in URG and DVE	14-16
Use Model	14-16
Filtering Covergroups in UCAPI.....	14-18

Coverage Reports	14-19
DVE Coverage Report	14-19
Low Power Coverage Report View in URG	14-22
Use Model	14-22
Low Power Functional Coverage Report Example	14-26
Examples for Different Scenarios.	14-27
Dumping Coverage Exclusion Files for Low Power Objects	14-34
 15. Gate-level Netlist Simulations (Simulating with Liberty (.db) Files)	
Using Connect Supply Net and Library Mapping	15-2
Use Model	15-2
Example.	15-3
DB Matching Criteria	15-3
Simulation Semantics for Cells.	15-4
Supported .db Attributes	15-5
Support for Nor-Style Cells	15-6
Power Aware and Non-Power Aware Models	15-7
Simulation Behavior for Power Aware Models	15-8
Simulation Behavior for Non-Power Aware Models.	15-8
Implicit Supply Net Connections	15-9
UPF2SV/SV2UPF Connections	15-10
SV2UPF Connection	15-10
UPF2SV Connection	15-11
Value Conversion Tables (VCTs).	15-11

Supported VCTs	15-12
Attributes Supported for Overriding the Default VCTs . . .	15-14
Precedence of VCT in High to Low Order	15-15
Limitations	15-15
Multi-Rail Macro Simulation	15-15
Use Model	15-15
Usage Example	15-17
Configuring Port-Based Corruption on Macros	15-20
Use Model	15-21
Example	15-22
Association of Pre-Inserted Isolation Cells	15-23
Support for Timing Simulation With SDF in Presence of UPF . . .	15-26
 16. Synopsys Low Power Flow Consistency	
Synopsys Low Power Flow Consistency Check	16-2
Use Model	16-2
-simulation_only Support	16-3
Error Checking for Isolation Strategy with -location parent on Top-level Ports	16-4
Handling Conflicting Port State Names in the add_port_state Command 16-5	
 17. Connecting UPF Supply Network to Wreal Supply Ports	
Connecting UPF Supply Nets to Wreal Input and Inout Ports . . .	17-2
Example	17-3

Connecting UPF Supply Nets to Wreal Output Ports	17-5
Example	17-6
18. Appendix	
Low Power Assertions	18-2
List of Automated Low Power Messages	18-2
Enabling Runtime Isolation Messages	18-18
Verbose Illegal PST State Change Messages	18-19
Supported UPF Query Commands	18-22
query_power_domain	18-22
query_supply_net	18-23
query_pst	18-24
query_pst_state	18-25
query_power_switch	18-26
query_isolation	18-28
query_retention	18-30
query_retention_control	18-32
Warning Messages for Unconnected Supply Ports	18-34
Root Supply Driver	18-36
TFIPC-L Lint Check in Presence of UPF Connections	18-37
19. VCS NLP Quick Reference	
Command Line Options	19-2
Compile-Time Options	19-2
List of “-power” Compile-Time Options	19-3
Simulation Option	19-12

Design Attributes to be Specified in UPF 19-15

 HDL Attributes 19-20

1

Getting Started

Power consumption is becoming an increasingly important aspect of circuit design. There are several different approaches that can be used to reduce power. This chapter introduces low power simulation, low power principles, power reduction strategies, and UPF commands supported by VCS Native Low Power (VCS NLP) in the following sections:

- [Introduction to Low Power Simulation](#)
- [Low Power Design Strategies](#)
- [Specifying Power Intent of the Design Using UPF](#)

Introduction to Low Power Simulation

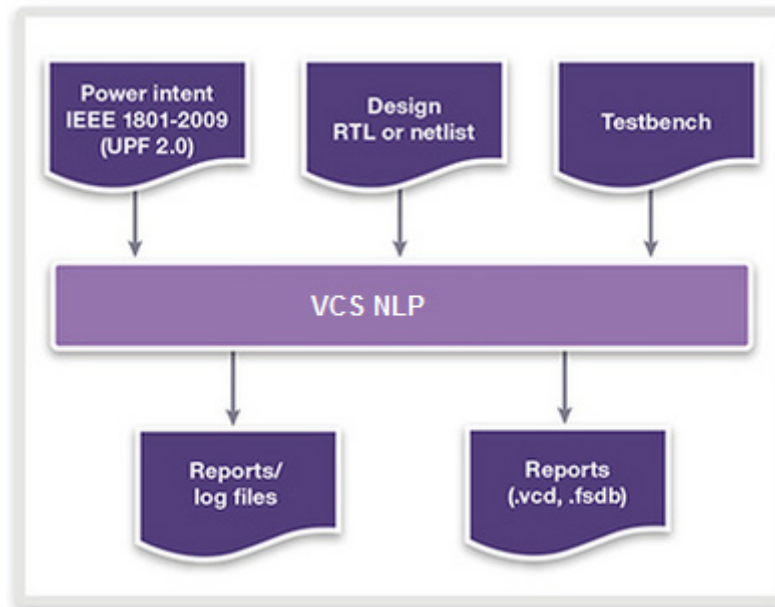
Low power designs have design components operating under different supply voltages from multiple supply rails. Traditional simulators don't have the intelligence to understand the relationship between different supply voltages and what they drive, causing potential silicon failures that go undetected in simulation.

VCS NLP equips VCS to natively perform voltage-level aware simulation with a complete understanding of the UPF-defined power network, including at RTL prior to implementation flows. This uniquely allows engineers to comprehensively verify correct behavior of designs that use advanced voltage control techniques for power management, and catch potentially design-killing low power bugs very early in the design process.

VCS NLP takes in the same Verilog or VHDL RTL or gate-level netlist representation of the design as in standard flows, and accepts the same testbench as in standard flows (optionally augmented for low power checks). However, the native low power flow requires power intent to be specified in an UPF-format file, which is loaded into VCS with the design and the testbench.

VCS NLP equips VCS to read this UPF, model the entire power network described in the UPF, and accurately understand the low power policies and voltage events. VCS in native low power mode will output a log file and an error and warnings report for all violations related to multi-voltage checks, as illustrated in [Figure 1-1](#).

Figure 1-1 VCS NLP Low Power Simulation Flow



Low Power Design Strategies

This section introduces power principles and power reduction strategies in the following sections:

- [Increasing Challenges of Power](#)
- [Types of Power Consumption](#)
- [Power Reduction Methods](#)

Increasing Challenges of Power

In earlier generations of IC design technologies, the main parameters of concern were timing and area. EDA tools were designed to maximize the speed while minimizing area. Power

consumption was a lesser concern. CMOS was considered a low power technology, with fairly low power consumption at the relatively low clock frequencies used at the time, and with negligible leakage current.

In recent years, however, device densities and clock frequencies have increased dramatically in CMOS devices, thereby increasing the power consumption dramatically. At the same time, supply voltages and transistor threshold voltages have been lowered, causing leakage current to become a significant problem. As a result, power consumption levels have reached their acceptable limits, and power has become as important as timing or area.

High power consumption can result in excessively high temperatures during operation. This means that expensive ceramic chip packaging must be used instead of plastic, and complex and expensive heat sinks and cooling systems are often required for product operation. Laptop computers and hand-held electronic devices can become uncomfortably hot to the touch. Higher operating temperatures also reduce reliability because of electromigration and other heat-related failure mechanisms.

High power consumption also reduces battery life in portable devices such as laptop computers, cell phones, and personal electronics. As more features are added to a product, power consumption increases and the battery life is reduced, requiring a larger, heavier battery or shorter life between charges. Battery technology has lagged behind the increased demands for power.

Another aspect of power consumption is the sheer cost of electrical energy used to power millions of computers, servers, and other electronic devices used on a large scale, both to run the devices themselves and to cool the machines and buildings in which they are used. Even a small reduction in power consumption of a

microprocessor or other device used on a large scale can result in large aggregate cost savings to users and can provide significant benefits to the environment as well.

Types of Power Consumption

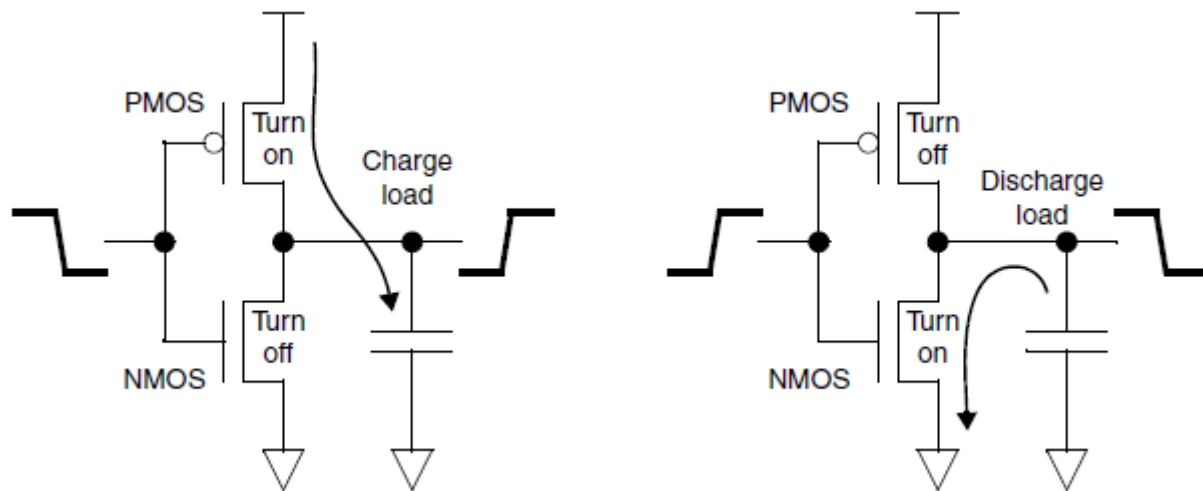
Designers must consider two types of power consumption, dynamic and static. Dynamic power is consumed during the switching of transistors, so it depends on the clock frequency and switching activity. Static power is the transistor leakage current that flows whenever power is applied to the device, so it is not related to the clock frequency or switching activity.

Dynamic Power

Dynamic power is the energy consumed during logic transitions on nets, consisting of two components, switching power and internal power. Switching power results from the charging and discharging of the external capacitive load on the output of a cell. Internal power results from the short-circuit (crowbar) current that flows through the PMOS-NMOS stack during a transition.

The cause of switching power is illustrated in [Figure 1-2](#). A transition from 0 to 1 on the output of the inverter charges the capacitive load of the output net through the PMOS transistor. A transition from 1 to 0 discharges the same capacitive load through the NMOS transistor.

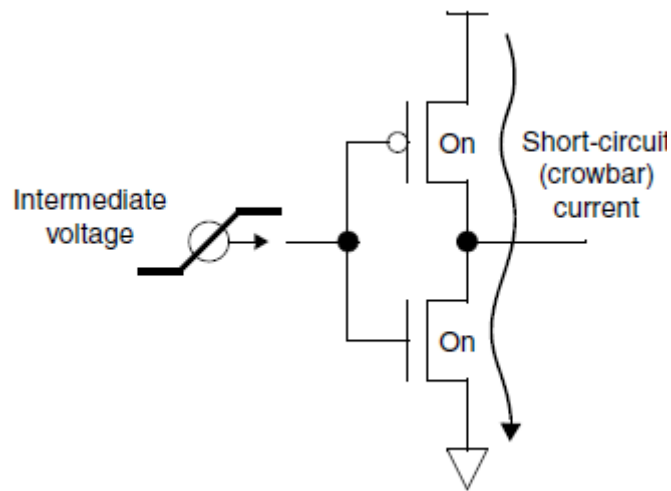
Figure 1-2 Switching Power



The amount of energy dissipated in each transition depends on the supply voltage and the capacitive load of the net. Also, because the current flows only during logic transitions on the net, the long-term dynamic power consumption depends on the clock frequency (possible transitions per second) and the switching activity (presence or absence of transitions actually occurring on the net in successive clock cycles).

Internal power is consumed during the short period of time when the input signal is at an intermediate voltage level, during which both the PMOS and NMOS transistors can be conducting. This condition results in a nearly short-circuit conductive path from VSS to ground, as illustrated in [Figure 1-3](#). A relatively large current, called the crowbar current, flows through the transistors for a brief period of time. Lower threshold voltages and slower transitions result in more internal power consumption.

Figure 1-3 Internal Power

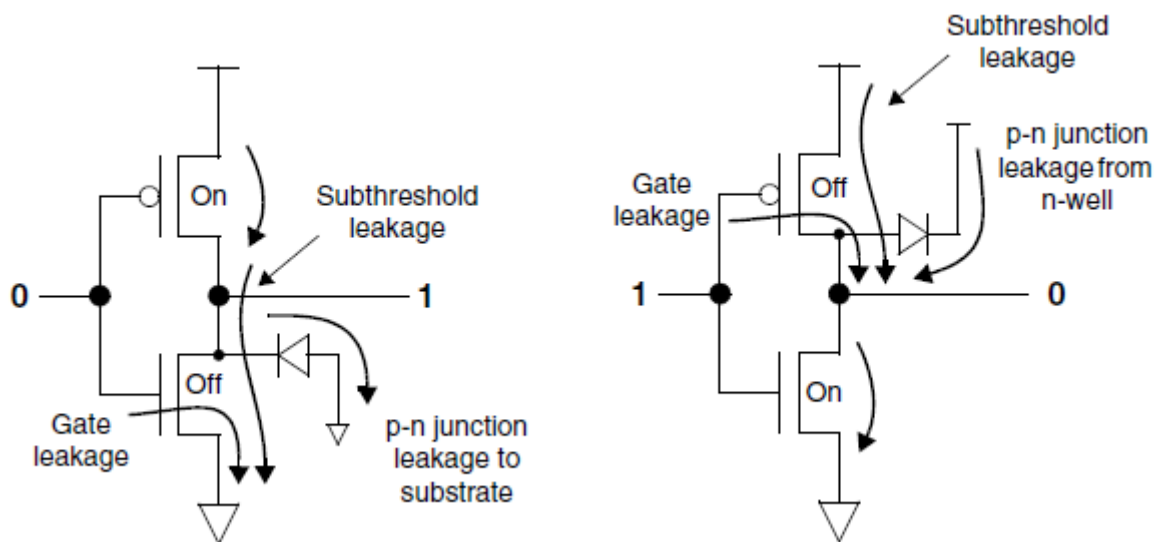


Static (Leakage) Power

Leakage current was negligible in earlier CMOS technologies. However, with shrinking device geometries and reduced threshold voltages, leakage power is becoming increasingly significant, sometimes approaching the levels of dynamic power dissipation.

The main causes of leakage power are reverse-bias p-n junction diode leakage, subthreshold leakage, and gate leakage. These leakage paths in a CMOS inverter are shown in [Figure 1-4](#).

Figure 1-4 Static Leakage Currents



Leakage at reverse-biased p-n junctions (diode leakage) has always existed in CMOS circuits. This is the leakage from the n-type drain of the NMOS transistor to the grounded p-type substrate, and from the n-well (held at VDD) to the p-type drain of the PMOS transistor. This leakage is relatively small.

Subthreshold leakage is the small source-to-drain current that flows even when the transistor is held in the “off” state. In older technologies, this current was negligible. However, with lower power supply voltages and lower threshold voltages, “off” gate voltages are getting close to “on” threshold voltages. Subthreshold leakage current increases exponentially as the gate voltage approaches the threshold voltage.

Gate leakage is the result of using an extremely thin insulating layer between the gate conductor and the MOS transistor channel. Gate oxides are becoming so thin that only a dozen or fewer layers of insulating atoms separate the gate from the source and drain. Under

these conditions, quantum-effect tunneling of electrons through the gate oxide can occur, resulting in significant leakage from the gate to the source or drain.

Leakage currents occur whenever power is applied to the transistor, irrespective of the clock speed or switching activity. Leakage cannot be reduced by slowing or stopping the clock. However, it can be reduced or eliminated by lowering the supply voltage or by switching off the power to the transistors entirely.

Power Reduction Methods

There are several different RTL and gate-level design strategies for reducing power. The following topics describe the power reduction methods:

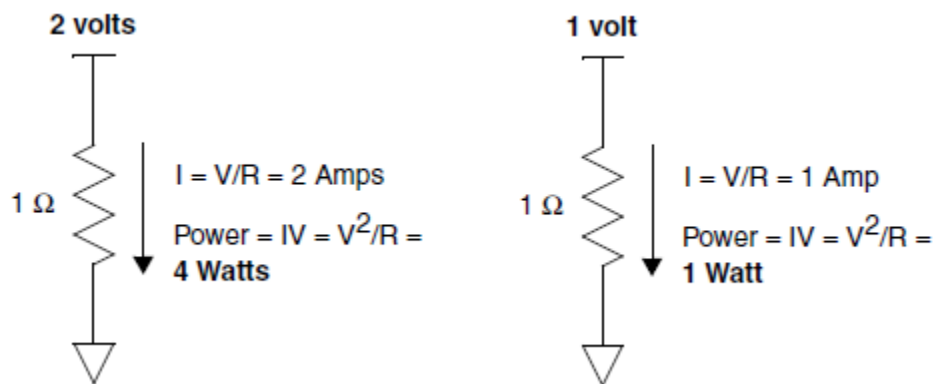
- [Supply Voltage Reduction](#)
- [Clock Gating](#)
- [Multiple-Vt Library Cells](#)
- [Multi-Voltage Design](#)
- [Power Gating](#)
- [Dynamic Voltage and Frequency Scaling](#)

Supply Voltage Reduction

The most basic way to reduce power is to reduce the supply voltage. Power usage is proportional to the square of the supply voltage, as the simple example in [Figure 1-5](#) demonstrates. A 50 percent reduction in the supply voltage results in a 50 percent reduction in

current and a 75 percent reduction in power. A similar degree of power reduction can be achieved in CMOS circuits for both dynamic and static power.

Figure 1-5 Power Versus Supply Voltage Example

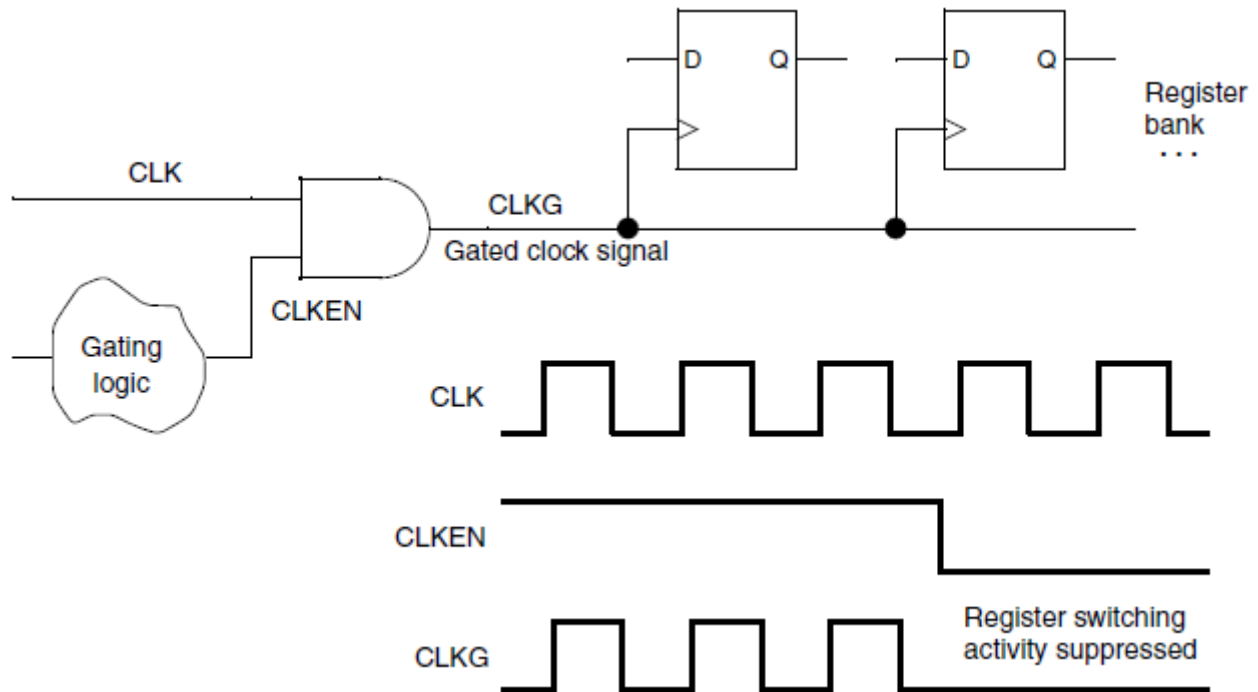


Successive generations of CMOS technologies have used lower and lower supply voltages to reduce power. Starting at 5 volts for compatibility with bipolar TTL circuits in the 1980s, supply voltages have fallen to about 1 volt for advanced technologies. Each lowering of the supply voltage reduces per-gate power consumption, but also lowers the switching speed. In addition, the transistor threshold voltage must be lowered, causing more problems with noise immunity, crowbar currents, and subthreshold leakage. The lower voltage swing makes it more difficult to interface the chip with external devices that require larger voltage swings.

Clock Gating

Clock gating is a dynamic power reduction method in which the clock signals are stopped for selected register banks during times when the stored logic values are not changing. One possible implementation of clock gating is shown in [Figure 1-6](#).

Figure 1-6 Clock Gating Example



Clock gating is particularly useful for registers that need to maintain the same logic values over many clock cycles. Shutting off the clocks eliminates unnecessary switching activity that would otherwise occur to reload the registers on each clock cycle. The main challenges of clock gating are finding the best places to use it and creating the logic to shut off and turn on the clock at the proper times.

Clock gating is a well-established power-saving technique that has been used for years. It is relatively simple to implement because it only requires a change in the netlist. No additional power supplies or power infrastructure changes are required.

Multiple-Vt Library Cells

Some CMOS technologies support the fabrication of transistors with different threshold voltages (V_t values). In that case, the cell library can offer two or more different cells to implement each logic function,

each using a different transistor threshold voltage. For example, the library can offer two inverter cells: one using low-Vt transistors and another using high-Vt transistors.

A low-Vt cell has higher speed, but higher subthreshold leakage current. A high-Vt cell has low leakage current, but less speed.

Multi-Voltage Design

Different parts of a chip might have different speed requirements. For example, the CPU and RAM blocks might need to be faster than a peripheral block. As mentioned earlier, a lower supply voltage reduces power consumption but also reduces speed. To get maximum speed and lower power at the same time, the CPU and RAM can operate with a higher supply voltage while the peripheral block operates with a lower voltage, as shown in [Figure 1-7](#).

Figure 1-7 Multi-voltage Chip Design

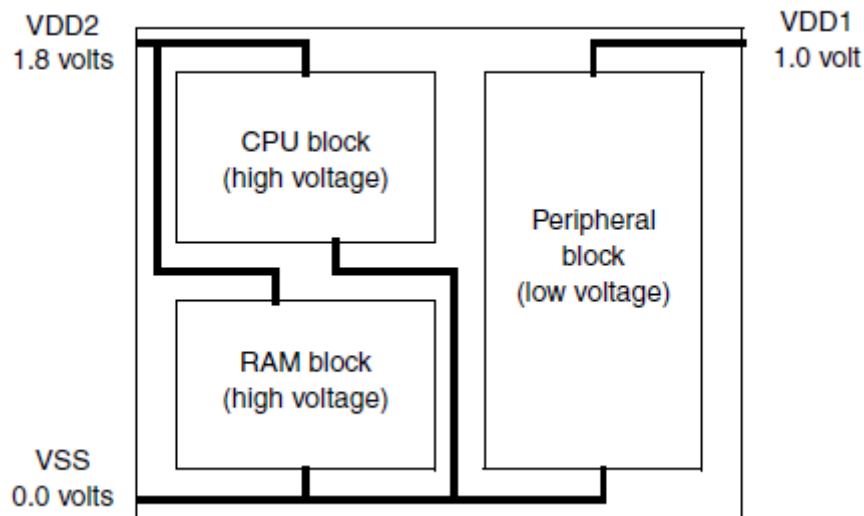
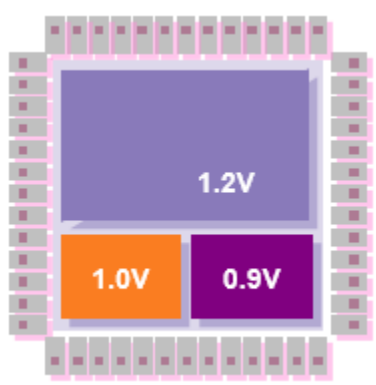


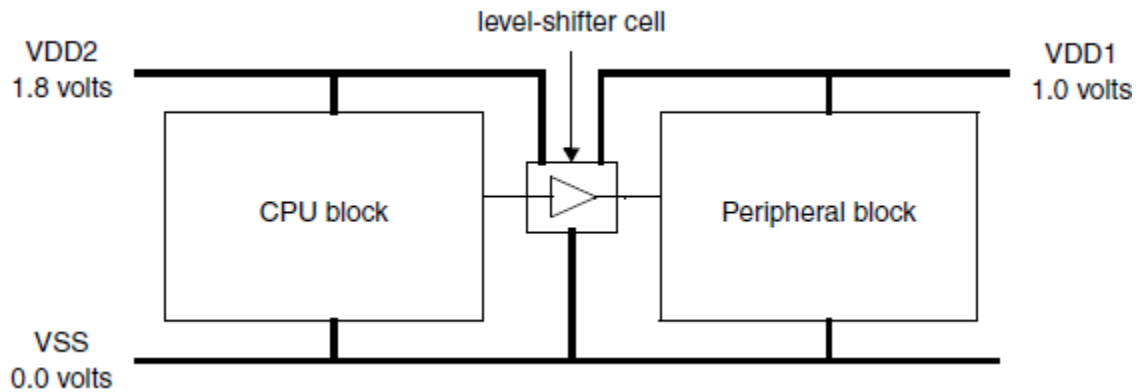
Figure 1-8 Multi-voltage Chip



Providing two or more supply voltages on a single chip introduces some complexities and costs. Additional device pins must be available to supply the chip voltages, and the power grid must distribute each of the voltage supplies separately to the appropriate blocks.

Where a logic signal leaves one power domain and enters another, if the voltages are significantly different, a level-shifter cell is necessary to generate a signal with the proper voltage swing. In the example shown in [Figure 1-9](#), a level shifter converts a signal with 1.8-volt swing to a signal with a 1.0-volt swing. A level-shifter cell itself requires two power supplies that match the input and output supply voltages.

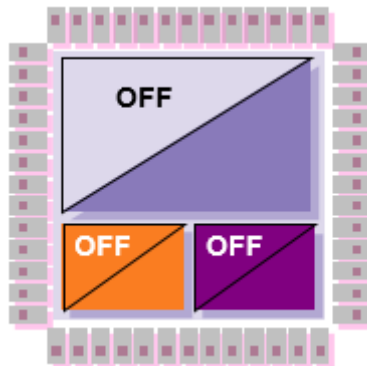
Figure 1-9 Level Shifter



Power Gating

Power gating is a power-saving technique in which portions of the chip are shut down completely during periods of inactivity, as illustrated in [Figure 1-10](#).

Figure 1-10 Power Gating

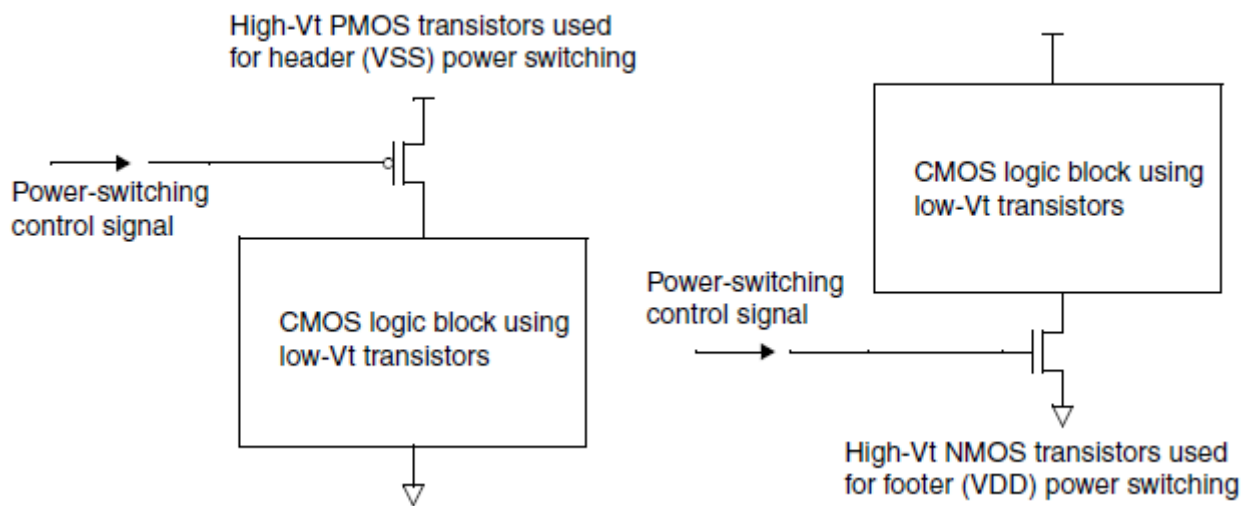


For example, in a cell phone chip, the block that performs voice processing can be shut down when the phone is in standby mode. When the user places a call or receives an outside call, the voice processing block must “wake up” from its powered-down state.

Power gating has the potential to reduce overall power consumption substantially because it lowers leakage power as well as switching power. It also introduces some additional challenges, including the need for a power controller, a power-switching network, isolation cells, and retention registers.

High-Vt transistors from a Multiple-Threshold CMOS (MTCMOS) technology are used for the power switches because they minimize leakage and their switching speed is not critical. PMOS header switches can be placed between VDD and the block power supply pins, or NMOS footer switches can be placed between VSS and the block ground pins, as illustrated in [Figure 1-11](#).

Figure 1-11 Power-Switching Network Transistors

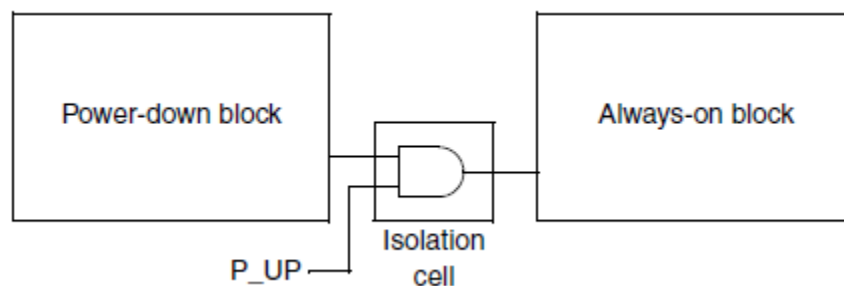


The switching strategy shown in [Figure 1-11](#) is called a coarse-grain strategy because the power gating is applied to the whole block. Multiple transistors in parallel drive a common supply net for the block. In a fine-grain strategy, each library cell has its own power switch, allowing fine-grain control over which cells are powered down. The fine-grain approach has better potential for power savings, but requires significantly more area.

Any use of power gating requires isolation cells where signals leave a powered-down block and enter a block that is always on (or currently powered up). An isolation cell provides a known, constant logic value to an always-on block when the power-down block has no power, thereby preventing unknown or intermediate values that could cause crowbar currents.

One simple implementation of an isolation cell is shown in [Figure 1-12](#). When the block on the left is powered up, the signal P_UP is high and the output signal passes through the isolation cell unchanged (except for a gate delay). When the block on the left is powered down, P_UP is low, holding the signal constant at logic 0 going into the always-on block. Other types of isolation cells can hold a logic 1 rather than 0, or can hold the signal value latched at the time of the power-down event. Isolation cells must themselves have power during block power-down periods.

Figure 1-12 Isolation Cell



The power gating can be combined with multi-voltage operation. Different blocks can be designed to operate at different voltages and also to be separately powered down when they are not needed. In that case, the interface cells between different blocks must perform both level shifting and isolation functions, depending on whether the two blocks are operating at different voltages or one is shut down. A

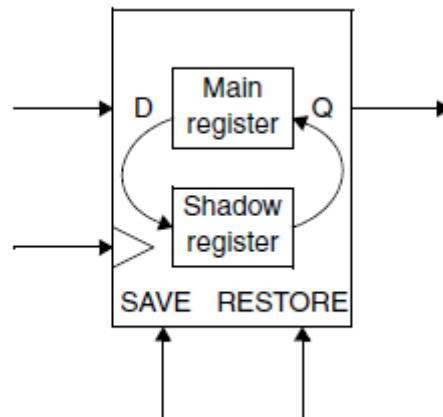
cell that performs both functions is called an enable level shifter. This cell must have two separate power supplies, just like any other level shifter.

When a block is powered down and then powered back up, it is often desirable for the block to be restored to the state it was in before the power-down event. There are several strategies for doing this. For example, the block register contents can be copied to RAM outside of the block before power-down, and then copied back after power-up.

Another strategy is to use retention registers in the power-down block. A retention register can retain data during power-down by saving the data into a shadow register (also known as the bubble register) before power-down. Upon power-up, it restores the data from the shadow register to the main register. The shadow register has an always-on power supply, but it is constructed with high-V_t transistors to minimize leakage during the power-down period. The main register is built with fast but leaky low-V_t transistors.

One type of retention register implementation is shown in [Figure 1-13](#). The SAVE signal saves the register data into the shadow register before power-down and the RESTORE signal restores the data after power-up. Instead of using separate, edge-sensitive SAVE and RESTORE signals, a retention register could use a single level-sensitive control signal.

Figure 1-13 Retention Register



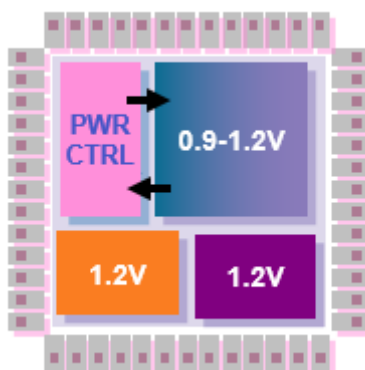
A retention register occupies a larger area than an ordinary register, and it requires an always-on power supply connection for the shadow register in addition to the power-down supply used by the rest of the device. However, restoring the data to the registers after power-up is fast and simple compared with other strategies.

Dynamic Voltage and Frequency Scaling

The principle of multi-voltage operation can be extended to allow the voltage to be changed during operation of the chip to match the current workload.

For example, a math processor chip in a laptop computer might operate at a lower voltage and lower clock frequency during simple spreadsheet computations, thereby saving power; and then at a higher voltage and higher clock frequency during 3-D image rendering when the highest performance is needed. The changing of supply voltage and operating frequency during operation to meet workload requirements is called dynamic voltage and frequency scaling. [Figure 1-14](#) illustrates the dynamic voltage and frequency scaling.

Figure 1-14 Dynamic Voltage and Frequency Scaling



The chip and voltage supply can be designed to use a number of established levels, or even a continuous range. Dynamic voltage scaling requires a multilevel power supply and a logic block to determine the best voltage level to use for a given task. Design, implementation, verification, and testing of the device can be especially challenging because of the ranges and combinations of voltage levels and operating frequencies that must be analyzed and accommodated.

Dynamic voltage scaling can be combined with power gating technology so that each block in the design can operate at multiple voltage levels for different performance requirements, or shut off completely when not needed at all.

Specifying Power Intent of the Design Using UPF

The IEEE 1801 Standard for Design and Verification of Low Power Integrated Circuits, also known as the Unified Power Format (UPF), consists of a set of commands used to specify the design intent for multi-voltage electronic systems.

Using UPF commands, you can specify the supply network, power switches, isolation, retention, and other aspects of power management for a chip design. A single set of low power design specification commands can be used throughout the design, analysis, verification, and implementation flow.

VCS NLP supports the industry-standard UPF 1.0 and 2.0 subset that are used to capture low power design requirements. Detailed information about the IEEE 1801 standard can be found in the IEEE 1801 specification itself, available as a download from the IEEE Standards Association at <http://standards.ieee.org>.

2

Low Power Design Concepts

This chapter describes the power elements that are used to specify the design intent for low power designs. It contains the following sections:

- [Power Architectural Elements](#)
- [Power Distribution Elements](#)
- [Power State Tables](#)

Power Architectural Elements

This section contains the following topics:

- [Library Requirements for Low Power Designs](#)
- [Power Domains](#)

Library Requirements for Low Power Designs

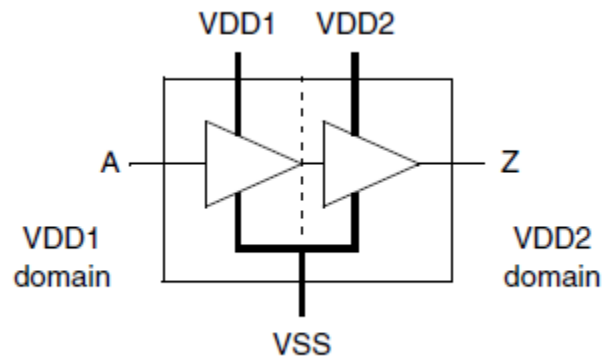
In order to use power-saving strategies such as clock gating, multi-voltage, multiple-V_t library cells, or power gating, the technology libraries used must conform to the Liberty syntax. The library should contain logic cells such as clock-gating cells, level-shifters, isolation cells, retention registers, always-on buffers, and inverters that support these strategies. The following topics describe the types of cells that support low power designs:

- [Level-Shifter Cells](#)
- [Isolation Cells](#)
- [Power Switch Cells](#)
- [Always-On Logic Cells](#)
- [Retention Register Cells](#)

Level-Shifter Cells

A level shifter is required where each signal crosses from one power domain to another. The level shifter operates as a buffer with one supply voltage at the input and a different supply voltage at the output. Thus, a level shifter converts a logic signal from one voltage swing to another, with a goal of having the smallest possible delay from input to output. See [Figure 2-1](#).

Figure 2-1 Level Shifter



Level-shifter cells are of three types:

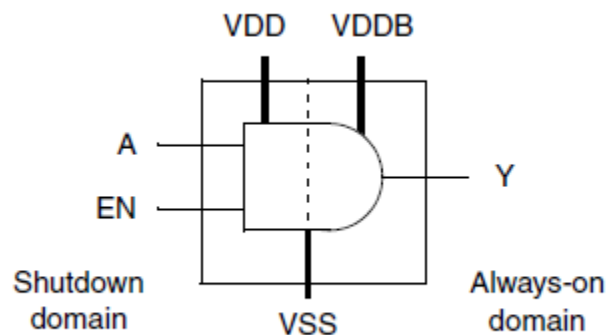
- Level shifters that convert from high voltage to low voltage (H2L)
- Level shifters that convert from low voltage to high voltage (L2H)
- Level shifter that can do both, high to low and low to high conversion

The library description of a level-shifter cell must have information about the type of conversion performed (high-to-low, low-to-high, or both), the supported voltage levels, and the identities of the respective power pins that must be connected to each power supply.

Isolation Cells

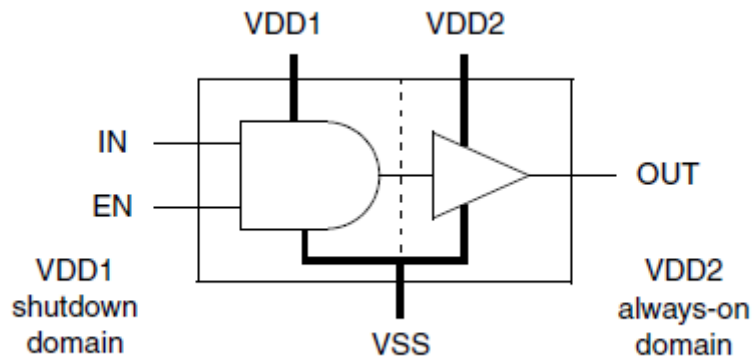
In a design with power gating, an isolation cell is required where each logic signal crosses from a power domain that can be powered down to a domain that is not powered down. The cell operates as a buffer when the input and output sides of the cell are both powered up, but provides a constant output signal during times that the input side is powered down. An enable input controls the operating mode of the cell. See [Figure 2-2](#).

Figure 2-2 Isolation Cell



A cell that can perform both level-shifting and isolation functions is called an enable level-shifter cell. This type of cell is used where a signal crosses from one power domain to another, where the two voltage levels are different and the first domain can be powered down. See [Figure 2-3](#).

Figure 2-3 Enable Level Shifter

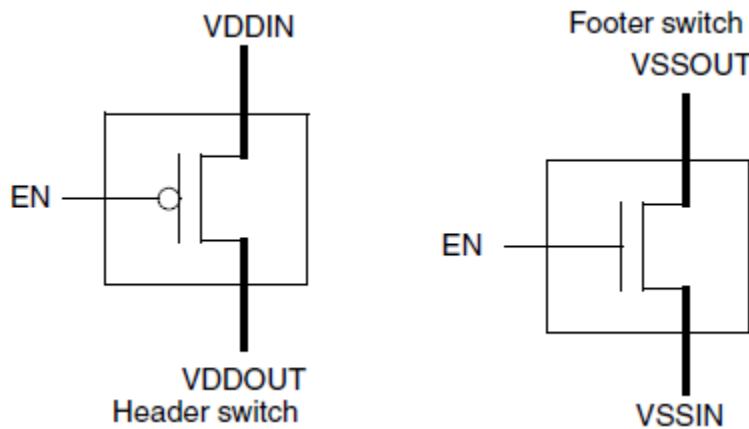


Power Switch Cells

In a design with power gating, either header or footer type power switch cells are required to supply power for cells that can be powered down. A header type power switch connects the power rail

to the power supply pins of the cells in the power-down domain. A footer type power switch connects the ground rail to the ground supply pins of the cells in the power-down domain. An input logic signal to the power switch controls the connection or disconnection state of the switch. See [Figure 2-4](#).

Figure 2-4 Power-Switch Cells



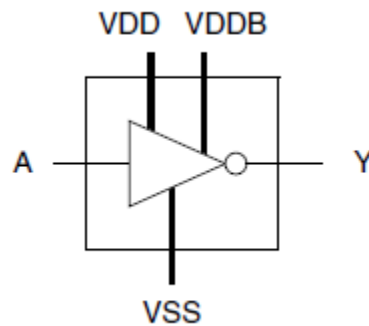
The power-switch cell can optionally have an output “acknowledge” signal indicating the current status of the switch.

Always-On Logic Cells

When dealing with shutdown domains, there can be some situations in which certain cells in the shutdown portion need to continuously stay active, such as for implementing retention registers, isolation cells, retention control paths, and isolation enable paths. For example, if a save signal or restore signal passing through a shutdown voltage area needs buffering, an always-on buffer cell must be used.

This type of logic is called always-on logic, which is built with always-on library cells. Compared to an ordinary cell, a functionally equivalent always-on cell has a backup power supply that operates continuously, even during the shutdown mode. See [Figure 2-5](#).

Figure 2-5 Always-On Logic Cell

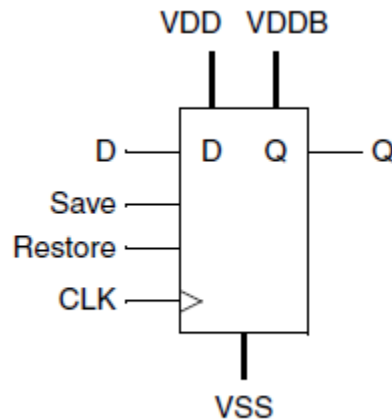


Retention Register Cells

In a design with power gating, there are several different ways to save register states before power-down and restore them upon power-up in the power-down domain. One method is to use retention registers, which are registers that can maintain their state during power-down by means of a low-leakage register network and an always-on power supply.

The library description of a retention register specifies the power pins and the input signals that control the saving and restoring of data. It also specifies which power pins are normal and can be powered down and which ones are the always-on pins used to maintain the data during power-down. See [Figure 2-6](#).

Figure 2-6 Retention Register



Power Domains

Power domain is a group of elements in the design that share a common power supply distribution network. Each power domain has a scope and an extent. The scope is the hierarchical level at which the domain is defined and is an ancestor of the elements belonging to the power domain, whereas the extent is the actual set of elements belonging to the power domain.

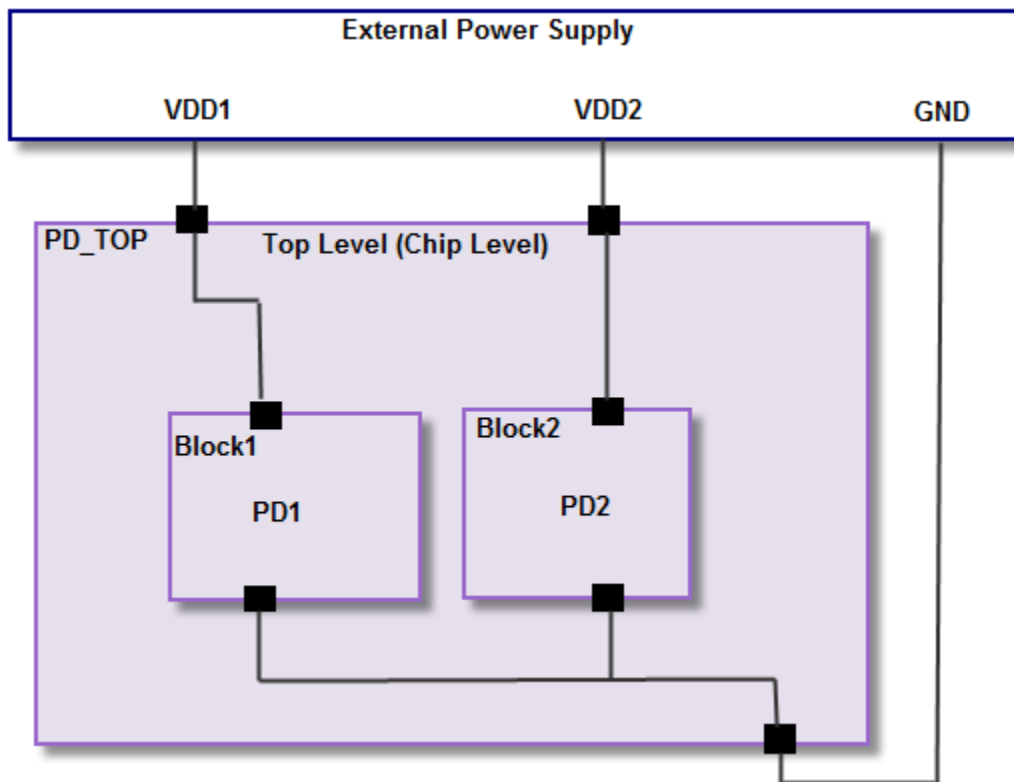
A power domain must have one primary supply net and one primary ground net, and might optionally have additional supply nets, supply ports, and power switches.

A power domain has the following characteristics:

- Name
- Level of hierarchy or scope where the power domain is defined or created
- The set of design elements that comprise the power domain

- Associated set of supply nets that are allowed to be used within the power domain
- Primary power supply and ground nets
- Synthesis strategies for isolation, level-shifters, always-on cells, and retention registers

Figure 2-7 Power Domains



In [Figure 2-7](#), the top-level chip occupies the top-level power domain, PD_TOP. The domain PD_TOP is defined to have three supply ports: VDD1, VDD2, and GND.

In addition to the top-level power domain, PD_TOP, there are two more power domains defined, called PD1 and PD2, created at the levels of two hierarchical blocks, Block1 and Block2, respectively.

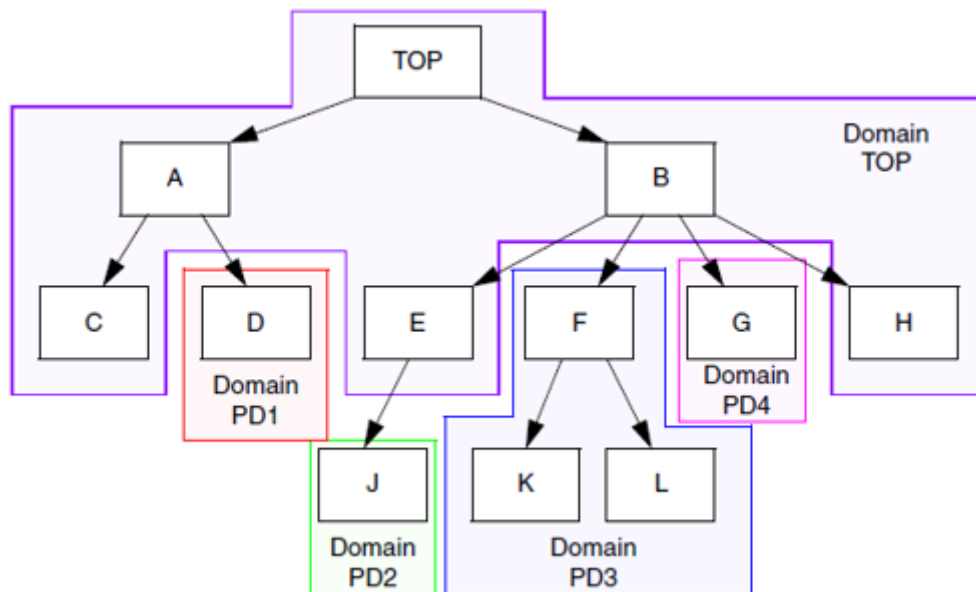
Each block has supply ports (shown as black squares in the diagram) to allow supply nets to cross from the top level down into the block level.

The following commands define the power domains of a design and assign hierarchical blocks to the domains:

```
create_power_domain TOP
create_power_domain PD1 -elements {A/D}
create_power_domain PD2 -elements {B/E/J}
create_power_domain PD3 -elements {B/F}
create_power_domain PD4 -elements {B/G}
```

The hierarchy and power domain membership of this example are illustrated in [Figure 2-8](#).

Figure 2-8 Power Domains and Hierarchy Example



In this example, the first `create_power_domain` command assigns the TOP design to the power domain called TOP, which causes all the lower-level blocks to also belong to the same power domain. The subsequent `create_power_domain` commands

assign some of the lower-level blocks to different power domains. Lower-level blocks not assigned to other power domains remain in the TOP power domain.

Power Distribution Elements

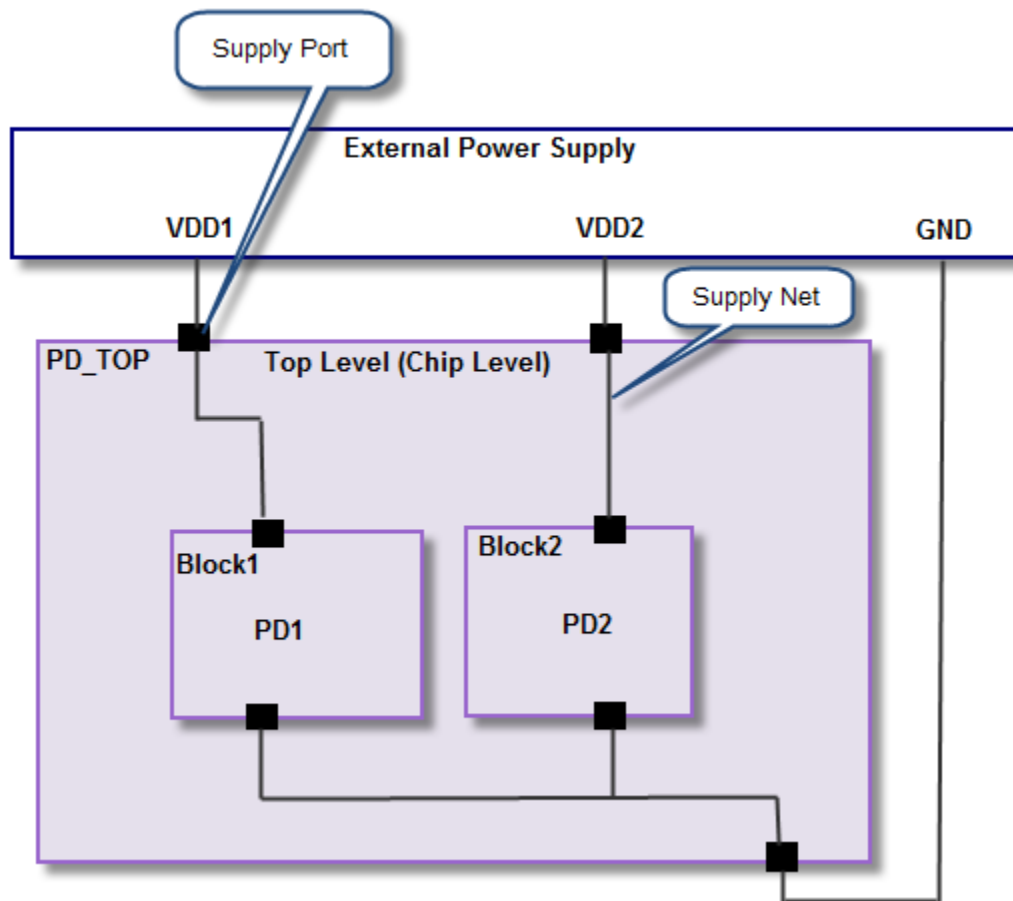
The following topics describe the power elements that allow you to specify the power intent of a design:

- [Supply Ports](#)
- [Supply Nets](#)
- [Power Switch](#)
- [Supply Sets](#)
- [Supply Set Handles](#)

Supply Ports

A supply port is a power supply connection point between two adjacent levels of the design hierarchy, between the parent and child blocks of the hierarchy. A supply net that crosses from one level of the design hierarchy to the next passes through a supply port.

Figure 2-9 Supply Ports



In [Figure 2-9](#), the black squares along the border of the power domain represent the supply ports of that domain.

To create the power supply and ground ports, use the `create_supply_port` command.

The name of the supply port should be a simple (non-hierarchical) name and unique at the level of hierarchy it is defined. Unless the `-domain` option is specified, the port is created in the current scope or level of hierarchy and all power domains in the current scope can use the created port. By default, the direction of the port is in (or input port).

For example, to create the supply ports VDD1, VDD2, and VDD3, and GND at the top level of design hierarchy or power domain PD_TOP, use the command as follows:

```
create_supply_port VDD1
create_supply_port VDD2
create_supply_port VDD3
create_supply_port GND
```

Supply Nets

A supply net is a conductor that carries a supply voltage or ground throughout a given power domain. A supply net connects supply ports. A supply net that spans more than one power domain is said to be “reused” in multiple domains. A supply net that crosses from one level of the design hierarchy to the next passes through a supply port. In [Figure 2-9](#), each block has supply ports to allow supply nets to cross from the top level down into the block level.

To create a supply net, use the `create_supply_net` command.

The supply net is created in the same scope or logic hierarchy as the specified power domain. When you use the `-reuse` option, the specified supply net is not created; instead an existing supply net with the specified name is reused.

```
create_supply_net GND_NET -domain PD1
create_supply_net GND_NET -domain PD2 -reuse
```

When a supply net is created, it is not considered a primary power supply or ground net. To make a specific power supply or ground net of a power domain, the primary supply or ground net, use the `set_domain_supply_net` command.

The `connect_supply_net` command connects the supply net to the specified supply ports or pins. The connection can be within the same level of hierarchy or to ports or pins down the hierarchy.

The following example shows the use of the `connect_supply_net` command to connect supply nets to various supply ports at different levels of hierarchy or power domains.

```
connect_supply_net VDD_SW -ports VDD
connect_supply_net VDD_SW -ports I1/I2/VDD
```

Power Switch

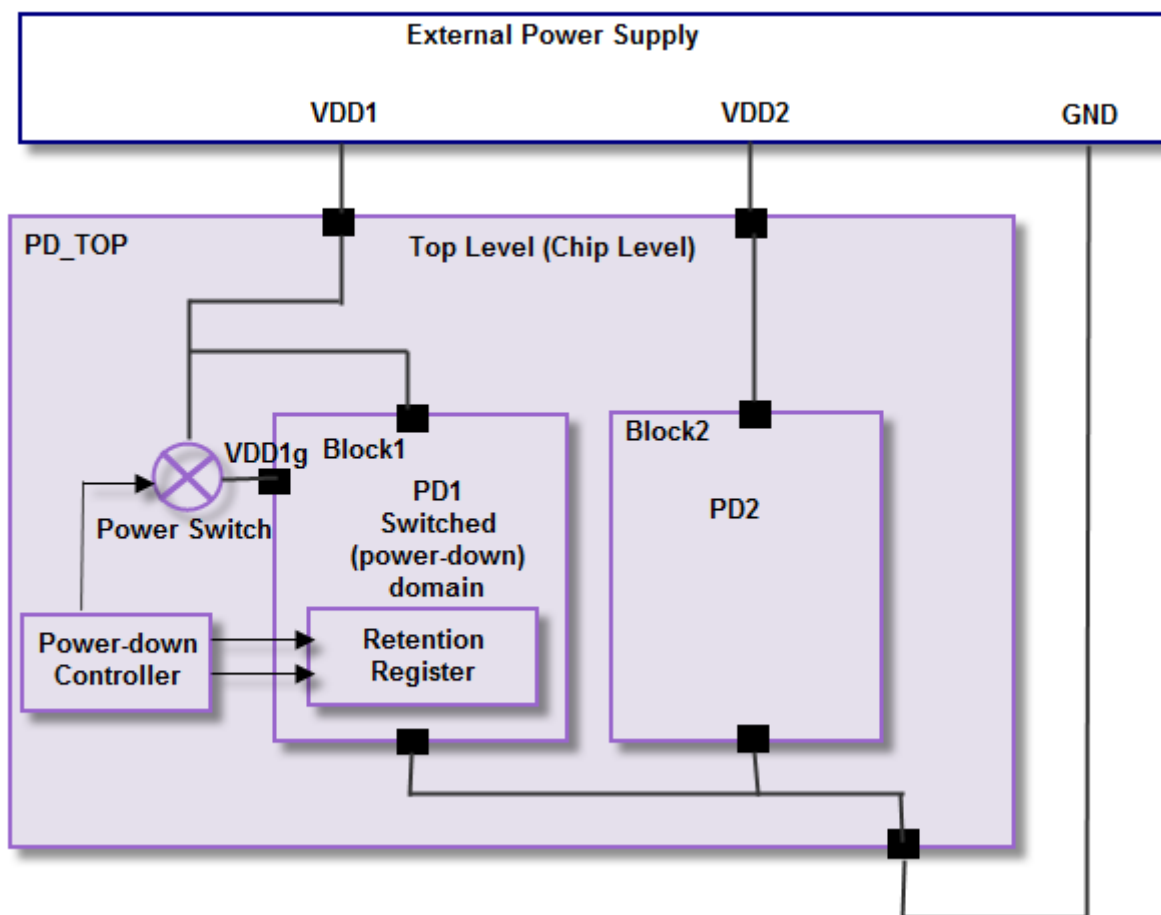
Power switch is a transistor or transistor array that can switch the power on and off for a portion of the chip, either at the VDD power supply (header switch) or at GND (footer switch), thereby cutting off power to portions of the chip during periods of inactivity.

Conceptually, it is a device that turns on and turns off power for a supply net. A switch has an input supply net, an output supply net that can be switched on or off, and at least one input signal to control switching. The switch can optionally have multiple input control signals and one or more output acknowledge signals.

In [Figure 2-10](#), PD1 is a power domain that has two supplies: a switchable supply called VDD1g and an always-on supply from VDD1. The always-on power supply maintains the domain's retention registers while VDD1g is powered down.

The power switch shuts off and turns on the power net VDD1g, either by connecting or disconnecting VDD1 and VDD1g. A power controller logic block at the top level generates the control signal for the switch. It also generates the save and restore signals for the retention registers in domain PD1.

Figure 2-10 Power Switch



The `create_power_switch` command creates a virtual instance of power switch in the scope of the specified power domain. This command lets the tool know that a generic power switch resides in the design at a specific scope or level of hierarchy. The off state of the power switch output is used in the power state table.

Supply Sets

A supply set is an abstract collection of supply nets, consisting of two supply functions, power and ground. A supply set is domain-independent, which means that the power and ground in the supply

set are available to be used by any power domain defined within the scope where the supply set was created. However, each power domain can be restricted to limit its usage of supply sets within that power domain.

You can use supply sets to define power intent at the RTL level, so you can synthesize a design even before you know the names of the actual supply nets.

You can use the `create_supply_set` command to create a supply set.

Supply Set Handles

A supply set handle is an abstract supply set implicitly created for a power domain. A newly created power domain has supply set handles for the domain's primary supply set, a default isolation supply set, and a default retention supply set.

These supply set handles let you synthesize a design even before you create any supply sets, supply nets, and supply ports for the power domain. Before such a design can physically implemented, its supply set handles must be refined, or associated with actual supply sets; and those supply sets must be refined so that they are associated with actual supply nets.

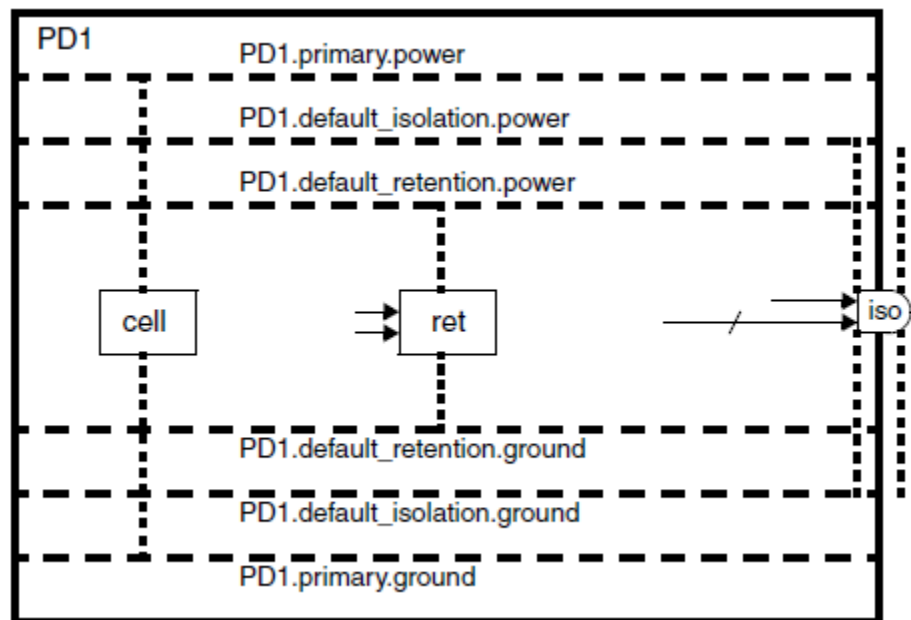
Creating a new power domain with the `create_power_domain` command implicitly creates the following supply set handles:

- `primary` – The primary supply set of the power domain.

- `default_isolation` – The default isolation supply set for any isolation strategy applied to this power domain, which is used when the isolation power or ground is not otherwise specified in the strategy.
- `default_retention` – The default retention supply set for any retention strategy applied to this power domain, which is used when the retention power or ground is not otherwise specified in the strategy.

For example, when you use the `create_power_domain` command to create a power domain called PD1, the command implicitly creates the supply set handles `PD1.primary`, `PD1.default_isolation`, and `PD1.default_retention`, providing the implicit power supply nets shown in [Figure 2-11](#).

Figure 2-11 Implicit Supply Set Handles



Power State Tables

A power state table (PST) lists the allowed combinations of voltage values and states of the power switches for all power domains in the design. It defines the legal combination of states that can exist simultaneously during the operation of the design.

A power state table is a set of power states of a design in which each power state is represented as an assignment of power states to individual power nets.

The `create_pst` command creates a new power state table and assigns a name to the table. The command lists the supply ports or supply nets in a particular order. The `add_port_state` command defines the names of the possible states for each supply port. The `add_pst_state` command lists the allowed combinations of states in the design.

Figure 2-12 Power State Table

	Supplies		
	VDD	VDD_SW_S	VDD_SW
States	s0	HV	LV
	s1	HV	OFF

3

Using UPF Commands to Specify Power Intent

This chapter describes the UPF commands supported by VCS NLP. These commands are used to specify the power intent for low power designs. This chapter contains the following sections:

- [UPF Commands Supported by VCS NLP](#)
- [Using add_power_state Command](#)
- [Using associate_supply_set Command](#)
- [Using the bind_checker Command](#)
- [Using create_power_switch Command](#)
- [Using create_supply_net Command](#)
- [Using map_isolation_cell Command](#)

- Using query_cell_mapped Command
- Using set_design_top Command
- Using set_equivalent Command
- Using set_isolation Command
- Using set_related_supply_net Command
- Using set_port_attributes Command
- Defining Retention Strategies
- Support for Voltage Triplets in the add_port_state and add_power_state Commands
- Using upf_version Command
- Using upf_tool Variable
- Using synopsys_program_name Variable
- Setting Supply Port Default Value
- Support for Auto-Completion of UPF Commands
- Support for Feedthrough Paths Inside Macro Blocks
- Generic Handles Support
- Bias Support for Low Power Designs
- UPF Structural Components
- UPF Script Examples

UPF Commands Supported by VCS NLP

The following sections describe the IEEE 1801 UPF commands supported by VCS NLP. These descriptions are intended to provide an overview of how the commands work in VCS NLP. For more information about using the UPF commands, see the IEEE 1801 (UPF) specification.

- [Basic Power Network Commands](#)
- [Level Shifter Commands](#)
- [Isolation Commands](#)
- [Retention Commands](#)
- [Power State Table Commands](#)
- [Logic Editing Commands](#)
- [Query Commands](#)
- [Simulation/Verification Extension](#)
- [Utility Commands](#)
- [Extending Switch](#)
- [Merging Power Domains](#)
- [Other Supported Power Intent Commands](#)

¹This command/command argument is parsed accepted and ignored by VCS NLP (that is, this command/command argument is not processed or saved in any output).

Basic Power Network Commands

The basic power commands define the power domains of the design and the supply ports, supply nets, and power switches of each domain. Following are the basic power network commands supported by VCS NLP:

- `associate_supply_set`
- `create_power_domain`
- `create_power_switch`
- `create_supply_port`
- `create_supply_net`
- `connect_supply_net`
- `connect_supply_set`
- `create_supply_set`
- `map_power_switch`
- `set_domain_supply_net`

associate_supply_set

The `associate_supply_set` command associates a supply set handle to another supply set. The `supply_set_name` specifies the name of the supply set that must be associated with the handle specified with the `-handle` option.

Syntax

```
associate_supply_set supply_set_name_list
```

```
[ -handle supply_set_handle ]
```

For more information on the `associate_supply_set` command, see [“Using associate_supply_set Command”](#).

create_power_domain

The `create_power_domain` command defines a power supply distribution network at the current scope (hierarchical level) or at the scope of a specified hierarchical instance. A power domain must have one primary supply net and one primary ground net, and can optionally have additional supply nets, supply ports, and power switches.

Syntax

```
create_power_domain domain_name
    [-simulation_only]
    [-elements element_list]
    [-include_scope]
    [-available_supplies supply_set_sef_list]
    [-supply {supply_set_handle [supply_set_ref]}*]
    [-scope instance_name]
    [-define_func_type {supply_function {pg_type_list}}]*1
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

create_power_switch

The `create_power_switch` command creates an instance of a power switch in the scope of a power domain. The switch has at least one input supply port and one output supply port.

Syntax

```
create_power_switch switch_name
```

```

-output_supply_port {port_name [supply_net_name]}
{-input_supply_port {port_name [supply_net_name]}}*
{-control_port {port_name [net_name]}}*
{-on_state {state_name input_supply_port
{boolean_function}}}*
[-off_state {state_name {boolean_function}}]*
[-supply_set supply_set_name]
[-on_partial_state {state_name input_supply_port
{boolean_function}}]*
[-ack_port {port_name net_name [{boolean_function}]]*
[-ack_delay {port_name delay}]*
[-error_state {state_name {boolean_function}}]*
[-domain domain_name]

```

* Asterisk indicates that the item in curly braces can be repeated in the command.

For more information, see [“Using create_power_switch Command”](#).

create_supply_port

The `create_supply_port` command creates a supply port at the current scope or at the scope of a power domain specified with the `-domain` option. A supply port is a power supply connection point between the current scope and the next higher-level (parent) scope. A supply port allows a supply net to cross from one hierarchical level to another.

Syntax

```

create_supply_port port_name
    [-domain domain_name]
    [-direction <in | out | inout>]

```


create_supply_net

The `create_supply_net` command creates a supply net to supply power or ground to a power domain.

Syntax

```
create_supply_net net_name
    [-domain domain_name] [-reuse]
    [-resolve <unresolved | one_hot | parallel
    | resolution_function_name>]
```

For more information on the `resolution_function_name` argument, see [“Specifying User-Defined Supply Net Resolution As a Verilog Function or Task in the Package”](#).

For more information on the `create_supply_net` command, see [“Using create_supply_net Command”](#).

connect_supply_net

The `connect_supply_net` command specifies an explicit connection of a supply net to a list of supply ports, thereby overriding any implicit connections that might otherwise apply. The command specifies the name of an existing supply net and lists the supply ports to be connected to the supply net.

Syntax

```
connect_supply_net net_name
    [-ports list]
    [-pg_type {pg_type_list element_list}] *1
    [-vct vct_name]
    [-pins list]1
    [-cells list]1
    [-domain domain_name]1
    [-rail_connection rail_type]1
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

connect_supply_set

The `connect_supply_set` command connects a supply set to the specified elements.

Syntax

```
connect_supply_set supply_set_ref1
    {-connect {supply_function {pg_type_list}}}*
    [-elements element_list]
    [-exclude_elements exclude_list]
    [-transitive <TRUE | FALSE>]
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

create_supply_set

The `create_supply_set` command creates a supply set at the current level of logic hierarchy. A supply set is a collection of supply nets. A supply set consists of the following two functions:

- Power
- Ground

Syntax

```
create_supply_set set_name
    [-function {func_name [net_name]}]*
    [-update]
```

*Asterisk indicates that the item in curly braces can be repeated in the command.

map_power_switch

The `map_power_switch` command specifies the primary power net and primary ground net for an existing power domain. Every power domain must have these supply nets defined. They are the default power nets connected to the logic elements (or inferred cells) of the power domain.

Syntax

```
map_power_switch {switch_name}1  
  -domain domain_name  
  -lib_cells {list}  
  [-port_map {{mapped_model_port  
    switch_port_or_supply_net_ref}*}]
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

set_domain_supply_net

The `set_domain_supply_net` command specifies the primary power net and primary ground net for an existing power domain. Every power domain must have these supply nets defined. They are the default power nets connected to the logic elements (or inferred cells) of the power domain.

Syntax

```
set_domain_supply_net domain_name  
  -primary_power_net supply_net_name  
  -primary_ground_net supply_net_name
```

Level Shifter Commands

The level shifter commands let you specify the strategy for inserting level shifters between power domains operating at different voltages. Following are the level shifter commands supported by VCS NLP:

- [set_level_shifter](#)
- [map_level_shifter_cell](#)
- [name_format](#)

set_level_shifter

The `set_level_shifter` command allows you to set a strategy for inserting level shifters during implementation. If a level shifter strategy is not specified on a particular power domain, the default level shifter strategy applies to all elements in the power domain, using the default strategy settings.

Syntax

```
set_level_shifter level_shifter_name1
    -domain domain_name
    [-elements element_list]
    [-exclude_elements exclude_list]
    [-source domain_name]
    [-sink domain_name]
    [-applies_to <inputs | outputs | both>]
    [-rule <low_to_high | high_to_low | both>]
    [-threshold value | list]
    [-no_shift]
    [-force_shift]
    [-location <self | parent | sibling | fanin | fanout |
automatic>]
    [-input_supply_set supply_set_name]
    [-output_supply_set supply_set_name]
```

```

[-internal_supply_set supply_set_name]
[-name_prefix string]
[-name_suffix string]
[-instance {{instance_name port_name}*}]
[-transitive <TRUE | FALSE>]
[-update]

```

map_level_shifter_cell

The `map_level_shifter_cell` command maps a particular level-shifter strategy to a library cell or range of library cells. All level-shifter cells belonging to the specified strategy are mapped to one of the library cells specified in the `-lib_cells` list.

Syntax

```

map_level_shifter_cell level_shifter_strategy1
    -domain domain_name
    -lib_cells list
    [-elements element_list]

```

name_format

The `name_format` command defines the format for constructing names of implicitly created objects.

Syntax

```

name_format1
    [-isolation_prefix string]
    [-isolation_suffix string]
    [-level_shift_prefix string]
    [-level_shift_suffix string]
    [-implicit_supply_suffix string]
    [-implicit_logic_prefix string]
    [-implicit_logic_suffix string]

```

Note:

As per UPF, the isolated net/port must be suffixed with `_UPF_ISO` by default. In case of back-to-back isolation on the same net/port, there will be two isolation cells inserted, and hence the suffixes are `_UPF_ISO_0` and `_UPF_ISO`.

Isolation Commands

The isolation commands specify the strategy for inserting isolation cells at the outputs of switched (power-down) domains. Following are the isolation commands supported by VCS NLP:

- [set_isolation](#)
- [set_isolation_control](#)
- [map_isolation_cell](#)

set_isolation

The `set_isolation` command specifies the isolation strategy for a power domain and the elements in that domain on which the strategy is applied. An isolation strategy includes specification of the enable signal net, the clamp value, and the location (inputs, outputs, or both).

Syntax

```
set_isolation isolation_name
  -domain ref_domain_name
  [-elements element_list]
  [-exclude_elements exclude_list]
  -source source_supply_ref
  -sink sink_supply_ref
  [-diff_supply_only <TRUE | FALSE>]
  [-applies_to <inputs | outputs | both>]
```

```

[-applies_to_clamp1 <0 | 1 | any | Z | latch | value>]
[-applies_to_sink_off_clamp1 <0 | 1 | any | Z | latch |
value>]
[-applies_to_source_off_clamp1 <0 | 1 | any | Z | latch
|
value>]
[-isolation_power_net net_name]
[-isolation_ground_net net_name]
[-no_isolation]
[-force_isolation]1
[-location <automatic | self | fanout | fanin1 | faninout1
| parent | sibling1 >]
[-clamp_value {<0 | 1 | Z | latch>*}]
[-isolation_signal signal_list <>]
[-isolation_sense <high | low | {<high | low>*}>]]
[-isolation_supply_set supply_set_list]
[-isolation_sense {<high | low>*}]
[-name_prefix string]1
[-name_suffix string]1
[-sink_off_clamp1 {<0 | 1 | any | Z | latch | value>
simstate_list}]]
[-source_off_clamp {<0 | 1 | any | Z | latch | value>
simstate_list}]]
[-instance {{instance_name port_name}*}]1
[-transitive <TRUE | FALSE>]1
[-update]

```

* Asterisk indicates that the item in curly braces can be repeated in the command.

For more information on the `set_isolation` command, see [“Using set_isolation Command”](#).

set_isolation_control

The `set_isolation_control` command allows the specification of the isolation control signal and the logical sense of that signal. The command identifies an existing isolation strategy and specifies the isolation control signal for that strategy.

Syntax

```
set_isolation_control isolation_name
    -domain domain_name
    -isolation_signal signal_name
    [-isolation_sense <high | low>]
    [-location <self | parent | sibling1 | fanout | automatic1>]
```

map_isolation_cell

The `map_isolation_cell` command maps a particular isolation strategy to a library cell or range of library cells. All isolation cells belonging to the specified strategy are mapped or remapped to one of the library cells specified in the `-lib_cells` list.

Syntax

```
map_isolation_cell isolation_name
    -domain domain_name
    [-elements element_list]1
    [-lib_cells lib_cells_list]1
    [-lib_cell_type lib_cell_type]
    [-lib_model_name model_name {-port {port_name
    net_name}}*]
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

For more information on the `map_isolation_cell` command, see [“Using map_isolation_cell Command”](#).

Retention Commands

The retention commands specify the strategy for inserting retention cells inside switched (power-down) domains. Following are the retention commands supported by VCS NLP:

- [set_retention](#)

- [set_retention_control](#)
- [set_retention_elements](#)
- [set_retention](#)

set_retention

The `set_retention` command specifies which registers in the domain are to be implemented as retention registers and identifies the save and restore signals for the retention functionality. Only the registers implied in the elements are given retention capabilities.

Syntax

```
set_retention retention_name
    -domain domain_name
    [-elements element_list]
    [-retention_power_net net_name]
    [-retention_ground_net net_name]
    [-retention_supply_set ret_supply_set]
    [-no_retention]
    [-save_signal {logic_net <high | low | posedge | negedge>}
    -restore_signal {logic_net <high | low | posedge |
    negedge>}]
    [-save_condition {{boolean_function}}]
    [-restore_condition {{boolean_function}}]
    [-retention_condition {{boolean_function}}]
    [-use_retention_as_primary]
    [-parameters {<RET_SUP_COR | NO_RET_SUP_COR | SAV_RES_COR
    | NO_SAV_RES_COR> *}]
    [-instance1 {{instance_name [signal_name]}*}]
    [-transitive <TRUE | FALSE>]
    [-update]
```

*Asterisk indicates that the item in curly braces can be repeated in the command.

For more information on the `set_retention` command, see [“Using set_retention command”](#).

Note:

This command supports all UPF LRM options except the options related to supply set.

set_retention_control

The `set_retention_control` command allows the specification of the retention control signal and the logical sense of that signal. The command identifies an existing retention strategy and specifies the save and restore signals and logical senses of those signals for that strategy.

Note:

The `set_retention_control` command is deprecated as per UPF 2.0 LRM (IEEE Std 1801-2013).

Syntax

```
set_retention_control retention_name
  -domain domain_name
  -save_signal {{net_name <high | low | posedge | negedge>}}
  -restore_signal {{net_name <high | low | posedge |
    negedge>}}
  [-assert_r_mutex {{net_name <high | low | posedge |
    negedge>}}]*
  [-assert_s_mutex {{net_name <high | low | posedge |
    negedge>}}]*
  [-assert_rs_mutex {{net_name <high | low | posedge |
    negedge>}}]*
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

Note:

The `-assert_r_mutex`, `-assert_rs_mutex`, and `-assert_s_mutex` arguments are not supported in VCS MX.

For more information on the `set_retention_control` command, see [“State Retention Support”](#).

set_retention_elements

The `set_retention_elements` command defines a list of state elements whose collective state shall be maintained coherently if retention is applied to any of these elements in the list.

Syntax

```
set_retention_elements retention_list_name
    [-elements element_list]
    [{-applies_to <required | not_optional | not_required |
optional>}]
    [-exclude_elements exclude_list]
    [-retention_purpose <>]1
    [-transitive <TRUE | FALSE>]1
```

For more information on the `-elements` argument, see [“Creating Retention List”](#).

map_retention_cell

The `map_retention_cell` command provides a mechanism for constraining the implementation choices for retention registers. The command must specify the name of an existing retention strategy and power domain.

Syntax

```
map_retention_cell retention_name_list1
```

```
-domain domain_name  
[-elements element_list]  
[-exclude_elements exclude_list]  
[-lib_cells lib_cell_list]  
[-lib_cell_type lib_cell_type]  
[-lib_model_name name -port_map {{port_name net_ref } *}]
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

Power State Table Commands

A Power State Table (PST) defines the legal combinations of states that can exist at the same time during operation of a design. The state of a power domain consists of the set of on-off states and voltage levels of its supply ports. The power state table is used for synthesis, analysis, and optimization. Following are the PST commands supported by VCS NLP:

- [create_pst](#)
- [add_port_state](#)
- [add_power_state](#)
- [create_power_state_group](#)
- [add_pst_state](#)

create_pst

The `create_pst` command provides a mechanism for constraining the implementation choices for retention registers. The command must specify the name of an existing retention strategy and power domain.

Syntax

```
create_pst table_name
    -supplies supply_list
```

add_port_state

The `add_port_state` command adds state information to a supply port. The command specifies the name of the supply port and the possible states of the port. Each state is specified as a state name and the voltage level for that state.

Syntax

```
add_port_state port_name
    {-state {name nom | min nom max | off}}*
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

Note:

You cannot use `add_port_state` on an RTL port tied to a supply net in the UPF. You can only use this command on the supply ports explicitly created in the UPF using `create_supply_port`.

add_power_state

The `add_power_state` command sets the power states on the nets of a supply set so that they can be used in the power state table.

Syntax

```
add_power_state object_name
    [-supply | -group]
    [-state {state_name
        [-logic_expr {boolean_expression}]
        [-supply_expr {boolean_expression}]
        [-power_expr {power_expression}]
```

```
[-simstate simstate]
[-legal | -illegal]
}] *
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

For more information on the `add_power_state` command, see [“Using add_power_state Command”](#).

create_power_state_group

The `create_power_state_group` command defines a group name that can be used in the `add_power_state` command. The group `group_name` is defined in the current scope.

Syntax

```
create_power_state_group group_name
```

For more information on the `create_power_state_group` command, see [“Creating Power State Group Using create_power_state_group Command”](#).

add_pst_state

The `add_pst_state` command defines the states of each of the supply nets for one possible state of the design.

Syntax

```
add_pst_state state_name
    -pst table_name
    -state supply_states
```

Logic Editing Commands

The logic editing commands create logic nets and logic ports and make connections between these nets and ports, irrespective of the power network. Following are the logic editing commands supported by VCS NLP:

- [create_logic_net](#)
- [create_logic_port](#)
- [connect_logic_net](#)

create_logic_net

The `create_logic_net` command creates a new logic net in the active scope.

Syntax

```
create_logic_net net_name
```

create_logic_port

The `create_logic_port` command creates a new logic port in the active scope and specifies the port direction (`in` for input, `out` for output, or `inout` for bidirectional).

Syntax

```
create_logic_port port_name  
    [-direction <in | out | inout1>]
```

connect_logic_net

The `connect_logic_net` command connects a specified logic net to one or more specified logic ports.

Syntax

```
connect_logic_net net_name
    -ports port_list
    [-reconnect]
```

Query Commands

Each query command returns a list of objects in the design that match the criteria you specify in the command.

Note:

Query commands are not supported in the UPF. You can specify query commands in a configuration file and pass it to the `vcs` command line using the `-lpa_bind <filename>` option.

Following are the query commands supported by VCS NLP:

- `find_objects`
- `query_cell_instances`
- `query_cell_mapped`
- `query_net_ports`
- `query_port_net`
- `query_pst`
- `query_pst_state`

- `query_power_switch`
- `query_upf`
- `query_design_attributes`
- `query_hdl2upf_vct`
- `query_isolation`
- `query_power_domain`
- `query_retention`
- `query_retention_control`
- `query_supply_net`

find_objects

The `find_objects` command finds instances, nets, or ports defined in the logic hierarchy in a specified scope and returns a list of object names that match a given search pattern.

Syntax

```
find_objects scope
  -pattern search_pattern
  [-object_type <inst | port | net | process1>]
  [-direction <in | out | inout>]
  [-transitive [<TRUE | FALSE>]]
  [-regexp | -exact]
  [-ignore_case]
  [-non_leaf | -leaf_only]
```

Note:

- Search is case-sensitive for Verilog objects and case-insensitive for VHDL objects.

- Hierarchical separators are not supported in the search pattern with `-pattern`. Search is limited to a non-hierarchical name in the given scope of search.

query_cell_instances

The `query_cell_instances` command returns a list of instance names for all instances of a given reference cell in the current scope of the design. You can optionally restrict the query to a given power domain.

Syntax

```
query_cell_instances cell_name  
    [-domain domain_name]
```

query_cell_mapped

The `query_cell_mapped` command returns the reference cell name of a given cell instance.

Syntax

```
query_cell_mapped instance_name
```

For more information on the `query_cell_mapped` command, see [“Using query_cell_mapped Command”](#).

query_net_ports

The `query_net_ports` command returns a list of the ports logically connected to a specified net.

Syntax

```
query_net_ports net_name1
```

```
[-transitive <TRUE | FALSE>]  
[-leaf]
```

query_port_net

The `query_port_net` command returns the name of the net logically connected to a specified port, if any such net exists.

Syntax

```
query_port_net port_name1  
    [-conn low | high]
```

query_pst

The `query_pst` command returns information about previously defined power state tables in the active scope. The command `query_pst *` returns a list of the power state table names.

Syntax

```
query_pst table_name  
    [-detailed]
```

query_pst_state

The `query_pst_state` command returns information about the states that have been previously defined for a specified power state table.

Syntax

```
query_pst_state state_name  
    -pst table_name  
    [-detailed]
```

query_power_switch

The `query_power_switch` command returns information about previously defined power switches. The command `query_power_switch *` returns a list of the power switch names.

Syntax

```
query_power_switch switch_name  
    [-detailed]
```

query_upf

The `query_upf` command searches for instances, nets, supply nets, ports, and supply ports in and below the scope or within the extent of a `domain_name`. This command works on the logic hierarchy and can be executed post-UPF annotation.

Syntax

```
query_upf <domain_name | scope>1  
    -pattern search_pattern  
    [-object_type <inst | port | supply_port | net | supply_net  
    | supply_set>]  
    [-inst_type <level_shifter | isolation_cell | switch_cell  
    | retention_cell | all>] |  
    [-direction <in | out | inout>]  
    [-transitive [<TRUE | FALSE>]]  
    [-regexp | -exact]  
    [-ignore_case]  
    [-non_leaf | -leaf_only]
```

query_design_attributes

The `query_design_attributes` command queries attribute information for a specified `element_name` or `model_name`.

Syntax

```
query_design_attributes  
    <-element element_name | -model model_name>  
    [-detailed]
```

query_hdl2upf_vct

The `query_hdl2upf_vct` command can list and query any previously defined value conversion table (VCT).

Syntax

```
query_hdl2upf_vct vct_name  
    [-detailed]
```

query_isolation

The `query_isolation` command can list the previously defined isolation strategies for the specified power domain `domain_name`. All elements returned are referenced to the current scope.

Syntax

```
query_isolation isolation_name  
    -domain ref_domain_name  
    [-detailed]
```

query_power_domain

The `query_power_domain` command queries the parameters of a power domain.

Syntax

```
query_power_domain domain_name  
    [-detailed]
```

query_retention

The `query_retention` command lists the previously defined retention strategies for the specified power domain `domain_name`. All elements returned are referenced to the current scope.

Syntax

```
query_retention retention_name
    -domain domain_name
    [-detailed]
```

query_retention_control

The `query_retention_control` command queries the retention control information for a retention strategy.

Syntax

```
query_retention_control retention_name
    -domain domain_name
    [-detailed]
```

query_supply_net

The `query_supply_net` command returns the information about a previously created supply net.

Syntax

```
query_supply_net net_name
    [-domain domain_name]
    [-detailed]
```

Simulation/Verification Extension

The simulation and verification extension commands support low-power checking by simulation and verification tools. Following are the simulation and verification extension commands supported by VCS NLP:

- [set_design_top](#)
- [set_partial_on_translation](#)

set_design_top

The `set_design_top` command specifies the root of the design.

Syntax

```
set_design_top root
```

For more information on the `set_design_top` command, see [“Using set_design_top Command”](#).

set_partial_on_translation

The `set_partial_on_translation` command specifies the translation of the `PARTIAL_ON` state to either `FULL_ON` or `OFF` for purposes of evaluating the power state of supply sets and power domains.

Syntax

```
set_partial_on_translation  
    [FULL_ON | OFF]  
    [-full_on_tools tool_list]  
    [-off_tools tool_list]
```

Utility Commands

These are the commands to load and execute UPF commands from a file, to write a set of UPF commands to a file, and to set the hierarchical scope for subsequent UPF commands. Following are the utility commands supported by VCS NLP:

- [load_upf](#)
- [set_scope](#)
- [set_design_attributes](#)
- [set_port_attributes](#)

load_upf

The `load_upf` command executes the UPF commands in a specified file.

Syntax

```
load_upf
  load_upf upf_file_name
  [-scope instance_name]
  [-simulation_only]
```

For more information on the `-simulation_only` attribute, see [“-simulation_only Support”](#).

set_scope

The `set_scope` command specifies the hierarchical scope of subsequent commands, including UPF commands.

Syntax

```
set_scope [instance]
```

set_design_attributes

The `set_design_attributes` command sets the attributes of one or more cells.

Syntax

```
set_design_attributes  
  [-models model_list | -elements element_list]  
  {-attribute name value}*  
  *
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

The `-models` option specifies a list of design models on which the attributes are set. The `-elements` option specifies a collection of cells or supply sets on which the attributes are set. In the absence of the `-models` or `-elements` option, the attribute is set on the current top-level design.

The `-attribute` option specifies the name and value of the attributes to be set on the cells specified with the `-elements` option.

set_port_attributes

The `set_port_attributes` command specifies a collection of ports where the attributes must be set for the source or sink, for the interface of the power domains, when used with the `set_isolation` command.

Syntax

```
set_port_attributes
  [-model name]
  [-elements element_list]
  [-exclude_elements1 element_exclude_list]
  [-ports port_list]
  [-exclude_ports port_exclude_list]
  [-applies_to <inputs | outputs | both>]
  [-attribute {name value}]*
  [-clamp_value <0 | 1 | any | Z | latch | value>]
  [-sink_off_clamp1 <0 | 1 | any | Z | latch | value>]
  [-source_off_clamp1 <0 | 1 | any | Z | latch | value>]
  [-driver_supply supply_set_ref]
  [-receiver_supply supply_set_ref]
  [-pg_type pg_type_value]
  [-related_power_port supply_port_name]
  [-related_ground_port supply_port_name]
  [-related_bias_ports supply_port_name_list]
  [-feedthrough]
  [-unconnected]
```

* Indicates that the item in square braces can be repeated in the command.

For more information on the `set_port_attributes` command, see [“Using set_port_attributes Command”](#).

Extending Switch

The following UPF command allows you to extend a switch:

set_power_switch

The `set_power_switch` command extends a switch by adding the input supply port(s), output supply port(s), and states to the switch. The supply ports are connected to the specified nets.

Syntax

```
set_power_switch switch_name1
  -output_supply_port {port_name [supply_net_name]}
  {-input_supply_port {port_name [supply_net_name]}}*
  {-control_port {port_name}}*
  {-on_state {state_name input_supply_port
    {boolean_function}}}*
  [-supply_set supply_set_name]
  [-on_partial_state {state_name input_supply_port
    {boolean_function}}]*
  [-off_state {state_name {boolean_function}}]*
  [-error_state {state_name {boolean_function}}]*
```

* Asterisk indicates that the item in curly braces can be repeated in the command

Merging Power Domains

The following UPF command allows you to merge power domains:

merge_power_domains

The `merge_power_domains` command merges two or more existing power domains into a single, new power domain. The merged domains can no longer be referenced separately.

Syntax

```
merge_power_domains new_domain_name1
  -power_domains list
  [-scope instance_name]
  [-all_equivalent]
```

Other Supported Power Intent Commands

Following are the other UPF commands supported by VCS NLP to specify power intent:

- `add_domain_elements`
- `bind_checker`
- `create_composite_domain`
- `create_hdl2upf_vct`
- `create_upf2hdl_vct`
- `describe_state_transition`
- `load_simstate_behavior`
- `load_upf_protected`
- `set_design_top`
- `set_equivalent`
- `set_simstate_behavior`
- `upf_version`

add_domain_elements

The `add_domain_elements` command provides the ability to separate the creation of a power domain from the specification of the elements contained within it.

Syntax

```
add_domain_elements domain_name
```

```
-elements element_list1
```

bind_checker

The `bind_checker` command inserts checker modules into a design without modifying the design code or introducing functional changes. The mechanism for binding the checkers to design elements relies on the SystemVerilog `bind` directive. The `bind` directive causes one module to be instantiated within another, without having to explicitly alter the code of either. This facilitates the complete separation between the design implementation and any associated verification code.

Syntax

```
bind_checker instance_name
  -module checker_name
  [-elements element_list]
  [-ports {{port_name net_name}*}}]
  [-parameters {{param_name param_value}*}}]
  [-name_prefix string]
  [-name_suffix string]
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

For more information on the `bind_checker` command, see [“Using the bind_checker Command”](#).

create_composite_domain

The `create_composite_domain` command defines a composite power domain composed of one or more subdomains. A composite power domain is a simple container for a set of power domains. Unlike a power domain, a composite domain has no corresponding physical region on the silicon.

Syntax

```
create_composite_domain composite_domain_name1
    [-subdomains subdomain_list]
    [-supply {supply_set_handle [supply_set_ref]}] *
    [-update]
```

* Asterisk indicates that the item in curly braces can be repeated in the command

create_hdl2upf_vct

The `create_hdl2upf_vct` command defines a VCT from an HDL logic type to the state type of the supply net value when that value is propagated from HDL port to a UPF supply net.

Syntax

```
create_hdl2upf_vct vct_name1
    -hdl_type {<vhdl | sv> [typename]}
```

create_upf2hdl_vct

The `create_upf2hdl_vct` command defines VCT that can be used in converting UPF `supply_net_type` values into HDL logic values.

Syntax

```
create_upf2hdl_vct vct_name1  
  -hdl_type {<vhdl | sv> [typename]}  
  -table {{from_value to_value}*}
```

* Asterisk indicates that the item in curly braces can be repeated in the command.

describe_state_transition

The `describe_state_transition` command specifies the legality of a transition from one object's named power state to another. The occurrence of an unnamed state during the transition from one state to another is ignored. `-from` and `-to` specify many-to-many transitions. `-paired` specifies one or more one-to-one transitions.

Syntax

```
describe_state_transition transition_name  
  -object object_name  
  -from {from_list} -to {to_list} -paired {{from_state  
  to_state}*}}  
  [-legal | -illegal]
```

* Asterisk indicates that the item in curly braces can be repeated in the command

load_simstate_behavior

The `load_simstate_behavior` command loads a UPF file that only contains `set_simstate_behavior` commands and applies these to the models in the library `lib_name`.

Syntax

```
load_simstate_behavior lib_name1  
    -file file_list
```

load_upf_protected

The `load_upf_protected` command loads a UPF file in a protected environment that prevents corruption of existing variables.

Syntax

```
load_upf_protected upf_file_name1  
    [-hide_globals]  
    [-scope scope_name]  
    [-version upf_version]  
    [-params param_list]
```

set_design_top

The `set_design_top` command specifies the root of the design.

Syntax

```
set_design_top root
```

set_equivalent

The `set_equivalent` command declares that the specified supplies are electrically or functionally equivalent. This command returns an empty string on success and an error on failure.

Syntax

```
set_equivalent  
    [-function_only]  
    [-nets supply_net_name_list]  
    [-sets supply_set_name_list]]
```


For more information on the `set_equivalent` command, see [“Using set_equivalent Command”](#).

set_simstate_behavior

The `set_simstate_behavior` command specifies the simstate behavior for models or instances. If `ENABLE` is specified, the simstate simulation semantics are applied for every supply set automatically connected to an instance of the model.

Syntax

```
set_simstate_behavior <ENABLE | DISABLE>
    [-lib name]1
    [-models list]
    [-elements element_list]
    [-exclude_elements exclude_list]
```

upf_version

The `upf_version` command returns a string value representing the UPF version currently being used by the tool reading the UPF file.

Syntax

```
upf_version [string]1
```

For more information on the `upf_version` command, see [“Using upf_version Command”](#).

Using add_power_state Command

The `add_power_state` command defines power states of a supply set. This section describes the simulation semantics of the `add_power_state` command.

The following sections describe the `add_power_state` support in VCS NLP:

- [Use Model](#)
- [Supply Expressions and Logic Expressions](#)
- [Power State Evaluation of Supply Set in Presence of -supply_expr/-logic_expr](#)
- [Defining Power State Tables Using the add_power_state Command](#)
- [Limitations of add_power_state Support](#)

Use Model

Following is the syntax of the `add_power_state` command:

```
add_power_state object_name
{-state state_name {[-supply_expr {boolean_function}]
  [-logic_expr {boolean_function}]
  [-simstate simstate] [-legal | -illegal] [-update]}}
[-simstate simstate] [-legal | -illegal]
[-update]
```

Where, `object_name` is the name of a supply set. You can use the `add_power_state` command to specify:

- Single or multiple states of a supply set

- Combination of supply expression and logic expression to define the state
- The simstate (NORMAL, CORRUPT, CORRUPT_ON_ACTIVITY) for any power domain driven by a supply set

Supply Expressions and Logic Expressions

The supply expression (-supply_expr) contains the supply set function (power, ground, nwell, pwell). The expression should contain the state and voltage value or a range of voltage values of a supply set function. The expression can be combination of multiple functions of a supply set (see [Example 3-1](#)).

Example 3-1 Supply Expressions

```
add_power_state SS_TOP -state HV {-supply_expr {power ==  
  ~{FULL_ON, 1.08} } }
```

The logic expression (-logic_expr) is a regular SystemVerilog Boolean expression. You must use logic signals from the design to create logic expressions (see [Example 3-2](#)).

Example 3-2 Logic Expressions

```
add_power_state TOP -state HV {-logic_expr {pd_top} -  
  simstate NORMAL }
```

A particular supply set state can contain a supply expression, logic expression, or both.

Power State Evaluation of Supply Set in Presence of `-supply_expr/-logic_expr`

In alignment with IEEE 1801-2013 (Section 9.3.2 Power state determination), the power state of a supply set is determined as follows:

1. If a power state has a supply expression, but no logic expression, then the power state is active if the supply expression is True
2. Else if a power state has a logic expression, but no supply expression, then the power state is active if the logic expression is True
3. Else if a power state has both a logic expression and a supply expression, then:
 - If the logic expression is True, the power state is active
 - If the supply expression is False, issue an error message

As indicated above, `-logic_expr` gets highest priority when both `-logic_expr` and `-supply_expr` are defined for a power state.

Defining Power State Tables Using the `add_power_state` Command

You can use the `add_power_state` command to perform the following:

- Define power state tables (PSTs) using the following two options:
 - 1) `-supply`
 - 2) `-group`

For more information, see [“Use Model”](#) and [“Defining PSTs Using add_power_state Options”](#) sections.

- Create power state group using the `create_power_state_group` command. For more information, see [“Creating Power State Group Using create_power_state_group Command”](#) section.
- Compose states hierarchically using the `-group` option. For more information, see [“Composing States Hierarchically”](#) section.

Use Model

You can use the `-supply` and `-group` options of the `add_power_state` command to define PSTs. Following is the syntax:

```
add_power_state [-supply | -group] object_name
                [-simstate simstate]
                {-state state_name { [-supply_expr {boolean_function}]
                                     [-logic_expr {boolean_function}]
                                     [-simstate simstate] }}
```

Where,

- `supply` — Allows you to specify the name of the supply set for which a state has to be created. This option is not mandatory. Even in the absence of this option, the given `object_name` is matched with an existing supply set.

When `-supply` is used:

- `object_name` must be the name of a supply set or a supply set handle
- The `-state` option can be used multiple times

- The following operators are supported for `-supply_expr`:
`==` and `&&` (this implies that both power and ground functions can be specified in the same Boolean expression)
- The `-supply_expr boolean_expression` references functions of supply sets or supply set handles
- `-group` — Allows you to specify the name of the already created group (at the given scope). For more information on group, see [“Creating Power State Group Using create_power_state_group Command”](#).

When `-group` is used:

- The `-state` option can be used multiple times
- The `-logic_expr boolean_expression` references supply set states, group states, or PST states. The Boolean expression can also be a mixture of these three types of states.
- Logic nets/ports/pins are not allowed in the Boolean expression. Specifying logic nets/ports/pins results in an error.
- Using `-supply_expr` is not supported (as per IEEE Std 1801-2009 LRM).
- Using `-simstate` (both within `-state` and outside `-state`) is not supported (as per IEEE Std 1801-2009 LRM).

Note:

You cannot use both `-supply` and `-group` options with the `add_power_state` statement. You can only use one of these options.

Creating Power State Group Using `create_power_state_group` Command

The `create_power_state_group` command defines a name for a group of related power states of the `add_power_state` command. Following is the syntax of the `create_power_state_group` command:

```
create_power_state_group group_name
```

Where, `group_name` is the simple name of the group that is defined in the current scope. The group name is used with the `-group` option of the `add_power_state` command.

Power states of a group may be defined in terms of power states of supply sets, power domains, composite domains, instances, and other groups. Power states of two or more different groups may refer to power states of the same object.

Defining PSTs Using `add_power_state` Options

To define PSTs using `add_power_state`, the following options are used:

- `-supply`
- `-group`

Consider that the following PST is to be built:

STATE\SUPPLY_SET	SS1	SS2
RUN	ON	ON
SLEEP	ON	OFF

Defining the PST is a three-step process:

1. Define the supply set states (ON and OFF listed in the above table)
2. Create a new group (also known as PST name)
3. Define the PST states (RUN and SLEEP listed in the above table)

Defining Supply Set States Using the `add_power_state` Command

Follow the guidelines listed below to define the supply set states:

1. Use `-supply` and specify the name of the supply set for which a state has to be created.
2. Use `-state` to specify the name of the supply set state.
3. Use `-supply_expr` to specify the Boolean expression for the supply set state.

The exact usage of `add_power_state` to define the supply set states is listed below:

```
# Creating power state ON for SS1
add_power_state -supply SS1 \
    -state ON {-supply_expr {(power == {FULL_ON 0.8}) &&
    (ground == {FULL_ON 0})}}
```

```
# Creating power states ON and OFF for SS2
add_power_state -supply SS2 \
    -state ON {-supply_expr {(power == {FULL_ON 0.8}) &&
    (ground == {FULL_ON 0})}}
```

```
add_power_state -supply SS2 \
    -state OFF {-supply_expr {(power == {OFF}) && (ground ==
    {FULL_ON 0})}}
```


Creating a Group Using the `create_power_state_group` Command

Create a new group (PST name) using the `create_power_state_group` command. This command takes only one argument which is the group name. The group is created at the given scope.

```
create_power_state_group GROUP1
```

Defining PST States Using the `add_power_state` Command

Follow the guidelines listed below to define the PST states:

1. Use `-group` and specify the name of the already created group (at the given scope).
2. Use `-state` to specify the name of the PST state.
3. Use `-logic_expr` to specify the Boolean expression for the PST state.

The exact usage of `add_power_state` to define the PST states is listed below:

```
add_power_state -group GROUP1
  -state RUN {-logic_expr {SS1 == ON && SS2 == ON}}
  -state SLEEP {-logic_expr {SS1 == ON && SS2 == OFF}}
```

Composing States Hierarchically

This section lists an example on how states present in multiple (for example, scope-level) groups (PSTs) can be composed hierarchically to create a top-level group. The `-group` option is used to achieve this. This avoids the exponential explosion of PST size.

Consider the following two scope-level groups (PSTs):

Table 3-1 Group Defined at SCOPE1

STATE\SUPPLY_SET	SS1	SS2
RUN	ON	ON
SLEEP	ON	OFF

Table 3-2 Group Defined at SCOPE2

STATE\SUPPLY_SET	SS1	SS2
RUN	ON	ON
IDLE	ON	OFF

For example, if your top-level group has the following states:

- When SCOPE1 is in RUN, SCOPE2 will be in RUN
- When SCOPE1 is in SLEEP, SCOPE2 will be in RUN
- When SCOPE1 is in SLEEP, SCOPE2 will be in IDLE

With the `-group` option, this intent can be easily captured with the following command:

```
create_power_state_group TOP_SCOPE_GROUP
add_power_state -group TOP_SCOPE_GROUP
    -state RUN_RUN {-logic_expr {SCOPE1/GROUP1 == RUN &&
SCOPE2/GROUP1 == RUN}}
    -state SLEEP_RUN {-logic_expr {SCOPE1/GROUP1 == SLEEP &&
SCOPE2/GROUP1 == RUN}}
    -state SLEEP_IDLE {-logic_expr {SCOPE1/GROUP1 == SLEEP &&
SCOPE2/GROUP1 == IDLE}}
```

Examples

This section lists the examples that show the usages that are syntactically and semantically correct and the usages that result in an error.

Example-1

```
create_supply_set SS1
create_power_domain PD_TOP
create_power_state_group SS1
```

The `create_power_state_group` command results in an error as there is another UPF object (supply set) with the name SS1.

Example-2

```
set_scope /
create_supply_set SS1
create_supply_set SS2
add_power_state -supply SS1 \
    -state ON {-supply_expr {(power == {FULL_ON 0.8}) &&
(ground == {FULL_ON 0})}}
add_power_state -supply SS2 \
    -state ON {-supply_expr {(power == {FULL_ON 0.9}) &&
(ground == {FULL_ON 0})}}

create_power_state_group GROUP1
add_power_state -group GROUP1
    -state RUN {-logic_expr {SS1 == ON && SS2 == ON}}

set_scope MID_INST
create_supply_set SS1
create_supply_set SS2
add_power_state -supply SS1 \
    -state ON {-supply_expr {(power == {FULL_ON 0.8}) &&
(ground == {FULL_ON 0})}}
add_power_state -supply SS2 \
    -state ON {-supply_expr {(power == {FULL_ON 0.9}) &&
(ground == {FULL_ON 0})}}

add_power_state -group GROUP1
    -state RUN {-logic_expr {SS1 == ON && SS2 == ON}}
```

The last `add_power_state` command results in an error because this command is referring to a group by name `GROUP1` that is not created at the current scope `MID_INST`.

Example-3

```
set_scope /
create_supply_set SS1
create_supply_set SS2
add_power_state -supply SS1 \
    -state ON {-supply_expr {(power == {FULL_ON 0.8}) &&
    (ground == {FULL_ON 0})}}
add_power_state -supply SS2 \
    -state ON {-supply_expr {(power == {FULL_ON 0.9}) &&
    (ground == {FULL_ON 0})}}

create_power_state_group GROUP1
add_power_state -supply SS1 -group GROUP1
    -state RUN {-logic_expr {SS1 == ON && SS2 == ON}}
```

The last `add_power_state` command results in an error because the options `-supply` and `-group` are used together. The syntax does not allow these two options to be used together.

Example-4

```
set_scope /
create_supply_set SS1
add_power_state -supply SS1 \
    -state {ON -supply_expr {(power == {FULL_ON 0.8}) &&
    (ground == {FULL_ON 0})}}
```

The `add_power_state` command has a curly brace before the state name. The curly brace should appear after the state name. This command results in an error because of the syntax error.

Example-5

```

set_scope /
create_supply_set SS1
add_power_state -supply SS1 \
    -state ON {-supply_expr {(power == {FULL_ON 0.8}) &&
    (ground == {FULL_ON 0})}}
add_power_state -supply SS1 \
    -state ON {-logic_expr {port1 && port2}}

```

Here, all the `add_power_state` commands run successfully.

Interaction Between `create_pst/add_pst_state` and `add_power_state` Syntax

This section describes the interaction that is allowed between the old syntax (defining PSTs using the `create_pst` and `add_pst_state` commands) and new syntax (defining PSTs using `-supply` and `-group` options) of the `add_power_state` command for defining PSTs.

The interaction style involves using old syntax at lower level and new syntax at higher level. Two cases are possible here.

- Case-1: Create supply set states using the old syntax, and create PST using the new syntax
- Case-2: Create PST using the old syntax, and create top-level PST using the new syntax

Case-1: Create supply set states using old syntax, and create PST using new syntax

Following is an example to illustrate this case.

The supply set states are created using the old syntax of `add_power_state` as follows:

```

add_power_state SS1 -state P_ON {-supply_expr {power ==
`{FULL_ON, 1.08}}}}
add_power_state SS1 -state G_ON {-supply_expr {ground ==
`{FULL_ON, 0}}}}

add_power_state SS2 -state P_ON {-supply_expr {power ==
`{FULL_ON, 0.08}}}}
add_power_state SS2 -state G_ON {-supply_expr {ground ==
`{FULL_ON, 0}}}}
add_power_state SS2 -state P_OFF {-supply_expr {power ==
`{OFF}}}}

```

For example, if you want to create the following PST using the above supply set states.

STATE\SUPPLY_SET	SS1.power	SS1.ground	SS2.power	SS2.ground
RUN	P_ON	G_ON	P_ON	G_ON
SLEEP	P_ON	G_ON	P_OFF	G_ON

This can be achieved using the `-group` option as follows:

```

create_power_state_group MY_GROUP
add_power_state -group MY_GROUP \
  -state RUN {-logic_expr {SS1 == P_ON && SS1 == G_ON && SS2
== P_ON && SS2 == G_ON}}
  -state SLEEP {-logic_expr {SS1 == P_ON && SS1 == G_ON &&
SS2 == P_OFF && SS2 == G_ON}}

```

Case-2: Create PST using old syntax, and create top-level PST using new syntax

Following is an example to illustrate this case.

Consider the following scope-level PSTs created using the old syntax:

Table 3-3 PST Defined at SCOPE1

STATE\SUPPLY_SET	SS1.power	SS1.ground	SS2.power	SS2.ground
RUN	P_ON	G_ON	P_ON	G_ON
SLEEP	P_ON	G_ON	P_OFF	G_ON

Table 3-4 PST Defined at SCOPE2

STATE\SUPPLY_SET	SS1.power	SS1.ground	SS2.power	SS2.ground
RUN	P_ON	G_ON	P_ON	G_ON
SLEEP	P_ON	G_ON	P_OFF	G_ON

The commands used to create the above PSTs (for example, PST defined at SCOPE1) are as follows:

```

add_power_state SS1 -state P_ON {-supply_expr {power ==
`{FULL_ON, 1.08}}}}
add_power_state SS1 -state G_ON {-supply_expr {ground ==
`{FULL_ON, 0}}}}

add_power_state SS2 -state P_ON {-supply_expr {power ==
`{FULL_ON, 0.08}}}}
add_power_state SS2 -state G_ON {-supply_expr {ground ==
`{FULL_ON, 0}}}}
add_power_state SS2 -state P_OFF {-supply_expr {power ==
`{OFF}}}}

create_pst MY_PST -supplies {SS1.power SS1.ground SS2.power
SS2.ground}
add_pst_state RUN -pst MY_PST -state {P_ON G_ON P_ON G_ON}
add_pst_state SLEEP -pst MY_PST -state {P_ON G_ON P_OFF G_ON}

```

If you want to create the following top-level PST by combining the states of the two scope-level PSTs:

STATE\PST	SCOPE1/MY_PST	SCOPE2/MY_PST
RUN_RUN	RUN	RUN
SLEEP_RUN	SLEEP	RUN
SLEEP_SLEEP	SLEEP	SLEEP

This can be achieved using the `-group` option in the following way:

```
create_power_state_group TOP_SCOPE_GROUP
add_power_state -group TOP_SCOPE_GROUP
    -state RUN_RUN {-logic_expr {SCOPE1/MY_PST == RUN &&
SCOPE2/MY_PST == RUN}}
    -state SLEEP_RUN {-logic_expr {SCOPE1/MY_PST == SLEEP &&
SCOPE2/MY_PST == RUN}}
    -state SLEEP_SLEEP {-logic_expr {SCOPE1/MY_PST == SLEEP
&& SCOPE2/MY_PST == SLEEP}}
```

Note:

The scope-level PSTs can be created either by using `add_power_state` or by using `add_port_state`. The main thing to note here is how scope-level PSTs can be used to create a top-level PST using the `-group` option.

Limitations

Following is the limitation of this feature:

- The `-legal/-illegal` options are ignored when specified with `-group`

Limitations of `add_power_state` Support

Following are the limitations of `add_power_state` support:

- Only one state can be defined per one `add_power_state` command
- Only one supply function is allowed in each supply expression
- `-legal/-illegal` options are not supported
- Interval function with `-logic_expr` is not supported
- `CORRUPT_STATE_ON_CHANGE`, `CORRUPT_STATE_ON_ACTIVITY`, and `NOT_NORMAL` simstates are not supported

Using `associate_supply_set` Command

The `associate_supply_set` command associates a supply set with a domain or strategy. The supply set includes the specific nets for each of the supplies (power, ground, and so on).

Following is the syntax of the `associate_supply_set` command:

```
associate_supply_set supply_set_name
    -handle supply_set_handle
```

Where,

- `supply_set_name` is a rooted name of a supply set to associate with a supply set handle
- `supply_set_handle` is the supply set handle with which the supply set is associated

When an implicit supply set (supply handle) is associated with another supply set, they are resolved to same real supply net Power/Ground pair, hence they have identical switching/corruption semantics. When two supply sets are associated, they are treated as virtually connected.

The following error checking applies to `associate_supply_set` command:

- Associating an implicit supply set (supply handle) either directly using `associate_supply_set` or some other command like `set_domain_supply_net` can be done only once
- It is an error to make circular associations
- You cannot use the `associate_supply_set` command with explicit supply sets in the `-handle` option
- Implicit supply set (supply handle) specified with the `-handle` option must be at or below the scope of supply set with which it is associated
- Associating supply handle of a domain to a supply set makes the supply set available in the domain to which the supply handle belongs to

Using the `bind_checker` Command

The `bind_checker` command inserts checker modules into a design without modifying the design code or introducing functional changes. The mechanism for binding the checkers to design elements relies on the SystemVerilog `bind` directive. The `bind` directive causes one module to be instantiated within another,

without having to explicitly alter the code of either. This facilitates the complete separation between the design implementation and any associated verification code.

Following is the syntax of the `bind_checker` command:

Syntax

```
bind_checker instance_name
-module checker_name
[-elements element_list/UPF object*]
[-ports {{port_name net_name/upf_name*}}]
[-parameters {{parameter_name constant_expression/
upf_name*}}]
[-name_prefix string]
[-name_suffix string]
```

Arguments

instance_name

The name used to instance the checker module in each design element.

-module *checker_name*

The name of a checker module containing the verification code. The verification code itself shall be written in SystemVerilog, but it can be bound to either a SystemVerilog or VHDL instance.

-elements *element_list*

The list of design elements or UPF objects like Power Domain, Isolation Strategy, Retention Strategy, Power Switch.

-ports *{{port_name net_name}}*

The association of signals to the checker's ports.

port_name is a port defined on the interface of *checker_name* and *net_name* is a name of a net relative to the scope where *checker_name* is being instantiated.

Signals in the target instance are bound by position to inputs in the bound checker module through the portlist. Thus, the bound module has access to all signals in the scope of the target instance by simply adding them to the port list, which facilitates sampling of the arbitrary design signals.

-parameters *{{ parameter_name constant_expression/
upf_name }}*

Allows parameter passing to the checker module.

-name_prefix *prefix*

In case the checker is bound to strategy, then *name_prefix* option suggests the prefix to be used for uniquely naming the checker instance.

-name_suffix *suffix*

In case the checker is bound to a strategy, then *name_prefix* option suggests the suffix to be used for uniquely naming the checker instance.

*** UPF Name Alias**

As part of the extensions to the UPF *bind_checker* command, the UPF names must be prefixed with '@' for distinction.

For example, consider the following custom bind file. Here, the checker to each power domain is bound using the `-elements` option, which uses named aliases (`@$PD`) .

```
proc primary_power_off {} {
    foreach PD [query_power_domain *] {
        .....
        bind_checker primary_power_off_msg \
            -module primary_power_off \
            -elements [list @$PD] \
            -parameters $params_list \
            -ports $ports_list
    }
}
primary_power_off
```

Since you are binding the checker to a power domain (extension of `bind_checker`), UPF name alias '@' must be used with the `-elements` option.

- If the ports list passed with the `-ports` option contains any of the UPF variables (returned by the UPF query command), then the target for the binding specified with `-elements` must be any UPF object like Power Domain, Power Switch, Isolation Strategy, and Retention Strategy.
- Cross-binding is supported between power domain, isolation and retention strategies only. For example, elements returned by `query_isolation` can be bound to a power domain.
- Binding of PST supplies returned by the `query_pst` command as ports is not supported.

Using `create_power_switch` Command

The `create_power_switch` command (see [“create_power_switch”](#) for syntax) creates an instance of a power switch in the scope of a power domain. The switch has at least one input supply port and one output supply port. When the switch is on, it connects the input supply port (or one of multiple input supply ports) to the output supply port. When the switch is off, the output supply port is shut down and has no power.

The command can optionally specify the characteristics of the acknowledge port or ports. The ports specified in the command are the connections points of the power switch.

The following sections describe this command:

- [Handling `create\_power\_switch -ack\_port` Connections](#)
- [Using Supply Set of the Power Switch for Acknowledge Port Corruption](#)
- [Driver Checks on Power Switch Acknowledge Port Signal](#)

Handling `create_power_switch -ack_port` Connections

The following points describe how the connections are handled between the `-ack_port` option of the `create_power_switch` command and its corresponding design nets and ports.

- For `-ack_port` of type variable whose resolution function is not defined, the connection is established if the design node does not contain any driver. VCS NLP issues an error message if the design node contains a driver.

- For `-ack_port` of type wire whose resolution function is defined, the connection is established if the design node does not contain any driver. VCS NLP issues a warning message and adds the acknowledge port connection as a multiple driver, if the design node contains a driver.

Using Supply Set of the Power Switch for Acknowledge Port Corruption

VCS NLP supports `-supply_set` option of the `create_power_switch` command.

The `-supply_set` option allows you to specify related supply set for both the control and acknowledge ports of the power switch. If any of the supply function of this related supply set goes OFF, then the acknowledge port is corrupted.

For backward compatibility, enable the `SKIP_ACK_PORT_CORRUPTION` variable in the Backward Compatibility configuration file and recompile. By default, this variable is disabled.

Driver Checks on Power Switch Acknowledge Port Signal

For a design node of type wire specified with `-ack_port` on the `create_power_switch` command, VCS NLP generates the “[UPF_MDOAPS] Multiple drivers on ack port signal” warning message by default, if the node is already connected in the design. Following is the sample warning message:

```
Warning- [UPF_MDOAPS] Multiple drivers on ack port signal
top.upf, 21
```

Multiple drivers on Power Switch 'tb/test/SW_1' ack port signal 'tb/test/out1'.
HDL Driver Source : test.v, 7.

Consider the following design and UPF files:

Example 3-3 test.v

```
module test (in1,out1);
  input in1;
  output out1;
  reg sw_en;
  ...
  core1 CORE1 (.in1(in1),.sw_en(sw_en),.out1(out1));
endmodule
module core1 (in1,sw_en,out1);
  input in1, sw_en;
  output reg out1;
  ...
endmodule
```

Example 3-4 top.upf

```
#Power Switch in scope "test"

create_power_switch SW_1 -domain TOP \
  -output_supply_port {VDD VDD} \
  -input_supply_port {VDDIN_1 VDDIN_1} \
  -control_port {sw_en sw_en} \
  -ack_port {ack out1 {!sw_en}} \
  -on_state {on1 VDDIN_1 {sw_en}}
```

In [Example 3-3](#), the acknowledge port signal `out1` is connected to instance `CORE1` through port map and has a driver in HDL. The acknowledge port signal `out1` is driven by the output port `out1` of type `reg` in `core1` module.

If you use the `-power=ackport_driver_check` compile-time option, VCS NLP checks for the drivers of the acknowledge port signal in the entire design hierarchy and reports the UPF_MDOAPS warning message (with driver information), as shown below, if there is a driver that can contribute to a value on the acknowledge port signal present in the design.

```
Warning-[UPF_MDOAPS] Multiple drivers on ack port signal
top.upf, 21
Multiple drivers on Power Switch 'tb/test/SW_1' ack port
signal 'tb/test/out1'.
HDL Driver Source : test.v, 7 Port Connect :- .out1(out1).
Please refer to "mvsim_native_reports/ackport_tracing.rpt"
for further details.
```

The `-power=ackport_driver_check` option dumps the `ackport_tracing.rpt` report file in the `mvsim_native_reports` directory as specified in the above warning message. The `ackport_tracing.rpt` file contains the tracing information from acknowledge port to real HDL driver.

VCS NLP does not consider undriven/unconnected ports of type net in HDL as driver for the acknowledge port signal under the `-power=ackport_driver_check` option, and issues the UPF_MDOAPS warning message.

You can use the `-error=UPF_MDOAPS` compile-time option to convert the above warning message into an error message as follows:

```
Error-[UPF_MDOAPS] Multiple drivers on ack port signal
top.upf, 21
Multiple drivers on Power Switch 'tb/test/SW_1' ack port
signal 'tb/test/out1'.
HDL Driver Source : test.v, 7 Port Connect :- .out1(out1).
Please refer to "mvsim_native_reports/ackport_tracing.rpt"
for further details.
```

Limitation

- Tracing through the feedthrough paths (simple assignments) is not supported in VHDL.

Using `create_supply_net` Command

The `create_supply_net` command (see [“create_supply_net”](#) for syntax) creates a supply net to supply power or ground to a power domain. You must specify the name for the new supply net and the name of the existing power domain in which the supply net will be used. The supply net is specified as a simple (not hierarchical) name.

The following sections describe this command:

- [Support for `-resolve parallel` Option](#)
- [Specifying User-Defined Supply Net Resolution As a Verilog Function or Task in the Package](#)

Support for `-resolve parallel` Option

VCS NLP supports the `-resolve parallel` option with the `create_supply_net` command.

Note:

If a `create_power_switch` command in the UPF and power switch instantiated in the design drive the same supply net, then the `create_supply_net` command for that supply net must have `-resolve parallel`.

Limitation

- VCS NLP does not check whether the root supply for the parallel switches is same.

Specifying User-Defined Supply Net Resolution As a Verilog Function or Task in the Package

VCS NLP supports custom resolution of supply nets. You can specify your own supply net resolution as a Verilog function or task in your package and use it in the UPF as follows:

```
create_supply_net net_name -resolve  
package_name::resolution_name
```

VCS NLP resolves the Verilog task or function and uses it to build the resolution gate of supply nets. This feature helps you to write your own supply net resolution function based on your supply network use model.

Note:

Resolution function must be commutative and associative.

Following is the example of UPF and resolution function:

Example 3-5 UPF Example Code

```
set_design_top top  
set_design_attributes -elements {.} -attribute enable_bias  
TRUE  
  
create_power_domain TOP -include_scope  
create_power_domain CORE1 -elements {c1}  
  
create_supply_net combined_vdd -resolve my_pkg::my_res
```

```
create_supply_net supply_net1
```

Example 3-6 Resolution Function Example Code

```
package my_pkg;
import UPF::*;

function state partOnPriority (state a, state b);
    if ((a == UNDETERMINED) || (b == UNDETERMINED))
        return UNDETERMINED;
    else if ((a == PARTIAL_ON) || (b == PARTIAL_ON))
        return PARTIAL_ON;
    else if ((a == FULL_ON) || (b == FULL_ON))
        return FULL_ON;
    else
        return OFF;
endfunction

task my_res(input supply_net_type a, input supply_net_type
b, output supply_net_type out);
    out = '{ partOnPriority(a.state, b.state), (a.voltage >
b.voltage) ? a.voltage : b.voltage }';
endtask
endpackage

module tb ;

import UPF::*;
import my_pkg::*;

endmodule
```

Limitations

- You can only write resolution function inside a package. Module, program-block, and classes are not supported.
- The resolution function must be in the form of 2-input supply nets and 1-output supply net.

Using map_isolation_cell Command

The `map_isolation_cell` command allows you to map a particular isolation strategy to the user-defined library cell or behavioral model, and instructs VCS NLP to use this isolation cell for clamping. You must specify the `set_isolation` and `set_isolation_control` commands. The `set_isolation_control` command is used to specify the location of the isolation strategy.

Following is the use model of the `map_isolation_cell` command:

```
set_isolation isolation_name
    -domain ref_domain_name
    [-applies_to <inputs | outputs | both>]
    [-isolation_power_net net_name]
    [-isolation_ground_net net_name]
    [-elements element_list]
    [-clamp_value {< 0 | 1 | Z | latch >}]
    [-location <self | parent | automatic>]
map_isolation_cell isolation_name
    -domain domain_name
    [-elements element_list]
    [-lib_cells lib_cells_list]
    [-lib_cell_type lib_cell_type]
    [-lib_model_name model_name {-port {port_name net_name}}*]
```

Key Points to Note

- If `-lib_model_name` is specified:
 - VCS NLP uses `model_name` for isolation modeling
 - Logic and supply ports are connected as specified by the `-port` options
 - All supply connections must be specified with `-port`

- Ports of type inout are not supported for supply connections
- Neither implicit nor automatic supply net connections apply
- No corruption semantics apply to the model specified using -lib_model_name
- VCS NLP generates an error if:
 - domain_name does not indicate a previously created power domain
 - isolation_name is not specified
 - -lib_model_name is not specified

Isolation Cell Modeling Example

Each `set_isolation` command contains one corresponding `map_isolation_cell` command. For example, consider the following user-defined isolation cell test case:

Example 3-7 User-defined Isolation Cell With Multiple Supplies and Enables

```
module Complex_ISO_Mod_Zero(input D, output Z, input power1,
    input power2, input backup, input ground1, input ground2,
    input EN1, input EN2, input EN3, input EN4 );

    wire temp;

    assign Z = ({power1,power2, backup, ground1, ground2} ==
5'b11100) ? temp : 1'bx;

    assign temp = (EN1 | EN2 | EN3 | EN4 | D) ;

endmodule
```

UPF Command

```
set_isolation ISO1 -domain PD1 -isolation_signal {iso_ctrl1
iso_ctrl2 iso_ctrl3 iso_ctrl4} -clamp_value 0 -applies_to
```

```
outputs -location parent -isolation_supply_set {TOP_prim
PD1_prim PD2_prim}
```

```
map_isolation_cell ISO1 -domain PD1 \
    -lib_model_name Complex_ISO_Mod_Zero \
    -port {EN1 isolation_signal[0]} \
    -port {EN2 isolation_signal[1]} \
    -port {EN3 isolation_signal[2]} \
    -port {EN4 isolation_signal[3]} \
    -port {power1 isolation_supply_set[0].power} \
    -port {power2 isolation_supply_set[1].power} \
    -port {backup isolation_supply_set[2].power} \
    -port {ground1 VSS} \
    -port {ground2 VSS}
```

Note:

- You must use the `-port` option to connect ports like enable, power, and ground.
- The `-port` option need not specify model input and output data port connection. Except these two data ports, all other ports must be connected using `-port`.
- It is assumed that the cell is the power aware model having PG pins in module interface, and manages its corruption semantics.

Using Value Conversion Table (VCT) for Model Supply Connection

- The default VCT for isolation model power port connection is UPF2SV_LOGIC. You can use the following syntax to change this VCT:

```
set_design_attributes -models <model_name>
-attribute SNPS_default_power_upf2sv_vct
<vct_name>
```

- The default VCT for isolation model ground port connection is UPF_GNDZERO2SV_LOGIC. You can use the following syntax to change this VCT:

```
set_design_attributes -models <model_name>
-attribute SNPS_default_ground_upf2sv_vct
<vct_name>
```

Note:

In cases where isolation supplies are not defined as part of the `set_isolation` command, and `default_isolation` supply set is picked up from the power domain, the usage of `<Domain>.<default_isolation>.<power/ground>` or `<Explicit Supply Set>.power` along with the `-ports` option for connecting power and ground pins of the isolation cell is supported.

```
create_supply_set PD2_iso
..
create_power_domain PD2 -supply {primary PD2_prim}
-supply {default_isolation PD2_iso}
..
set_isolation iso_pd1 -domain PD2 -clamp_value 1
-source PD1_prim
set_isolation_control iso_pd1 -domain PD2
-isolation_signal iso_enable -isolation_sense high
-location parent

map_isolation_cell iso_pd1 -domain PD2 \
    -lib_model_name ISO_Mod_Zero \
    -port {enable iso_enable} \
    -port {power PD2_iso.power} \
    -port {ground PD2_iso.ground}
```

Or,

```
map_isolation_cell iso_pd1 -domain PD2 \
    -lib_model_name ISO_Mod_Zero \
```



```
-port {enable iso_enable} \
-port {power PD2.default_isolation.power} \
-port {ground PD2.default_isolation.ground}
```

Also, generic name `isolation_supply_set` can be used.

```
map_isolation_cell iso_pd1 -domain PD2 \
-lib_model_name ISO_Mod_Zero \
-port {enable iso_enable} \
-port {power isolation_supply_set.power} \
-port {ground isolation_supply_set.ground}
```

Limitations

Following are the limitations of the `map_isolation_cell` command:

This feature does not support the following:

- Non power-aware models are not supported
- In case of MX designs, the `map_isolation_cell` command is supported only for the isolation cells inserted in Verilog
- `-elements` is not supported with the `map_isolation_cell` command
- Auto corruption is not supported

Using `query_cell_mapped` Command

The `query_cell_mapped` command identifies the cell that is used for the named instance. If the specified instance module has a db cell mapped to it, then this command returns the cell name in the `<library>/<cell>` format, else it returns null string.

Following is the syntax of the `query_cell_mapped` command:

```
query_cell_mapped instance_name
```

Where, `instance_name` is the name of the instance.

Example:

```
query_cell_mapped top/a/my_inst
```

Limitations

Following are the limitations of the `query_cell_mapped` command:

- This command supports only Verilog instances.
- Wildcards are not supported.

Using `set_design_top` Command

VCS NLP allows you to specify either the module name or instance name as an argument to the `set_design_top` command. For any string passed with the `set_design_top` command, VCS NLP first checks for the existence of the module name.

If the specified string is a module, VCS NLP finds the first instance of that module and sets it as design top. If the specified string is not a module, VCS NLP checks whether it is an instance or not. If it is an instance, VCS NLP sets it as design top. If the specified string is not found in the design, then VCS NLP generates an error message.

Using set_equivalent Command

The `set_equivalent` command declares that the specified supplies are electrically or functionally equivalent. This command returns an empty string on success and an error on failure.

Following is the syntax of the `set_equivalent` command:

```
set_equivalent
  [-function_only]
  [-nets supply_net_name_list]
  [-sets supply_set_name_list]
```

Where,

`-function_only`

Specifies that the supplies are functionally equivalent rather than electrically equivalent.

If `-function_only` is specified, then the supplies are declared to be functionally equivalent only; otherwise the supplies are declared to be electrically equivalent, which implies that they are also functionally equivalent.

`-nets`

Defines equivalence for a list of supply port, supply net, and/or macro internal PG pin names. When supply ports are listed, equivalence applies to the supply nets connected to the port.

`-sets`

Defines equivalence for a list of supply sets and/or supply set handles.

Note:

You cannot use both `-nets` and `-sets` options with the `set_equivalent` statement. You can only use one of these options.

Example

```
set_equivalent -nets { vss1 vss2 ground }
set_equivalent -function_only -nets { vdd_wall vdd_battery }
set_equivalent -function_only -sets {sys/aon_ss mem/
PD.primary}
```

Limitations

- Macros are not supported for electrical equivalence
- Error message is not issued for a supply source without supply pad or macro port
- For `-function_only`, no assertion is issued during the simulation when functionally equivalent nets go to different states

Using `set_isolation` Command

The `set_isolation` command allows you to specify an isolation strategy for ports on the interface of a power domain. The isolation strategy for a power domain is applied at the domain boundary, as required to ensure correct electrical and logical functionality when domains are in different power states.

The following sections describe the `set_isolation` command:

- [Use Model](#)

- [Examples](#)
- [Precedence Details](#)
- [Defining Isolation Strategies](#)

Use Model

Following is the syntax of the `set_isolation` command:

```
set_isolation isolation_name
  -domain ref_domain_name
  [-elements element_list]
  [-source source_supply_ref | -sink sink_supply_ref | -applies_to
    <inputs | outputs | both>]
  [-isolation_power_net net_name] [-isolation_ground_net net_name]
  [-no_isolation]
  [-isolation_signal signal_list [-isolation_sense {<high | low>*}]]
  [-clamp_value {< 0 | 1 | any | Z | latch | value>*}]
  [-location <automatic | self | parent>]
  [-diff_supply_only <TRUE | FALSE>]
```

The `set_isolation` command specifies the ports on `ref_domain_name` to isolate using a specified strategy. The `-source`/`-sink`/`-diff_supply_only` options are filters similar to `-applies_to`. These options are used to filter the set of elements for a given `set_isolation` command invocation.

For any port in the `prefilter_element_list` (elements specified in the `-elements` list), the following filtering functions are applied:

- `-source` filters the ports receiving a net that is driven by logic powered by the supply set
- `-sink` filters the ports driving a net that fans-out to logic powered by the supply set

- When both `-source` and `-sink` are specified, a port is included if it has a source and a sink as specified
- With `-diff_supply_only`, no isolation is introduced into the path from the driver to the receiver for an isolation strategy defined for a port on the interface of `ref_domain_name`, where the driver is powered by the same supply as a receiver of the port
- `-applies_to` filters the ports within a domain for which the isolation strategy with specified mode is defined
- When one or more of the above filters are used together, then `effective_element_list` contains only those elements that satisfy all the filters. If an element in `effective_element_list` is not on the interface of `ref_domain_name`, then it is not isolated.

The `-source` and `-sink` options require supply sets. The `-diff_supply_only` option can be used without supply sets as the difference is evaluated on a supply net basis.

If `-applies_to` is not specified, and if at least one filter (`-source/-sink/-diff_supply_only`) or `-elements` is specified, the default value of `-applies_to` is taken as `both`. This is as per the IEEE Standard 1801.

If a filter (`-source/-sink/-diff_supply_only`) or `-elements` is not specified, the default value of `-applies_to` is taken as `outputs`. This behavior exists in order to be backward compatible with UPF 1.0. However, this is not compatible with the latest IEEE Standard 1801.

To get the IEEE Standard 1801 compliant behavior, use the `-power=default_applies_to_dont_filter` compile-time option. This option sets the default filter as `both`. Therefore, the strategy that has no filters and `-elements` applies to both input and output ports.

Examples

Basic Source–Sink Policy

The basic usage of `-source/-sink` is illustrated here. The `-source` and `-sink` options use different supply sets. Power domain specified with `-domain` is considered as a reference domain for the isolation strategy. The location (`self/parent/automatic`) of isolation cell is being evaluated with the reference to the domain name mentioned with `-domain`.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -source <source_supply_ref> \  
    -sink    <sink_supply_ref> \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

Source–Sink with Elements

With `-source/-sink/-elements` usage, only those pins/ports (mentioned in the `-elements` option) whose source/sink criteria is satisfied, are isolated.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -source <source_supply_ref> \  
    -sink <sink_supply_ref> \  
    -elements {elements_list} \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

Isolation Policy with Source only

The `set_isolation` command can be used only with `-source <source_supply_ref>` defined. In this case, all ports of reference domain that are driven by the supply set specified in the `-source` option should be isolated, irrespective of knowledge of sink supply set.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -source <source_supply_ref> \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

Source only with -elements

If `-source` and `-elements` options are used together in the `set_isolation` command, isolation policy applies only to the ports that satisfy both the filters.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -source <source_supply_ref> \  
    -elements {elements_list} \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

Isolation Policy with Sink only

The `set_isolation` command can be used only with `-sink <sink_supply_ref>` defined. In this case, all ports of reference domain for which the sink (receiver logic) is supplied by the supply set specified in the `-sink` switch should be isolated, irrespective of knowledge of source supply set.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -sink <sink_supply_ref> \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```


Sink only with -elements

If `-sink` and `-elements` options are used together in the `set_isolation` command, isolation policy is applied to the ports that satisfy both the filters.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -sink <sink_supply_ref> \  
    -elements {elements_list} \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

Source-Sink with -no_isolation

No isolation is applied on the elements mentioned in the `-elements` option when `-no_isolation` is used in the `set_isolation` command. Generally, this policy is used in conjunction with other isolation policy having same `reference_domain` and `-source/-sink` filters. Specific elements listed in `-elements` are not isolated.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -source <source_supply> -sink<sink_supply> \  
    -no_isolation \  
    -elements {elements_list} \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

-diff_supply_only TRUE

The `-diff_supply_only` option determines the isolation behavior between driver and receiver supply sets. It applies isolation if the supplies of driver and receiver are different. Default value of `diff_supply_only` is `FALSE`.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -diff_supply_only TRUE \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

```
-clamp_value <1/0/Z/latch>
```

VCS NLP also supports conservative `diff_supply_only`. For more information, see [“Support for Conservative `diff\_supply\_only` Isolation”](#).

Source–Sink with `-diff_supply_only`

As per the IEEE 1801 standard, an error message is issued if the `-diff_supply_only`, `-source`, or `-sink` arguments are used together. VCS NLP supports using `-source`, `-sink`, and `-diff_supply_only` options together. In this case, isolation policy must be applied to the ports from source to sink domain, if both source and sink supplies are different.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -source <source_supply> -sink <sink_supply> \  
    -diff_supply_only TRUE \  
    -location <self/parent/automatic> \  
    -clamp_value <1/0/Z/latch>
```

`-diff_supply_only` with `-no_isolation`

No isolation is applied on the elements mentioned in the `-elements` option when `-no_isolation` is used in the `set_isolation` command. Generally, this policy is used in conjunction with other isolation policy having same `reference_domain` and `-diff_supply_only TRUE` filters. Specific elements listed in `-elements` are not isolated. Other elements fulfilling the `-diff_supply_only TRUE` condition are isolated.

```
set_isolation <isolation_name> -domain <domain_name> \  
    -diff_supply_only TRUE \  
    -location <self/parent/automatic> \  
    -no_isolation \  
    -elements <elements>
```

```
-elements {elements_list} \  
-clamp_value <1/0/Z/latch>
```

Precedence Details

Precedence among various options is listed below. Strategies that are applied directly on pins/ports have highest precedence, followed by the strategies defined for the whole domain (with no `-elements`). The general rule is that specific strategies have higher precedence over general strategies. The `-source/-sink/-diff_supply_only` rule has higher precedence than the `-applies_to` rule. The effective precedence is as follows (precedence decreases from top to bottom):

- Domain level strategy matching `-source && -sink && -diff_supply_only && -nonsocial`
- Domain level strategy matching `-source && -sink && -diff_supply_only`
- Domain level strategy matching `-source && -sink && -no_isolation`
- Domain level strategy matching `-source && -sink`
- Domain level strategy matching `-sink && -no_isolation`
- Domain level strategy matching `-source && -no_isolation`
- Domain level strategy matching `-sink`
- Domain level strategy matching `-source`
- Domain level strategy matching `-diff_supply_only && -no_isolation`
- Domain level strategy matching `-diff_supply_only`

Note:

- If multiple isolation strategies refer to a single port, a warning message is issued to indicate that multiple isolation strategies are applied. It also indicates the policy that is applied to the port out of multiple strategies.
- Top domain boundary ports are not considered as source or sink. If you want to use top domain boundary port as source or sink, you must use the `set_port_attributes` command.
- By default, Verilog continuous assignment statements are considered as feedthrough in VCS NLP. Therefore, continuous assignment statements or buffers (Verilog) in the design are not treated as driver or load.
- If an isolation policy refers to any specific member of SystemVerilog union, then it applies to all the members of the union.
- VCS NLP supports using `-diff_supply_only`, `-source`, or `-sink` together on the command line.

Defining Isolation Strategies

Isolation strategy can be defined using both `set_isolation` and `set_isolation_control` commands or using the `set_isolation` command only (as per the IEEE 1801-2013).

VCS NLP supports the `-isolation_signal`, `-isolation_sense`, and `-location` options in the `set_isolation` command. Following is the syntax:

```
set_isolation isolation_name
...
[-location <self | parent | automatic |>]
[-isolation_signal signal_list [-isolation_sense {<high | low>}]]
```

For every isolation strategy created, you can specify the above options with the `set_isolation` command. There is no need to define `set_isolation_control` for such isolation strategy.

If you use any of the `-isolation_signal`, `-isolation_sense`, and `-location` options in the `set_isolation` command, then the `set_isolation_control` command is ignored.

Using `set_related_supply_net` Command

The `set_related_supply_net` (SRSN) is a non-UPF command. You can use this command to associate an external supply net to the design ports.

This command is mainly used in hierarchical UPF flow to specify always-on paths. It works on power domain instance boundary ports, PG cell pins, and the design top boundary ports. The design top is the top specified using the `-power_top` compile-time option or the `set_design_top` UPF command.

Following is the use model of the `set_related_supply_net` command:

```
set_related_supply_net [-object_list objects]
                        [-ground ground_net_name]
                        [-power power_net_name]
```

You must use either `-power` or `-ground`. If `-power` or `-ground` is not specified, corresponding supply is treated as ON.

If you do not specify `-object_list`, all the ports of design top specified by `set_design_top` or `-power_top` are taken as the default object list. `objects` is the hierarchical reference to the logic ports relative to the current scope.

Example

The following example demonstrates the usage of the `set_related_supply_net` command:

```
create_power_domain TOP
create_power_domain A -elements {instA1}

create_supply_port VDD
create_supply_port VDDA
create_supply_port VSS

create_supply_net VDD -domain TOP
create_supply_net VDDA -domain A
create_supply_net VSS -domain TOP
create_supply_net VSS -domain A -reuse

connect_supply_net VDD -ports VDD
connect_supply_net VDDA -ports VDDA

set_domain_supply_net TOP -primary_power_net VDD -
primary_ground_net VSS
set_domain_supply_net A -primary_power_net VDDA -
primary_ground_net VSS

set_related_supply_net -object_list {instA1/out1} -power VDD -
ground VSS
```

In the above example UPF, consider that the power domain A is ON. If VDD is switched off, then `out1` of `instA1` corrupts according to the above specified `set_related_supply_net` command. However, other ports remain ON as the power domain A of `instA1` is ON.

Using set_port_attributes Command

The `set_port_attributes` (SPA) command allows you to specify the boundary conditions of the design under test. During block level verification, this command allows you to specify the power information about the logic driving the primary input ports and logic driven by the primary output ports of the design.

The following sections describe the `set_port_attributes` support in VCS NLP:

- [Use Model](#)
- [Comparison with set_related_supply_net \(SRSN\)](#)
- [set_port_attributes on Non-Boundary Ports](#)
- [Using the -repeater_supply Option](#)
- [Support for Unconnected Ports Inside Macro Blocks](#)
- [Inserting SPA and SRSN Buffers on HighConn](#)

Use Model

Following is the syntax of the `set_port_attributes` command:

```
set_port_attributes
[-model name]
[-elements element_list]
[-exclude_elements element_exclude_list]
[-ports port_list]
[-exclude_ports port_exclude_list]
[-applies_to <inputs | outputs | inout | {<inputs | outputs
| inout >*>}]
[-attribute {name value}]*
[-clamp_value <0 | 1 | any | Z | latch | value>]
[-driver_supply supply_set_ref]
```

```
[-receiver_supply supply_set_ref]  
[-repeater_supply supply_set_ref]  
[-feedthrough]  
[-unconnected]
```

Supported attributes with `-attribute` are `iso_source`, `iso_sink`, and `related_supply_default_primary`.

Options

- `-driver_supply/-receiver_supply`

When `-driver_supply` is attributed on a port, it specifies the supply of the logic driving the port. The port is corrupted when the driver supply is in a simstate other than NORMAL. Any isolation strategy with the driver supply as `-source` applies on the port as long as all other filtering criteria on strategy are met. That is, VCS NLP uses the `-driver_supply` for strategy association for the port.

Similarly, when `-receiver_supply` is attributed on a port, it specifies the supply of the logic reading the port. The port is corrupted when the receiver supply is in a simstate other than NORMAL. Any isolation strategy with the receiver supply as `-sink` applies on the port as long as all other filtering criteria on strategy are met. That is, VCS NLP uses `-receiver_supply` for strategy association for the port.

Attributes

- `iso_source/iso_sink`

The `iso_sink` attribute indicates the supply set that drives the eventual sink (or receiving logic) of a primary output port. The `iso_source` attribute indicates the supply set that drives the source (or driving logic) of a primary input port.

The supply set specified with this attribute is used by VCS NLP for strategy association. It is not used for determining the corruption on the port. If a port contains both `iso_source` and `-driver_supply`, `iso_source` is used for strategy association and `-driver_supply` is used for corruption. Similarly, for `iso_sink` and `-receiver_supply`, `iso_sink` is used for strategy association and `-receiver_supply` is used for corruption.

- `related_supply_default_primary`

This attribute indicates that the driver/receiver supply for a particular port is same as the primary power of its corresponding power domain. Note that this attribute applies only if there is no `set_port_attributes` or `set_related_supply_net` command on the port.

Comparison with `set_related_supply_net` (SRSN)

The `set_related_supply_net` command continues to have the same corruption and strategy association semantics as before. However, if there is a `set_port_attributes` command on the same port, the `set_port_attributes` command might override the `set_related_supply_net` specification. The priority (decreasing order) in which all these attributes apply are as follows:

- Corruption semantics

```
driver/receiver_supply > set_related_supply_net >
related_supply_default_primary
```

- Strategy association

```
iso_source/iso_sink > driver_supply/receiver_supply >
set_related_supply_net > related_supply_default_primary
```

Also, note that `set_port_attributes` can be applied on all ports of an instance using the `-elements` option. If a particular port figures both in `-elements` list and `-ports` list, the attribute with `-ports` list is applied.

set_port_attributes on Non-Boundary Ports

VCS NLP applies corruption semantics, but ignores `set_port_attributes` for policy association. Instead, it uses the actual source/sink logic in the design.

Note:

- Applying both `set_port_attributes` and `set_related_supply_net` on the same port is not recommended.
- Wildcards are not supported with the `set_port_attributes` command.

Using the -repeater_supply Option

VCS NLP supports the `-repeater_supply` option with the `set_port_attributes` UPF command which has the semantic to force a repeater at the port having the repeater supply attribute in addition to specifying the supply to be used by that repeater.

The following syntax describes the usage of this option:

```
set_port_attributes
  [-ports {port_list}]
  [{-elements {element_list} [-applies_to <inputs | outputs
    | both>]}]
  [-repeater_supply supply_set_ref]
```

Where, *supply_set_ref* is the supply set used by a repeater driving the port.

You can also use the following attributes to specify ground and power net, instead of specifying supply set.

```
set_port_attributes -attribute repeater_power_net  
<supply_net_power> -ports ...
```

```
set_port_attributes -attribute repeater_ground_net  
<supply_net_ground> -ports ...
```

The output information is displayed in the following files:

File Name	Description
isolation_association.rpt	Displays information for each boundary port of a node.
repeaters.rpt	Lists all the ports which are treated as having repeaters.
xovers.rpt	Displays information on changes to the crossings/ source/sink.

Note:

- If the `-power=verbose1` compile-time option is specified, then VCS NLP generates the `boundary_port_partitioning.rpt` report file. This file displays the repeaters implemented and its associated supplies.
- You cannot apply repeater supply on the top domain input ports and macro output ports.

Limitations

Following are the limitations of this feature:

- Repeater supply on interface/modports is not supported
- `set_port_attributes` is not supported for 'inout' ports, `set_related_supply_net` can be used instead

Support for Unconnected Ports Inside Macro Blocks

An unconnected port represents a port on the interface of a module or cell that is not connected to either a source or sink supply.

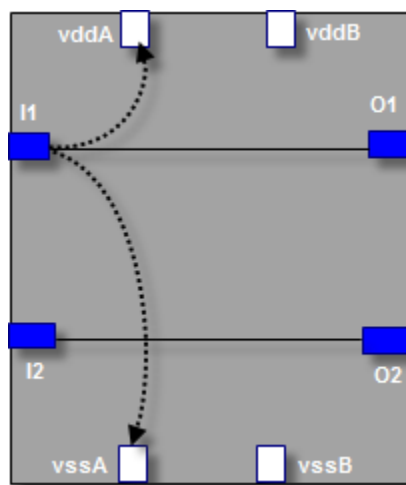
VCS NLP allows you to specify the logic ports of a macro block that are not connected to any supply ports using the `-unconnected` option of the `set_port_attributes` command.

Following is the syntax of the `-unconnected` option:

```
set_port_attributes -model <model> -unconnected -ports  
<port_list>
```

Consider the macro cell in [Figure 3-1](#) with two set of supplies: `vddA` / `vssA` and `vddB` / `vssB`.

Figure 3-1 Macro Cell



In [Figure 3-1](#), the input port I1 drives the logic powered by vddA / vssA and connection to port O1. The input port I2 is not connected to any supply port. You can use the following command to attribute port I2 as unconnected:

```
set_port_attributes -ports {I2} -model cellX -unconnected
```

Key Points to Note

- If `-model` is specified, only ports that belong to the given model can be specified as an argument to the command. Hierarchical names are not supported
- UPF specified unconnected port ignores `related_power_pin`, `related_ground_pin`, and `power_down_function` of that port
- You must use `-unconnected` in combination with the `-ports` option
- VCS NLP issues an error message if `-model` is used in combination with the `-elements` option

- `-model` can refer to the PAD or macro library cells and black box elements
- Multiple instances of a command are allowed for the same `-model` reference. Overlapped instances of referenced port with previous settings are parsed and ignored
- At least one element must be specified in `-ports`
- `set_related_supply_net` (SRSN) overrides the `set_port_attributes -unconnected` setting on a port

Limitation

Following is the limitation:

- Wildcards are not supported with the `-ports` option.

Specifying Clamp Value for Ports

VCS NLP supports specifying clamp value for ports. Following is the syntax:

```
set_port_attributes [-clamp_value <0 | 1 | any | Z
| latch | value>]
```

Where, `-clamp_value` specifies the clamp value to be used for the port with isolation strategy applied to it.

If different clamp values are specified on the same port using the `set_isolation` and `set_port_attributes` commands, then VCS NLP issues the `UPF_SPACVM` warning message, and considers the clamp value provided by the isolation policy.

Example

Consider the following UPF code:

```
set_isolation iso2 -domain TOP -isolation_supply_set SSTOP
-clamp_value 1 -elements { dataout1 dataout2 dataout3 }

set_isolation_control iso2 -domain TOP -isolation_signal
iso_enable -isolation_sense high -location self

set_port_attributes -clamp_value 0 -ports {dataout1} -
attribute iso_sink SSTOP
```

Here, `set_isolation` and `set_port_attributes` are on the same port `dataout1`. The clamp value from the isolation policy is 1, and from `set_port_attributes` is 0. Therefore, VCS NLP issues the `UPF_SPACVM` warning message and considers the clamp value from the isolation strategy. Following is the warning message:

```
Warning-[UPF_SPACVM] Clamp Value Mismatch
sanity_test_port_base.upf, 30
  Clamp value '1' specified in isolation strategy
  'topunit/TOP_U/TOP/isolation/iso2' for design port
  'topunit.TOP_U.dataout1' does not match with the value '0'
  specified with 'set_port_attributes -clamp_value'. Clamp
  value from isolation strategy is considered. Isolation
  Strategy: 'sanity_test_port_base.upf, 41'.
```

Inserting SPA and SRSN Buffers on HighConn

VCS NLP inserts the `set_port_attributes` (with the `-driver_supply` and `-receiver_supply` attributes) and `set_related_supply_net` buffers on HighConn for the regular models and macros. [Table 3-5](#) and [Table 3-6](#) describes the placement of SRSN and SPA buffers.

Table 3-5 Placement of SRSN Buffers

SRSN	Ports	Location
Hierarchical Models	Input, Output, and Inout ports	HighConn
Black box Models	Input, Output, and Inout ports	LowConn
Top Power Domain Boundary	Input, Output, and Inout ports	LowConn

Table 3-6 Placement of SPA Buffers

SPA	Ports	Location
Hierarchical Models	Input ports with -driver_supply	HighConn
	Output ports with -receiver_supply	
Black box Models	Input ports with -receiver_supply	LowConn
	Output ports with -driver_supply	
Top power domain boundary	Input, output	LowConn

Precedence Rules for SRSN and SPA

If SRSN and SPA are specified on the same port, then SPA takes precedence over SRSN. Also, policy that is specified at more granular level takes precedence over the policy specified at less granular level. That is, if the policy P1 is specified on a scope, policy P2 is specified on a port, and policy P3 is specified on a bit of the port, then the precedence order is P3 > P2 > P1.

Defining Retention Strategies

Retention strategies can be defined using the combination of the `set_retention` and `set_retention_control` commands or using the `set_retention` command only (as per the IEEE 1801-2013).

VCS NLP supports the `-save_signal` and `-restore_signal` options in the `set_retention` command. Following is the syntax:

```
set_retention retention_name
...
[-save_signal {logic_net <high | low | posedge | negedge>}
-restore_signal {logic_net <high | low | posedge | negedge>}]
```

If you use any of the `-save_signal` and `-restore_signal` options in the `set_retention` command, then the `set_retention_control` command is ignored.

Note:

The following mutex options are supported only with the `set_retention_control` command:

```
[-assert_r_mutex {{net_name <high | low | posedge | negedge>}}]*
[-assert_s_mutex {{net_name <high | low | posedge | negedge>}}]*
[-assert_rs_mutex {{net_name <high | low | posedge | negedge>}}]*
```

Support for Voltage Triplets in the `add_port_state` and `add_power_state` Commands

VCS NLP supports the following features in the `add_port_state` command:

- Specifying minimum and maximum voltage pair (*Vmin*, *Vmax*)
- Interpreting the voltage pair (*Vmin*, *Vmax*) or triplet (*Vmin*, *Vnom*, *Vmax*) as a voltage range similar to the `add_power_state` command

You can use the `-power=discreteportstates` compile-time option to change the interpretation of the voltages to discrete values, rather than a range of voltages. This option is supported for the `add_port_state` and `add_power_state` commands.

If two voltage values (for example, `voltage_1` and `voltage_2`) are specified, then *Vmin* is set to `voltage_1`, *Vmax* is set to `voltage_2`, and *Vnom* is set to $(\text{voltage_1} + \text{voltage_2})/2$.

If all the three voltage values are specified, then *Vmin* is set to `voltage_1`, *Vnom* is set to `voltage_2`, and *Vmax* is set to `voltage_3`.

VCS NLP generates an error message if:

`voltage_2 < voltage_1 or voltage_3 < voltage_2`

Using `upf_version` Command

VCS NLP supports the `upf_version` UPF command. Support for `upf_version` is limited to parse and ignore only. No version compatibility checks or semantics are applied for UPF commands based on it.

Syntax

```
upf_version [string]
```

Irrespective of the version string, the return value of the `upf_version` command is IEEE-1801.

Using upf_tool Variable

VCS NLP supports the usage of Tcl variable `upf_tool` in the UPF file. For VCS NLP, the value of `upf_tool` is internally set to `MVSIM_NATIVE`, whereas for MVTOOLS, it is set to `MVSIM`. You can use this variable to have two different UPF commands for VCS NLP or MVSIM in the same UPF file.

Usage Example

The following is the usage example:

```
if {[info exists upf_tool]} {  
  
    if {$upf_tool == "MVSIM_NATIVE"} {  
  
        set_retention V1_RETENTION_CLK_FREE \  
            -domain Island_V1 \  
            -retention_power_net V1_ret \  
            -restore_condition {{!reset}} \  
            -save_signal {save posedge} \  
            -restore_signal {save high} \  
            -retention_condition {save}  
  
    } elseif {$upf_tool eq "MVSIM"} {  
  
        set_retention V1_RETENTION_CLK_FREE -domain Island_V1 -  
            retention_power_net V1_ret  
        set_retention_control V1_RETENTION_CLK_FREE -domain  
            Island_V1 -save_signal {save posedge} -restore_signal {save  
            high}  
  
    }  
}
```

You can also use this variable to add or skip UPF commands for VCS NLP or MVSIM.

```
if {$upf_tool == "MVSIM_NATIVE"} {  
  set_design_attributes ...  
}  
  
if {$upf_tool ne "MVSIM_NATIVE"} {  
  set_related_supply_net ...  
}
```

Using `synopsys_program_name` Variable

VCS NLP supports the usage of Tcl variable `$synopsys_program_name` in the UPF file. For VCS NLP, the value of `$synopsys_program_name` is internally set to `vcs`. You can use this variable for conditional compilation in UPF similar to `$upf_tool`.

Usage Example

The following is the usage example:

```
if {[info exists synopsys_program_name]} {  
  
    if {$synopsys_program_name == "vcs"} {  
  
        set_retention V1_RETENTION_CLK_FREE \  
            -domain Island_V1 \  
            -retention_power_net V1_ret \  
            -restore_condition {{!reset}} \  
            -save_signal {save posedge} \  
            -restore_signal {save high} \  
            -retention_condition {save}  
  
    } elseif {  
  
        set_retention V1_RETENTION_CLK_FREE -domain Island_V1 -  
        retention_power_net V1_ret  
        set_retention_control V1_RETENTION_CLK_FREE -domain  
        Island_V1 -save_signal {save posedge} -restore_signal {save  
        high}  
  
    }  
}
```

You can also use this variable to add or skip UPF commands for VCS NLP or other tools.

```
if {$synopsys_program_name == "vcs"} {  
    set_design_attributes ...  
}  
  
if {$synopsys_program_name ne "vcs"} {  
    set_related_supply_net ...  
}
```

Setting Supply Port Default Value

This feature allows you to set the default state on the supply port using `set_design_attributes`. Once the default state is applied on a supply port which is a supply pad, it starts from that state.

Use Model

Set the default state for the given port using `set_design_attributes`, as shown below, which is defined in `add_port_state`.

Syntax:

```
add_port_state port_name  
{-state {name <nom | min max | min nom max | off>}}
```



```
set_design_attributes -element { <supply_port | supply_net>  
} -attribute SNPS_default_supply_net_state  
<state_in_add_port_state>
```

Example

Consider the following example code. The supply port `V3` starts with its default state `ON_D`, as defined in `set_design_attributes`.

```
set_design_top testbench/inst  
..  
add_port_state V3 -state {ON 4.0} \  
                    -state {ON1 3.5} \  
                    -state {ON2 3.0} \  
                    -state {ON3 2.0} \  
                    -state {ON_D 2.1}  
    set_design_attributes -elements { V3 } -attribute  
    SNPS_default_supply_net_state ON_D
```

Note:

If `set_design_attributes` is used to define default state of the supply port, and at the same time if `supply_on()` sets value of that supply port to different value, then `supply_on()` gets higher priority.

Support for Auto-Completion of UPF Commands

By default, the auto-completion for the UPF commands and command options is disabled.

- The following error message is issued for incomplete UPF command:

```
Error-[UPF_IUC] Invalid UPF command
```

- The following error message is issued for incomplete UPF command option:

```
Error-[UPF_UUCO] Unknown UPF command option
```

You can use the `-power=auto_complete` compile-time option to enable the auto-completion for the UPF commands and command options. It is recommended to correct the UPF command or command option instead of using `-power=auto_complete`.

Support for Feedthrough Paths Inside Macro Blocks

Feedthrough ports are the set of ports on the interface of a module or cell that are directly connected to each other inside a module or cell to create a feedthrough path. In a macro block, a pure feedthrough path is represented by a wire connecting two or more ports of the block (logical feedthrough).

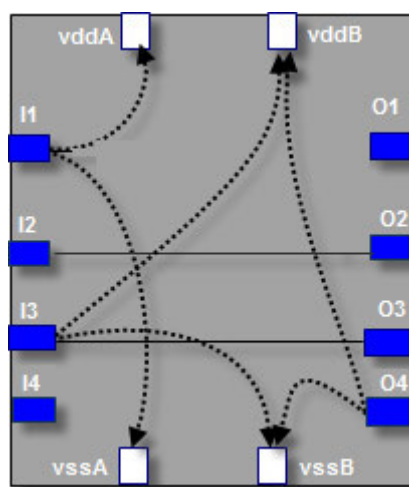
VCS NLP supports creating pure feedthrough paths inside a macro block by allowing you to directly connect set of ports of a model to each other using the `-feedthrough` option of the `set_port_attributes` command.

Following is the syntax of the `-feedthrough` option:

```
set_port_attributes -model <model> -feedthrough -ports  
<port_list>
```

Consider the following macro cell with two set of supplies: `vddA / vssA` and `vddB / vssB`.

Figure 3-2 Macro Cell



In [Figure 3-1](#), I1 drives logic powered by vddA / vssA, I2 is directly connected to port O2, I3 drives logic powered by vddB / vssB and the connection to port O3, port O4 is driven by logic powered by vddB / vssB.

Example:

```
set_port_attributes -ports {I2 O2} -model cellX -feedthrough
```

Key Points to Note

- If `-model` is specified, only ports that belong to the given model can be specified as an argument to the command. Hierarchical names are not supported.
- You must use `-feedthrough` in combination with the `-ports` option. At least two elements must be specified in `-ports`.
- VCS NLP issues an error message if `-model` is used in combination with the `-elements` option.

- `-model` can refer to the PAD or macro library cells, and black box elements.
- Multiple instances of a command are allowed for the same `-model` reference. Overlapped instances of the referenced port with previous settings are parsed and ignored.
- VCS NLP issues an error message if you use the `-unconnected` and `-feedthrough` options on the same `-port` element.
- `set_related_supply_net` (SRSN) overrides the `set_port_attributes -feedthrough` setting on a port.

Limitations

- Wildcards are not supported with the `-ports` option.
- MX isolation is not supported.

Generic Handles Support

VCS NLP allows you to use generic handles in the retention strategy and the `bind_checker` command. This section includes the following topics:

- [Generic Handles Support in Retention Strategy](#)
- [Generic Handles Support in the `bind\_checker` Command](#)

Generic Handles Support in Retention Strategy

VCS NLP supports specifying generic names for the clock and asynchronous load as part of `-save_condition`, `-restore_condition`, and `-retention_condition` of the `set_retention` command. You can define the conditions using the generic handles `UPF_GENERIC_CLOCK` and `UPF_GENERIC_ASYNC_LOAD`.

Inference of the generic clock and asynchronous load follow the below guidelines:

1. Coding style used must be valid as per the supported templates for synthesis.
2. Both signals acting as clock and asynchronous load must be in the sensitivity list, and must be edge-sensitive.
3. `UPF_GENERIC_ASYNC_LOAD` is inferred based on the sense specified in the sensitivity list and the conditions specified inside the always block.

Following are the list of scenarios and `GENERIC_CLOCK/ASYNC_LOAD` inferred:

Example-1: `GENERIC_CLK: clk, ASYNC_LOAD: ~reset`

```
always @ (posedge clk or negedge reset)
  if (~reset)
    dataout1 <= 0;
  else
    dataout1 <= data_in;
```

Example-2: `GENERIC_CLK: clk, ASYNC_LOAD: 1'b1`

```
always @ (posedge clk or negedge reset)
```

```

if (reset)
    dataout1 <= 0;
else
    dataout1 <= data_in;

```

Example-3: GENERIC_CLK: clk, ASYNC_LOAD: 1'b1

If any of the expression is not edge sensitive, for example:

```

always @ (posedge clk or reset)
    if (reset)
        dataout1 <= 0;
    else
        dataout1 <= data_in;

```

Example-4: GENERIC_CLK = 1'b1, ASYNC_LOAD = 1'b1

ASYNC_LOAD is not inferred, for example:

```

always @ (posedge clk or posedge reset or posedge set)
    if (reset || set) // or if (reset || 1'b1)
        dataout1 <= 0;
    else
        dataout1 <= data_in;

```

Specify generic (canonical) design signal names of clock (UPF_GENERIC_CLOCK) and asynchronous load (UPF_GENERIC_ASYNC_LOAD) as part of save/restore/retention condition of the retention policy, as shown below:

```

set_retention RET1 -domain PD1
    -save_signal {save_sig} -restore_signal(restore_sig)
    -retention_power_net VDD_RET_NET
    -retention_ground_net VSS_RET_NET
    -save_condition {! UPF_GENERIC_ASYNC_LOAD}
    -retention_condition
        {UPF_GENERIC_CLOCK && UPF_GENERIC_ASYNC_LOAD}

```

Note:

- Usage of `UPF_GENERIC_ASYNC_LOAD` should be limited to the retention strategies that have asynchronous reset flops as elements.
- `UPF_GENERIC_ASYNC_LOAD` is taken as value 1 for the synchronous reset flops, non-resettable flops, and latches.
- Usage of `UPF_GENERIC_CLOCK` should be limited to the retention strategies that do not have any latches as elements.
- `UPF_GENERIC_CLOCK` is taken as value 1 for latches.

Limitations

- Supported for retention in Verilog context only.

Generic Handles Support in the `bind_checker` Command

This section includes the following topics:

- [Generic Handles Support for Isolation in the `bind\_checker` Command](#)
- [Generic Handles Support for Retention in the `bind\_checker` Command](#)

Generic Handles Support for Isolation in the `bind_checker` Command

VCS NLP supports binding of the user-defined checker modules for isolation with the generic handles of the signals like isolation `DATA` and `OUTPUT`.

VCS NLP allows you to bind the checker module per boundary port on which the isolation is applied with UPF_GENERIC_DATA (input of the clamp) and UPF_GENERIC_OUTPUT (output of the clamp) as part of a generic port or parameter list. For more information, see [“Isolation Example”](#).

Use Model

For VCS:

```
% vcs -sverilog -upf <upf_file> -lpa_bind file_name.tcl  
    file_name.sv
```

```
% simv
```

Example:

```
% vcs -sverilog design.v -upf test.upf -lpa_bind  
    checker.tcl checker.sv  
% simv
```

For VCS MX:

```
% vlogan -sverilog file_name.sv  
  
% vcs testbench -debug_all -upf <upf_file> -sverilog  
    -lpa_bind file_name.tcl
```

```
% simv
```

Example:

```
% vlogan -sverilog design.v checker.sv  
% vcs testbench -debug_all -upf test.upf -sverilog  
    -lpa_bind checker.tcl  
% simv
```

Isolation Example

Example 3-8 Testcase checker.sv

```
module iso_checker (isoSupply, isoGround, ISO_IN, ISO_OUT,
ISOEN);
    input isoGround, isoSupply, ISO_IN, ISO_OUT, ISOEN;
    bit isopoweroff;
    import UPF::*;
    import SNPS_LP_MSG::*;
    supply_net_type isoGround, isoSupply;
    parameter string PD_NAME = "";
    parameter string ISOPOLICY1 = "";
    parameter string ISO_GRND_NET_NAME = "";
    parameter string ISO_OUT_GEN = "";
    parameter string ISO_IN_GEN = "";
    parameter string ISO_PWR_NET_NAME = "";
    assign isopoweroff = ((isoSupply.state == OFF) ||
(isoGround.state == OFF));

    always @(ISO_OUT)
        begin : my_msg_iso_checker
            my_assert : assert(ISO_OUT !== 1'bx) else
            begin
                lp_msg_print("LP_MSG_ISO_OUT_CORRUPT",
                $sformatf( lp_msg_get_format("LP_MSG_ISO_OUT_CORRUPT"),
                ISO_OUT_GEN, ISOPOLICY1));
            end
        end : my_msg_iso_checker

    always @ (ISO_IN, ISO_OUT)
        begin
            $display (" %m %0t [CHECKER] ISO_EN = %d
            %s = %d, %s = %d for policy %s \n", $time,
            ISOEN,ISO_IN_GEN,ISO_IN, ISO_OUT_GEN,ISO_OUT, ISOPOLICY1);
        end
endmodule
```

Example 3-9 Tcl Script checker.tcl

```
proc iso_check {} {
    foreach PD [query_power_domain *] {
        foreach ISO_POLICY [query_isolation * -domain $PD] {
```

```

    array set iso_detail [query_isolation $ISO_POLICY -domain
$PD -detailed]
    set PD_NAME [list PD_NAME $iso_detail(domain)]
    set isoSupply [list isoSupply
$iso_detail(isolation_power_net)]
    set isoGround [list isoGround
$iso_detail(isolation_ground_net)]
    set ISO_IN [list ISO_IN UPF_GENERIC_DATA]
    set ISO_OUT [list ISO_OUT UPF_GENERIC_OUTPUT]
    set ISOEN [list ISOEN $iso_detail(isolation_signal)]
    set ISO_PWR_NET_NAME [list ISO_PWR_NET_NAME
$iso_detail(isolation_power_net)]
    set ISO_GRND_NET_NAME [list ISO_GRND_NET_NAME
$iso_detail(isolation_ground_net)]
    set ISO_IN_GEN [list ISO_IN_GEN UPF_GENERIC_DATA]
    set ISO_OUT_GEN [list ISO_OUT_GEN UPF_GENERIC_OUTPUT]
    set ISO_FULLNAME @$PD.isolation.$ISO_POLICY
    set ISOPOLICY1 [list ISOPOLICY1 [query_original_name
$ISO_FULLNAME]]
    set ports_list {}
    lappend ports_list $isoGround $isoSupply $ISO_IN $ISO_OUT
$ISOEN
    set params_list {}
    lappend params_list $PD_NAME $ISO_PWR_NET_NAME
$ISO_GRND_NET_NAME $ISOPOLICY1 $ISO_IN_GEN $ISO_OUT_GEN

    bind_checker no_iso_msg \
        -module iso_checker \
        -elements @$PD.isolation.$ISO_POLICY \
        -parameters $params_list \
        -ports $ports_list
    array unset iso_detail
}
}
}
iso_check

```

Limitation

- For MX designs, this feature is supported only for the isolation inserted in Verilog.

Generic Handles Support for Retention in the bind_checker Command

VCS NLP supports binding of the user-defined checker modules for retention with the generic handles `CLOCK` and `ASYNC_LOAD`.

VCS NLP supports the following to provide access to the design signals like `CLOCK` and `ASYNC_LOAD` load of the sequential element for which the UPF retention policy is applied:

- Instantiation of the checker module in the design scope where the corresponding UPF policy is applied.
- Identification of the type of the generic signals (`UPF_GENERIC_CLOCK`, `UPF_GENERIC_ASYNC_LOAD`) present in the design scope.

For more information, see [“Retention Example”](#).

Use Model

For VCS:

```
% vcs -sverilog -upf <upf_file> -lpa_bind file_name.tcl  
file_name.sv
```

```
% simv
```

Example:

```
% vcs -sverilog design.v -upf test.upf -lpa_bind  
checker.tcl checker.sv  
% simv
```

For VCS MX:

```
% vlogan -sverilog file_name.sv
```

```
% vcs testbench -debug_all -upf <upf_file> -sverilog
-lpa_bind file_name.tcl
```

```
% simv
```

Example:

```
% vlogan -sverilog design.v checker.sv
% vcs testbench -debug_all -upf test.upf -sverilog
-lpa_bind checker.tcl
% simv
```

Retention Example

Example 3-10 Testcase checker.sv

```
module ret_checker (RET_GRND_NET, RET_PWR_NET, GEN_CLK,
GEN_RST);
    input RET_GRND_NET, RET_PWR_NET, GEN_CLK, GEN_RST;

    import UPF::*;
    import SNPS_LP_MSG::*;

    supply_net_type RET_GRND_NET, RET_PWR_NET;
    parameter string PD_NAME = "";
    parameter string RET_GRND_NET_NAME = "";
    parameter string RET_PWR_NET_NAME = "";

    always @(GEN_CLK or GEN_RST)
        begin
            <User Checker Code>
        end
endmodule
```

Example 3-11 Tcl Script checker.tcl

```
proc ret_check {} {
    foreach PD [query_power_domain *] {
        foreach RET_POLICY [query_retention * -domain $PD] {
            array set ret_detail [query_retention $RET_POLICY -domain
$PD -detailed]
```

```

        set PD_NAME [list PD_NAME $ret_detail(domain)]
        set RET_PWR_NET [list RET_PWR_NET
$ret_detail(retention_power_net)]
        set RET_PWR_NET_NAME [list RET_PWR_NET_NAME
$ret_detail(retention_power_net)]
        set RET_GRND_NET [list RET_GRND_NET
$ret_detail(retention_ground_net)]
        set RET_GRND_NET_NAME [list RET_GRND_NET_NAME
$ret_detail(retention_ground_net)]
        set GEN_CLK [list GEN_CLK UPF_GENERIC_CLOCK]
        set GEN_RST [list GEN_RST UPF_GENERIC_ASYNC_LOAD]
        set ports_list {}
        lappend ports_list $RET_GRND_NET $RET_PWR_NET
$GEN_CLK $GEN_RST
        set params_list {}
        lappend params_list $PD_NAME $RET_GRND_NET_NAME
$RET_PWR_NET_NAME
        bind_checker ret_check \
            -module ret_checker \
            -elements @$PD.retention.$RET_POLICY \
            -parameters $params_list \
            -ports $ports_list
        array unset ret_detail
    }
}
}
ret_check

```

Limitation

Following is the limitation of this feature:

- This feature is supported only for Verilog in case of MX designs.

Bias Support for Low Power Designs

Leakage current was negligible in earlier CMOS technologies. However, with shrinking device geometries and reduced threshold voltages, leakage power is becoming increasingly significant, sometimes approaching the levels of dynamic power dissipation. The main causes of leakage power are reverse-bias p-n junction diode leakage, sub-threshold leakage, and gate leakage.

Substrate-biasing (back-biasing) is a low power design technique which enables dynamic tuning of performance and static power consumption of a CMOS device by applying a variable voltage on the nwell/pwell terminal of the device.

VCS NLP allows you to specify a bias supply network for the design and its intended usage through the following UPF commands:

- Using `enable_bias` design attribute in the UPF to enable the bias feature. Transitive effect is applied on the child blocks under the top bias block
- If a supply set is created (using `create_supply_set`) under the scope of bias enabled module, then nwell and pwell function is automatically created for this supply set
- If a power domain is created (using `create_power_domain`) under the scope of bias enabled module, then bias is automatically enabled for all instances under top bias block

Enabling Bias Feature

The bias feature is disabled by default. To enable bias feature for the design, you must set the `enable_bias` design attribute to `TRUE` as follows:

```
set_design_attributes -attribute enable_bias TRUE
-elements {.}
```

The elements list can be any hierarchical instance. The value of the attribute can be `TRUE` or `FALSE`. Setting `enable_bias` on a design effectively designates all instances in the design as bias blocks. You can disable the bias feature for individual sub-blocks of the design by setting the `enable_bias` attribute to `FALSE` on those blocks, thereby designating them as non-bias blocks.

A design whose top-level block has an `enable_bias` attribute is called a bias design, and the top-level block is called a bias block. All supply sets created under the scope of a bias block have four functions: `power`, `ground`, `pwell` and `nwell`. This includes implicit supply sets created for power domains under the scope of a bias block.

VCS NLP performs the following checks for the `enable_bias` attribute:

- The value of the attribute can only be set to `TRUE` if it is `TRUE` for all hierarchical blocks above the current scope. Setting the value to `TRUE` is always permitted if the current scope is top.

- The root elements of a power domain must be either all bias blocks or all non-bias blocks. That is, if the value of the `enable_bias` attribute is set to `FALSE`, then all parts of the block with the attribute should belong to the power domains scoped within the block. The only exception to this rule shall be:
 - The top power domain of the block maybe scoped outside the block as long as all root cells of the power domains are user-marked non-bias blocks.

Setting the `enable_bias` attribute on an instance has the following effects:

- All instances at or below the current scope automatically inherits the value of the attribute
- All power domains inside a non-bias block in a bias design are converted to fully-restricted domains, that is, they cannot use any bias supply set declared in the scopes above the block. This prevents a supply set with bias rails from being used inside a non-bias block
- The conversion is performed by the implementation tools automatically for any power domain created under the scope of a non-bias block, and is written to the output UPF.

For example, the following command:

```
create_power_domain PD -elements U1
```

is converted to:

```
create_power_domain PD -elements U1 -supply
{extra_supplies " "}
```

Bias Support for add_power_state

VCS NLP supports bias functions and negative floating point voltage value on pwell and nwell supply nets in the `add_power_state` command.

Example:

```
add_power_state TOP_SS -state STBY {-supply_expr {nwell ==  
`{FULL_ON, -0.8} } -simstate CORRUPT_ON_ACTIVITY}
```

Establishing Connections to Bias Pins/Supplies

Bias Support for create_supply_set

The `create_supply_set` command is enhanced to accept the following predefined supply set functions, if the current scope is a bias block.

- `power`
- `ground`
- `pwell`
- `nwell`

If the current scope is not a bias block, VCS NLP generates an error message if you try to define the bias functions of a supply set. Following is an example of how a bias supply set function is used.

Example 3-12 Bias Support for create_supply_set

```
create_supply_port TOP_VNW_SP  
  
create_supply_net TOP_VNW
```

```
connect_supply_net TOP_VNW -port TOP_VNW_SP  
  
create_supply_set TOP_SS -function {nwell TOP_VNW}
```

In this example, the supply net `TOP_VNW` is defined for the nwell function of the supply set `TOP_SS`. Any standard or power management instance powered by `TOP_SS` will have its nwell bias pin connected to the supply net `TOP_VNW`.

Bias Support for `create_power_domain`

If the current scope is a bias block, the `create_power_domain` command is enhanced to implicitly create a supply set with bias functions if implicit supply sets are used. If explicit supply sets are used, a supply set with bias functions can be supplied through the `-supply` option of the `create_power_domain` command.

For example, consider the following command where `TOP_SS` is the supply set defined in [Example 3-12](#). This command implicitly connects the bias supply `TOP_VNW` to all normal instances using the primary power supply set.

```
create_power_domain TOP_PD -supply {primary  
TOP_SS}
```

Any standard or power management instance powered by `TOP_SS` will have its nwell bias pin connected to the supply net `TOP_VNW`.

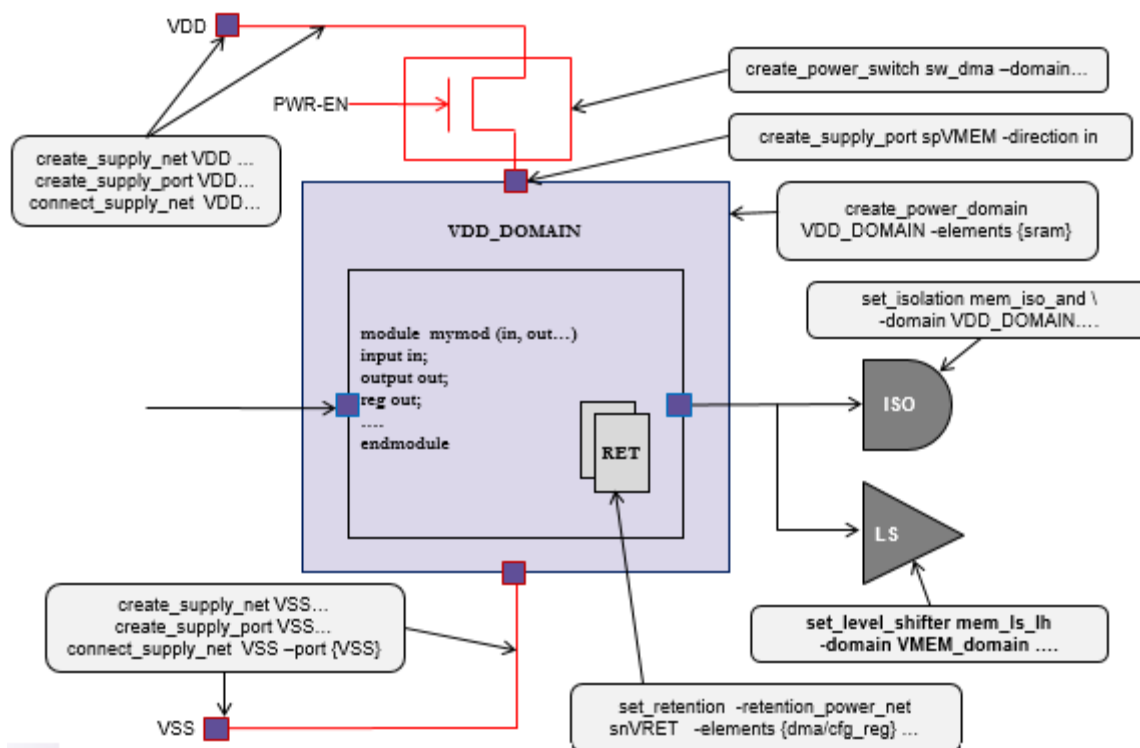
UPF Structural Components

HDL do not support low power details and semantics. UPF compliments the HDL by capturing structure and semantics of power domains in the design.

Table 3-7 UPF and HDL Structural Components

HDL	UPF
Module	Power domains
Logic ports	Supply ports
Wires	Supply nets
Logic gates	Power switches
Assertions	Power State Table
Behavioral logic	Isolation/Retention/Level shifter policies

Figure 3-3 UPF Structural Components



UPF Script Examples

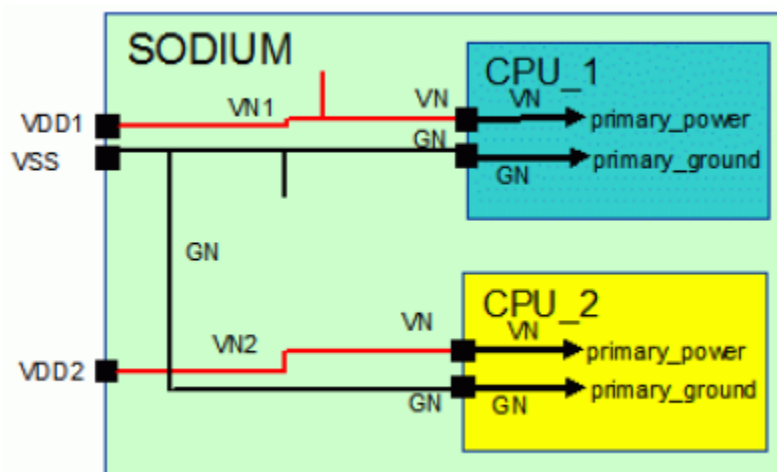
The following topics demonstrate the UPF syntax used for specifying power intent:

- [Simple Multi-Voltage Design](#)
- [Bottom-Up Power Intent Specification](#)
- [Top-Down Power Intent Specification](#)
- [Supply Set Example](#)

Simple Multi-Voltage Design

The simple multi-voltage chip design shown in [Figure 3-4](#) has two power supplies, `VDD1` and `VDD2`, at different voltage levels. The chip has two internal blocks, `CPU_1` and `CPU_2`, both of which are instances of the same CPU block. Block `CPU_1` and the top level of the chip use `VDD1`, whereas block `CPU_2` uses `VDD2`. Both `VDD1` and `VDD2` operate as always-on power throughout the chip. The same ground, `VSS`, is used throughout the chip.

Figure 3-4 Simple Multi-Voltage Chip Design



Bottom-Up Power Intent Specification

In a bottom-up flow, the lower-level CPU blocks are designed independently from the top-level chip. Therefore, their power intent must be specified at the block level and then later integrated into the chip-level power intent specification.

Following is the power intent script at the block level, `CPU.upf`:

```
create_power_domain PD
create_supply_net VN -domain PD
create_supply_net GN -domain PD
set_domain_supply_net PD -primary_power_net VN -
primary_ground_net GN
create_supply_port VN
create_supply_port GN
connect_supply_net VN -ports {VN}
connect_supply_net GN -ports {GN}
```

Following is the power intent script at the top level, `SODIUM.upf`:

```
load_upf CPU.upf -scope CPU_1
load_upf CPU.upf -scope CPU_2
```

```

# still at scope SODIUM
create_supply_port VDD1
create_supply_port VDD2
create_supply_port VSS
create_power_domain PD
create_supply_net VN1 -domain PD
connect_supply_net VN1 -ports {VDD1 CPU_1/VN}
create_supply_net VN2 -domain PD
connect_supply_net VN2 -ports {VDD2 CPU_2/VN}
create_supply_net GN -domain PD
connect_supply_net GN -ports {VSS CPU_1/GN CPU_2/GN}
set_domain_supply_net PD -primary_power_net VN1 -
primary_ground_net GN
# PD, CPU_1/PD and CPU_2/PD are different power domains.

```

Top-Down Power Intent Specification

If the chip is designed from the top down, the power intent of the design can be specified directly from the top level, if desired, as demonstrated by the following example:

```

create_power_domain PD_CPU_1 -elements {CPU_1}
create_power_domain PD_CPU_2 -elements {CPU_2}
create_power_domain PD_SODIUM
create_supply_port VDD1
create_supply_port VDD2
create_supply_port VSS
create_supply_net VN1 -domain PD_CPU_1
create_supply_net VN1 -domain PD_SODIUM -reuse
connect_supply_net VN1 -ports {VDD1}
create_supply_net VN2 -domain PD_CPU_2
connect_supply_net VN2 -ports {VDD2}
create_supply_net GN -domain PD_CPU_1
create_supply_net GN -domain PD_CPU_2 -reuse
create_supply_net GN -domain PD_SODIUM -reuse
connect_supply_net GN -ports {VSS}
set_domain_supply_net PD_CPU_1 \
-primary_power_net VN1 -primary_ground_net GN
set_domain_supply_net PD_CPU_2 \

```

```

-primary_power_net VN2 -primary_ground_net GN
set_domain_supply_net PD_SODIUM \
-primary_power_net VN1 -primary_ground_net GN

```

Supply Set Example

The following script demonstrates the usage of explicit supply sets:

```

# UPF section for Explicit Supply Set definition
## Power Domain Supply Sets
#set_scope /
create_supply_set VDD_VSS
create_supply_set VDD_LOW_VSS
create_supply_set VDDS_VSS_p0
create_supply_set VDDS_VSS_p1
create_supply_set VDDS_VSS_misc
create_supply_set VDD_LOWS_VSS_p2
create_supply_set VDD_LOWS_VSS_p3
# UPF section for Power Domain definition
## Number of PDs: 6
create_power_domain TOP \
-supply {extra_supplies_0 VDD_VSS} \
-supply {extra_supplies_1 VDD_LOW_VSS}
set_domain_supply_net TOP -primary_power_net VDD_VSS.power \
-primary_ground_net VDD_VSS.ground
create_power_domain LEON3_p0 \
-elements {u0_0/pd_switchable} \
-supply {extra_supplies_0 VDDS_VSS_p0} \
-supply {extra_supplies_1 VDD_VSS}
set_domain_supply_net LEON3_p0 -primary_power_net VDDS_VSS_p0.power \
-primary_ground_net VDDS_VSS_p0.ground
create_power_domain LEON3_p1 \
-elements {u0_1/pd_switchable} \
-supply {extra_supplies_0 VDDS_VSS_p1} \
-supply {extra_supplies_1 VDD_VSS}
set_domain_supply_net LEON3_p1 -primary_power_net VDDS_VSS_p1.power \
-primary_ground_net VDDS_VSS_p1.ground
create_power_domain LEON3_p2 \
-elements {u0_2/pd_switchable} \
-supply {extra_supplies_0 VDD_LOWS_VSS_p2} \
-supply {extra_supplies_1 VDD_VSS} \
-supply {extra_supplies_2 VDD_LOW_VSS}
set_domain_supply_net LEON3_p2 -primary_power_net VDD_LOWS_VSS_p2.power \
-primary_ground_net VDD_LOWS_VSS_p2.ground
create_power_domain LEON3_p3 \
-elements {u0_3/pd_switchable} \
-supply {extra_supplies_0 VDD_LOWS_VSS_p3} \
-supply {extra_supplies_1 VDD_VSS} \
-supply {extra_supplies_2 VDD_LOW_VSS}
set_domain_supply_net LEON3_p3 -primary_power_net VDD_LOWS_VSS_p3.power \
-primary_ground_net VDD_LOWS_VSS_p3.ground
create_power_domain LEON3_misc \

```

```

-elements {u_m/u_m_pd} \
-supply {extra_supplies_0 VDDS_VSS_misc} \
-supply {extra_supplies_1 VDD_VSS} \
set_domain_supply_net LEON3_misc -primary_power_net VDDS_VSS_misc.power
-primary_ground_net VDDS_VSS_misc.ground
# UPF section for port and supply set connections
## High Voltage supply 1.08
create_supply_port VDD
connect_supply_net VDD_VSS.power -ports VDD
## Low Voltage Supply 0.7
create_supply_port VDD_LOW
connect_supply_net VDD_LOW_VSS.power -ports VDD_LOW
## Ground nets
create_supply_port VSS
connect_supply_net VDD_VSS.ground -ports VSS
# Power Switches
create_power_switch leon3_p0_sw \
-domain LEON3_p0 \
-input_supply_port {in VDD_VSS.power} \
-output_supply_port {out VDDS_VSS_p0.power} \
-control_port {p0_sd u_m/u_power_controller_top/p0_sd} \
-on_state {p0_on_state in {!p0_sd}}
create_power_switch leon3_p1_sw \
-domain LEON3_p1 \
# Updates
create_supply_set VDD_LOW_VSS -function {ground VDD_VSS.ground} -update
create_supply_set VDDS_VSS_p0 -function {ground VDD_VSS.ground} -update
create_supply_set VDDS_VSS_p1 -function {ground VDD_VSS.ground} -update
create_supply_set VDDS_VSS_misc -function {ground VDD_VSS.ground} -update
create_supply_set VDD_LOWS_VSS_p2 -function {ground VDD_VSS.ground} \
-update
create_supply_set VDD_LOWS_VSS_p3 -function {ground VDD_VSS.ground} \
-update
# UPF Power States
## Top Supply States
add_power_state VDD_VSS -state TOP_HV \
{-supply_expr {power == `{FULL_ON, 1.08}}}}
add_power_state VDD_VSS -state GND \
{-supply_expr {ground == `{FULL_ON, 0.0}}}}
add_power_state VDD_LOW_VSS -state TOP_LV \
{-supply_expr {power == `{FULL_ON, 0.7}}}}
## Off Supply states
add_power_state VDDS_VSS_p0 -state P0_HV \
{-supply_expr {power == `{FULL_ON, 1.08}}}}
add_power_state VDDS_VSS_p0 -state P0_OFF \
{-supply_expr {power == `{OFF}}}}
add_power_state VDDS_VSS_p1 -state P1_HV \
{-supply_expr {power == `{FULL_ON, 1.08}}}}
add_power_state VDDS_VSS_p1 -state P1_OFF \
{-supply_expr {power == `{OFF}}}}
add_power_state VDD_LOWS_VSS_p2 -state P2_LV \
{-supply_expr {power == `{FULL_ON, 0.7}}}}
add_power_state VDD_LOWS_VSS_p2 -state P2_OFF \
{-supply_expr {power == `{OFF}}}}
add_power_state VDD_LOWS_VSS_p3 -state P3_LV \
{-supply_expr {power == `{FULL_ON, 0.7}}}}
add_power_state VDD_LOWS_VSS_p3 -state P3_OFF \
{-supply_expr {power == `{OFF}}}}

```

```

add_power_state VDDS_VSS_misc -state MISC_HV\
{-supply_expr {power == {FULL_ON,1.08}}}\
add_power_state VDDS_VSS_misc -state MISC_OFF\
{-supply_expr {power == {OFF}}}\
# PST
create_pst LEON3_MP PST -supplies \
{VDD_VSS.power VDD_LOW_VSS.power VDDS_VSS_p0.power VDDS_VSS_p1.power \
VDD_LOWS_VSS_p2.power VDD_LOWS_VSS_p3.power VDDS_VSS_misc.power \
VDD_VSS.ground}
add_pst_state INIT -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_OFF P1_OFF P2_OFF P3_OFF MISC_OFF GND}
add_pst_state P0_BOOT -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_HV P1_OFF P2_OFF P3_OFF MISC_OFF GND}
add_pst_state P0_P1_BOOT -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_HV P1_HV P2_OFF P3_OFF MISC_OFF GND}
add_pst_state P0_P1_P3_BOOT -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_HV P1_HV P2_LV P3_OFF MISC_OFF GND}
add_pst_state ALL_ON -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_HV P1_HV P2_LV P3_LV MISC_HV GND}
add_pst_state HIGH_PERF -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_HV P1_HV P2_OFF P3_OFF MISC_HV GND}
add_pst_state LOW_PERF -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_OFF P1_OFF P2_LV P3_LV MISC_HV GND}
add_pst_state P3_DROWSY -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_OFF P1_OFF P2_OFF P3_LV MISC_HV GND}
add_pst_state P2_DROWSY -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_OFF P1_OFF P2_LV P3_OFF MISC_HV GND}
add_pst_state P1_DROWSY -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_HV P1_OFF P2_OFF P3_LV MISC_HV GND}
add_pst_state P0_DROWSY -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_OFF P1_HV P2_OFF P3_LV MISC_HV GND}
add_pst_state ULTRA_DROWSY -pst LEON3_MP PST -state {TOP_HV TOP_LV \
P0_OFF P1_HV P2_OFF P3_OFF MISC_OFF GND}

```


4

Wildcard Support in UPF Commands

VCS NLP supports wildcard characters to refer design objects in the UPF. Support for wildcard characters is only applicable for the design (HDL)/UPF objects relevant to the option of the command. You can use ? and * wildcard characters in the string pattern to match any single character or a sequence of zero or more characters, respectively.

The supported wildcard characters are:

? — Matches any single character

* — Matches any sequence of zero or more characters

You can use these wildcard characters in a string or different hierarchies. A wildcard character does not match:

- The hierarchy separator or patterns involving the hierarchy separator.

- The * wildcard character does not expand to the generate instances. Generate labels are treated as a separate hierarchy, hence explicit wildcard per generate label name is needed for a match.

Examples

Verilog Example:

Consider the following generate instances:

```
U1/gen1[0]/inst  
U1/gen1[1]/inst  
U1/gen1[2]/inst
```

Where, gen1 is the generate label.

You must use the following wildcard pattern in VCS NLP to match the above generate instances:

```
u1/gen1[*]/* or u1/gen1*/*
```

VHDL Example:

Consider the following generate instances:

```
U1/gen1(0)/inst  
U1/gen1(1)/inst  
U1/gen1(2)/inst
```

Where, gen1 is the generate label.

You must use the following wildcard pattern in VCS NLP to match the above generate instances:

```
u1/gen1(*)/*
```

- Wildcard in index for arrays/vectors.

For example:

`inst*` — Matches instance `inst[0]`, `instA[1]`, `inst2[0]` ..

`array*[0]` — Does not match `array[3][0]`

`array[*][1]` — Does not match `array[0][1]`, `array[1][1]`

Table 4-1 *UPF Commands Supporting Wildcard Characters*

Commands	Options
<code>create_power_domain</code>	<code>-elements</code>
<code>set_isolation</code>	<code>-elements</code> , <code>-exclude_elements</code>
<code>set_retention</code>	<code>-elements</code>
<code>set_related_supply_net</code>	<code>-object_list</code>
<code>set_design_attributes</code>	<code>-elements</code> , <code>-models</code>
<code>set_level_shifter</code>	<code>-elements</code> , <code>-exclude_elements</code>
<code>connect_logic_net</code>	<code>-ports</code>
<code>set_port_attributes</code>	<code>-elements</code> and <code>-ports</code>
<code>map_isolation_cell</code>	<code>-elements</code>
<code>connect_supply_net</code>	<code>-ports</code> , <code>-exclude_elements</code>

Examples

Wildcard characters:

? – For single character

* – For zero or more characters

Following is the basic example with `create_power_domain`:

Consider two instances: Muxa, Muxb

To add both the instances in PD1:

```
create_power_domain PD1 -elements {Mux?}
```

To add only Muxa using wildcard:

```
create_power_domain PD1 -elements {*xa}
```

Wildcard Usage with Isolation

The following example illustrates the usage of wildcard in isolation policy.

List of elements to be isolated:

```
alu_i/Add1/MUXCY_L0/muxcy_inst1/CI,  
alu_i/Add1/MUXCY_L0/muxcy_inst1/DI,  
alu_i/Add2/MUXCY_L0/muxcy_inst1/CI,  
alu_i/Add2/MUXCY_L0/muxcy_inst1/DI
```

You can write the above list of elements using wildcard as follows:

```
set_isolation iso_dom3  
-domain <ref_domain_name>  
-applies_to both  
-elements {alu_i/Add*/M*Y_L0/muxcy_i*1/?I }  
-location self
```

Wildcard Usage with Retention

For the above mentioned elements, you can write the retention policy as follows:

```
set_retention ret_dom3
-domain domain3
-retention_power_net VDD -retention_ground_net VSS
-elements {alu_i/Add1/M*Y_L0/muxcy_i*1/?I }
```

Wildcard Usage with set_design_attributes

You can write model names in similar way, as illustrated above, using wildcard in UPF_dont_touch policy.

```
set_design_attributes -attribute UPF_dont_touch TRUE -
models sta*
set_design_attributes -attribute UPF_dont_touch TRUE -
models MUXCY_?
```

Wildcard Usage with set_related_supply_net

The following example shows the usage of wildcard with set_related_supply_net:

```
set_related_supply_net -object_list {u_wrap_A/
ou?_*[1]}
```

Wildcard Usage with connect_supply_net

The following example shows the usage of wildcard with connect_supply_net:

```
connect_supply_net VDD -ports {A/*/VDD*}
```

Matches both supply ports in the UPF and ports in HDL. The above expression matches A/inst/VDD, A/inst/VDD1, A/inst1/VDDPP, and so on.

Treating Generate Instance as a Single Token for Wildcard Matching

By default, VCS NLP treats each level of generate block name as a separate token while matching the wildcard patterns in the UPF.

You can use the `-power=dccompat_gen_inst_name_match` compile-time option to treat the generate instance name (any depth) as a single token for matching the wildcard character “*” specified in the UPF.

Note:

The `-power=dccompat_gen_inst_name_match` option is not needed for wildcard matching in the `find_objects` command.

For example, consider the following instance path where `gen_inv[0].i_inv` is a generate instance:

```
i_wrap/i_bloc/gen_inv[0].i_inv
```

Consider the following command in the UPF file to match the above instance:

```
create_power_domain PD_INV -elements {i_wrap/  
i_bloc/gen*.i_inv}
```

By default, the above wildcard pattern does not match `i_wrap/i_bloc/gen_inv[0].i_inv` as `gen_inv[0]` and `i_inv` are treated as separate instances.

With `-power=dccompat_gen_inst_name_match`, the generate instance `gen_inv[0].i_inv` is treated as a single instance name and single token for wildcard matching, thus resulting in a match.

Limitations of Wildcard Support

- Multi-select expressions cannot be specified with wildcard characters:

Example: `a/*/b/c[0][1]`
`a/b/c/d[*][1]`

The following case is treated as single select, and is supported:

`a/b/c/d[0][*]`

- Wildcard characters to access members of struct, union, interface, or modports are supported only for the `set_isolation` and `set_retention` commands.
- Wildcard characters are not supported with `-elements` if the elements refer to the internals of the module specified in `-models`. Wildcard characters in `-elements` are supported for matching hierarchical references only.
- Union members are not supported as part of `-elements`.

Note:

VCS NLP generates the following warning message when a wildcard expression does not find any matching object:

`"Warning-[UPF_WFF] Wildcard flattening failed"`

5

Supply-Set-Based Power Intent Specification

VCS NLP supports supply-set-based specification of power intent. Supply set is a collection of supply functions that in aggregate provide a complete power source for the design elements. VCS NLP supports the following predefined supply net functions:

- Power
- Ground
- User-defined functions

The supply set approach is a more abstract way of specifying the power network than the explicit supply port or supply net based format. This feature allows you to specify different states of supply set and power domains using the `add_power_state` command.

VCS NLP supports the following supply-set-based UPF features:

- [Explicit Supply Sets with Explicit Supply Nets](#)
- [Explicit Supply Sets with Implicit Supply Nets](#)
- [Implicit Supply Sets](#)
- [Supply Network Initialization](#)
- [Supply Set Association](#)
- [Disabling the Propagation of Supply Set State Names to Functional Nets](#)
- [Supply Sets in Strategies](#)
- [Setting Power State Using set_supply_set_state Function from Testbench](#)
- [Simstates](#)
- [Setting Default Power State for a Supply Set](#)

Explicit Supply Sets with Explicit Supply Nets

VCS NLP allows you to convert an existing supply-net-based UPF to a supply set. To enable this feature, create supply sets and connect the supply set handles to the existing supply nets using the `create_supply_set` command. Supply ports can be driven using the `supply_on` and `supply_off` calls. Consider the code shown in [Example 5-1](#).

Example 5-1 Existing UPF

```
create_supply_port VDD
create_supply_net VDD
connect_supply_net VDD -ports VDD
create_supply_port VSS
```

```

create_supply_net VSS
connect_supply_net VSS -ports VSS
#Define supply set and explicit nets
create_supply_set SS_TOP -function {power VDD} -function
{ground VSS}
create_supply_port VDD
create_supply_net VDD
connect_supply_net VDD -ports VDD
create_supply_port VSS
create_supply_net VSS
connect_supply_net VSS -ports VSS

#add_power_state command for supply set SS_TOP

add_power_state SS_TOP -state HV {-supply_expr {power ==
`{FULL_ON, 1.08} }}
add_power_state SS_TOP -state TOP_OFF {-supply_expr {power
== OFF}}

```

You can choose to replace the `create_power_domain` and `set_domain_supply_net` commands with a single `create_power_domain -supply` command, as shown in [Example 5-2](#).

Example 5-2 Existing UPF with Create Power Domain

```

create_power_domain PD_TOP
set_domain_supply_net PD_TOP -primary_power_net VDD -
primary_ground_net VSS

#Supply set based
create_power_domain PD_TOP -supply {primary SS_TOP}

```

Using the explicitly created supply nets and ports, you can drive different voltage values on to the supply set functions with the `supply_on` and `supply_off` calls.

Explicit Supply Sets with Implicit Supply Nets

With this approach, there are no explicit supply ports or supply nets. The power domain is associated with a supply set as shown in [Example 5-3](#).

Example 5-3 Explicit Supply Sets

```
create_supply_set SS_TOP
create_power_domain PD_TOP -supply {primary SS_TOP}
```

The power domain is driven to different states based on logic expressions defined in the `add_power_state` command (see [Example 5-4](#)).

Example 5-4 Add Power State Command

```
add_power_state TOP -state HV {-logic_expr {pd_top}
-simstate NORMAL }
add_power_state TOP -state TOP_OFF {-logic_expr {!pd_top}
-simstate CORRUPT }
```

Implicit Supply Sets

This feature allows you to use default power domain handles in the UPF (see [Example 5-5](#)), thereby avoiding the need to create supply sets.

Example 5-5 Implicit Supply Sets

```
create_power_domain TOP
add_power_state TOP.primary -state HV { -logic_expr {pd_top}
-simstate NORMAL }
add_power_state TOP.primary -state TOP_OFF { -logic_expr
{!pd_top} -simstate CORRUPT }
```

In [Example 5-5](#), the power domain name is treated as a supply set handle. Following are the available sets:

- `TOP.primary` — Primary of domain
- `TOP.default_retention` — Used as retention power when a retention strategy for the domain has no retention power specified.
- `TOP.default_isolation` — Used as isolation power when an isolation strategy for the domain has no isolation power specified.

All power domain state transitions happen based on the value of the specified logic expression.

Supply Network Initialization

All the supply ports and supply set functions with no explicit nets are initialized to an `ON` state by default based on the `add_port_state` or `add_power_state` command. If there are multiple `ON` states, VCS NLP selects any one of the `ON` states. The default `ON` state can also be selected by specifying the following command in the UPF or in the runtime configuration file:

```
set_design_attributes -elements {  
  hierarchical_name_of_supply_net} -attribute  
  SNPS_default_supply_net_state STATE_NAME
```

If no port or power states are defined, then all the supply ports and supply set functions with no explicit nets start in `FULL_ON` state with 1.0V. Any undriven supply net is initialized to `OFF`.

Use the `-power=undriven_supplies_off` compile-time option or the `SNPS_treat_undriven_as_off` runtime design attribute to initialize all the supply ports (except supply pads) and supply set functions with no explicit nets to `OFF` state.

Use the `SNPS_treat_undriven_as_undetermined` runtime design attribute to initialize all the undriven supply ports/nets and supply set functions with no explicit nets to `UNDETERMINED` state.

Use the `SNPS_treat_undriven_supply_nets_as_undetermined` runtime design attribute to initialize all the undriven supply nets to `UNDETERMINED` state.

Following is the use model to use the runtime configuration file:

```
% simv -power power_config.tcl
```

Where, `power_config.tcl` has the attribute defined. For example, as follows:

```
set_design_attributes -elements {  
hierarchical_name_of_supply_net} -attribute  
SNPS_default_supply_net_state STATE_NAME
```

Supply Set Association

Following are the semantics of supply set association in alignment with IEEE 1801-2013 (Section 4.6.2, Power state of a supply set):

- Power states will not be merged between the explicit supply set and associated supply set handle

- You can define power states for explicit supply set and supply set handle (`PD.primary`) separately
- Reporting and coverage on the explicit supply sets is enabled only when the power states are specified. Both reporting and coverage are turned off for implicit supply sets
- Multiple power states can be active and most conservative simstate is applied in case of a conflict (`CORRUPT > CORRUPT_ON_ACTIVITY > NORMAL`)

Disabling the Propagation of Supply Set State Names to Functional Nets

As per the IEEE Std 1801 LRM, the state names defined for the supply set in the `add_power_state` command should not be propagated to the functional net.

VCS NLP supports the above LRM behavior. The state name of the supply set specified in the `add_power_state` command is not propagated to the functional nets defined in the `-supply_expr` expression by default.

This feature disables the propagation of supply set state names to the associated supply nets and stops the creation of an incorrect system PST. It helps you to write Power State Table (PST) as per the IEEE Std 1801 LRM.

Case-1: Supply set implicit net is used in the PST with the state defined in the `add_power_state` command.

Example:

```

create_supply_set SS1
add_power_state SS1 -state ON1 {-supply_expr {power ==
{FULL_ON 2.0}}}}

create_pst MY_PST -supplies      {SS1.power}
add_pst_state STATE -pst MY_PST {ON1}

```

VCS NLP issues the following error message for this case:

```

Error-[UPF_PST_STUNRSLV] Pst entry could not be resolved
test.upf, 39
Pst entry 'ON1' corresponding to SupplyNet/SupplyPort 'tb/
top/SS1.power' in State 'STATE' of PST 'tb/top_inst/core1/
MYPST' could not be resolved.

```

Case-2: Supply port is used in the PST with the state defined in the `add_power_state` command.

Example:

```

create_supply_port VDD
create_supply_set SS1 -function {power VDD}
add_power_state SS1 -state ON1 \
{-supply_expr {power == {FULL_ON 1.0}}}}

create_pst MY_PST -supplies      {VDD}
add_pst_state STATE -pst MY_PST {ON1}

```

VCS NLP issues the following error message for this case:

```

Error-[UPF_PST_STUNRSLV] Pst entry could not be resolved
test.upf, 39
Pst entry 'ON1' corresponding to SupplyNet/SupplyPort 'tb/
top/VDD' in State 'STATE' of PST 'tb/top_inst/core1/MYPST'
could not be resolved.

```

Case-3: Supply ports without the state defined using the `add_port_state` command.

Consider the case where the supply ports are specified without the `add_port_state` command and without explicit initialization in the testbench.

```
create_supply_net VDD1
create_supply_port VDD1

connect_supply_net VDD1 -ports {VDD1}

create_supply_set SS1 -function {power VDD1} ...

add_power_state SS1 \
  -state ON {-supply_expr {power == {FULL_ON 1.2}}}}
```

The states of the supply net function are not propagated to the connected ports, in this case from `SS1.power` to `VDD1`. Hence they remain uninitialized at time 0, and supply set `SS1` is initialized to the `DEFAULT_NORMAL` state.

```
[0] [INFO] [LP_PORT_STATE_INIT] Supply Source 'tb/top_inst/
VSS1' initialized to 'ON' state with root net 'tb/top_inst/
VSS1' at state 'FULL_ON' and voltage 0 V.
```

```
[0] [INFO] [LP_SS_INIT] Supply Set 'tb/top_inst/SS1'
initialized to state 'DEFAULT_NORMAL' with simstate
'NORMAL'.
```

You must explicitly set the default start state using runtime calls from the testbench.

Backward Compatibility

For backward compatibility, you must specify the

```
set_design_attributes
enable_state_propagation_in_add_power_state TRUE
attribute at the scope for which you want to enable the supply set
state propagation.
```

Note:

- Only the `-elements` option of the `set_design_attributes` command is supported.

Only top design module or design module instances are allowed in `-elements`.

- The `-models`, `-exclude_elements`, and `-transitive` options of the `set_design_attributes` command are not supported.

VCS NLP issues an error message for the following cases where the `enable_state_propagation_in_add_power_state` design attribute is set to `TRUE`.

- The `create_power_state_group` command is used.
- The `-group` option is used in the `add_power_state` command.

Supply set state names are propagated to the supply net functions for the following scenario:

```
set_design_attributes -elements {..} \  
-attribute enable_state_propagation_in_add_power_state  
TRUE
```

```
create_supply_set SS1 -function {power VDD}  
add_power_state SS1 -state ON1 \  
{-supply_expr {power == {FULL_ON 1.0}}}
```

```
create_pst MY_PST -supplies {SS1.power}  
add_pst_state STATE -pst MY_PST {ON2}
```

The `enable_state_propagation_in_add_power_state` attribute value will not be allowed to change in the current scope once the state is defined for a Supply Set using the `add_power_state` command. For example:

```

set_design_attributes -elements {.} \
-attribute
enable_state_propagation_in_add_power_state
TRUE
create_supply_set SS1 -function {power VDD}
add_power_state SS1 -state ON1 \
{-supply_expr {power == {FULL_ON 1.0}}}

//NOT ALLOWED
set_design_attributes -elements {.} \
-attribute
enable_state_propagation_in_add_power_state
FALSE

```

Valid Usage Examples

Following are the supported usage scenarios for the `enable_state_propagation_in_add_power_state` design attribute:

- The design attribute is set on a design module instance named `block`

```

set_design_attributes -elements {block} \
-attribute enable_state_propagation_in_add_power_state
TRUE

```

- The design attribute is set on a design module instance named `block`

```

set_scope block
set_design_attributes -elements {.} \
-attribute enable_state_propagation_in_add_power_state
TRUE

```

- If the design attribute is set to `TRUE` for the `block` element in the following example, it is applicable for all its lower scopes.

```

set_design_attributes -elements {block} \
-attribute enable_state_propagation_in_add_power_state
TRUE

set_scope block/mid
create_supply_set SS2
add_power_state SS2 -state ON2 {-supply_expr {power ==
{FULL_ON 2.0}}}}

create_pst MY_PST -supplies {SS2.power}
add_pst_state STATE -pst MY_PST {ON2}

```

Error Scenarios

VCS NLP issues an error message for the following usages of the `enable_state_propagation_in_add_power_state` design attribute:

- Empty elements list specified with the design attribute

```

set_design_attributes -elements { } \
-attribute enable_state_propagation_in_add_power_state
TRUE

```

- The `-elements` list not specified with the design attribute

```

set_design_attributes \
-attribute enable_state_propagation_in_add_power_state
TRUE

```

- `-models/-exclude_elements` specified with the design attribute

```

set_design_attributes -models {M1} -exclude_elements
{U1} \
-attribute enable_state_propagation_in_add_power_state
TRUE

```

- Attribute value other than `true/TRUE` specified with the design attribute

```
set_design_attributes -elements {.} \
-attribute enable_state_propagation_in_add_power_state
JUNK
```

- An invalid object or object other than design top/design module instance specified with the design attribute

```
set_design_attributes {JUNK clk}\
-attribute enable_state_propagation_in_add_power_state
TRUE
```

- An upper scope has value specified as `TRUE` and lower scope has value specified as `FALSE`

```
set_design_attributes -elements {.} \
-attribute enable_state_propagation_in_add_power_state
TRUE
```

```
set_scope block
set_design_attributes -elements {.} \
-attribute enable_state_propagation_in_add_power_state
FALSE
```

Supply Sets in Strategies

Isolation and retention strategies support using supply set functions instead of power and ground nets.

Limitation

Following is the limitation:

- Only NORMAL, CORRUPT and CORRUPT_ON_ACTIVITY simstates are supported.

Setting Power State Using set_supply_set_state Function from Testbench

VCS NLP allows you to control supply set states through testbench using the `set_supply_set_state` function.

Syntax:

```
set_supply_set_state (input string supply_set_name, input
string power_state_name);
```

Where, `supply_set_name` is the name of the supply set, and `power_state_name` is the name of the power state.

The `set_supply_set_state` call is applicable for supply sets with power state definitions using `-supply_expr` with supply set function, which is either undriven or has a single undriven root supply driver (see [Root Supply Driver](#)).

For supply sets matching the above criteria, the following semantics apply when `set_supply_set_state` is called:

1. Supply set function in the `supply_expr` is driven to the state/voltage specified in `supply_expr`.
2. If only `-supply_expr` is specified, then power state is active.
3. If `-logic_expr` is also specified, the power state will be active only if the logic expression evaluates to TRUE.

Consider the following example where VDD is undriven:

```

## Power Domain Associated with Supply Set ##
create_supply_set SS -function {power VDD} -function {ground VSS}
create_power_domain PD -supply SS

## Power States ##
add_power_state PD.primary -state HV {-supply_expr {power ==
`{FULL_ON, 1.2}} -simstate NORMAL}
add_power_state PD.primary -state LV {-supply_expr {power ==
`{OFF, 0.0}} -simstate CORRUPT}

initial begin
    set_supply_set_state("tb/dut/PD.primary", "HV");
    #100
    set_supply_set_state("tb/dut/PD.primary", "LV");
end

```

The above `set_supply_set_state` calls result in VDD to be `{FULL_ON, 1.2V}` at time 0 and `{OFF, 0V}` at time 100. For the supply set, PD.primary, HV and LV are the active power states at time 0 and 100 respectively.

Simstates

The following are the two default simstates when there is no explicit simstate specified:

- DEFAULT_NORMAL
- DEFAULT_CORRUPT

A supply set is in DEFAULT_NORMAL if all functions are in FULL_ON state. It is in DEFAULT_CORRUPT, if any one supply function (power/ground) is OFF. Also, the state of a supply set is DEFAULT_CORRUPT when at least one of the functions defined for the supply set is not associated with a supply net.

If multiple power state specifications of a supply set match at the same time, the simstate of a domain is based on precedence order, as specified in the LRM. The order is `DEFAULT_CORRUPT`, `CORRUPT`, `NORMAL`, and `DEFAULT_NORMAL`.

The default simstate for a power state is taken as `CORRUPT` if a supply expression uses `OFF` supply state for a supply function power or ground.

In alignment with IEEE 1801-2013 (Section 4.6.2 Power state of a supply set), simstate defined on the explicit supply set is not propagated to the supply set handle, and `add_power_state` with `-simstate` must be defined on supply set handle `PD.primary`.

```
## Power States on Supply Set Handle (domain.primary) ##
add_power_state PD.primary -state HV {-supply_expr {power ==
'{FULL_ON, 1.2}} -simstate NORMAL}
add_power_state PD.primary -state STBY {-supply_expr {power ==
'{FULL_ON, 0.8}} -simstate CORRUPT_ON_ACTIVITY}
add_power_state PD.primary -state LV {-supply_expr {power ==
'{OFF, 0.0}} -simstate CORRUPT}
```

Setting Default Power State for a Supply Set

For a supply set with only one ON power state defined, the `-power=ssdefaulton` compile-time option is used to set the power state of a supply set to that state.

The following criteria has to be met for the usage of this option:

- Supply set must be undriven
- The power state is the only state defined for a supply set, and has the power function set to ON state using `-supply_expr` with no `-logic_expr`

- The power state is the only state defined for a supply set, and has the ground function set to ON state using `-supply_expr` with `no -logic_expr`

Example

In the following example, `PD3_primary` qualifies as it has only one ON state for power/ground defined:

```
add_power_state ss_PD1_primary -state V12 {-supply_expr
{power == `{FULL_ON, 1.2}}}}
add_power_state ss_PD1_primary -state PSO {-supply_expr
{power == `{OFF}}}}
add_power_state ss_PD2_primary -state V12 {-supply_expr
{power == `{FULL_ON, 1.2}}}}
add_power_state ss_PD2_primary -state PSO {-supply_expr
{power == `{OFF}}}}
add_power_state ss_PD3_primary -state V12 {-supply_expr
{power == `{FULL_ON, 1.2}}}}
add_power_state ss_PD3_primary -state GND {-supply_expr
{ground == `{FULL_ON, 0.0}}}}
```


6

Low Power Simulation

The following sections describe the low power design simulation in VCS NLP:

- [Running Low Power Designs](#)
- [HDL Constructs and Techniques in VCS](#)
- [Support for UPF Supply Functions](#)
- [Voltage-Based supply_on](#)
- [Modeling Delay Specification for a Power Switch](#)
- [Monitoring the Simstate of a Power-Managed Instance](#)
- [Reporting Write Attempt on OFF Domain Element](#)
- [Initial Block Retriggering](#)

Running Low Power Designs

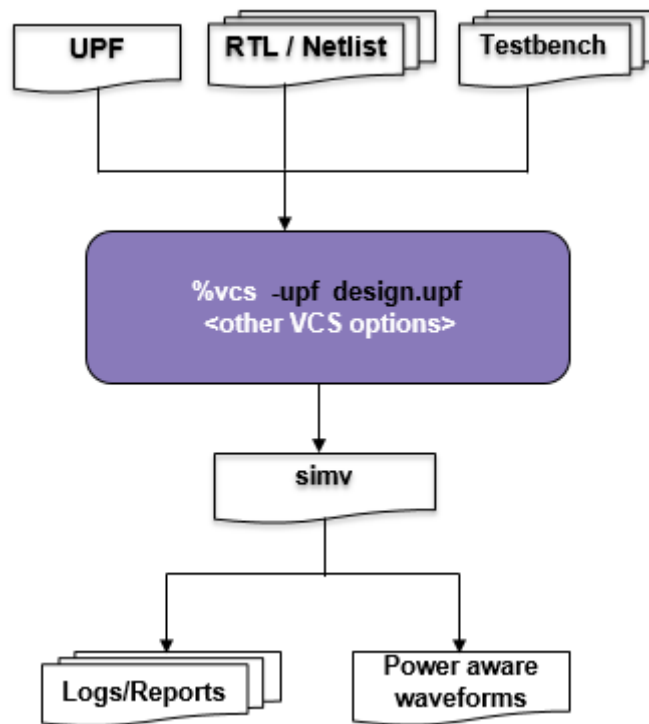
This section describes the VCS NLP low power simulation flow and the options required to compile and run low power designs in the following sections:

- [VCS NLP Low Power Simulation Flow](#)
- [Using Low Power in VCS MX](#)

VCS NLP Low Power Simulation Flow

Specifying the `-upf` option at compile time automatically enables VCS NLP. You must specify the UPF file using the `-upf` option at `vcs` step as shown in [Figure 6-1](#).

Figure 6-1 VCS NLP Low Power Simulation Flow



Using Low Power in VCS MX

To use low power mixed designs in VCS MX

1. Analyze your design:

VHDL files

```
% vhdlan [vhdlan_options] file1.vhd file2.vhd
```

Verilog files

```
% vlogan [vlogan_options] file1.v file2.v
```

SystemVerilog files

```
% vlogan -sverilog [vlogan_options] file1.sv file2.sv  
file3.v
```

2. Elaborate your design using the -upf option:

```
% vcs [elab_options] [libname].design_unit -upf  
<upf_file>
```

3. Simulate your design:

```
% simv
```

Power Techniques Supported in VCS MX

VCS MX flow supports the following power techniques:

- Power gating
- Shutdown
- Isolation
- Retention

Limitations

- Shadow retention registers are not visible in DVE for VHDL as in Verilog
- UPF name formatting is not honored for output isolation in VHDL scope
- HDL constructs not supported for isolation are enum, record, int, real, and boolean
- HDL constructs not supported for retention are real, int, and record

HDL Constructs and Techniques in VCS

VCS supports the following HDL constructs in syntax and in low power semantics:

1. Verilog 95
2. Verilog 2K
3. SystemVerilog design constructs
 - Not corrupted:
 - Time
 - sv-string and character
 - event
 - Dynamic data types: class objects (dynamic arrays, smart queues, and so on.)

Note:

- In VHDL, all data types are corrupted with ``left`. The only exception is `std_logic`, which is corrupted by `X`.
- All data types having ``left` as 0 are corrupted by 0.
Example: real, natural, and so on.
- The corrupted value of enum depends on the enum base type.
- Corrupted with `SIZE'X`:
 - Integer
 - logic

- Corrupted with `SIZE'0`:
bit, byte, real, int, short int, and long int

Support in syntax, but not in semantics

The SVTB HDL syntax is supported in VCS. VCS issues a parse warning if any SystemVerilog testbench construct is encountered. However, low power semantics are not applied on this construct during simulation.

Support for UPF Supply Functions

VCS NLP supports the following functions from the UPF package:

```
function bit supply_on( string pad_name, real value);  
function bit supply_off( string pad_name );  
function bit supply_partial_on( string pad_name, real value );  
function supply_net_type get_supply_value( string name );  
function real get_supply_voltage( supply_net_type arg );  
function bit get_supply_on_state( supply_net_type arg );  
function state get_supply_state( supply_net_type arg );
```

You must import the UPF package, as shown below, before using these functions:

```
import UPF::*;
```

The names of the supply nets, supply ports, and supply set functions in the following function calls can be full hierarchical names from the simulation top or from the power top scope.

The `supply_on`, `supply_partial_on`, and `supply_off` calls work only on undriven supply nets, supply ports, or supply set functions.

The `get_supply_value` call works on any supply nets, supply ports, or supply set functions.

- `supply_on/supply_partial_on/supply_off` — Sets the state and voltage value on a supply net/port.

Verilog Example:

```
import UPF::*;
initial begin
    supply_on("VDD", 1.08);
    supply_on("testbench/TOP_U/u2/V_U", 1.06);
    supply_on("SS1.power", 1.0); //Explicit supply set
    supply_on("PD1.primary.power", 1.0); //Implicit supply set
    supply_off("VDD");
    supply_off("testbench/TOP_U/u2/V_U");
    supply_partial_on("testbench/TOP_U/VDD", 1.00);
end
```

VHDL Example:

```
use IEEE.upf.all;
entity testbench is
end testbench;
architecture behave of testbench is
...
    signal dummy :boolean;
begin
    simulate:process
    begin
        dummy <= supply_on("testbench/TOP_U/Vdd", 1.08);
        dummy <= supply_on("testbench/TOP_U/V1", 1.08);
        wait for 100 ns;
        dummy <= supply_off("testbench/TOP_U/Vdd");
        ...
    end process;
```

Note:

You can also specify the `-liblist` UPF compile-time option as an alternative.

- `get_supply_value` — Return value is of type `UPF::supply_net_type`
- `get_supply_voltage` — Gets the voltage value of the supply. Return value is of type 'real'.
- `get_supply_on_state` — Return value is of type 'bit'. Returns 1 if the state of the supply is `FULL_ON`, else 0.
- `get_supply_state` — Gets the information on the current state (`FULL_ON/OFF/PARTIAL_ON/UNDETERMINED`) of the supply. Return value is of type `UPF::state`.

Example:

```
module test;
  import UPF::*;
  supply_net_type snt;
  real voltage;
  bit isOn;
  state currState;

  initial begin
    snt = get_supply_value("testbench/TOP_U/u2/V_U");
    voltage = get_supply_voltage(snt);
    isOn = get_supply_on_state(snt);
    currState = get_supply_state(snt);
  end
  ..
endmodule
```

UCLI Support for Supply Functions

For UCLI, the hierarchical path to the supply port must be absolute path. Relative paths are not supported.

Use Model:

```
% vcs file_name.v -upf test.upf -debug_all
```

```
% simv -ucli
ucli% lp_show -supply_on <port name> <voltage value>
ucli% run <time>
ucli% lp_show -supply_off <port name>
```

Example:

```
ucli% lp_show -supply_on testbench/TOP_U/Vdd 1.08
ucli% lp_show -supply_on testbench/TOP_U/V2 1.08
ucli% run 300ns
ucli% lp_show -supply_off testbench/TOP_U/Vdd
```

Limitations

- This feature is not supported for VHDL testbench.

Voltage-Based supply_on

By default, a UPF `supply_on` function call changes the state of the supply pad to `FULL_ON` irrespective of the voltage specified. If the `SNPS_voltage_based_supply_on` attribute is set to `TRUE`, as shown below, in the UPF or runtime configuration file (`-power <config_file>`), then the `supply_on("<SUPPLY_PAD>," 0.0)` function with voltage as 0 on a supply pad of type `power` changes the state of the supply to `OFF`.

```
set_design_attributes -attribute
SNPS_voltage_based_supply_on TRUE
```

The `SNPS_voltage_based_supply_on` attribute has no impact on the supply of type `ground`. Supply is determined as ground type, if the supply pad is driving a supply net:

- which is `primary_ground_net` of a domain

or

- which is `isolation_ground_net` or `retention_ground_net`

or

- which is a related ground supply used in `set_related_supply_net`

or,

- tied to a macro PG pin of type `primary_ground`

Modeling Delay Specification for a Power Switch

To model the realistic behavior of a power switch, VCS NLP has a facility to specify a delay value to be used before the power switch turns ON after the control signal and the input power are valid. Use the `SNPS_power_switch_on_delay` attribute of the `set_design_attributes` command to specify the delay value as follows:

```
set_design_attributes -elements \  
{<Hierarchical name of the power switch policy>} \  
-attribute SNPS_power_switch_on_delay <value>
```

Add this command to the configuration file you specify at runtime using `-power <config file>` with the simulation executable. For example:

```
% simv -power lpconfig.tcl <other runtime options>
```

The delay units are based on the simulator timescale. For example, if you specify `value` as 100, then for a timescale of 1ns/1ns the power switch powers up after 100ns, after control is active and input supply is ON.

Monitoring the Simstate of a Power-Managed Instance

VCS NLP provides a way to monitor the simstate of the power-managed instances from the testbench using the `upf_simstate` variable of type `UPF::power_state_simstate`. Where, `power_state_simstate` is a predefined type in the UPF package.

```
typedef enum {NORMAL,
              CORRUPT,
              CORRUPT_ON_ACTIVITY,
              CORRUPT_STATE_ON_CHANGE,
              CORRUPT_STATE_ON_ACTIVITY
            }
power_state_simstate;
```

You must import the UPF package and declare a variable of type `power_state_simstate` by name `upf_simstate` in the power-managed module. Once declared, the user-defined `upf_simstate` variable mirrors the instance's simstate during simulation. You can access this variable from the testbench using the XMR.

Example

```
module testbench;
  M1 dut(..);

  always @(dut.upf_simstate)
  begin
    if (dut.upf_simstate == NORMAL)
      $display("Domain PD1 is in NORMAL state");
    else if (dut.upf_simstate == CORRUPT)
      $display("Domain PD1 is in CORRUPT state");
    end
  ..
endmodule

module M1(...);
  ..
  import UPF::*

  power_state_simstate upf_simstate;
  ..
endmodule
```

Note:

- Variable name must be `upf_simstate` in order to mirror the `simstate` of an instance.
- `upf_simstate` can only be read and cannot be written to.
- Access the `upf_simstate` from the testbench or an always-on module using XMR since accessing it locally in the power-managed module results in the monitoring logic (for example, always block) to be inactive during the CORRUPT `simstate`.

Reporting Write Attempt on OFF Domain Element

VCS NLP generates the runtime warning messages listed in [Table 6-1](#) when an attempt to write on a power domain element is detected and the domain is in `CORRUPT` simstate. VCS NLP blocks the write on the power domain element.

You must use the `-power=check_power_down_write` compile-time option to enable this feature.

Table 6-1 Warning Messages

Action	Warning Message
XMR write to a Verilog signal from the Verilog scope	LP_HDL_XMR_WRITE_BLOCKED
Signal write through vpi/acc (PLI call)	LP_PLI_SIGNAL_WRITE_BLOCKED
Signal write from UCLI	LP_UCLI_SIGNAL_WRITE_BLOCKED

Supported Scenarios

This feature is supported for the following writes on elements of an OFF domain:

Scenario-1

`$deposit` on a signal

Example:

```
$deposit("chipTb.chipDut.domain.myreg"...)
```

Scenario-2

Writing to a reg through an XMR in the initial block

Example:

```
initial chipTb.chipDut.domain.myreg = 1'b0;
```

Scenario-3

force statement through an XMR

Example:

```
initial force chipTb.chipDut.domain.myout[9:0] = 10'b0;
```

Scenario-4

Changing the value of a signal using \$hdl_xmr

Example:

```
reg mysig;  
initial $hdl_xmr("chipTb.chipDut.domain.myreg", "mysig");
```

Use Model

```
% vcs -sverilog -debug_all -upf file.upf test.v  
-power=check_power_down_write -P pli.tab pli.c
```

```
% simv -ucli -i ucli_file
```

Following is the sample output:

```
[WARNING] [LP_HDL_XMR_WRITE_BLOCKED] XMR write from scope  
'test' to signal/variable 'test.t.inv.mem' is blocked as
```



```
the simstate of the corresponding power domain 'test/t/A' is 'CORRUPT'. File : test.v, Line : 21
```

```
[WARNING] [LP_PLI_SIGNAL_WRITE_BLOCKED] XMR write from scope 'test' to signal/variable 'test.t.inv.mem[1]' is blocked as the simstate of the corresponding power domain 'test/t/A' is 'CORRUPT'. File : test.v, Line : 49
```

```
ucli% force {test.t.inv.mem[1]} 1  
[WARNING] [LP_UCLI_SIGNAL_WRITE_BLOCKED] XMR write from scope 'PowerNetworkModel.\ps_test/t .A' to signal/variable 'test.t.inv.mem[1]' is blocked as the simstate of the corresponding power domain 'test/t/A' is 'CORRUPT'. File : UCLI_CMD, Line : 0.
```

Limitations

This feature does not support the following:

- Writing through the VHPI call
- XMR write through the continuous assign statement
- The following runtime force semantics:

`FORCE_PRIORITY`, `FORCE_AND_CORRUPT`, and `DEFER_FORCES`

Initial Block Retriggering

VCS NLP supports initial block retriggering (`reinit`). You can use initial blocks in your design to assign power-up values to the storage elements, write behavioral models of analog macros for the digital simulations, handle memory reads through tasks, and so on.

However, with multiple power domains in the design, when you remove voltage from the power domain containing these initial blocks, VCS NLP simulations corrupt the variables used in the initial blocks. This might eventually cause the simulation to fail, as these initial blocks are executed only once by the simulator and not every time the power is restored.

Enabling Initial Block Retriggering in VCS NLP

Use the `set_design_attributes` command to enable initial block retriggering (`reinit`) on a design element. Following is the syntax of this command:

Syntax

```
set_design_attributes
  -elements element_list
  -models model_list
  [-attribute name value]*
  -transitive <TRUE|FALSE>
```

Arguments

- `-elements element_list` — List of design elements or named initial blocks.
- `-models model_list` — List of models to be attributed.
- `-attribute name value` — For the enumerated design element or model, associate the attribute `SNPS_reinit` or `SNPS_dont_reinit` with a value of `TRUE` or `FALSE`.
- `-transitive <TRUE|FALSE>` — When `-transitive` is `TRUE`, the command applies to the descendants of the elements. Default value is `FALSE`.

Initial Block Retriggering Supported Attributes

Initial block retriggering supports the attributes shown in [Table 6-2](#).

Table 6-2 Supported Attributes for Initial Block Retriggering

Attribute Name	Description
<code>SNPS_reinit</code>	The supported values for this attribute are <code>TRUE</code> and <code>FALSE</code> : • <code>TRUE</code> value enables <code>reinit</code> on the initial blocks targeted by the command. • <code>FALSE</code> value resets the <code>reinit</code> marking on the initial blocks targeted by the command.
<code>SNPS_dont_reinit</code>	The supported values for this attribute are <code>TRUE</code> and <code>FALSE</code> : • <code>TRUE</code> value enables <code>don't reinit</code> property on the initial blocks targeted by the command. • <code>FALSE</code> value resets the <code>don't reinit</code> marking on the initial blocks targeted by the command.

Reinit Property

Based on the values passed to the `SNPS_reinit` and `SNPS_dont_reinit` attributes, VCS applies one of the following three properties to the initial blocks in the power-managed part of the design:

- `uninitialized` — By default, every initial block has an `uninitialized` property. These initial blocks are not retriggered on power-up.
- `reinit` — Initial blocks having this property are retriggered on power-up.
- `don't reinit` — Initial blocks having this property are not retriggered on power-up.

VCS NLP applies the `reinit` or `don't reinit` property in descending order of priority, as listed below:

1. Named initial blocks in a module
2. Design instance
3. Design module
4. Power domain
5. Entire design

Transitive Reinit

You can apply the `reinit` property on a design instance or module transitively on all power-managed descendants of the design module instances. VCS NLP applies only the `transitive reinit` property on the initial blocks which have the `uninitialized` property.

Examples for Different Reinit Scenarios

- Module-level support. That is, initial blocks of all instances of a module are retriggered on power-up.

```
set_design_attributes -models ModuleForReInit \  
-attribute SNPS_reinit TRUE
```

- Named initial block support at module level.

```
set_design_attributes -elements named_initial_1 \  
-models ModuleForReInit \  
-attribute SNPS_reinit TRUE
```

- Transitive support for an instance. That is, initial blocks of descendants are also retriggered on wakeup.

```
set_design_attributes -elements U_Inst \
-attribute SNPS_reinit TRUE \
-transitive TRUE
```

- Transitive support for a module. That is, initial blocks of descendants are also retriggered on wakeup.

```
set_design_attributes \
-models ModuleForReInit \
-transitive TRUE \
-attribute SNPS_reinit TRUE
```

- Non-transitive support for an instance.

```
set_design_attributes -elements U_Inst \
-attribute SNPS_reinit TRUE
```

- Non-transitive support for a module.

```
set_design_attributes \
-models ModuleForReInit \
-attribute SNPS_reinit TRUE
```

- Power domain level support. That is, initial blocks of all instances in a power domain are retriggered on wakeup.

```
set_design_attributes -elements PD_Core \
-attribute SNPS_reinit TRUE
```

- Design-wide reinit

```
set_design_attributes -attribute SNPS_reinit TRUE
```

Transitive Reinit Limitations

Hierarchical named initial blocks are not supported for `reinit` or `don't reinit`. Only named initial blocks in a module can be specified for `reinit` or `don't reinit`.

7

Compile-Time Static Reports

VCS NLP compile-time static reports are generated in the compile-time static reports directory. The default name of this directory is `mvsim_native_reports`, and its default location is your current working directory. The following sections describe compile-time static reports:

- [Redirecting/Renaming mvsim_native_reports Directory](#)
- [Compile-Time Static Report for set_design_attributes](#)
- [Default Reports Dumped Under mvsim_native_reports Directory](#)

Redirecting/Renaming mvsim_native_reports Directory

You can use the `-mvrpt` compile-time option to redirect or rename the compile static reports directory. Following is the syntax of the `-mvrpt` option:

```
-mvrpt=<full_path_to_directory>
```

Key Points to Note

- The `-mvrpt` option supports both absolute paths and relative paths
- If the specified directory does not exist, a new directory is created and the reports are dumped in that directory
- If the write permissions are not available for the specified directory, a warning message is issued, and the reports are dumped in the default directory `mvsim_native_reports` in the current working directory

Example

The following option dumps the VCS NLP compile static reports in the `/my_path/dir_test/reports` directory:

```
-mvrpt=/my_path/dir_test/reports
```

Compile-Time Static Report for `set_design_attributes`

VCS NLP dumps the report files with the information about all the design attributes used in the UPF with the `set_design_attributes` command.

All the reports are dumped in the `SDA` folder in the `mvsim_native_reports` directory. If any attribute is declared at the design level, it is dumped in the common file `SDA_attribute_summary.rpt`. Some of the attributes are defined at the power domain or design model level; for such attributes, individual reports are generated in the same `SDA` folder with the respective names of the attributes (`<attribute>.rpt`).

VCS NLP dumps the static report files for the following compile-time design attributes, if they are specified in the UPF:

- `deselect_coverage_object`
- `enable_bias`
- `iso_nor`
- `is_soi`
- `lower_domain_boundary`
- `conservative_diff_supply_only_isolation`
- `SNPS_default_supply_net_state`
- `SNPS_voltage_based_supply_on`
- `SNPS_force_semantics`
- `SNPS_default_power_upf2sv_vct`

- SNPS_default_power_upf2vh_vct
- SNPS_default_ground_upf2sv_vct
- SNPS_default_ground_upf2vh_vct
- SNPS_default_power_sv2upf_vct
- SNPS_default_ground_sv2upf_vct
- SNPS_macro_port_based_corruption
- SNPS_override_pbp_corruption
- SNPS_random_corruption
- SNPS_reinit
- SNPS_dont_reinit
- UPF_dont_touch

Default Reports Dumped Under mvsim_native_reports Directory

[Table 7-1](#) lists all the reports that are dumped by default by VCS NLP under the mvsim_native_reports directory.

Table 7-1 *Default Reports Dumped Under mvsim_native_reports Directory*

Report Name	Description
<code>hierarchical_flop_inference.rpt</code>	Compile-time static report for Verilog. Captures the details of the inferred retention registers.
<code>hierarchical_flop_inference_vhdl.rpt</code>	Compile-time static report for VHDL. Captures the details of the inferred retention registers.
<code>isolation_association_summary.rpt</code>	Captures the details of the associated isolation cell instances and their associated isolation strategies
<code>library_mapping.rpt</code>	Captures the cell information.
<code>library_match.rpt</code>	Captures the matching/non-matching cell information.
<code>library_mapping_after_upf.rpt</code>	Captures the port-based corruption information of a cell.
<code>SDA/SDA_attribute_summary.rpt</code>	Captures information of the <code>set_design_attributes</code> command specified at element or model level.
<code>srsn_spa_association.rpt</code>	Captures information about the SRSN and SPA buffers associated with a port
<code>upf_state_machine.rpt</code>	Captures the objects with legal/illegal states/transitions being covered.
<code>upf_control_port_coverage.rpt</code>	Captures the functional coverage on the control ports/signals of a power switch, isolation strategy, and retention strategy.
<code>upf_powerswitch_mapping.rpt</code>	Captures the associated/Unassociated Power Switch Instances information
<code>upf_supply_set_power_state.rpt</code>	Captures information on all the power states of all the Supply Sets in the design.

Report Name	Description
undefined_db_cell.rpt	Lists the cells modules which are defined within <code>`celldefine</code> and <code>`endcelldefine</code> , but not having a corresponding matched liberty db cell.
upf_supply_source_rsd.rpt	Captures information on all the type of supply sources like root supply of a supply source and net/ports connected the source.
upf_construct_statistics.rpt	Captures information on the number of low power constructs used in the UPF file.

8

Handling Corruption in Low Power Designs

The following sections describe VCS NLP corruption semantics for low power designs:

- [Simulation Semantics for Corruption](#)
- [Random Corruption](#)
- [Preventing Corruption of Memories During Power Shutdown](#)
- [Corruption of Bound Modules](#)
- [Controlling the Corruption and Wake-up Semantics of the Initial and Always Blocks](#)
- [Treating Continuous Assignments as Pass Through](#)
- [Treating Continuous Assignment Statements as Feedthrough](#)
- [CORRUPT_ON_ACTIVITY Simstate \(Low-Vdd Standby\)](#)

- [Excluding Parts of Design from Being Power Managed](#)
- [Skipping Corruption](#)

Simulation Semantics for Corruption

The following semantics apply when the domain simstate transitions to CORRUPT:

- All 2-state variables are corrupted to 0
- All 4-state variables are corrupted to x
- All the queued events including HDL forces are flushed (see the [“Overriding Corruption with HDL Forces”](#) section to configure the behavior of HDL forces during corruption)
- PLI/VPI/UCLI forces do not take effect (see the [“Treating Continuous Assignment Statements as Feedthrough”](#) section to configure the behavior of runtime forces during corruption)
- All events scheduled in future time are flushed and no new events are allowed
- All the always blocks are disabled
- All the initial blocks and their child threads are terminated
- A domain turned off at time 0 enables corruption at time 0

Overriding Corruption with HDL Forces

Use the `-power=force_priority` compile-time option to override corruption with the HDL force. The following are the simulation semantics for HDL forces and corruption under `-power=force_priority`:

- Force happens in `NORMAL` state — When the signal goes to `CORRUPT` state, the forced value overrides corruption until released. On wake-up, force continues to override until released
- Force happens in `CORRUPT` state — Forced value overrides corruption. On wake-up, force continues to override until released
- Force is released in `CORRUPT` state — Back to the corruption value and normal simulation semantics apply

Table 8-1 Semantics for HDL Forces and Corruption under `-power=force_priority`

Current Value = A Forced Value = F Corruption Value = X	Simstate=NORMAL (Domain=ON)	Simstate=CORRUPT (Domain=OFF)	(CORRUPT -> NORMAL) (Domain=ON)
Force in <code>NORMAL</code>	A -> F	F	F (until released)
Force in <code>CORRUPT</code>	A	X -> F	F (until released)
Force in <code>NORMAL</code> , release in <code>CORRUPT</code>	A -> F	F -> X	X

Note:

If the force is active on a part (bit or part select) of the node or memory, corruption on the whole node or memory is skipped when the simstate goes from `NORMAL` to `CORRUPT`.

Corruption of XMR Based on its Domain Simstate

Use the `-power=xmr_corrupt` compile-time option, as shown below, if the corruption of the LHS XMR must be based on the simstate of its target domain.

```
% vcs file_name.v -upf <upf_file> -power=xmr_corrupt
```

Note:

The code with XMR reference must be in the power-managed hierarchy. This option does not have any impact on the XMR in testbench.

Example

Consider the following example where `child2` instance is in the power domain PD2, and the `parent` and `child1` instance are in the power domain PD1.

```
top -> parent (PD1) -> child1 (PD1)
                        -> child2 (PD2)
```

Element of `child2` instance is assigned in the `parent` using the XMR.

```
assign child2.sig = in_sig;
```

or

```
assign child2.sig = child1.out;
```


By default, when the PD1 power domain is in CORRUPT simstate, then `child2.sig` is corrupted. Use `-power=xmr_corrupt` if the corruption of the LHS XMR has to be based on the simstate of its target domain (PD2 in case of `child2`).

With `-power=xmr_corrupt`, `child2.sig` is corrupted when the power domain PD2 is in the CORRUPT simstate.

Limitations

Following is the limitation of this feature:

- XMR reference using `hdl_xmr/$hdl_xmr` is not supported

Random Corruption

By default, VCS NLP corrupts 4-state variables to x and 2-state variables to 0 in a power-managed module during shutdown.

The random corruption feature enables you to select a different random value for corruption for each shutdown event. If a power domain is shut down multiple times during simulation, the corruption value during each shutdown is selected randomly from the set of values you specify.

Usage

Use the following options to enable this feature in VCS NLP:

- `-pa_random_corrupt` compile-time option
- `-power <Tcl filename>` runtime option

Where,

`<Tcl filename>` — Tcl file which sets the `SNPS_random_corruption` attribute using the `set_design_attributes` command. You can use this command to set the corruption value to any of 0/1/X/Z during shutdown. Use the following syntax:

```
set_design_attributes -elements {<power domain>} \  
-attribute SNPS_random_corruption <value>
```

You must specify one of the following values with the `SNPS_random_corruption` attribute:

- 0 - Corruption to 0
- 1 - Corruption to 1
- X - Corruption to X
- Z - Corruption to Z
- 01 - Randomly choose either 0 or 1 per shutdown event
- 01X - Randomly choose one of 0/1/X per shutdown event
- 01XZ - Randomly choose one of 0/1/X/Z per shutdown event

The values not in the above list are illegal.

Note:

- If X or Z is chosen randomly as the corruption value, 2-state variables are corrupted to 0.
- The hierarchical separator for a power domain specified with `-elements` is '/'.

You can also specify a seed using the `set_design_attributes` command. A seed is a number that initializes a pseudo-random generator used for corruption during shutdown. You can use it to recreate the same shutdown corruption pattern from different runs of the same test case. Use the following syntax to specify a seed:

```
set_design_attributes  
-attribute SNPS_random_seed <value>
```

Key Points to Note

- The random seed applies to the entire design. Specifying random seed per domain level using the `SNPS_random_seed` design attribute with `-elements` is not supported
- Random corruption is applicable to elements that are corrupted based on domain simstate going to `CORRUPT/`
`CORRUPT_ON_ACTIVITY`
- Random corruption is not applicable to corruption on elements due to exception connections like `set_related_supply_net`, `set_port_attributes`
- Random corruption is not applied for port-based corruption in case of macros
- Random corruption is not applied on retention registers
- Random corruption is not applied for corruption due to isolation/retention supplies being OFF

- SNPS_random_corruption and SNPS_random_seed are the runtime design attributes

Following are the only design attributes that can be used at runtime. The rest of the design attributes mentioned in this document are for compile time.

- SNPS_random_corruption
- SNPS_random_seed
- SNPS_treat_undriven_as_off
- SNPS_treat_undriven_as_undetermined
- SNPS_treat_undriven_supply_nets_as_undetermined
- SNPS_keep_unassociated_iso_on
- SNPS_power_switch_on_delay
- SNPS_voltage_based_supply_on

Examples

Compile:

```
% vcs -sverilog test.v -upf test.upf -pa_random_corrupt
```

Run:

```
% simv -power lpconfig.tcl
```

Examples for lpconfig.tcl file:

1. Random corruption on a power domain

```
set_design_attributes -elements \
{power_pa_tb/noRandomCorruption} \
-attribute SNPS_random_corruption 01X
```

2. Random corruption on the entire design

```
set_design_attributes -attribute \
SNPS_random_corruption 01
```

3. Applying a random seed on the entire design

```
set_design_attributes -attribute SNPS_random_seed 37
```

Note:

- Enum variables, when corrupted with 0, take the value 0. When they are corrupted to 1, they take the value of all ones (32'hffff_ffff).
- VCS NLP supports random corruption of all individual bits of a vector and all the vectors in a given domain. That is, all the bits of a vector and different vectors gets the values which are independent of each other. For example, as shown in [Table 8-2](#).

Table 8-2 *Randomized Corruption of Vectors and Individual Bits of a Vector*

a[2:0]	b[2:0]
3'b110	3'b01x
3'b001	3'b1z1
3'b010	3'bx1x
3'b001	3'b11z

Limitations

Following is the limitation of this feature:

- Vector/Bit-level randomization is supported only for the Verilog portion of the design. It is not applicable to the VHDL elements.

Preventing Corruption of Memories During Power Shutdown

Use the `-power=dont_touch_mem` compile-time option to prevent the corruption of memories that are initialized using `$readmemh*` and have no other structural drivers.

```
% vcs -sverilog -upf <upf_file> file_name.sv
-power=dont_touch_mem
```

Limitations

Following is the limitation of this feature:

- Partial assignment (where only some locations of memory are initialized with `$readmemh`) is not supported.

Corruption of Bound Modules

By default, VCS NLP avoids corruption of modules bound to an RTL design. You can use the `-power=corrupt_binds` compile-time option to allow corruption of modules bound to an RTL design using the `bind` statement.

Controlling the Corruption and Wake-up Semantics of the Initial and Always Blocks

You can use the attributes listed in [Table 8-3](#) to control the corruption and wake-up semantics of the initial and always blocks. You must use the `-power=attributes_on` compile-time option to enable these attributes.

Table 8-3 *List of Attributes Used to Control the Corruption and Wake-up Semantics of the Initial and Always Blocks*

Attribute	Description
<code>vcs_reinit</code>	Allows you to retrigger the selected initial blocks on power-up without using the UPF commands. You can mark an initial block by specifying <code>(* vcs_reinit *)</code> attribute in the RTL. For example, as shown below: <pre> (* vcs_reinit *) initial begin:block_reinit #5 mem[0] = 8'b000000001; mem[1] = 8'b000000010; mem[2] = 8'b000000100; mem[3] = 8'b000001000; end </pre>
<code>vcs_dont_reinit</code>	Disables the retriggering of the selected initial block after power-up. Specify <code>(* vcs_dont_reinit *)</code> before an initial block in the RTL. For example, as shown below: <pre> (* vcs_dont_reinit *) initial begin:block_reinit #5 mem[0] = 8'b000000001; mem[1] = 8'b000000010; mem[2] = 8'b000000100; mem[3] = 8'b000001000; end </pre> This attribute does not retrigger the selected initial block on power-up even if it is marked for reinit using the <code>SNPS_reinit</code> design attribute in the UPF.

Attribute	Description
vcs_always_on	Marks the selected always block as always-on. Specify (<code>* vcs_always_on *</code>) before an always block in the RTL. For example, as shown below: <pre>(* vcs_always_on *) always @(posedge clk) begin q <= d; end</pre> The above always block is marked as always-on, and is not affected by power-down. No corruption is observed on this part of code when the domain is powered off.
vcs_dont_trigger_on_wakeup	Disables the automatic retriggering of the selected always block on power-up. Specify (<code>* vcs_dont_trigger_on_wakeup *</code>) before an always block in the RTL. For example, as shown below: <pre>(* vcs_dont_trigger_on_wakeup *) always @ (ISO_IN, ISO_OUT) begin OUT = ISO_IN & ISO_OUT; end</pre>

Treating Continuous Assignments as Pass Through

VCS NLP treats the following statements as pass through and skips corruption:

- Simple Verilog continuous assignments (`assign a=b`)

- `buf` primitives
- Simple VHDL signal assignments (`temp1 <= temp2`)
- Signal assignments with composite data type (records)
- Simple assignment in VHDL procedures
- Assignments with simple concatenations

Limitations

Following are the limitations of this feature:

- Only corruption is skipped on Verilog `buf` primitives
- The following are not treated as pass through for isolation analysis:
 - Assignments inside VHDL process statements
 - VHDL signal assignments with delay

Treating Continuous Assignment Statements as Feedthrough

VCS NLP treats the simple Verilog continuous assignment statements (no expression on RHS) as feedthrough for the isolation crossover analysis. Also, the LHS of a simple continuous assignment statement is not corrupted during power shutdown.

By default, VCS NLP skips corruption on the feedthrough paths for Verilog and VHDL. You can use the `-power=nofeedthru` compile-time option to allow corruption on the feedthrough paths for Verilog and VHDL.

Configuring Semantics for Runtime Forces During Power Shutdown and Wake-up

VCS NLP supports the following modes to configure force semantics for runtime forces:

- [Force Priority](#)
- [Force and Corrupt](#)
- [Flush Forces](#)
- [Defer Forces](#)

Key Points to Note:

- This feature is supported for `simstate CORRUPT` only. It is not supported for any other simstates with corruption semantics.
- The above force semantics for runtime forces are applicable for:
 - The design as a whole. There will be no granularity with respect to the design instances, signals or power domains.
 - All types of runtime forces (PLI, VPI (both `deposit` and `vpiForceFlag`), UCLI, DVE, and so on) follow the same semantics.

Force Priority

- Force happens in `NORMAL` state — When the signal goes to the `CORRUPT` state, the forced value overrides the corrupt value until released. On wake-up, the force continues to override until released.
- Force happens in `CORRUPT` state — Forced value overrides corrupt value. On wake-up, the force continues to override until released.
- Force is released in `CORRUPT` state — Back to corrupt value and normal simulation semantics apply.

See [Table 8-4](#) for more information.

Force and Corrupt

- Force happens in `NORMAL` state — When the signal goes to the `CORRUPT` state, VCS NLP corrupts it, but keeps the forced state. On wake-up, the force overrides with the corrupt value, until released
- Force happens in `CORRUPT` state — VCS NLP continues with the corrupt value, but keeps the forced state. On wake-up, the force overrides with the corrupt value, until released
- Force is released in `CORRUPT` state — Forced state -> Normal simulation semantics

Note:

Forced state is a state that the simulator tracks for each signal to determine whether a force is active or not.

See [Table 8-4](#) for more information.

Flush Forces

- Force happens in `NORMAL` state — When the signal goes to the `CORRUPT` state, VCS NLP releases force and continues with the corruption semantics
- Force happens in `CORRUPT` state — Force is ignored
- Force is released in `CORRUPT` state — Not applicable

See [Table 8-4](#) for more information.

Defer Forces

- Force happens in `NORMAL` state — When the signal goes to the `CORRUPT` state, VCS NLP corrupts it, but keeps the forced state. On wake-up, the signal is forced with the earlier forced value, until released
- Force happens in `CORRUPT` state — VCS NLP continues with the corrupt value, but keeps the forced state. On wake-up, the force overrides the corrupt value, until released
- Force is released in `CORRUPT` state — Forced state -> Normal simulation semantics

See [Table 8-4](#) for more information.

Table 8-4 Force Semantics for Corrupt

Option	Current Value = A Forced Value = B	NORMAL State (Domain= ON)	CORRUPT State (Domain= OFF)	WAKEUP (Domain=ON)
Force Priority (Continue with force)	Force at NORMAL	A -> B	B	B (until released)
	Force at CORRUPT	A	X-> B	B (until released)
	Force released at CORRUPT	A -> B	B -> X	X
Force and Corrupt (Corrupt, but keep the forced state)	Force at NORMAL	A -> B	X	X (until released)
	Force at CORRUPT	A	X	X (until released)
	Force released at CORRUPT	A -> B	X	X
Flush Forces (Release force and corrupt)	Force at NORMAL	A -> B	X	X
	Force at CORRUPT	A	X	X
	Force released at CORRUPT	Not Applicable	Not Applicable	Not Applicable
Defer Forces (Corrupt on power down and restore forced value on power up)	Force at NORMAL	A->B	X	B (until released)
	Force at CORRUPT	A	X	B (until released)
	Force released at CORRUPT	A->B	X	X

Use Model

To enable the configurations mentioned above, you must specify the following UPF command in the `upf` file:

```
set_design_attributes -attribute  
SNPS_force_semantics <CONFIGURATION NAME>
```

Where, CONFIGURATION NAME can be one of the following:

FORCE_PRIORITY	FORCE_AND_CORRUPT
FLUSH_FORCES	DEFER_FORCES

Limitations

- This feature is currently limited to the ‘runtime forces’ only. That is, only those forces which appear during runtime by UCLI/VPI or DVE
- Language forces are not supported
- Forces on SV variables of type real are not supported
- Forces on SV variables of type longint are not supported
- Forces on Memories and MDAs are not supported
- While forces on Verilog part of the design in an MX design are supported, the forces on VHDL signals (either in MX design or pure VHDL design) are not supported
- Forces on ports which have exception supply connections using `set_port_attributes` or `set_related_supply_net` are not supported

- This feature is supported for simstate CORRUPT only. It is not supported for any other simstates with corruption semantics

CORRUPT_ON_ACTIVITY Simstate (Low-Vdd Standby)

In simstate, CORRUPT_ON_ACTIVITY, the active state of nets and state elements driven by an element remain unchanged at the transition. The processes modeling the behavior of the element remain enabled for activation (evaluation). Any net or state element that is actively driven after transitioning to this state is corrupted.

-simstate of add_power_state command supports CORRUPT_ON_ACTIVITY.

Examples

- Using Supply Expression

```
add_power_state Island_V1.primary \  
-state V1_COA {-supply_expr {power == `{FULL_ON,0.7}} -  
simstate CORRUPT_ON_ACTIVITY}
```

- Using Logic Expression

```
add_power_state Island_V1.primary \  
-state V1_COA {-logic_expr {psw_ctrl == 1} -simstate  
CORRUPT_ON_ACTIVITY}
```

Simulation Behavior

[Table 8-5](#) describes the simulation behavior of CORRUPT_ON_ACTIVITY.

Table 8-5 *Simulation Behavior of CORRUPT_ON_ACTIVITY*

Construct		Behavior
Sequential Logic (always/process)	Active clock edge	Data corrupted
	Clock Stable, Change on Data	No corruption
	Async reset active edge	Data corrupted
	Clock Stable, Sync reset active	No corruption
Combinatorial Logic	Change on sensitivity list elements of always/process block	LHS corrupted
	Cont-assign, Signal Assignments	LHS corrupted only if change on RHS results in a change on LHS
	Primitive Gates	Output corrupted only if change on inputs result in a change on output
Assignments/Gates with Delays	Change on RHS/inputs	Delay ignored for corruption
UDPs	Change on Inputs	Corrupted

Limitations

- Corruption is not applied for generating Verilog instances under VHDL
- In case of gates with delays, output is not corrupted if the change on output in `CORRUPT_ON_ACTIVITY` is due to a future event scheduled from `NORMAL` simstate
- Corruption of bit/part-selects

Excluding Parts of Design from Being Power Managed

The `UPF_dont_touch` attribute is used to exclude certain sections of the design from being power managed. This can be applied only at a module level or to specific elements of a module. This attribute supports Always ON behavior. It denotes that the signals are “always on” during power down and would be simulated as an ON logic.

Use Model

You can use the `UPF_dont_touch` attribute at compile-time, as shown below, on a module or module signals transitively. For modules, all its instances and the hierarchy under the module instance is marked `UPF_dont_touch` transitively.

```
set_design_attributes -attribute UPF_dont_touch TRUE
                    -models <module list>
                    -elements <signals>
```

Note:

For backwards compatibility, use the `-power=UPF_dont_touch_BC` compile-time option.

Key Points to Note

- All the instances of the don't touch model does not get corrupted during shut down
- When a model is marked as don't touch, the model and its child instances, if any, are marked to be non-power managed

- If any child instance under a module marked `UPF_dont_touch` is partitioned into different power domain, the `UPF_dont_touch` attribute does not apply to that child instance
- Any specific pin of don't touch model cannot be marked as `UPF_dont_touch FALSE`
- Initial block retriggering does not replay the initial blocks in the don't touch model

Module Support

The following points describe the module support:

- The recommended usage of `UPF_dont_touch` is on leaf level modules
- All instances of the don't touch model do not get corrupted during the shut down
- Isolation and retention are inserted in the don't touch module
- `set_related_supply_net/set_port_attributes` on the ports of the `dont_touch` module is ignored for corruption
- When a model is marked as don't touch, the model and its child instances, if any, are also marked as don't touch
- Any specific pin of don't touch model cannot be marked as `UPF_dont_touch FALSE`
- Initial block retriggering does not replay the initial blocks in the don't touch model

Module Named Block Support

- You can mark a complete named block (for example, always/ process block) as don't touch using `-elements`
- All the signals modified in the always block are disabled for shutdown corruption
- Signals are active even during shutdown so that the value propagation occurs

Module Signal Support

- Signals from the always block can be selectively marked as don't touch
- Corruption is not performed on the signal due to power activity
- `set_related_supply_net/set_port_attributes` on the port is ignored for corruption
- `UPF_dont_touch` marking does not impact isolation insertion, retention, and level shifter inference

Support for a Module Inside a Library

- You can specify don't touch model selectively from a particular library using the `-models` option of `set_design_attributes`
- The library name should be prefixed to model name as `lib.model`

Example: `set_design_attributes -attribute UPF_dont_touch TRUE -models {lib.buffer}`

Usage Examples

Model Level Support

```
set_design_attributes -models LEVEL_LEAF -  
attribute UPF_dont_touch TRUE
```

Model Signal Level Support

```
set_design_attributes -models LEVEL_LEAF -elements  
{sig1 sig2} -attribute UPF_dont_touch TRUE
```

Limitations

- `-elements <element_list>` without `-models` is not supported
- Instance level and hierarchical signal level `UPF_dont_touch` is not supported
- Specifying bit/part selects with `-elements` is not supported
- Transitive property for a module level attribute applies as long as the child instance is in the same power domain as the parent that is marked `UPF_dont_touch`. If the child is partitioned in a different power domain, the instance and the hierarchy under it is not marked `UPF_dont_touch` transitively

Skipping Corruption

You can use the `-power=apfcompat` compile-time option to:

- Skip corruption of reals

- Skip corruption of constants
- Skip corruption of signals assigned only inside initial blocks

You can use the design attribute `SNPS_apf_compat` at runtime to skip the corruption of VHDL variables, real and integer types.

To use `SNPS_apf_compat`, pass the below design attribute through the power configuration file at runtime:

config.tcl file:

```
set_design_attributes -attribute SNPS_apf_compat TRUE
```

```
% simv -power config.tcl
```

9

Isolation Support

This chapter describes the VCS NLP isolation support in the following sections:

- [Isolation Support for MX Designs](#)
- [Isolation Support for SystemVerilog Design Constructs](#)
- [Skipping Isolation on the Undriven/Floating Ports](#)
- [NOR-Style Isolation Support](#)
- [Support for Conservative diff_supply_only Isolation](#)
- [Name-Based Isolation Cell Association in Netlist](#)
- [Support for Isolation Terminal Points at the Hierarchical Boundaries](#)
- [Path-Based Isolation Support for Heterogeneous Loads](#)

- Support for `-exclude_elements` and `-update` Options
- Support for `-applies_to` with `-elements` for Isolation
- Support for Specifying Instances with `set_isolation` `-elements`
- Generating Virtual Isolation Reports
- Isolation/Corruption on Ports with Logical Expressions in Port Map
- Supporting Lower Domain Boundary for Isolation

Isolation Support for MX Designs

VCS NLP supports all VHDL data types for isolation.

Limitations

Following are the limitations of isolation support for MX designs:

- Isolation value for enum is always ``left` value
- Isolation is not applied for a crossover between VHDL record and SystemVerilog struct type
- Isolation is not applied for a crossover between VHDL integer type and Verilog reg type
- Isolation is not applied if there is a logic expression in port map
- The following specifications in `-elements` are not supported:
 - Part selects
 - Member of a record

- Element of MDA

Isolation Support for SystemVerilog Design Constructs

VCS NLP supports virtual isolation of the following SystemVerilog Design (SVD) constructs:

- logic, int, struct, enum, reg, shortint, and union
- multidimensional arrays

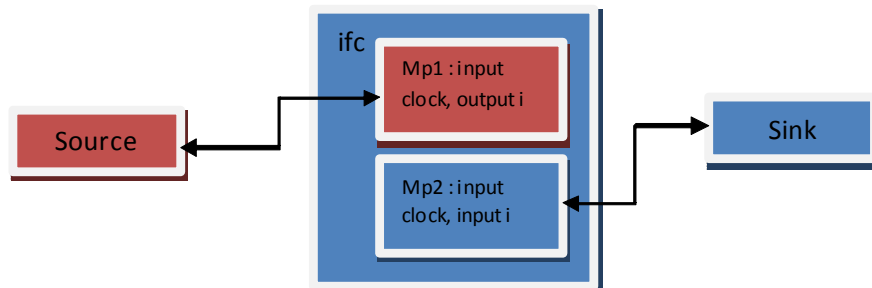
Limitations

Following are the limitations of this feature:

- Real data type is not supported
- Supply0, supply1, and tri are not supported

Isolating Interface Modports

You can isolate input/output of interface modports in SVD designs. This feature is enabled by default.



Limitations

- Isolation on interfaces without modports do not work as there is no direction associated with the signals
- Only `-self` and `-parent` locations are supported
- Clamp value `0/1/z` are supported
- Simple directional ports in the interfaces are not supported

Skipping Isolation on the Undriven/Floating Ports

VCS NLP isolates the undriven/floating boundary ports if the isolation strategy is specified on them.

You can use the `-power=dont_isolate_floating_nets` compile-time option to skip isolation on the undriven/floating boundary ports. VCS NLP captures the undriven/floating boundary port information in the `isolation_association.rpt` file generated in the [mvsim_native_reports](#) directory.

Following is the sample report of the `isolation_association.rpt` file:

```
# Number of potential locations for Instrumentation: 1
set ISO_SIGNET_PORT(1) { { dut/core/in -policy PD_AAA/
isolation/iso_in -domain PD_A }
{ -isolation_signal ISO_ENA -isolation_sense low -
clamp_value 0 -location parent }
{ -instrumented_node dut/in_core -driver_supply SS_AAA -
receiver_supply SS_BBB -src_undriven} }
```

```
# Number of potential locations for Instrumentation: 1
set ISO_SIGNET_PORT(2) { { dut/core/out -policy PD_AAA/
```

```
isolation/iso_out -domain PD_AAA }
{ -isolation_signal ISO_ENB -isolation_sense low -
clamp_value 0 -location parent }
{ -instrumented_node dut/out_core -driver_supply SS_AAA -
receiver_supply SS_BBB -receiver_floating } }
```

NOR-Style Isolation Support

NOR-style isolation is applicable to the isolation strategies with clamp value 0. In general, these are single rail isolation cells with switched supply. The output of the isolation cell is 0 when the power is OFF and isolation enable is active.

You can use the `iso_nor` design attribute in the UPF to indicate that a certain isolation strategy is defining a NOR-style isolation. Following is the use model:

```
set_design_attributes -elements
<scope_of_the_domain> -attribute iso_nor
{<domain_name.strategy_name>}
```

Example:

```
set_isolation V1_iso -domain PD1 -isolation_power_net VDD
-isolation_ground_net VSS -clamp_value 0 -source
PD1.primary -sink TOP.primary

set_isolation_control V1_iso -domain PD1
-isolation_signal iso_sig -location self
-isolation_sense high

set_design_attributes -elements { . } -attribute iso_nor
{ PD1.V1_iso }
```

Key points to note

- You can specify only one scope in `-elements`
- You can specify multiple isolation strategies as space separated list with the `iso_nor` attribute. Following is the example:

```
iso_nor {PD_SW.sw_iso_c0 PD_SW1.sw_iso_c1}
```

- The `iso_nor` attribute with `TRUE` value, as shown in the following command, applies NOR-style isolation to the whole design. Here, `-elements` is not mandatory

```
set_design_attributes -attribute iso_nor TRUE
```

- The semantics of `iso_nor` are ignored when a user-defined isolation model is mapped to the isolation strategy using `map_isolation_cell -lib_model_name`
- VCS NLP issues a compile time error message, if isolation strategies with clamp value other than 0 are specified with the `iso_nor` attribute
- The output is corrupted only if the ground supply of the NOR-style isolation strategy is switched off

VCS NLP reports all the marked NOR-style isolation strategies in the `mvsim_native_reports/isolation_association.rpt` file.

Following is a sample `isolation_association.rpt` file:

```
set ISO_SIGNET_PORT(3) { { CORE1/out1 -policy PD1/  
isolation/V1_iso -domain PD1}  
{-isolation_signal iso_sig -isolation_sense high  
-clamp_value nor -location self}  
{-instrumented_node CORE1/out1 -driver_supply  
PD1.primary -receiver_supply TOP.primary} }
```

Corruption Semantics for NOR-Style Isolation

[Table 9-1](#) lists the corruption semantics for NOR-style isolation.

Table 9-1 Corruption Semantics for NOR-Style Isolation

Input pin of isolation	Enable pin of isolation	VDD	VSS	Output pin of isolation
Don't care	Don't care	Don't care	OFF	X
Don't care	Active	OFF	ON	0
Don't care	Inactive	OFF	ON	Same as input
Don't care	Active	ON	ON	0
Don't care	Inactive	ON	ON	Same as input

Limitations

Following is the limitation of this feature:

- Wildcards are not supported in the arguments of the `iso_nor` attribute.

```
set_design_attributes -elements {.} -attribute  
iso_nor {PD_SW.A*}
```

Support for Conservative `diff_supply_only` Isolation

When an isolation strategy uses the `-diff_supply_only` option of the `set_isolation` command, as shown below, then VCS NLP inserts an isolation cell for the port only if none of its global loads/sinks use the same supply as its driver/source.

```
set_isolation iso -domain P1 -applies_to outputs
-diff_supply_only TRUE -location self
```

You can use the `conservative_diff_supply_only_isolation` design attribute in the UPF to insert an isolation cell for the `-diff_supply_only` isolation strategies in the cases where one or more of the loads/sinks might have the same supply as the driver/source, but at least one of them uses a different supply. Specify the following in the UPF:

```
set_design_attributes -attribute
conservative_diff_supply_only_isolation TRUE
```

Under this attribute, for the `-diff_supply_only` strategies on a port fanning out to multiple power domains, VCS NLP inserts an isolation cell if **at least one** logic sink power domain is different from the logic source domain.

Consider the following example where `-diff_supply_only` isolation strategy is applied to the output ports of power domain P1 with location “self”. The logic in power domains P1 and P2 are powered by their respective primary supplies. In Figure-1, VCS NLP does not insert isolation cell as one of the loads uses the same supply as the driver.

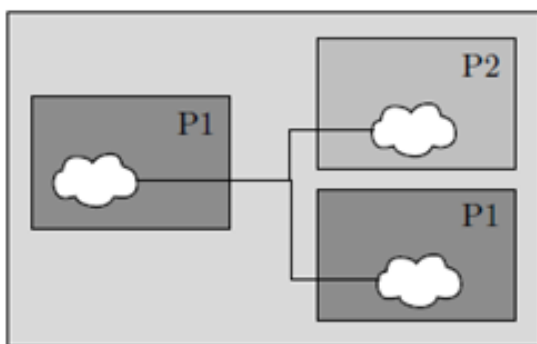


Figure-1

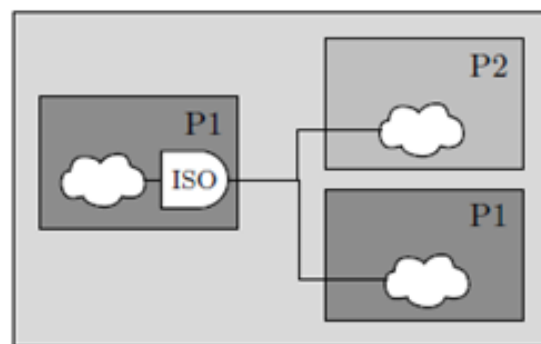


Figure-2

If you use the `conservative_diff_supply_only_isolation` design attribute, then VCS NLP inserts an isolation cell, as shown in Figure-2.

The `conservative_diff_supply_only_isolation` design attribute is a global attribute and applies to all isolation strategies with `-diff_supply_only`.

Note:

- The `conservative_diff_supply_only_isolation` attribute applies to the `-diff_supply_only` option of the `set_isolation` command only; it does not apply to the `-sink` option.
- The `conservative_diff_supply_only_isolation` attribute impacts isolation strategies with location `self` or `parent` only.

Name-Based Isolation Cell Association in Netlist

VCS NLP supports name-based association of isolation cells in the netlists written using DC/ICC with the `mv_upf_strategy_in_inst_name` variable.

Following is the naming convention of the isolation cell instance in the netlist:

```
ISO <name_prefix>_snps_<power_domain_name>__<iso_strategy_name>_snps_<pin_name>_<inst_index>_<name_suffix>
```

The key concept of this feature is that if VCS NLP finds an isolation instance whose name matches the above naming convention, it associates this isolation instance with the corresponding isolation strategy.

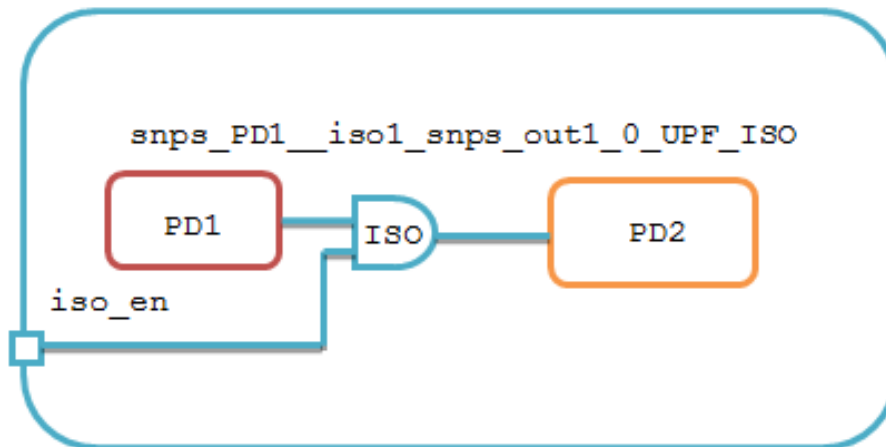
For example, in [Figure 9-1](#) the crossover starts from PD1 to PD2.

Following are the isolation strategies on the source (PD1/iso1) and sink (PD2/iso2) domains:

```
set_isolation iso1 -domain PD1
-source PD1.primary
-sink TOP.primary

set_isolation iso2 -domain PD2
-source PD1.primary
-sink PD2.primary
```

Figure 9-1 *Name-based Isolation Cell Association*



The source domain strategy PD1/iso1 has
-source PD1.primary and -sink TOP.primary, which does not fulfill the requirement for the path-based strategy association.

The sink domain strategy `PD2/iso2` has `-source PD1.primary` and `-sink PD2.primary`, which fulfills the requirement for the path-based strategy association.

However, `PD1/iso1` strategy name matches with the isolation cell present on the path. Therefore, VCS NLP associates isolation cell `snps_PD1__iso1_snps_out1_0_UPF_ISO` with the isolation strategy `PD1/iso1`.

Support for Isolation Terminal Points at the Hierarchical Boundaries

This section describes a method to mark hierarchical boundaries as terminal points for isolation strategies using `-source`, `-sink`, `-diff_supply_only`, or `-location automatic`. This section contains the following topics:

- [Introduction](#)
- [Use Model](#)
- [Examples](#)
- [Limitations](#)

Introduction

The `set_design_attributes` command on the hierarchical boundary is considered for isolation analysis. You can use the design attribute `terminal_boundary` to change the behavior of the hierarchical boundaries; any hierarchical boundary which contains

this attribute is considered as a terminal point for the global net. This effectively changes the behavior of `-source`, `-sink`, `-diff_supply_only`, or `-location` automatic strategies.

Use Model

You can use the `terminal_boundary` design attribute to mark hierarchical boundaries as terminal points for isolation strategies.

Syntax:

```
set_design_attributes -elements {instance_name} -attribute  
terminal_boundary true
```

Key Points to Note about `terminal_boundary`

- The default value of this attribute is `false`
- VCS generates an error if any element in the elements list is not a power domain boundary
- VCS generates an error if any element in the elements list is a power domain boundary which is a leaf cell (macros/pads)
- The `terminal_boundary` attribute is supported at instance level only. It is not supported at port level. When this attribute is set on an instance, the boundary ports of that instance becomes a terminal point for any isolation global net.

The purpose of using the `terminal_boundary` attribute is to give flexibility and simplify the use of `source`, `sink`, `diff_supply_only`, or the `location` automatic strategies in a bottom up flow; any design which has such attribute is expected to be integrated into a bigger design.

Note:

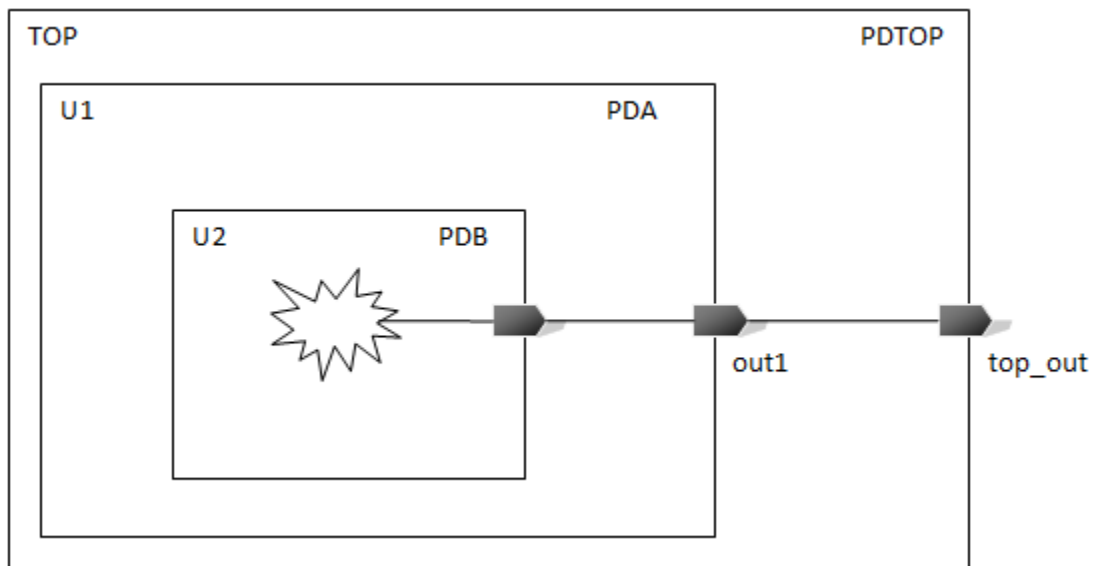
For `-location automatic` strategy specified on terminal boundary ports, VCS NLP always inserts virtual isolation cell at `-location self`.

Enhancement to the `-driver_supply` and `-receiver_supply` Port Attributes

VCS NLP supports both `-driver_supply` and `-receiver_supply` attributes for the terminal boundary ports.

Examples

Example-1: Isolation with `-source`



Base UPF

```
create_supply_set SSTOP
create_supply_set SSA
create_supply_set SSB
```

```

create_power_domain TOP -supply {primary SSTOP}
create_power_domain PDA -elements {U1} -supply {primary SSA}
-scope U1
create_power_domain PDB -elements {U1/U2} -supply {primary
SSB}

set_isolation iso1 -domain PDTOP -source SSB
set_isolation_control iso1 -domain PDTOP -location self
set_isolation iso2 -domain PDTOP -source SSA
set_isolation_control iso2 -domain PDTOP -location self

```

Additional UPF

```

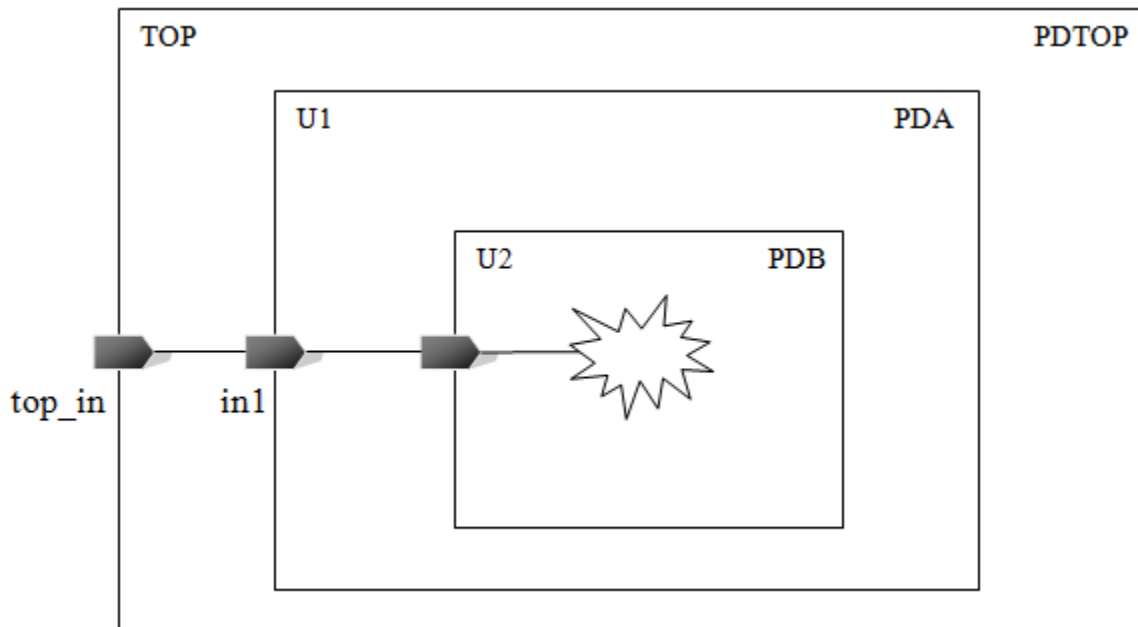
set_design_attributes -elements {U1} -attribute
terminal_boundary true

```

Description

If only the base UPF is used, then only the `iso1` strategy is implemented as `SSB` is the driver of the single path shown. If additional UPF is included, this means that `U1` is implemented standalone, and two paths are considered rather than just one. No strategies apply to the path from the source to the hierarchical port `out1`. However, on the path from `out1` to `top_out`, the effective source supply is `SSA`, so strategy `iso2` is implemented.

Example-2: Isolation with -sink



Base UPF

```
create_supply_set SSTOP
create_supply_set SSA
create_supply_set SSB
create_power_domain TOP -supply {primary SSTOP}
create_power_domain PDA -elements {U1} -supply {primary SSA}
-scope U1
create_power_domain PDB -elements {U1/U2} -supply {primary
SSB}
```

```
set_isolation iso1 -domain PDTOP -sink SSB
set_isolation_control iso1 -domain PDTOP -location self
set_isolation iso2 -domain PDTOP -sink SSA
set_isolation_control iso2 -domain PDTOP -location self
```

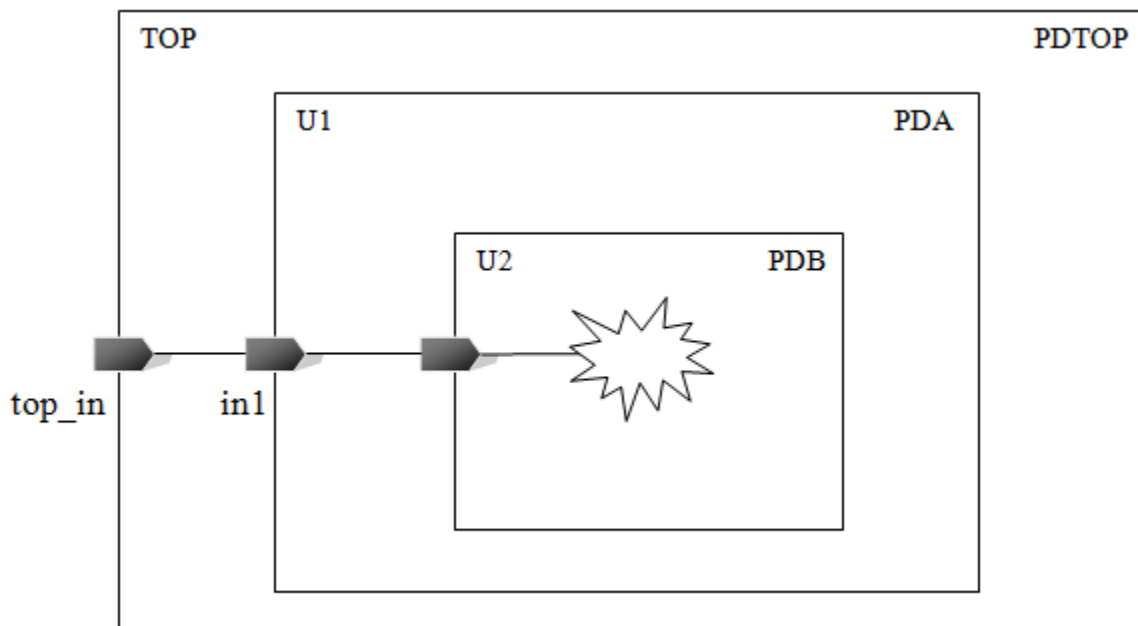
Additional UPF

```
set_design_attributes -elements {U1} -attribute
terminal_boundary true
```

Description

If only the base UPF is used, then only the `iso1` strategy is implemented as `SSB` is the sink of the single path shown. If additional UPF is included, this means that `U1` is implemented stand-alone, and two paths are considered rather than just one. No strategies apply to the path from the source to the hierarchical port `in1` to the sink. However, on the path from `top_in` to `in1`, the effective source supply is `SSA`, so strategy `iso2` is implemented.

Example-3: Isolation with -driver_supply



Base UPF

```
create_supply_set SSTOP
create_supply_set SSA
create_supply_set SSB
create_power_domain TOP -supply {primary SSTOP}
create_power_domain PDA -elements {U1} -supply {primary SSA}
-scope U1
create_power_domain PDB -elements {U1/U2} -supply {primary
```

```
SSB}
```

```
set_isolation iso1 -domain PDTOP -sink SSB  
set_isolation_control iso1 -domain PDTOP -location self  
set_isolation iso2 -domain PDTOP -sink SSTOP  
set_isolation_control iso2 -domain PDTOP -location self  
set_port_attributes -elements{U1} -ports{U1/in1}  
-driver_supply SSTOP
```

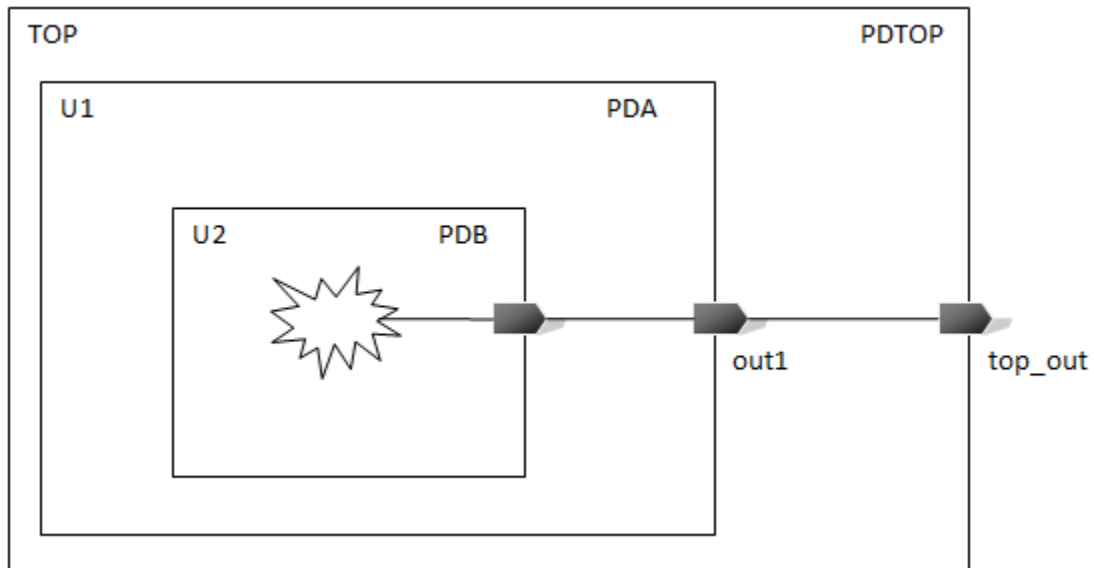
Additional UPF

```
set_design_attributes -elements {U1} -attribute  
terminal_boundary true
```

Description

If only the base UPF is used, then only the `iso1` strategy is implemented as `SSB` is the sink of the single path shown. If additional UPF is included, this means that `U1` is implemented stand-alone, and two paths are considered rather than just one. No strategies apply to the path from the hierarchical port `in1` to the sink. However, on the path from `top_in` to `in1`, the effective sink supply is `SSTOP` due to `-driver_supply`, so strategy `iso2` is implemented.

Example-4: Isolation with -receiver_supply



Base UPF

```
create_supply_set SSTOP
create_supply_set SSA
create_supply_set SSB
create_power_domain TOP -supply {primary SSTOP}
create_power_domain PDA -elements {U1} -supply {primary SSA}
-scope U1
create_power_domain PDB -elements {U1/U2} -supply {primary
SSB}
```

```
set_isolation iso1 -domain PDTOP -source SSB
set_isolation_control iso1 -domain PDTOP -location self
set_isolation iso2 -domain PDTOP -source SSTOP
set_isolation_control iso2 -domain PDTOP -location self
set_port_attributes -elements{U1} -ports {U1/out1}
-receiver_supply
```

Additional UPF

```
set_design_attributes -elements {U1} -attribute
terminal_boundary true
```


Description

If only the base UPF is used, then only the `iso1` strategy is implemented as `SSB` is the driver of the single path shown. If additional UPF is included, then this means that `U1` is implemented stand-alone, and two paths are considered rather than just one. No strategies apply to the path from the hierarchical port `out1`. However, on the path from `out1` to `top_out`, the effective source supply is `SSTOP` due to `-receiver_supply`, so strategy `iso2` is implemented.

Limitations

- Terminal boundary cannot be set on the leaf level instance for DB
- Terminal Boundary cannot be set on non-power domain boundary hierarchy
- Terminal boundary attribute cannot be set on scope where the domain is scoped at top level (specifying `-scope` option for `create_power_domain`)
- Isolation with location parent and fanout is not allowed on terminal boundary ports

Path-Based Isolation Support for Heterogeneous Loads

The location specified by the isolation strategy indicates the domain of the logic hierarchy where the cell must be inserted. For example, the `self` location implies that the isolation could be inserted in the extent of the reference domain. The `parent` location implies that the isolation could be inserted in the extent of the domain of the parent hierarchy/instance.

Isolation cell insertion is not constrained to the immediate self/parent hierarchies. You can use the `hetero_fanout_isolation` design attribute, as shown below, to insert isolation cell in the extent of the domain of the self or parent hierarchy/instance.

```
set_design_attributes -elements {.} -attribute  
hetero_fanout_isolation true
```

The following behavior is supported under the `hetero_fanout_isolation` design attribute:

- Isolation cell insertion at location `self/parent` with heterogeneous loads for the `-diff_supply_only` isolation strategies. That is, for a port with multiple sinks, with at least one sink having different supply than the source, and at least one sink having similar supply as the source, you can insert isolation cell in extent of the domain of the self or parent hierarchy/instance, but not crossing the domain boundary
- Isolation cell insertion at location `self/parent` with heterogeneous loads for the `-source/-sink` or `-sink` isolation strategies. That is, for a port with multiple sinks that are driven by different power supplies, you can insert an isolation cell in the extent of the domain of the self or parent hierarchy/instance, but not crossing the domain boundary

Key Points to Note

- With this support, specifying multiple path-based (`-source/-sink/-diff_supply_only`) isolation strategies is supported on a single target port
- Isolation strategies specified with the `-source` or `-sink` options support all paths, including those whose loads have heterogeneous supplies

- Port-based isolation rule (other than `-no_isolation`) is evaluated as per the isolation strategy on the individual driver-load pairs. Therefore, presence of both port-based and path-based isolation specified with the same granularity results in the combination of path-based isolation on specific driver-loads pairs of the path, and port-based isolation on the remainders of the path. If there are no remainders, then the port-based isolation does not apply. For more information, see [“Precedence Rule Scenarios”](#)

Heterogeneous Load Scenarios

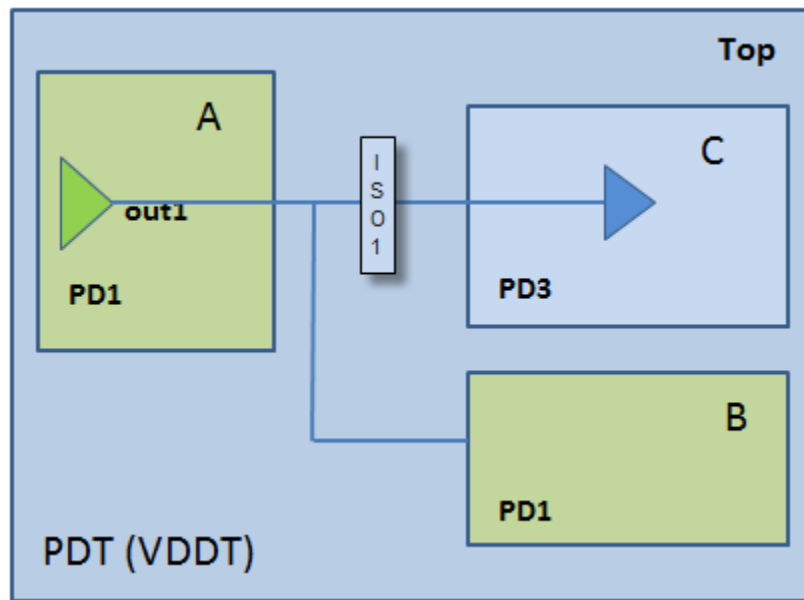
- `-location parent` with `-diff_supply_only`

Consider the following example where isolation strategy `ISO1` with `-diff_supply_only` and `location parent` is applied to the output ports of power domain `PD1`.

```
set_isolation ISO1 -location parent -  
diff_supply_only -domain PD1
```

In [Figure 9-2](#), one of the loads uses the same supply as the driver. Path from **A** to **C** gets isolated without isolating path from **A** to **B**.

Figure 9-2 -location parent with -diff_supply_only



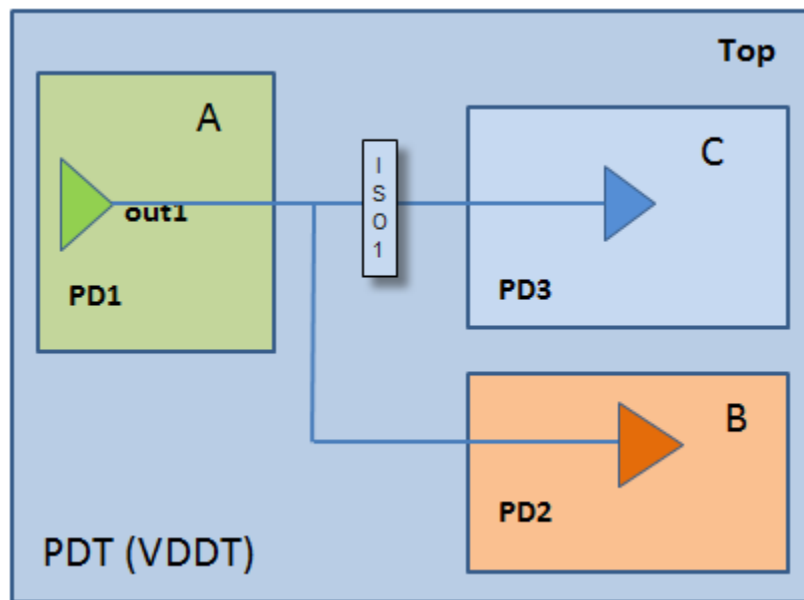
- -location parent with -source/-sink or -sink

Consider the following example where isolation strategy ISO1 with -source/-sink and location parent is applied to the output ports of power domain PD1.

```
set_isolation ISO1 -location parent -source PD1  
-sink PD3 -domain PD1
```

In [Figure 9-3](#), isolation is placed in the extent of PD3 such that the path from **A** to **C** gets isolated. The path from **A** to **B** is not isolated.

Figure 9-3 -location parent with -source/-sink

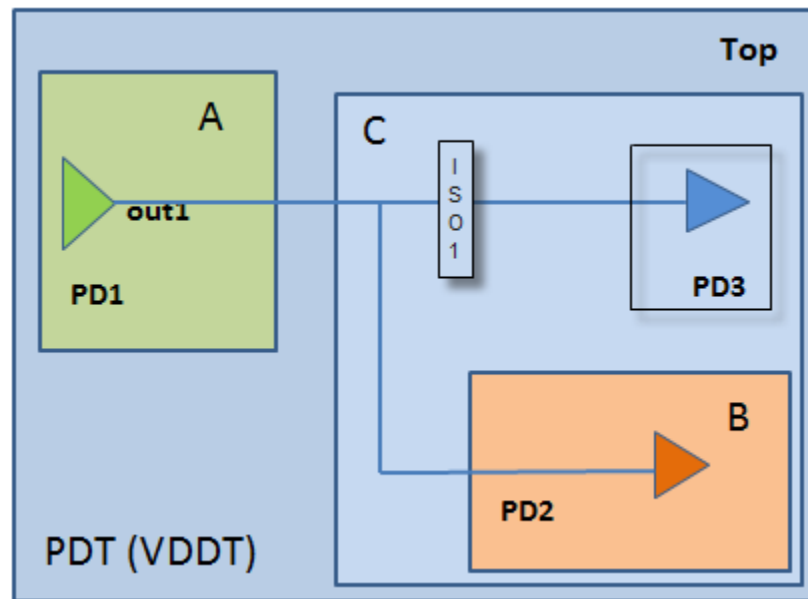


- -location self with -sink

Consider the following example where the isolation strategy ISO1 with -sink and location self is applied to the input port of power domain PD3.

```
ISO1 -location self -sink PD3 -domain PD3
```

Figure 9-4 -location self with -sink

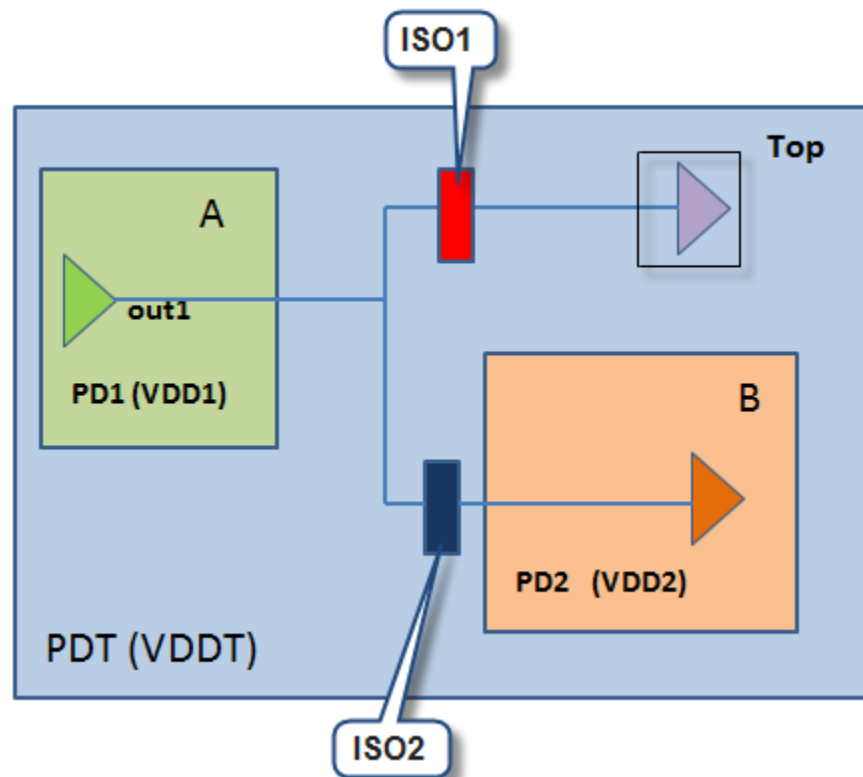


In [Figure 9-4](#), isolation cell is placed in the extent of the power domain PD3 such that the path from PD1 to PD3 gets isolated leaving the path from PD1 to PD2 unisolated.

Precedence Rule Scenarios

Case-1

Figure 9-5 Scenario-1: All isolation strategies are on domain PD1



Scenario-1: All isolation strategies are on domain **PD1**

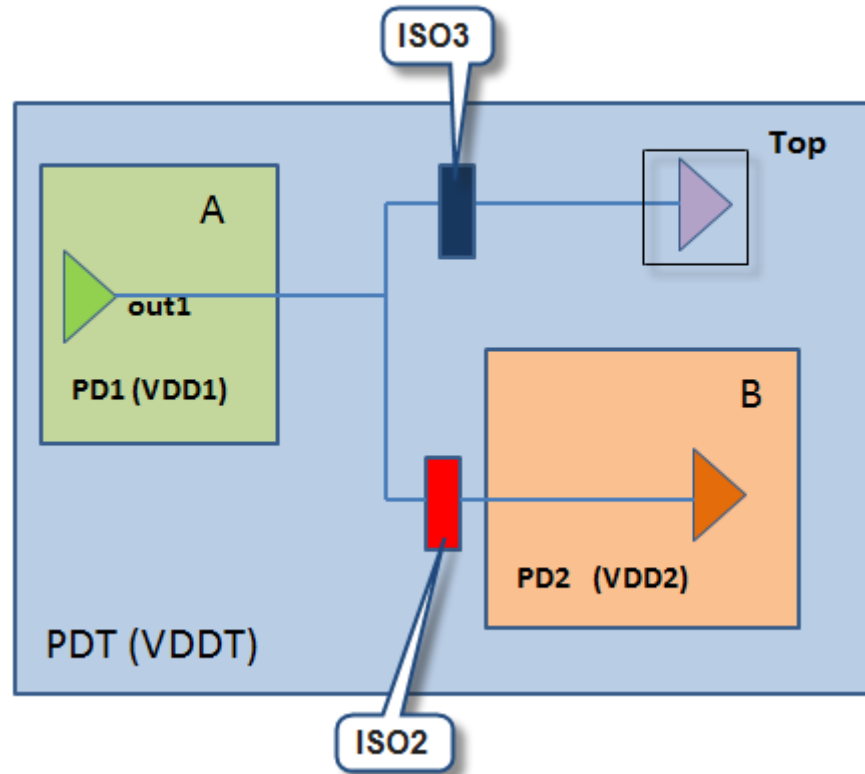
ISO1: -elements A -location parent

ISO2: -elements A -sink VDD2 -location parent

Precedence order: ISO2 == ISO1

Path-based ISO2 takes more specific path (VDD1 -> VDD2), while ISO1 takes the remainder paths (VDD1 -> VDDT).

Figure 9-6 Scenario-2: All isolation strategies are on domain PD1



Scenario-2: All isolation strategies are on domain **PD1**

ISO1: -elements A -location parent

ISO2: -elements A -sink VDD2 -location parent

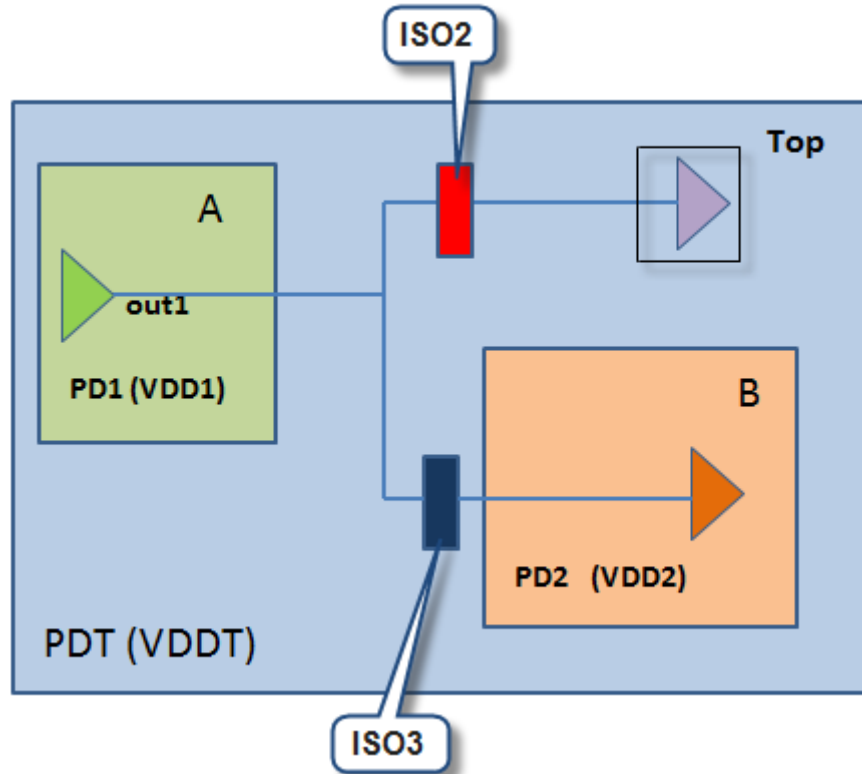
ISO3: -elements -sink VDDT -location parent

Precedence order: ISO2, ISO3, ISO1 - ISO1 dropped

ISO1 is dropped because ISO2 and ISO3 already covered all possible paths of ISO1.

Case-2

Figure 9-7 Scenario-1: All isolation strategies are on domain PD1



Scenario-1: All isolation strategies are on domain **PD1**

ISO1: -elements A -src VDD1 -location self

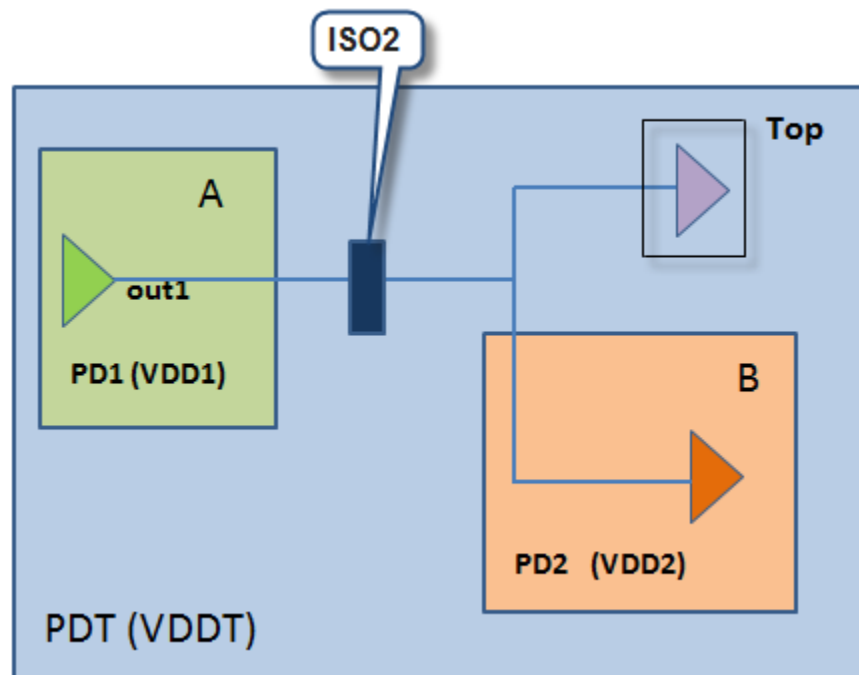
ISO2: -elements A -location parent

ISO3: -elements A -sink VDD2 -location parent

Precedence order: ISO3, ISO1, ISO2 - ISO1 dropped

ISO1 is dropped because it cannot be implemented in the presence of ISO3.

Figure 9-8 Scenario-2: All isolation strategies are on domain PD1



Scenario-2: All isolation strategies are on domain **PD1**

ISO1: -src VDD1 -location self

ISO2: -elements A -location parent

ISO3: -sink VDD2 -location parent

Precedence order: ISO2, ISO3, ISO1 - ISO1 and ISO3 dropped

ISO1 and ISO3 are dropped because ISO2 already covered all paths.

Support for `-exclude_elements` and `-update` Options

VCS NLP supports the following options with the `set_isolation` command:

- `-exclude_elements`
- `-update`

The `-update` option provides additional information for a previous command with the same strategy name and domain name, and executed in the same scope.

The `-exclude_elements` option is used to exclude any port or element from the strategy.

Use Model

The following topics describe the use model:

- [Using `-exclude\_elements` with `set\_isolation` Command](#)
- [Using `-update` with `set\_isolation` Command](#)

Using `-exclude_elements` with `set_isolation` Command

The `-exclude_elements` option explicitly identifies the set of ports to which this strategy does not apply.

Syntax:

```
set_isolation strategy_name -exclude_elements  
{exclude_list}
```

The `exclude_list` might contain rooted names of instances or ports in the specified domain. If an instance name is specified in the `exclude_list`, it is equivalent to specifying all the ports of that instance in the `exclude_list`.

Using `-update` with `set_isolation` Command

The `-update` option adds information to the base command executed in the same scope.

Syntax:

```
set_isolation strategy_name
    -domain domain_name
    [-elements element_list]
    [-exclude_elements exclude_list]
    [-update]
```

Example-1: Adding elements to the isolation strategy

```
set_isolation test_strategy -domain pda
-elements {a b c d}
-isolation_signal {iso_en}
-isolation_sense {LOW}
set_isolation test_strategy -domain pda -update
-elements {e f g}
```

The `-update` option adds elements `e`, `f`, and `g` to `test_strategy`.

Example-2: Excluding elements from the isolation strategy

```
set_isolation iso_pd_top -domain pd_top -applies_to output
...
set_isolation iso_pd_top -domain pd_top -update -
exclude_elements {out1}
```

Isolation strategy `iso_pd_top` is applied on all the output ports except `out1` because `out1` is excluded from the `iso_pd_top` strategy.

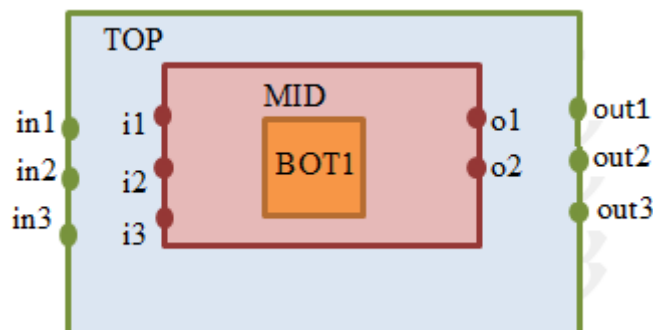
Support for `-applies_to` with `-elements` for Isolation

VCS NLP supports the `-applies_to` option as a filter in presence of the `-elements` option of the `set_isolation` command. It allows you to filter the elements specified in the `-elements` list of the `set_isolation` command as per the direction specified by the `-applies_to` option.

Key Points to Note

- The `-applies_to` option is always interpreted relative to the domain for which the strategy is being defined (also known as reference domain)
- HighConn elements are considered for filtering only when the lower domain boundary feature is enabled. When lower domain boundary feature is disabled, HighConn elements result in an error for not being on the boundary of the domain

Consider the following design structure:



The isolation strategy ISO is applied to out1, out2, out3, MID/i1, MID/i2, and MID/i3.

```
set_scope /
set_design_attributes -elements { . } -attribute
lower_domain_boundary true
create_power_domain PD_TOP
create_power_domain PD_MID -elements { MID }
create_power_domain PD_BOT -elements { MID/BOT1 }
set_isolation ISO -domain PD_TOP -elements { * } -applies_to
outputs ...
```

For the following example, lower domain boundary feature is enabled. Therefore, the isolation strategy ISO is applied to MID/o1 and MID/o2 since they are the input boundaries of the domain PD_TOP.

```
set_scope /
set_design_attributes -elements { . } -attribute
lower_domain_boundary true
create_power_domain PD_TOP
create_power_domain PD_MID -elements { MID }
set_isolation ISO -domain PD_TOP -elements { MID } -
applies_to inputs ...
```

VCS NLP supports this feature by default. For backwards compatibility, use the DISABLE_APPLIESTO_WITH_ELEMENTS variable in the synopsys_bc.setup file, as shown below:

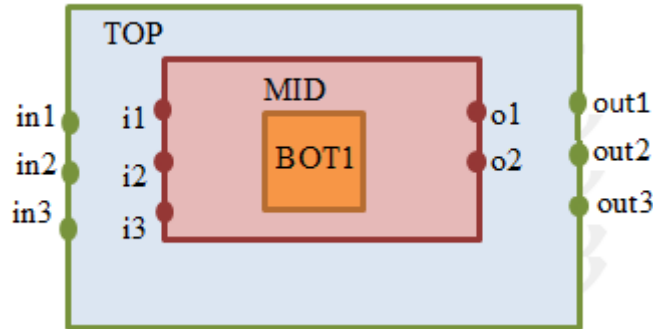
```
enable DISABLE_APPLIESTO_WITH_ELEMENTS
```

When the above variable is enabled, the -applies_to option is ignored in presence of -elements of the set_isolation command.

Support for Specifying Instances with `set_isolation` -elements

VCS NLP allows you to specify the design instances in the `-elements` list of the `set_isolation` command. The isolation strategy is applied only to the boundary elements of the specified instance in the reference domain. With this support, you can easily specify the target elements for the strategy.

Consider the following design structure:



The instance `MID` contains input ports `i1`, `i2`, `i3` and output ports `o1` and `o2`. The isolation strategy `ISO` is applied to all the input and output ports of `MID`.

```
set_scope /  
set_design_attributes -elements { . } -attribute  
lower_domain_boundary true  
create_power_domain PD_TOP  
create_power_domain PD_MID -elements { MID }  
set_isolation ISO -domain PD_TOP -elements { MID } ...
```

VCS NLP allows you to specify wildcards for the instances in `-elements`. If wildcards are specified in `-elements`, it will match any ports or instances at the current level of hierarchy.

VCS NLP supports instances in the `-elements` list by default. For backwards compatibility, use the `DISABLE_INSTANCE_WITH_ELEM` variable in the `synopsys_bc.setup` file, as shown below:

```
enable DISABLE_INSTANCE_WITH_ELEM
```

This variable disables the instances support in `-elements`.

Generating Virtual Isolation Reports

You can use the `-power=write_mvinfo` compile-time option to generate the following reports:

- [Virtual Isolation Insertion Report](#)
- [Virtual Isolation Inference Report](#)

Following is the use model of the `-power=write_mvinfo` option:

```
% vcs -upf <upf_file> -sverilog file_name.v  
-power_top top -power=write_mvinfo
```

Virtual Isolation Insertion Report

VCS NLP generates an isolation insertion report at compile time which provides the details such as signal being isolated and its corresponding isolation control, isolation sense and clamp value.

Reporting Format

```
{isolation_control  isolation_sense  clamp_value  
isolated_signal}
```

1. The report is dumped in the [mvsim_native_reports/](#)
`isolation_insertion_info.txt` file as a Tcl list.
2. The `isolation_control` and `isolation_sense` are the
isolation enable and the sense specified in UPF for the isolation
strategy.
3. The `clamp_value` is “high” if 1 and “low” if 0 in the UPF. “Z” is
printed as it is.
4. Hierarchy of the `isolation_control` and `isolated_signal`
are relative to power top.
5. “/” is used as the hierarchical separator.
6. The file is sorted with `isolated_signal` name field.
7. If the isolation is already implemented in the design, it is not written
into the insertion report.

Sample Report

```
set mvsim_iso_list {  
  {p1/iso high low f2/f2d4/in1\[1\]}  
  {p1/iso high high t22/d2/in1\[1\]}  
  {p1/iso high high t42/d4/in1\[3\]}  
}
```

Virtual Isolation Inference Report

You can use the `-power=write_mvinfo` option to generate a report for the virtual isolation inference from the RTL based on the UPF. The generated report can be used to compare with the isolation inference report dumped by Formality on the same RTL and UPF.

The isolation inference report from VCS NLP is generated as `new_isolation_association.rpt` file in the `mvsim_native_reports` directory.

Following is the sample report from VCS NLP:

```
set mvsim_iso_list {
  { /inst/X/Y/out[0] /Island_V1 V1_iso /inst/X/Y/out[0] L 1 }
  { /inst/X/Y/out[1] /Island_V1 V1_iso /inst/X/Y/out[1] L 1 }
  { /inst/X/Y/out[2] /Island_V1 V1_iso /inst/X/Y/out[2] L 1 }
  { /inst/X/Y/out[3] /Island_V1 V1_iso /inst/X/Y/out[3] L 1 }
  { /inst/X/Y/out[4] /Island_V1 V1_iso /inst/X/Y/out[4] L 1 }
  { /inst/X/Y/out[5] /Island_V1 V1_iso /inst/X/Y/out[5] L 1 }
  { /inst/X/Y/out[6] /Island_V1 V1_iso /inst/X/Y/out[6] L 1 }
  { /inst/X/Y/out[7] /Island_V1 V1_iso /inst/X/Y/out[7] L 1 }
```

Isolation Inference Report Format

Each inferred isolation cell consists of the following fields:

```
<boundary_port_path> <power_domain_name>
<isolation_strategy_applied> <location_port_path>
<L|H> <order>
```

Where,

```
<boundary_port_path>
```

Path to the boundary ports for the specified domain in `set_isolation`.

`<power_domain_name>`

Path to the specified domain name in `set_isolation`.

`<isolation_strategy_applied>`

The specified strategy name from `set_isolation`.

`<location_port_path>`

The path to the port being isolated. This can be the same as the boundary port.

`<L|H>`

The isolation cell is located on the low or high side of the location port.

`<order>`

Distinguishes multiple isolation cells back to back. It will be a number starting at 1 and incrementing to the total number of back to back isolation cells at that location.

Number 1 represents the closest isolation cell to the boundary port path, and the number increments for each back to back isolation cell. The largest number is the isolation cell farthest away from the boundary port path.

Isolation/Corruption on Ports with Logical Expressions in Port Map

You can use the `-power=portconnect_exprs` compile-time option to enable isolation/corruption on the logical expressions in the port map. By default, isolation/corruption is not enabled on the logical expressions in the port map.

For example, consider the following code:

```
child child_inst (.data_in(~data));
```

You can use the `-power=portconnect_exprs` option to enable isolation/corruption on `child_inst/data_in`.

Note:

Logical expressions that are part of a concatenation expression are handled for isolation/corruption by default.

Limitations

Following are the limitations of this feature:

- Expressions in the port map of a user-defined* type `port` are not supported for isolation/corruption.

*Refers to any type other than basic Verilog types (reg, wire, logic, and so on). For example: struct, union, enum, and so on.
- Expressions containing streaming and assignment pattern operators are not supported.

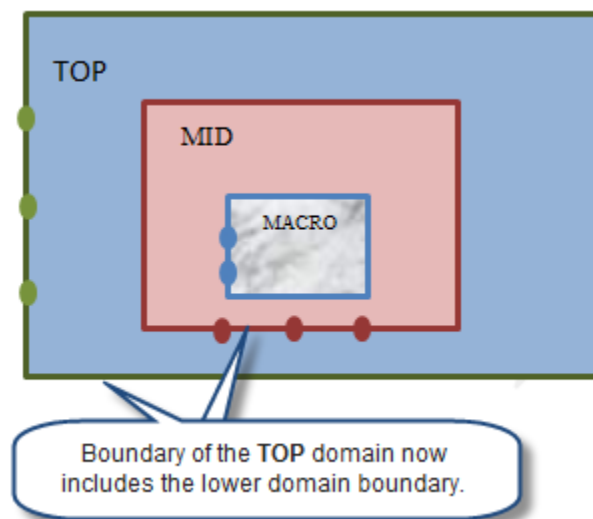
Supporting Lower Domain Boundary for Isolation

You can use the `lower_domain_boundary` design attribute to extend the definition of a power domain boundary to include the lower domain boundary ports/pins, as shown in [Figure 9-9](#), where the boundary `TOP` includes both the interface colored green and interface colored red.

However, the boundary of `TOP` does not extend to the interface colored blue of the domain `MACRO`. The boundary of the domain `MID` extends to the boundary of the nested domain `MACRO`.

For hierarchical models, this feature allows you to insert isolation cells in the parent domain.

Figure 9-9 Boundary of TOP which Includes the Lower Domain Boundary



Use Model

Use the `lower_domain_boundary` design attribute, as shown below, to enable this feature:

```
set_design_attributes -elements {element_list} -  
attribute lower_domain_boundary TRUE/true/FALSE/false
```

For example, consider the following command. This feature is enabled for all domains scoped at and below the current scope where the command is executed.

```
set_design_attributes -elements {.} -attribute  
lower_domain_boundary TRUE
```

Key Points to Note

- The default value of `lower_domain_boundary` is `FALSE`. The definition of the lower boundary is enabled or disabled for the entire design irrespective of the scope specified in the `set_design_attributes` command
- The `lower_domain_boundary` attribute can be set on the top scope before defining any power domains for the design and the value matches with any explicit setting elsewhere in the design
- The `lower_domain_boundary` attribute can be set on the lower scope with `FALSE` value when the top scope is set to `TRUE`. For more information, see [“Block-Level Control of Lower Domain Boundary”](#)
- If there are no power domains and explicit settings in the design, the `lower_domain_boundary` attribute is allowed to change on the top scope

Examples

Example-1

Consider the following code:

```
set_scope /  
set_design_attributes -elements { mid_inst } -attribute  
lower_domain_boundary FALSE
```

In this example, attribute is set on `mid_inst` since the value `false` matches with the default value on the top scope.

Example-2

Consider the following code:

```
set_scope /  
set_design_attributes -elements { . } -attribute  
lower_domain_boundary FALSE  
set_design_attributes -elements { mid_inst } -attribute  
lower_domain_boundary FALSE
```

In this example, attribute will be set on `mid_inst` since the value `false` matches with the explicit setting on the top scope.

Example-3

Consider the following code:

```
set_scope /  
set_design_attributes -elements { . } -attribute  
lower_domain_boundary TRUE  
create_power_domain TOP  
set_scope mid_inst  
  
set_design_attributes -elements { . } -attribute  
lower_domain_boundary TRUE
```

```
create_power_domain MID
set_scope /
```

In this example, attribute is set on both the top scope and `mid_inst` since the attributes are set before creating the power domains at the respective scopes.

Example-4

Consider the following code:

```
set_scope /
set_design_attributes -elements { . } -attribute
lower_domain_boundary TRUE
set_design_attributes -elements { mid_inst } -attribute
lower_domain_boundary TRUE
create_power_domain MID -elements { mid_inst } -scope {
mid_inst }
set_design_attributes -elements { . } -attribute
lower_domain_boundary FALSE
```

In this example, you cannot change the attribute value at the top scope to `FALSE` as there is a power domain below the top scope. Even if there is no power domain `MID`, attribute value does not change as there is a setting on `mid_inst`.

Block-Level Control of Lower Domain Boundary

VCS NLP allows you to set the `lower_domain_boundary` attribute on the lower scope with `FALSE` value when the top scope is set to `TRUE` (OFF-within-ON block).

Use Model

Following is the use model of the OFF-within-ON block configuration:


```
set_design_attributes -elements {.} -attribute  
lower_domain_boundary TRUE
```

```
set_design_attributes -elements <scope> -attribute  
lower_domain_boundary FALSE
```

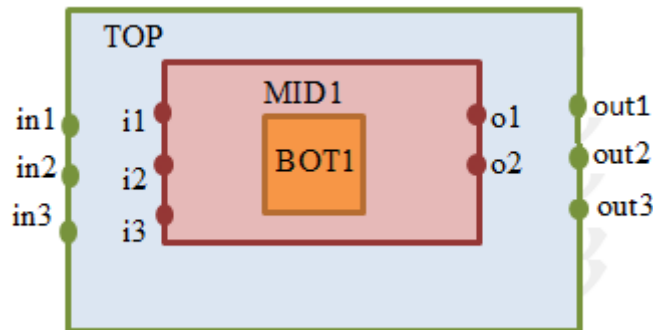
Key Points to Note

- You can set the `lower_domain_boundary` attribute on a design or scope with `FALSE` value only if none of its descendant blocks are set to `TRUE`
- You can set the `lower_domain_boundary` attribute on a scope with `TRUE` value only if none of its ancestor blocks are set to `FALSE`
- The power domains of all the descendant blocks of an OFF-within-ON block must be scoped at or below that OFF-within-ON block. That is, any descendant block of an OFF-within-ON block cannot be the extent (root cell) of a domain, which is scoped above that OFF-within-ON block
- A scope without the `lower_domain_boundary` attribute setting inherits the setting from its attributed ancestor
- The `lower_domain_boundary` attribute setting applies to all the power domains scoped at or below the design or the scope having the setting
- The `lower_domain_boundary` attribute is not allowed to change once it is set
- While processing the `set_isolation` command, you can refer to the lower domain boundary port only if the `lower_domain_boundary` attribute is set to `TRUE`

- Isolation strategy with `-location` parent is not allowed on the following:
 - Domains where the `lower_domain_boundary` attribute is set to `TRUE`.
 - Domain boundaries where the `lower_domain_boundary` attribute value is changed to `FALSE`.

Examples

Consider the following design structure:



Example-1

```
set_scope /
set_design_attributes -elements { . } -attribute
lower_domain_boundary TRUE
set_design_attributes -elements { MID1/BOT1 } -attribute
lower_domain_boundary FALSE
create_power_domain PDT -include_scope
create_power_domain PDB -elements { MID1/BOT1 } -scope {
MID1/BOT1 }
```

In this example, MID1/BOT1 is a valid OFF-within-ON block since the domain PDB is scoped at MID1/BOT1 and all the scopes above MID1/BOT1 have implicit/explicit `TRUE` setting.

Example-2

```
set_scope /  
set_design_attributes -elements { . } -attribute  
lower_domain_boundary TRUE  
set_design_attributes -elements { MID1 } -attribute  
lower_domain_boundary FALSE  
create_power_domain PDT -include_scope
```

This is an invalid OFF-within-ON block configuration since MID1 does not have a domain of its own, which is a requirement if MID1 has to be a valid OFF-within-ON block. The above scenario results in a compile-time error.

Example-3

```
set_scope /  
set_design_attributes -elements { . } -attribute  
lower_domain_boundary FALSE  
set_design_attributes -elements { MID1 } -attribute  
lower_domain_boundary TRUE
```

This example issues an error message since ON-within-OFF block is not allowed.

Limitation

Isolation strategies with `-location fanout` are not supported.

Behavior of the Isolation Commands

The `set_isolation` command identifies the specified pin and port elements at the lower boundary interface of the specified domain. Also, the `-applies_to` option considers the lower boundary pins of the domain while filtering the objects of the domain-level strategies.

Commands Supported for Delayed Error Check

Domain boundary error checks on elements specified in the `set_isolation` command are performed at the end of reading UPF. This allows you to specify the lower domain boundary ports in the isolation element list even before the lower power domain is defined.

Consider the following example:

```
create_power_domain TOP
set_isolation out_iso -domain TOP -elements { BOT/* } ...
create_power_domain BOT -elements { BOT }
```

In this example, pins of `BOT` are mentioned as lower domain boundary pins of `TOP` before the creation of the domain `BOT`. VCS NLP does not issue an error message due to the delayed error check.

Impact on Isolation Engine

This section describes the following topics:

- [General isolation strategies \(without `–elements`\)](#)
- [Specific Isolation Strategies \(with `–elements`\)](#)
- [Inserting Back-to-Back Isolation Cells in the Lower Domain Boundary](#)
- [Order of Precedence of Isolation Strategies](#)

General isolation strategies (without –elements)

Isolation strategies of a power domain apply to its boundary pins. This feature expands the definition of the domain boundary pins to the lower domain-boundary pins. This means that the isolation strategies of a domain also apply to its lower boundary.

Output pins of a nested power domain are the input pins for the domain surrounding it, and the input pins of the nested domain are the output pins of the surrounding domain. Therefore, any isolation strategies that apply to the input pins of a domain also applies to the output pins of the root cells of the domains nested inside it, and any isolation strategies that apply to the output pins of a domain also applies to the input pins of the root cells of the domains nested inside it.

Figure 9-10 Output Isolation Strategy of TOP Applies to the Input of BOT

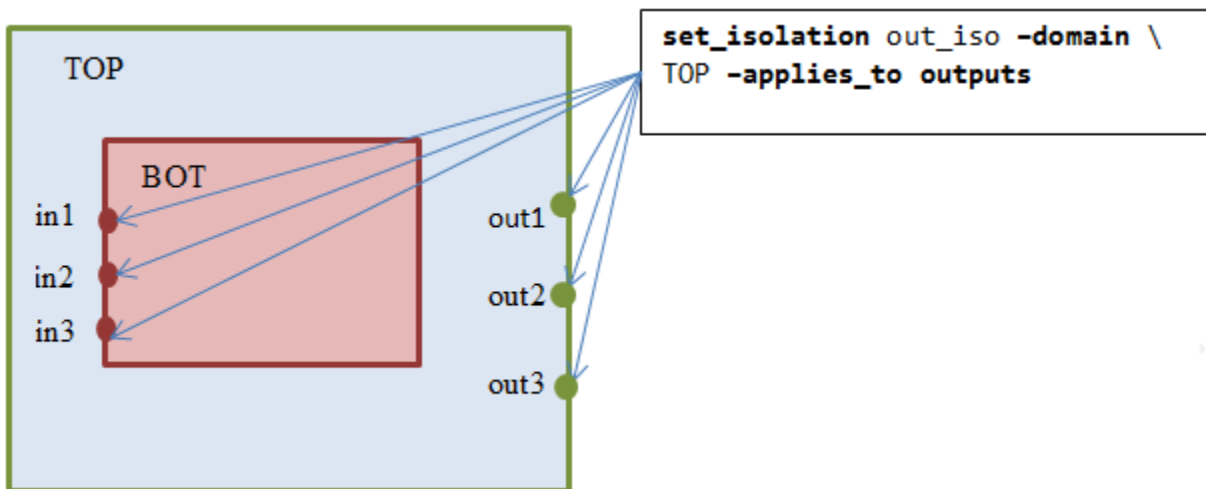


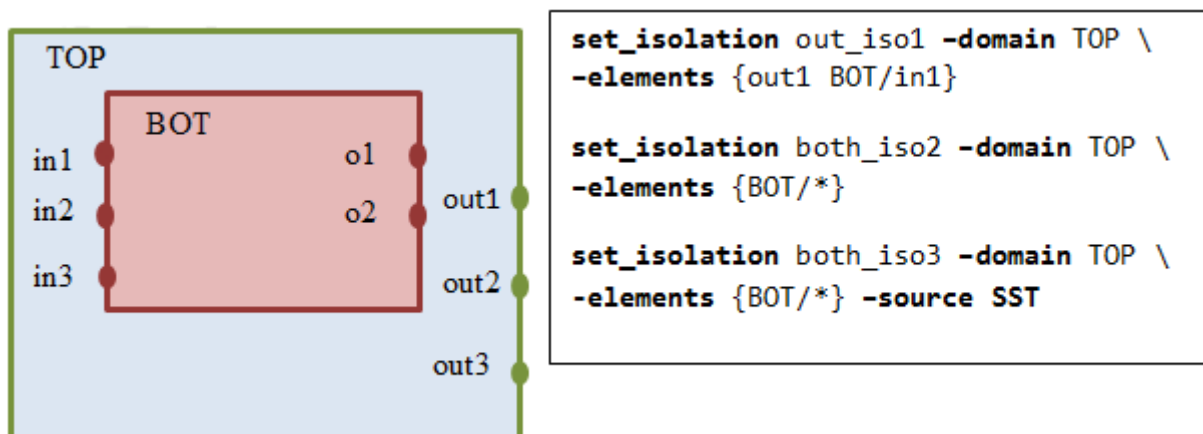
Figure 9-10 shows that the isolation strategy `out_iso` of the power domain `TOP` also applies to the input pins of the `BOT` block. Those pins are considered as outputs of the `TOP` domain. This means that with `lower_domain_boundary` enabled, `out_iso` results in 6

isolation cells for domain TOP, whereas without `lower_domain_boundary`, `out_iso` results in 3 isolation cells only.

Specific Isolation Strategies (with `-elements`)

Isolation strategies that specifically choose the pins to which they are applied do not change. They continue to apply to the pins and ports in the `-elements` option of the strategy. However, the isolation strategies are able to refer to the pins on the lower boundary of the domain in their `-elements` option. This enables you to target an isolation strategy specifically to the lower boundary pin(s).

Figure 9-11 Isolation Strategies of TOP Refer to the Lower Boundary Pins



In [Figure 9-11](#), `out_iso1` shows how the pin `BOT/in1` can be specified as one of the elements of the isolation strategies of the TOP power domain.

`both_iso2` shows how all the pins of the BOT domain can be specified as the lower boundary of the domain TOP by specifying all pins of the cell BOT in `-elements`. This strategy applies to all the pins in the element list.

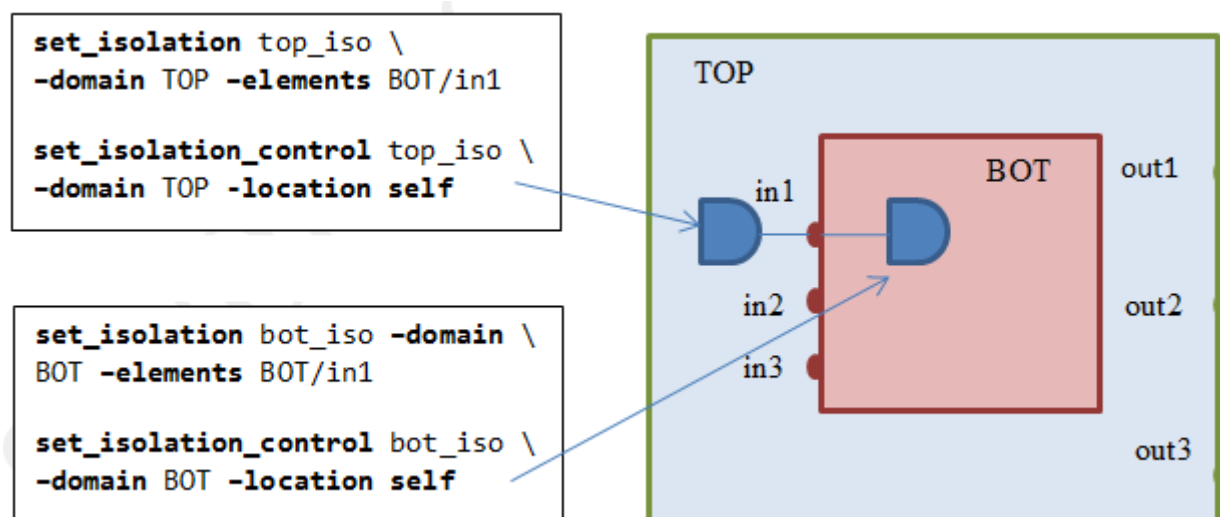
`both_iso3` is a `-source`, `-sink`, or `-diff_supply_only` based strategy with no `-applies_to`. The target objects for this strategy are all the pins (both input and output) of `BOT`, and the strategy finally applies to only those pins which satisfy the `-source`, `-sink`, or `-diff_supply_only` filter.

Inserting Back-to-Back Isolation Cells in the Lower Domain Boundary

This feature allows you to specify isolation strategies such that two isolation cells can be inserted on the same power domain boundary pin. One of these cells is inserted in the power domain and the other is inserted in its surrounding domain.

Consider [Figure 9-12](#). In this case, the boundary pin `BOT/in1` of power domain `BOT` is subjected to two isolation strategies. One is the isolation strategy `bot_iso` specified on the domain `BOT`, and the other is the isolation strategy `top_iso` specified for `TOP`. The isolation strategy `top_iso` applies to `BOT/in1` because `BOT/in1` lies on the lower boundary of the `TOP` domain.

Figure 9-12 Inserting Back-to-Back Isolation Cells



Order of Precedence of Isolation Strategies

This feature does not affect the precedence order. The precedence order is applied to the strategies on each of HighConn and LowConn sides of a domain boundary.

Limitations

- Isolation strategies with only `-location self` are supported
- VCS NLP does not support placing the isolation cells of the modport type ports on the lower domain boundary

10

Handling of Constants for Corruption and Isolation

VCS NLP supports the following behaviors for corruption and isolation of constants. Literals `1'b1` and `1'b0` in port maps or RHS of continuous assignments are treated as constants. The following topics describe this feature:

- [Default Behavior](#)
- [Supply Based Corruption of Constants](#)
- [Skip Corruption on Constants](#)

Default Behavior

All literal constants in port map or all variables that are assigned to the literal constants using the continuous assign statements are powered by the power domain where the declaration or assignment is made.

All constants are corrupted like any other logic when either Power or Ground of the domain goes OFF.

Since constants are corrupted like any other logic, they would also need isolation. Isolation strategy can be specified on all constant driven paths which cross the domain boundary. Constants in the port map of cell mapped (macro) instances are not corrupted unless there is isolation on that port.

Supply Based Corruption of Constants

Supply based corruption of constants applies to the constants in port map. `1'b0` is corrupted if `VSS` (ground) is OFF, and `1'b1` gets corrupted if `VDD` (power) is OFF. The old behavior can be achieved by using the `-power=const_corruption_old` compile-time option. LHS of continuous assignments with literal constants in RHS are corrupted based on the domain simstate.

Note:

Isolation is not applied on constants.

Skip Corruption on Constants

The `-power=pass_thru_const` compile-time option allows you to skip corruption on constants.

Note:

Any isolation strategy defined is applied on constants.

11

State Retention Support

The retention commands specify the strategy for inserting retention cells inside switched (power-down) domains. VCS NLP supports the `set_retention` and `set_retention_control` commands to model the state retention schemes.

This chapter describes the state retention support in the following sections:

- [Introduction](#)
- [Using `set\_retention` command](#)
- [Supported Retention Schemes](#)
- [Conditions](#)
- [Mutexes](#)
- [Parameters](#)

- [Creating Retention List](#)
- [Automated Assertions](#)
- [Generic Handles](#)
- [Debug Hooks for Retention](#)
- [Limitations of Retention Support](#)

Introduction

The `set_retention` command specifies which registers in the domain are to be implemented as retention registers and identifies the save and restore signals for the retention functionality. Only the registers implied in the elements are given retention capabilities.

Following is the syntax of the `set_retention` command:

```
set_retention retention_name
-domain domain_name
[-elements element_list] [-exclude_elements exclude_list] **
[-retention_power_net net_name] [-retention_ground_net net_name]
[-retention_supply_set ret_supply_set] [-no_retention]
[-save_signal {{logic_net <high | low | posedge | negedge>}}]
[-restore_signal {{logic_net <high | low | posedge | negedge>}}]
[-save_condition {{boolean_function}}]
[-restore_condition {{boolean_function}}]
[-retention_condition {{boolean_function}}]
[-use_retention_as_primary]
[ -parameters {< <RET_SUP_COR | NO_RET_SUP_COR> |
<SAV_RES_COR | NO_SAV_RES_COR> >}]
[-instance {{instance_name [signal_name]}}] **
[-transitive <TRUE | FALSE>]
[-update]
```

****** — Not supported

***** — Non-LRM Extensions

Using `set_retention` command

The `set_retention` command specifies which registers in the domain are to be implemented as retention registers and identifies the save and restore signals for the retention functionality.

If a design element is specified, all registers within the design element acquire the specified retention strategy. If a process is specified, all registers inferred by the process acquire the specified retention strategy. If a reg, signal, or variable is specified, and the object is a sequential element, then the implied register acquires the specified retention strategy. Any specified reg, signal, or variable that does not infer a sequential element is not changed by this command.

If `-elements` is specified, the element names outside the extent of `domain_name` are excluded. When `-elements` is not specified, this is equivalent to using the elements list that defines the power domain.

If the retention power and retention ground nets are specified, an implicit retention supply set is created and used with the specified strategy.

The retention power net serves the power function in the retention supply set and the retention ground net serves the ground function in the retention supply set.

If the retention power net is specified, but the retention ground net is not specified, then the domain's primary supply set ground function is used as the retention ground. If the retention ground net is specified, but the retention power net is not specified, then the domain's primary supply set power function is used as the retention power.

VCS NLP generates an error message, if a retention supply set is specified and a retention supply net is individually defined.

Support for the -update Option

VCS NLP supports the `-update` option with the `set_retention` command.

The `-update` option provides additional information for a previous command with the same strategy name, domain name, and executed in the same scope.

Use Model

The `-update` option adds information to the base command executed in the same scope of the power domain for which the inferred cells are defined.

Syntax:

```
set_retention retention_name
    -domain domain_name
    [-elements element_list]
    [-update]
```

Example:

```
set_retention test_ret
    -domain PD1
    -elements i_core/q_dff
    -save_signal {en posedge}
    -restore_signal {en negedge}

set_retention test_ret
    -domain PD1
```



```
-update  
-elements i_core/q_tff
```

The `-update` option adds two elements (`i_core/q_dff` and `i_core/q_tff`) to the retention strategy `test_ret`.

Using `-no_retention` Option

Do not apply retention on the elements specified. Useful to skip retention on a subset of registers in a power domain.

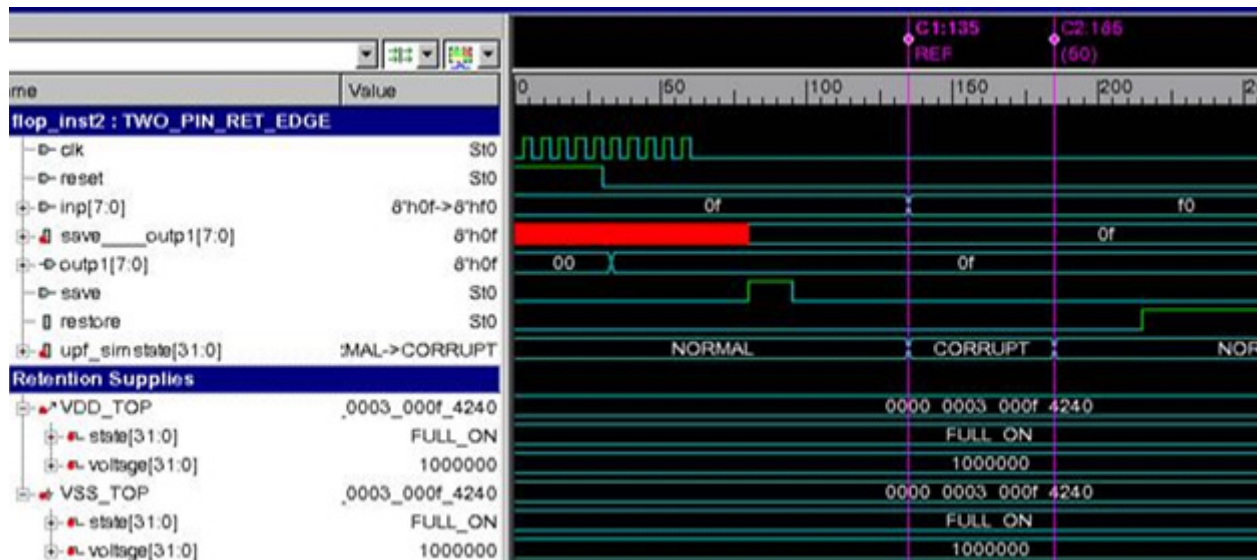
```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd  
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP  
  
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd  
-save_signal {save_level high} -restore_signal  
{restore_level high}  
  
set_retention NO_RET -domain Island_Vdd -elements  
{flop_inst3} -no_retention
```

Using `-use_retention_as_primary` Option

If specified, storage element and the output drivers of the register are powered using the retention supply. As a result, simstate for the retention supply set is applied to the register's output.

```
set_retention TWO_PIN_RET_EDGE -domain Island_Vdd  
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP  
-elements {flop_inst2/outp1} -use_retention_as_primary  
  
set_retention_control TWO_PIN_RET_EDGE -domain Island_Vdd  
-save_signal {save posedge} -restore_signal {restore  
posedge}
```

Figure 11-1 TWO_PIN_RET_EDGE



Supported Retention Schemes

Using the supported commands/options mentioned in the [“Introduction”](#) section, the supported retention schemes are categorized as follows:

1. [Two-pin Retention](#)
2. [Single-pin Retention](#)
3. [Zero-pin Retention](#)
4. [Save Signal Only](#)
5. [Restore Signal Only](#)

Note:

Use the `-power=partial_single_pin_retention` compile-time option to enable both Save Signal Only and Restore Signal Only retention schemes.

Two-pin Retention

Separate save and restore signals are used to control save and restore operations. Both level and edge sensitive are supported.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd -  
retention_power_net VDD_TOP -retention_ground_net VSS_TOP -  
elements {flop_inst1/outp1}
```

```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd  
-save_signal {save_level_high} -restore_signal  
{restore_level_high}
```

```
set_retention TWO_PIN_RET_EDGE -domain Island_Vdd -  
retention_power_net VDD_TOP -retention_ground_net VSS_TOP -  
elements {flop_inst2/outp1}
```

```
set_retention_control TWO_PIN_RET_EDGE -domain Island_Vdd -  
save_signal {save_posedge} -restore_signal {restore_posedge}
```

Semantics

1. Level-sensitive save/restore

Save operation is performed as long as the save signal is at active level.

Restore operation is performed as long as the restore signal is at active level.

2. Edge-sensitive save/restore

Save event is the specified edge of the save signal.

Restore event is the specified edge of the restore signal.

Example: `outp1` is the primary register, `save_outp1` is the shadow register

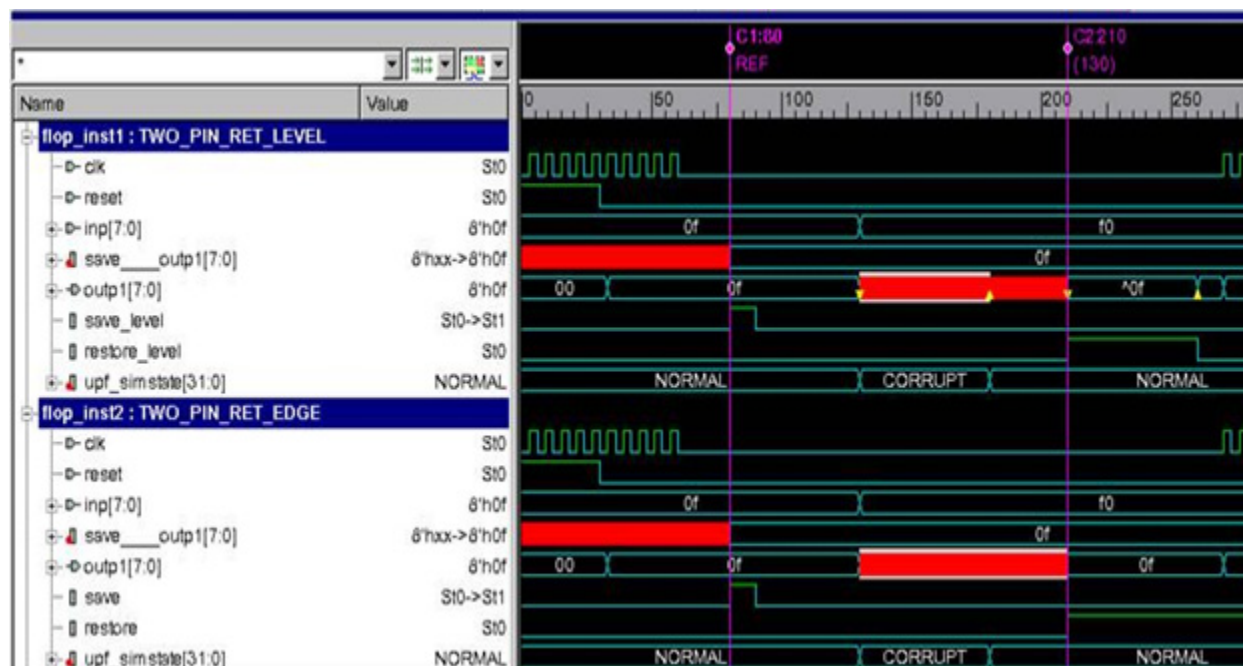
`TWO_PIN_RET_LEVEL`

- Save operation from time 80-90, since `save_level` is high
- Restore operation from time 210-260, since `restore_level` is high

`TWO_PIN_RET_EDGE`

- Save event at time 80, posedge on save
- Restore event at time 210, posedge on restore

Figure 11-2 Two-pin Retention



Single-pin Retention

Same signal is used as save and restore signal for controlling save and restore operations. Both level and edge sensitive are supported.

```
set_retention SINGLE_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst3/outp1}
```

```
set_retention_control SINGLE_PIN_RET_LEVEL -domain
Island_Vdd -save_signal {save_restore_level high}
-restore_signal {save_restore_level low}
```

```
set_retention SINGLE_PIN_RET_EDGE -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst4/outp1}
```

```
set_retention_control SINGLE_PIN_RET_EDGE -domain
Island_Vdd -save_signal {save_restore_posedge}
-restore_signal {save_restore negedge}
```

Semantics

1. Level-sensitive save/restore

Save operation is performed on the trailing edge specified with `-save_signal`, that is, negedge for high and posedge for low.

Restore operation is performed as long as the restore signal is at active level.

2. Edge-sensitive save/restore

Save event is the specified edge of `-save_signal`.

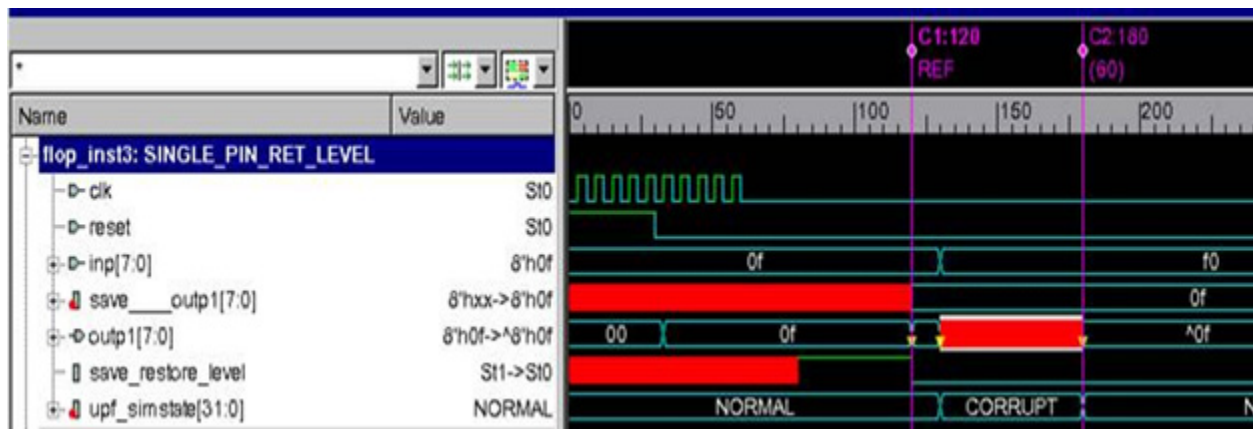
Restore event is the specified edge of `-restore_signal`.

Example: `outp1` is the primary register, `save_outp1` is the shadow register

SINGLE_PIN_RET_LEVEL

- Save event at time 120, trailing edge of `save_restore_level`
- Restore operation from time 120-260 since `save_restore_level` is low

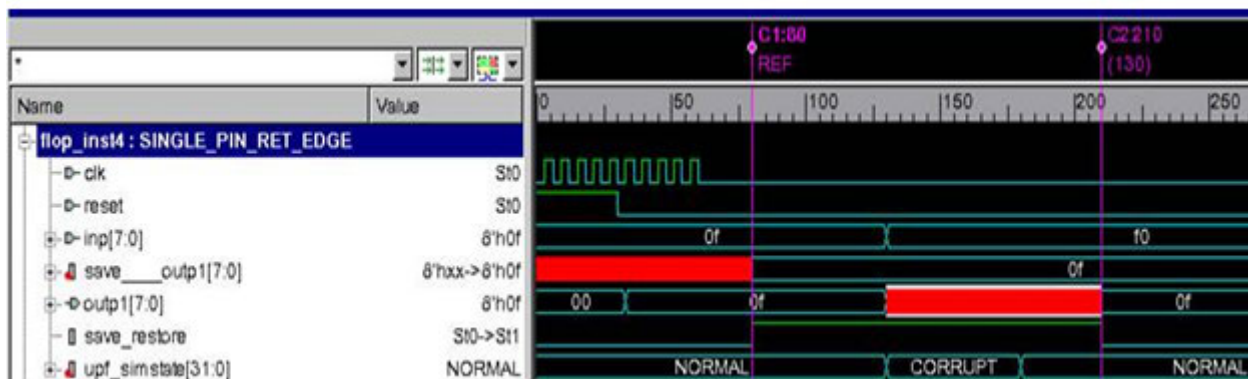
Figure 11-3 SINGLE_PIN_RET_LEVEL



SINGLE_PIN_RET_EDGE

- Save event at time 80, posedge on `save_restore`
- Restore event at time 210, negedge on `save_restore`

Figure 11-4 SINGLE_PIN_RET_EDGE



Zero-pin Retention

No `-save_signal` and `-restore_signal` specified.

```
set_retention ZERO_PIN_RET -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
```

```
-elements {flop_inst5/outp1}
```

Semantics

Power-down event (NORMAL -> CORRUPT) for the domain acts as the save event.

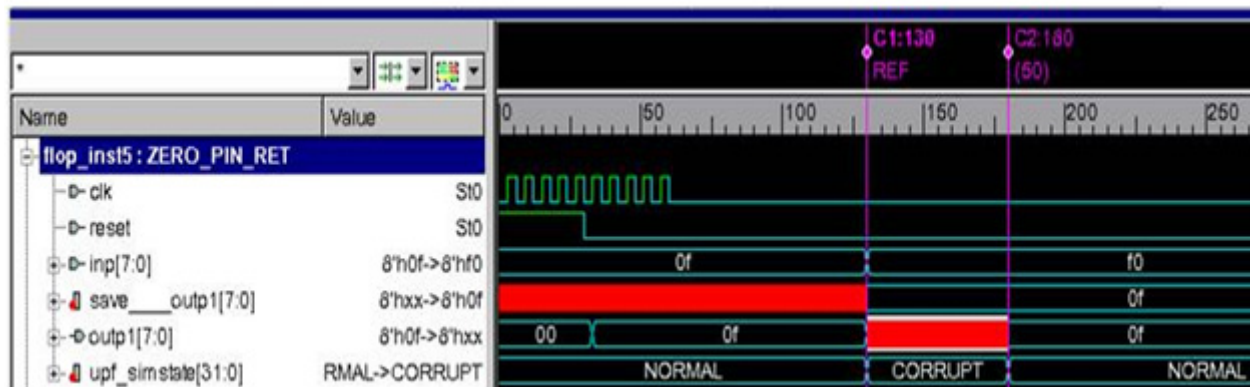
Power-up event (CORRUPT -> NORMAL) for the domain acts as the restore event.

Example: outp1 is the primary register, save_outp1 is the shadow register

ZERO_PIN_RET

- Save event at time 130, domain simstate transition from NORMAL -> CORRUPT
- Restore operation at time 180, domain simstate transition from CORRUPT -> NORMAL

Figure 11-5 ZERO_PIN_RET



Note:

Zero-pin retention is not supported for VHDL designs.

Clock-Gated Zero-Pin Retention

Use the `-power=clk_gated_zero_pin_retention` compile-time option to enable the clock-gated zero-pin retention. This option is applicable for the retention strategies that are specified at domain or instance level. The following semantic is applied when you use this option:

- During power-up (restore), if `-retention_condition` is true, the saved value is restored to the register, but stops the execution of always block (actual process), so reset/clock is not honored. If the `-retention_condition` is false, the execution of always block is allowed.

Limitations

Following are the limitations:

- You cannot use `-power=clk_gated_zero_pin_retention`, if the retention strategy is defined on the registers using `-elements`.
- Clock-gated zero-pin retention is not supported for VHDL designs.

Save Signal Only

Only `-save_signal` is specified and no `-restore_signal` is specified.

Use the `-power=partial_single_pin_retention` compile-time option to enable the save signal only retention scheme.

```
set_retention RET_SAVE_ONLY -domain Island_Vdd  
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP  
-elements {flop_inst6/outp1} -save_signal {save posedge}
```

Semantics

Save event is the active edge or level specified for the save signal.

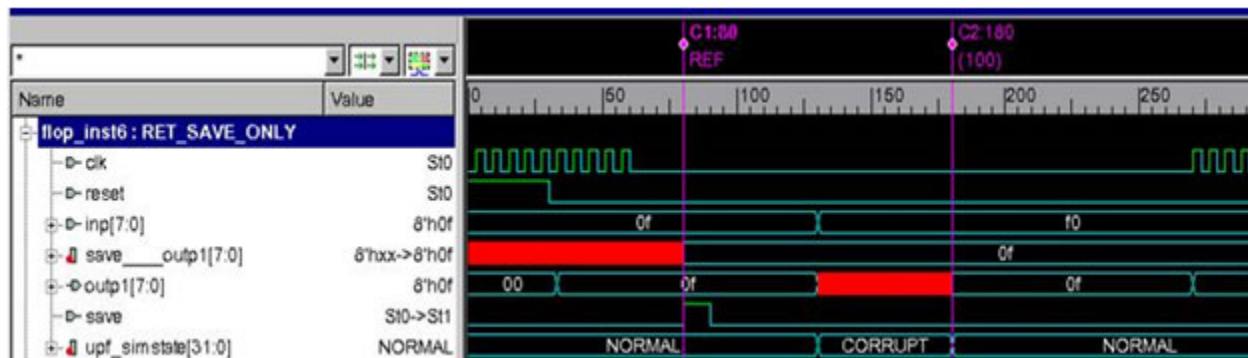
Power-up event (CORRUPT -> NORMAL) for the domain acts as the restore event.

Example: outp1 is the primary register, save_outp1 is the shadow register

RET_SAVE_ONLY

- Save event at time 80, posedge on save
- Restore operation at time 180, domain simstate transition from CORRUPT -> NORMAL

Figure 11-6 RET_SAVE_ONLY



Restore Signal Only

Only -restore_signal is specified and no -save_signal is specified.

Use the `-power=partial_single_pin_retention` compile-time option to enable the restore signal only retention scheme.

```
set_retention RET_RESTORE_ONLY -domain Island_Vdd  
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP  
-elements {flop_inst7/outp1} -restore_signal {restore  
posedge}
```

Semantics

Power-down event (NORMAL -> CORRUPT) for the domain acts as the save event.

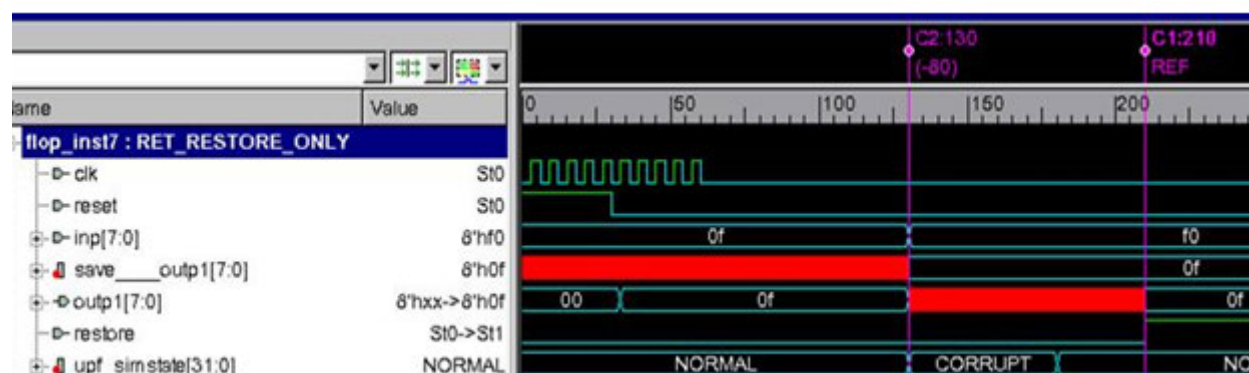
Restore event is the active edge or level specified for the restore signal.

Example: outp1 is the primary register, save_outp1 is the shadow register

RET_RESTORE_ONLY

- Save event at time 130, domain simstate transition from NORMAL -> CORRUPT
- Restore operation at time 180, posedge on restore

Figure 11-7 RET_RESTORE_ONLY



Conditions

This section describes the following topics:

- [-save_condition/-restore_condition](#)
- [-retention_condition](#)

-save_condition/-restore_condition

You can further specify gating conditions for the save/restore events using the `-save_condition` and `-restore_condition` options. Restore event has highest priority over any other operation of the retention element (clock/set/reset). Restore condition can be used to give priority to clock/set/reset.

Condition must be a Boolean expression and save/restore events are gated if the Boolean expression evaluates to `FALSE`. Default value is `TRUE`.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd
```

```
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst1/outp1} -save_condition {clk}
-restore_condition {!reset}
```

```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd
-save_signal {save_level high} -restore_signal
{restore_level high}
```

Example: outp1 is the primary register, save_outp1 is the shadow register

TWO_PIN_RET_LEVEL

- No save operation between time 80-90 as the `save_condition` is FALSE. Shadow register retains its old value 8'hxx
- Restore operation between time 210-260 as `restore_condition` is TRUE. Restores the content of the shadow register

Figure 11-8 TWO_PIN_RET_LEVEL



```
set_retention TWO_PIN_RET_EDGE -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst2/outp1} -save_condition {!clk}
-restore_condition {reset}
```

```
set_retention_control TWO_PIN_RET_EDGE -domain Island_Vdd -
```

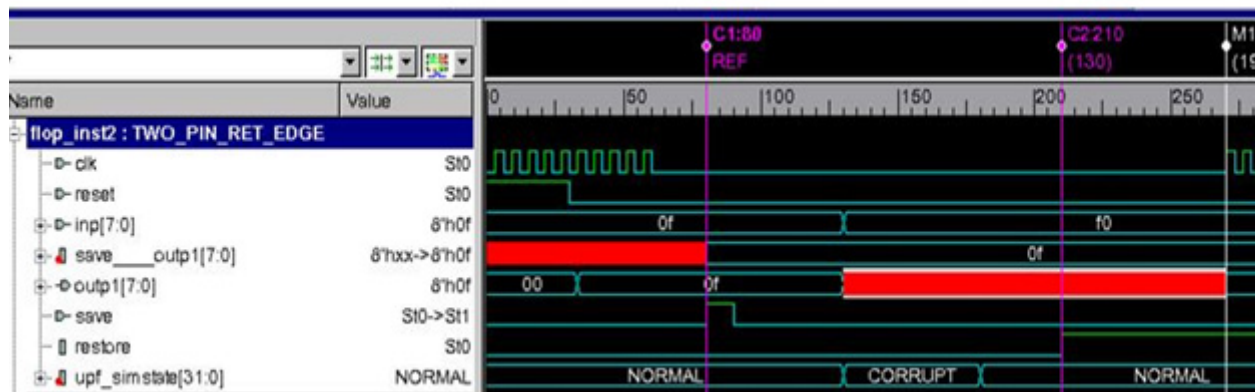
```
save_signal {save posedge} -restore_signal {restore posedge}
```

Example: outp1 is the primary register, save_outp1 is the shadow register

TWO_PIN_RET_EDGE

- Save operation at time 80 as the `save_condition` is TRUE
- No Restore operation at time 210 as the `restore_condition` is FALSE. Corrupts the value on the primary register till the active clock edge

Figure 11-9 TWO_PIN_RET_EDGE



-retention_condition

Boolean expression can be specified and the shadow register contents are corrupted, if the expression evaluates to FALSE when the primary supplies are OFF. The default value is TRUE.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst1/outp1} -retention_condition {clk}
```

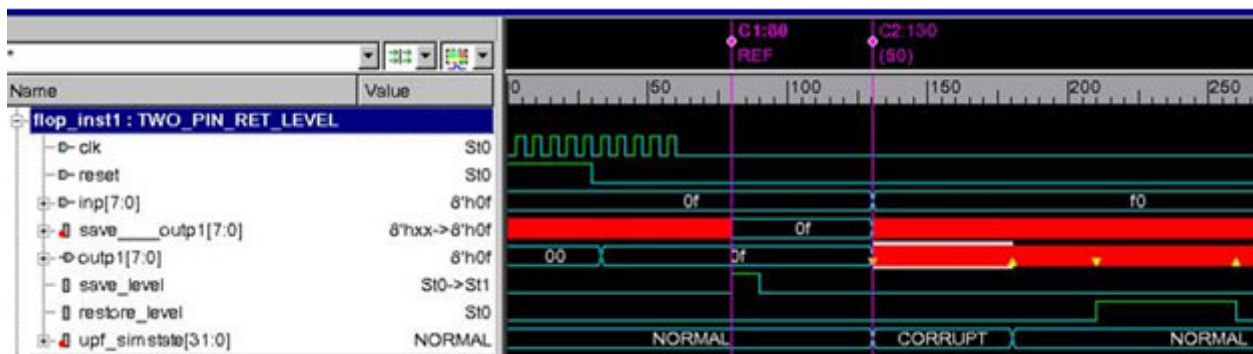
```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd
-save_signal {save_level high} -restore_signal
{restore_level high}
```

Example: outp1 is the primary register, save_outp1 is the shadow register

TWO_PIN_RET_LEVEL

- Save operation from time 80-90, since save_level is high
- Shadow register corrupted on power down at time 130 as the retention_condition is FALSE

Figure 11-10 TWO_PIN_RET_LEVEL



Mutexes

This section contains the following topics:

- [assert_s_mutex](#)
- [assert_r_mutex](#)
- [assert_rs_mutex](#)

assert_s_mutex

1. Defines mutual exclusion between save event and an active level/edge of other signal.
2. Assertion is reported.
3. In case of level sensitive `save_signal`, shadow register is corrupted if the mutual exclusion fails.
4. No corruption of shadow register in case of edge-sensitive `save_signal`.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd  
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP  
-elements {flop_inst1/outp1}
```

```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd  
-save_signal {save_level_high} -restore_signal  
{restore_level_high} -assert_s_mutex {clk low}
```

Example: outp1 is the primary register, save_outp1 is the shadow register

TWO_PIN_RET_LEVEL

- `assert_s_mutex` failure at time 80, shadow register corrupted
- Assertion reported at time 80 indicating mutex failure

```
[80] [ERROR] [LP_ASSERT_S_MUTEX] 'assert_s_mutex' defined  
for retention strategy 'TWO_PIN_RET_LEVEL' of power domain  
'testbench/top/Island_Vdd' failed. Save signal 'testbench/  
top/save_level' ('high') and signal 'testbench/top/clk'  
('low') are not mutually exclusive.
```


Figure 11-11 TWO_PIN_RET_LEVEL



```
set_retention TWO_PIN_RET_EDGE -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst2/outp1}
```

```
set_retention_control TWO_PIN_RET_EDGE -domain Island_Vdd
-save_signal {save posedge} -restore_signal {restore
posedge} -assert_s_mutex {clk low}
```

Example: outp1 is the primary register, save_outp1 is the shadow register

TWO_PIN_RET_EDGE

- assert_s_mutex failure at time 80, primary register content saved to shadow register
- Assertion reported at time 80 indicating mutex failure

```
[80] [ERROR] [LP_ASSERT_S_MUTEX] 'assert_s_mutex' defined
for retention strategy 'TWO_PIN_RET_EDGE' of power domain
'testbench/top/Island_Vdd' failed. Save signal 'testbench/
top/save' ('posedge') and signal 'testbench/top/clk' ('low')
are not mutually exclusive.
```

Figure 11-12 TWO_PIN_RET_EDGE



assert_r_mutex

1. Defines mutual exclusion between restore event and an active level/edge of other signal.
2. Assertion is reported.
3. In case of level sensitive `restore_signal`, primary register is corrupted if the mutual exclusion fails.
4. No corruption in case of edge-sensitive `restore_signal`.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst1/outp1}
```

```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd
-save_signal {save_level high} -restore_signal
{restore_level high} -assert_r_mutex {clk low}
```

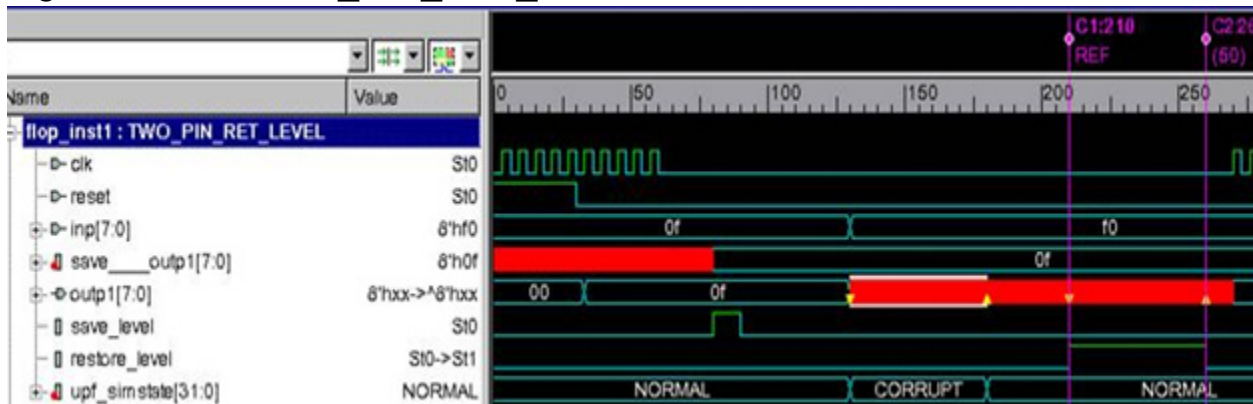
Example: `outp1` is the primary register, `save_outp1` is the shadow register

TWO_PIN_RET_LEVEL

- Save operation from time 80-90 since `save_level` is high
- `assert_r_mutex` failure at time 210, primary register corrupted
- Assertion reported at time 210 indicating mutex failure

[210] [ERROR] [LP_ASSERT_R_MUTEX] 'assert_r_mutex' defined for retention strategy 'TWO_PIN_RET_LEVEL' of power domain 'testbench/top/Island_Vdd' failed. Restore signal 'testbench/top/restore_level' ('high') and signal 'testbench/top/clock' ('low') are not mutually exclusive.

Figure 11-13 TWO_PIN_RET_LEVEL



```
set_retention TWO_PIN_RET_EDGE -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst2/outp1}
```

```
set_retention_control TWO_PIN_RET_EDGE -domain Island_Vdd
-save_signal {save posedge} -restore_signal {restore
posedge} -assert_r_mutex {clk low}
```

Example: outp1 is the primary register, save_outp1 is the shadow register

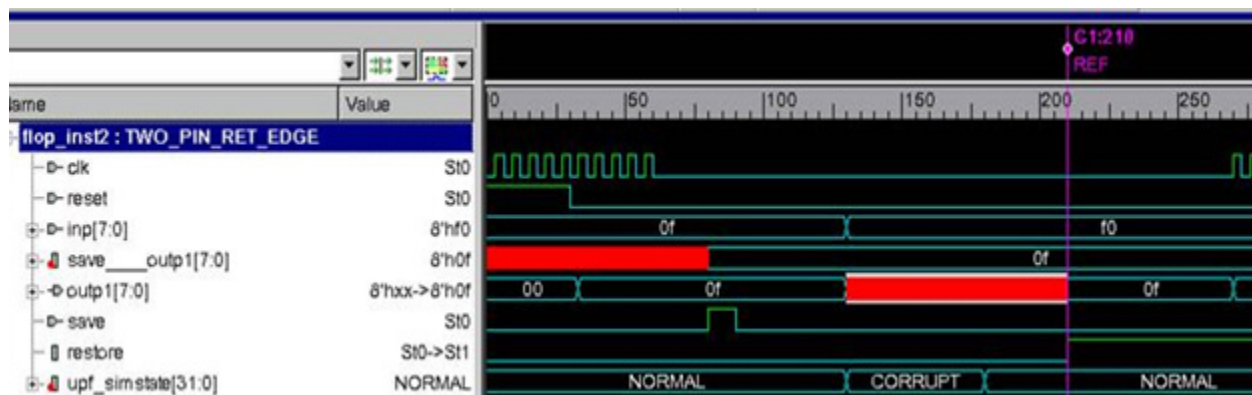
TWO_PIN_RET_EDGE

- Save operation from time 80-90, posedge on save

- `assert_r_mutex` failure at time 210, restore operation performed
- Assertion reported at time 210 indicating mutex failure

[210] [ERROR] [LP_ASSERT_R_MUTEX] 'assert_r_mutex' defined for retention strategy 'TWO_PIN_RET_EDGE' of power domain 'testbench/top/Island_Vdd' failed. Restore signal 'testbench/top/restore' ('posedge') and signal 'testbench/top/clock' ('low') are not mutually exclusive.

Figure 11-14 TWO_PIN_RET_EDGE



assert_rs_mutex

1. Defines mutual exclusion between save and restore events and an active level/edge of other signal.
2. Assertion is reported for mutual exclusion failures on either/both save or/and restore.
3. In case of level-sensitive `save_signal`, shadow register is corrupted if the mutual exclusion fails with respect to the save event.
4. In case of level sensitive `restore_signal`, primary register is corrupted if the mutual exclusion fails with respect to the restore event.

```
set_retention SINGLE_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst3/otp1}
```

```
set_retention_control SINGLE_PIN_RET_LEVEL -domain
Island_Vdd -save_signal {save_restore_level high}
-restore_signal {save_restore_level low} -assert_rs_mutex
{clk low}
```

Example: otp1 is the primary register, save_otp1 is the shadow register

SINGLE_PIN_RET_LEVEL

- assert_s_mutex failure at time 120 on trailing edge of save_restore_level, shadow register corrupted
- Assertion reported at time 120 indicating assert_s_mutex failure
- assert_r_mutex failure from time 120-260, primary register corrupted
- Assertion reported at time 120 indicating assert_r_mutex failure

```
[120] [ERROR] [LP_ASSERT_S_MUTEX] 'assert_s_mutex' defined
for retention strategy 'SINGLE_PIN_RET_LEVEL' of power
domain 'testbench/top/Island_Vdd' failed. Save signal
'testbench/top/save_restore_level' ('negedge') and signal
'clk' ('low') are not mutually exclusive.
```

```
[120] [ERROR] [LP_ASSERT_R_MUTEX] 'assert_r_mutex' defined
for retention strategy 'SINGLE_PIN_RET_LEVEL' of power
domain 'testbench/top/Island_Vdd' failed. Restore signal
'testbench/top/save_restore_level' ('low') and signal 'clk'
('low') are not mutually exclusive.
```

Figure 11-15 SINGLE_PIN_RET_LEVEL



Parameters

This section describes the following topics:

- [Supported Parameters with set_retention_control](#)
- [Supported Parameters with set_retention](#)

Supported Parameters with set_retention_control

COR_AUTO

Set by default. Enables corruption on primary and shadow registers due to mutex failures on level-sensitive restore and save signals respectively.

COR_NONE

Disables corruption on the primary and shadow registers due to mutex failures on level-sensitive restore and save signals respectively.

```

set_retention SINGLE_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst3/outp1}
set_retention_control SINGLE_PIN_RET_LEVEL -domain
Island_Vdd -save_signal {save_restore_level high}
-restore_signal {save_restore_level low} -assert_r_mutex
{clk low} -parameters COR_NONE

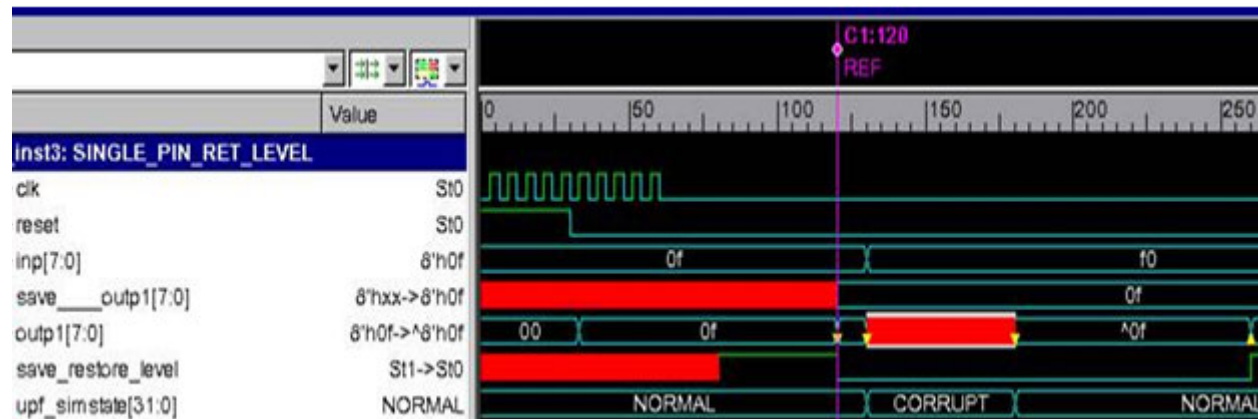
```

Example: outp1 is the primary register, save_outp1 is the shadow register

SINGLE_PIN_RET_LEVEL

- assert_s_mutex failure at time 120 on trailing edge of save_restore_level, value saved to the shadow register and no corruption on the shadow register
- Assertion reported at time 120 indicating assert_s_mutex failure
- assert_r_mutex failure from time 120-260, shadow register content restored to the primary register and no corruption on primary register
- Assertion reported at time 120 indicating assert_r_mutex failure

Figure 11-16 SINGLE_PIN_RET_LEVEL



COR_RET

Enables corruption on the shadow register due to mutex failures on level-sensitive save signal.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst1/outp1}
```

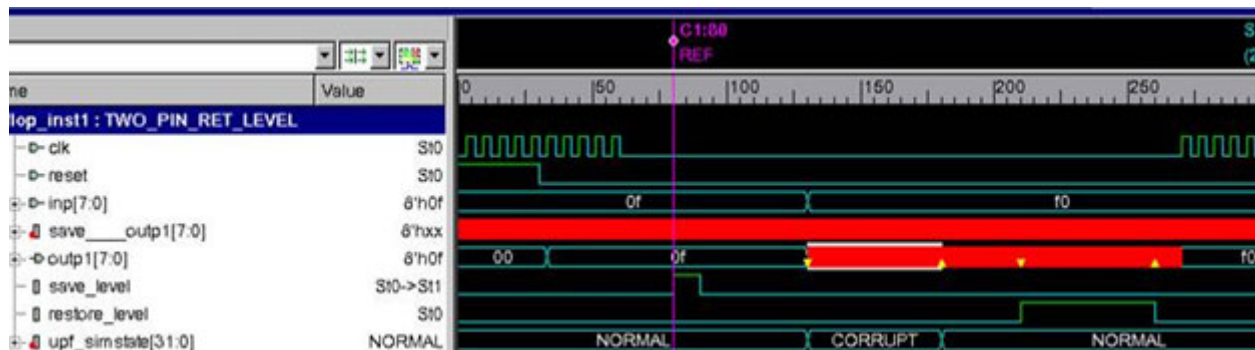
```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd
-save_signal {save_level_high} -restore_signal
{restore_level_high} -assert_rs_mutex {clk low} -parameters
COR_RET
```

Example: outp1 is the primary register, save_outp1 is the shadow register

TWO_PIN_RET_LEVEL

- assert_s_mutex failure at time 80, shadow register corrupted
- Assertion reported at time 80 indicating mutex failure

Figure 11-17 TWO_PIN_RET_LEVEL



COR_NORMAL

Enables corruption on primary register due to mutex failures on level-sensitive restore signal.

```
set_retention SINGLE_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst3/outp1}
```

```
set_retention_control SINGLE_PIN_RET_LEVEL -domain
Island_Vdd -save_signal {save_restore_level high}
-restore_signal {save_restore_level low} -assert_rs_mutex
{clk low} -parameters COR_NORMAL
```

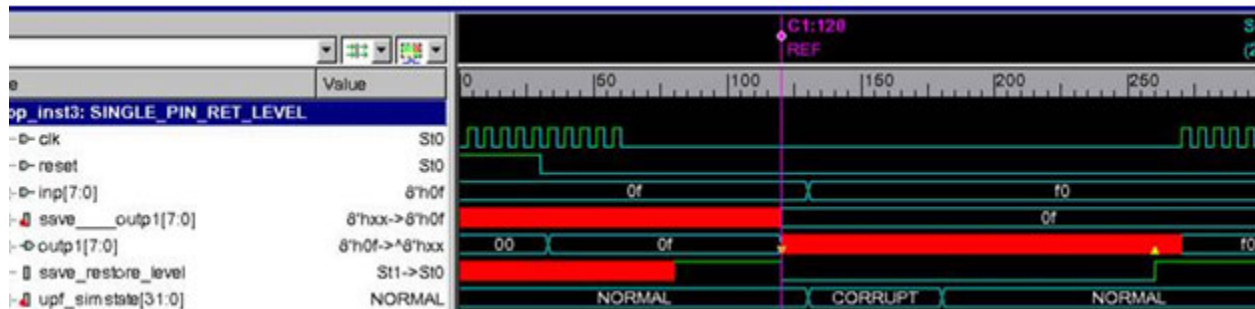
Example: outp1 is the primary register, save_outp1 is the shadow register

SINGLE_PIN_RET_LEVEL

- assert_s_mutex failure at time 120 on trailing edge of save_restore_level, value saved to the shadow register and no corruption on the shadow register
- Assertion reported at time 120 indicating assert_s_mutex failure

- `assert_r_mutex` failure from time 120-260, primary register corrupted
- Assertion reported at time 120 indicating `assert_r_mutex` failure

Figure 11-18 `SINGLE_PIN_RET_LEVEL`



COR_BOTH

Enables corruption on both primary and shadow registers due to any mutex failures on level-sensitive restore and save signals.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst1/outp1}
```

```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd
-save_signal {save_level_high} -restore_signal
{restore_level_high} -assert_s_mutex {clk low} -parameters
COR_BOTH
```

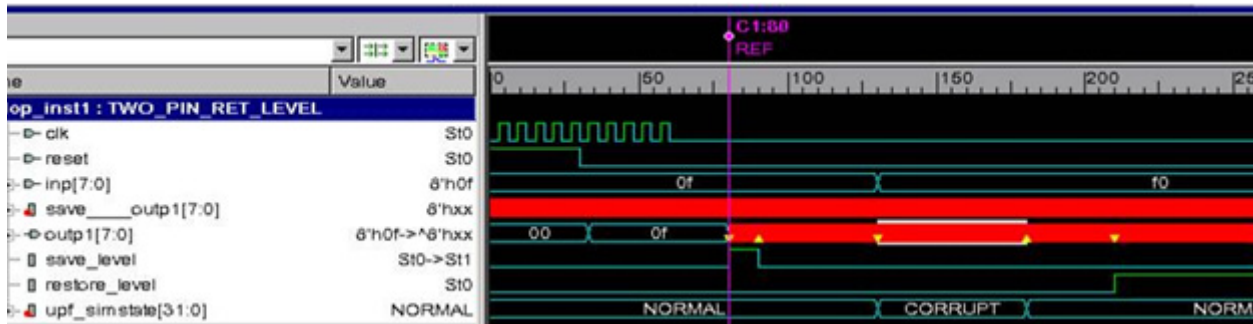
Example: `outp1` is the primary register, `save_outp1` is the shadow register

TWO_PIN_RET_LEVEL

- `assert_s_mutex` failure at time 80, shadow register and primary register corrupted

- Assertion reported at time 120 indicating `assert_s_mutex` failure

Figure 11-19 TWO_PIN_RET_LEVEL



Supported Parameters with `set_retention`

SAVE_RES_COR/NO_SAVE_RES_COR

`SAVE_RES_COR` enables corruption of primary register during concurrent assertion of `save`, `save_condition`, `restore`, or `restore_condition`. It is default.

`NO_SAVE_RES_COR` disables the corruption.

```
set_retention TWO_PIN_RET_LEVEL -domain Island_Vdd
-retention_power_net VDD_TOP -retention_ground_net VSS_TOP
-elements {flop_inst1/outp1}
```

```
set_retention_control TWO_PIN_RET_LEVEL -domain Island_Vdd
-save_signal {save_level high} -restore_signal
{restore_level low}
```

Example: `outp1` is the primary register, `save_outp1` is the shadow register

TWO_PIN_RET_LEVEL

- Both save and restore events at time 85, primary register corrupted

Figure 11-20 TWO_PIN_RET_LEVEL



RET_SUP_COR/NO_RET_SUP_COR

RET_SUP_COR enables corruption of the primary registers when the retention supplies are OFF.

NO_RET_SUP_COR disables the primary register when the retention supplies are OFF. It is default.

```
set_retention TWO_PIN_RET_LEVEL_P -domain Island_top  
-retention_power_net V1SW -retention_ground_net VSS_TOP  
-elements {flop_inst8/outp1} -parameters RET_SUP_COR
```

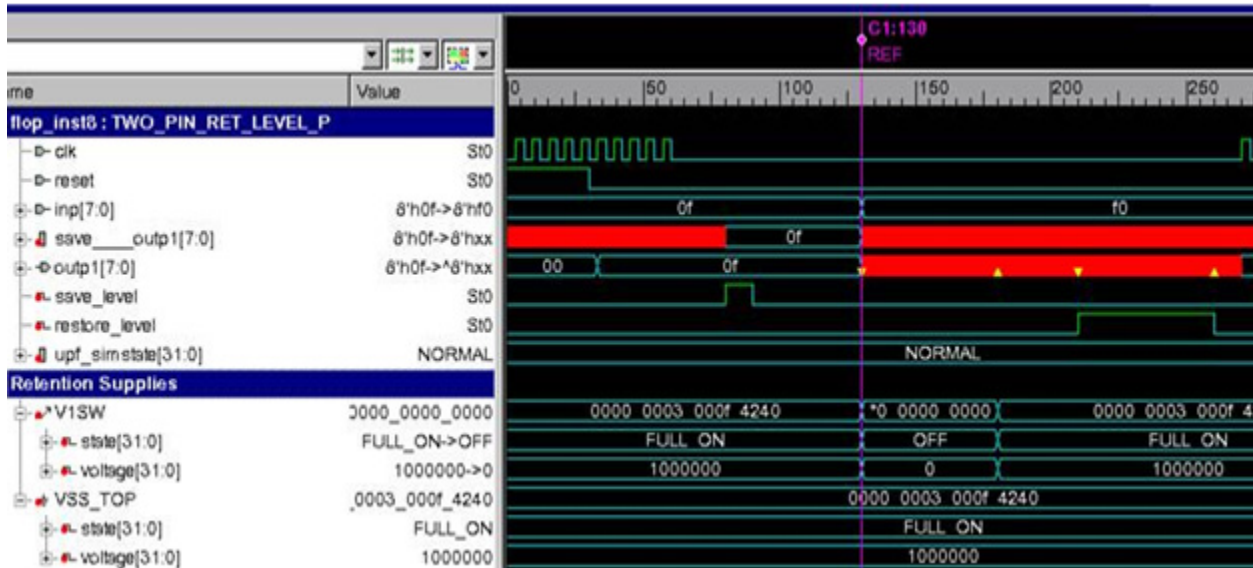
```
set_retention_control TWO_PIN_RET_LEVEL_P -domain  
Island_top -save_signal {save_level high} -restore_signal  
{restore_level high}
```

Example: **outp1** is the primary register, **save_outp1** is the shadow register

TWO_PIN_RET_LEVEL_P

- Retention supplies turning OFF at time 130, primary register corrupted even though the primary supplies are ON

Figure 11-21 TWO_PIN_RET_LEVEL_P



Creating Retention List

The `set_retention_elements` command (see [“set_retention_elements”](#) for syntax) creates a named list of elements to be used in the `set_retention` command.

VCS NLP supports the `-elements` option of the `set_retention_elements` command.

This feature allows you to create a named list of elements to be used in the `set_retention` command. This feature helps you to create the list of registers which need to be retained during power shutdown.

Following is the syntax:

```
set_retention_elements <retention_list_name>  
-elements <element_list>
```

Where,

- `retention_list_name` is a simple name; this shall be unique within the current scope.
- `element_list` is a list of rooted names: instances, named processes, sequential registers, or signal names.

Example:

```
set_retention_elements RET_LIST1 -elements {u1}  
set_retention RET2 -domain PD2 -elements {RET_LIST1 u2}
```

Key Points to Note

- Retention list should be defined before it is used in the `set_retention` command.
- Wildcards are supported in the `-elements` option.
- You can use the same retention list in multiple `set_retention` commands.
- The behavior of the `set_retention` command matches as if the retention list has been written in the `-elements` option. For example, consider the following command:

```
set_retention_elements rlist1 -elements {u1 u2}  
set_retention ret -domain PD1 {rlist1 u3}
```

This command is equivalent to the following:

```
set_retention ret -domain PD1 {u1 u1 u3}
```

Usage Examples

- Using `retention_element_list` along with other elements in `set_retention -elements`

```
set_retention_elements RET_LIST1 -elements {u1}  
set_retention RET2 -domain PD2 -elements {RET_LIST1 u2}
```

- Using elements of `set_retention_elements` individually

```
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}  
set_retention RET1 -domain PD1 -elements {R1}  
set_retention RET2 -domain PD2 -elements {u1/R2 u2}
```

- Specifying `set_retention_elements` in multiple `set_retention -elements`

This example also shows that not all the elements in the retention list belong to the same power domain.

```
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}  
set_retention RET1 -domain PD1 -elements {RET_LIST1}
```

`u1/R2, u2` do not belong to power domain `PD1`.

```
set_retention RET2 -domain PD2 -elements {RET_LIST1}
```

`R1` does not belong to the power domain `PD2`.

VCS NLP issues a warning message in this case. Following is the sample warning message:

```
Warning- [UPF_RET_ELEM_ODS] Retention element outside  
domain scope  
ret_list_contains_inst_sig_process_dpr_sc1.upf, 41  
Ignoring retention on element 'tb/top_inst/u1' specified
```

with retention strategy 'tb/top_inst/PD1/retention/RET1' as it is outside the extent of power domain 'tb/top_inst/PD1'. Element is in the extent of power domain 'tb/top_inst/PD2'.
Please specify only elements that are in the extent of the domain specified with '-domain' option.

Invalid Usage Scenarios

VCS NLP issues error message for the following invalid usage scenarios:

- **Multiple retention_element_lists at a given scope with a matching name**

```
set_scope /  
set_retention_elements RET_LIST1 -elements {u2}  
set_retention_elements RET_LIST1 -elements {u1}
```

Following is the sample error message:

```
Error-[UPF_SNNU] Strategy/Object name not unique  
./Source/mvsim_nlp/nlp_pv/set_retention_elements/  
set_ret_ele_neg_scl/set_ret_ele_neg_scl.upf, 40  
  'tb/top_inst/RET_LIST1' is already used as 'retention  
elements list' name in  
  UPF/design scope 'tb/top_inst'.  
  Strategy/Object names must be unique in one UPF/design  
scope.
```

- **Using retention_element_list in another set_retention_elements command**

```
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}  
set_retention_elements RET_LIST2 -elements {RET_LIST1}
```

- **Using retention_list_name before definition**

```
set_retention RET1 -domain PD1 -elements {RET_LIST1 u1}
```


- Referring to `retention_element_list` defined outside the current scope

```
set_scope /
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}
set_scope u1
set_retention RET1 -domain PD1 -elements {RET_LIST1 R1}
```

- Defining multiple retention strategies on an element

```
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}
set_retention RET1 -domain PD1 -elements {RET_LIST1}
set_retention RET2 -domain PD1 -elements {R1}
```

- Defining `-no_retention` strategy for an element in `retention_element_list`

Case-1:

```
set_retention RET1 -domain PD1 -elements {R1} -
no_retention
set_retention RET2 -domain PD2
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}
```

Case-2:

```
set_retention_elements RET_LIST1 -elements {R1 u1/R2 u2}
set_retention RET1 -domain PD1 -elements {R1} -
no_retention
```

Limitations

VCS NLP does not issue an error message in the following cases:

- Element that belongs to `retention_element_list` is not retained when any other element in the same `retention_element_list` is retained.

- Element that belongs to `retention_element_list` is not retained when any other element in the same power domain extent is retained.

Ignoring Latches From Retention

By default, VCS NLP considers flops and latches for retention. You can use the `-power=ignore_latch_retention` compile-time option to ignore latches from retention. This option allows granular control on type of logic (flop, latch) that needs to be retained.

Automated Assertions

For the list of automated assertions supported for retention, refer to the Retention context in [Table 18-1](#).

Generic Handles

VCS NLP supports `UPF_GENERIC_CLOCK` and `UPF_GENERIC_ASYNC_LOAD` generic handles for accessing the clock and the asynchronous load per retention register respectively.

These generic handles can be used to specify the retention functionality by using them in `boolean_expression` for `-save_condition`, `-restore_condition`, or `-retention_condition` of the `set_retention` command.

Along with specifying the retention functionality, generic handles can also be used in the `bind_checker` command to write the clock or asynchronous load related assertions on the retention registers.

For more information, refer to [“Generic Handles Support”](#).

Debug Hooks for Retention

This section describes the following topics:

- [Hierarchical Flop Inference Report](#)
- [Shadow Register in DVE](#)
- [Retention Strategy in Power Pane](#)

Hierarchical Flop Inference Report

VCS NLP generates the `hierarchical_flop_inference.rpt` compile-time static report for Verilog in the [mvsim_native_reports](#) directory. This report captures the details of the inferred retention registers. This report is dumped by default. Data in the report is dumped as Tcl list as follows:

```
set sr_elements {
{retention_register clock reset save_signal save_sense
restore_signal restore_sense primary_power_net
primary_ground_net retention_power_net
retention_ground_net}\
{retention_register clock reset save_signal save_sense
restore_signal restore_sense primary_power_net
primary_ground_net retention_power_net
retention_ground_net}\
{ .. }\
}
```

Following is the example:

```
set sr_elements { {testbench.top.flop_inst1.outp1 {{7 0}}
testbench.top.flop_inst1.clk
testbench.top.flop_inst1.reset testbench/top/save_level
high testbench/top/restore_level high testbench/top/V1SW
testbench/top/VSS_TOP testbench/top/VDD_TOP testbench/top/
VSS_TOP } \

{testbench.top.flop_inst2.outp1 {{7 0}}
testbench.top.flop_inst2.clk
testbench.top.flop_inst2.reset testbench/top/save_level
high testbench/top/restore_level high testbench/top/V1SW
testbench/top/VSS_TOP testbench/top/VDD_TOP testbench/top/
VSS_TOP } \
}
```

Similarly, VCS NLP generates the
hierarchical_flop_inference_vhdl.rpt compile-time
static report for VHDL in the mvsim_native_reports directory.

Shadow Register in DVE

VCS NLP dumps a shadow register named
save__<primary_register> to debug the save/restore
sequence. It is available in the Data Pane along with the primary
register. For more information, refer to [“Shadow Registers”](#).

Retention Strategy in Power Pane

Retention Strategy is shown under the reference power domain in
the Power Pane. The following data members are populated in the
Data Pane. The data members can be added to the Wave View.

- Save Signal

- Restore Signal
- Retention Power Net
- Retention Ground Net

For more information, refer to [Figure 13-53](#) and “[Low Power Objects in the Data Pane](#)” .

Limitations of Retention Support

Following are the limitations of the state retention support:

- Part-selects are not allowed in `-elements` of `set_retention`. This is a Synopsys Low Power Flow restriction, and no tool in the flow supports part-selects in `-elements`
- No parser warnings for invalid input for `set_retention -parameters`
- Only clean-edges (0->1,1->0) are considered as active edges for save/restore signals
- Corruption for mutex failures is applied only for level-sensitive cases. No corruption for edge-sensitive

Example

```
typedef struct { reg ret_reg; reg no_ret_reg;} my_str;

my_str q_str;
reg [2:0] out, q_vec1, q_vec2;

always @(posedge clk) begin
    q_vec1      <= d;
    q_vec2      <= d;
    q_str.ret_reg <= d[0];
end
```

```
        q_str.no_ret_reg <= d[1];  
    end
```

Partial Retention -- Supported

```
set_retention RETAIN_DFF_1 -domain DFF_PD -elements  
{q_vec1}
```

```
set_retention RETAIN_DFF_2 -domain DFF_PD -elements  
{q_vec2}
```

Part/Bit-level retention for a vector -- Not supported

```
set_retention RETAIN_DFF_1 -domain DFF_PD \  
-elements {q_vec1[0] q_vec2[2:1] }
```

```
set_retention RETAIN_DFF_2 -domain DFF_PD -elements  
{q_vec1[1]}
```

-no_retention on structure member -- Not supported

```
set_retention RETAIN_DFF_1 -domain DFF_PD -elements  
{q_str}
```

```
set_retention NO_RETAIN -domain DFF_PD -no_retention \  
-elements {q_str.no_ret_reg}
```

12

Unified Low Power Messaging

This chapter describes the VCS NLP low power messaging in the following sections:

- [Message Format](#)
- [Message Control](#)
- [Wildcard Support](#)
- [User-Defined Messages in LP_MSG Format](#)
- [Assertions Related Clock/Reset Wiggle During Shutdown/ Standby](#)
- [Checking for Assertion of Isolation Enable When Source Supply is OFF](#)
- [Power-on Reset Assertions Support](#)
- [Power Aware Verification Environment](#)

- [Compile-Time UPF Parser Checks](#)

Message Format

Following is the format of the low power runtime messages that capture the power events during the simulation:

```
[time] [severity] [msg_id] <Message>
```

Example 12-1 Low Power Runtime Message

```
[0] [INFO] [LP_PD_INIT_STATE] Power domain 'testbench/inst/Island_VDD' started in 'NORMAL' state.  
[500] [INFO] [LP_PD_STATE_CHANGE] Power domain 'testbench/inst/Island_VDD' state changed from 'NORMAL' to 'CORRUPT'.
```

See “[List of Automated Low Power Messages](#)” section for the list of all low power runtime messages.

Message Control

The following two modes of message control are supported:

- [Runtime Configuration File](#)
- [Function Calls from Testbench](#)

The options specified through the runtime configuration file are used for initialization. The function calls from the testbench overrides these initial settings.

The following message control options are supported:

- Choice of message logging modes
- Turn ON/OFF all/selected messages
- Override default severity of messages
- Filter messages based on severity

Runtime Configuration File

Use the `-power <config_file>` runtime option to specify the runtime configuration file. The following topics describe the options supported in the configuration file:

- [Logging Mode](#)
- [Turn ON/OFF Messages](#)
- [Override the Default Severity of Messages](#)
- [Filter Messages Based on Severity](#)

Logging Mode

```
set_lp_msg_log_mode <DEFAULT/CUSTOM/  
DEFAULT_CUSTOM> -file {filename}
```

- **DEFAULT** — All the messages are logged into the simulation log file specified with the `-l` runtime option. Without `-l`, messages are displayed on the console.
- **CUSTOM** — All the messages are logged into the log file specified using `-file` with `set_lp_msg_log_mode`.

- **DEFAULT_CUSTOM** — All the messages are logged into the simulation log file specified with the `-l` runtime option and `-file` with `set_lp_msg_log_mode`.

Note:

`-file` is optional for **CUSTOM** and **DEFAULT_CUSTOM**. If `-file` is not specified, all messages are logged in the `vcs_lpmsg.log` file.

Example:

```
set_lp_msg_log_mode CUSTOM -file {test.log}
```

All the low power messages are dumped in the `test.log` file.

Turn ON/OFF Messages

```
set_lp_msg_onoff <ON/OFF> -msg {msgList}
```

- Default value is **ON** for all the messages.
- `-msg` is optional. Not specifying `-msg` results in the **ON/OFF** value being applied to all the messages.
- `msgList` is the list of space separated messages.

Example:

Turn OFF all the messages:

```
set_lp_msg_onoff OFF
```

Turn ON specific message ID:

```
set_lp_msg_onoff ON -msg { LP_SS_INIT}
```

Turn OFF specific message ID:

```
set_lp_msg_onoff OFF -msg { LP_ISOGN_INIT}
```

Turn OFF multiple message IDs:

```
set_lp_msg_onoff OFF -msg {LP_SS_INIT  
LP_PST_INIT_ILLEGAL LP_PSW_CTRL_INIT_INVALID}
```

Override the Default Severity of Messages

```
set_lp_msg_severity <INFO/WARNING/ERROR/FATAL> -  
msg { msgList }
```

Severity must be one of INFO/WARNING/ERROR/FATAL.

Example:

```
set_lp_msg_severity ERROR -msg {LP_SS_INIT}  
set_lp_msg_severity INFO -msg {LP_ISOEN_INVALID}  
set_lp_msg_severity WARNING -msg {LP_ISOGN_OFF }
```

Filter Messages Based on Severity

```
set_lp_msg_log_severity <INFO/WARNING/ERROR/FATAL>
```

All the messages with severity less than the specified severity value are filtered.

- INFO — Default severity level. No messages are filtered.
- WARNING — Messages with severity INFO are filtered.
- ERROR — Messages with severity INFO and WARNING are filtered.
- FATAL — Messages with severity FATAL are ON and the rest are filtered.

Example:

To turn OFF all the INFO messages:

```
set_lp_msg_log_severity WARNING
```

Function Calls from Testbench

You must import the `SNPS_LP_MSG` package before using the message control functions.

```
import SNPS_LP_MSG::*;
```

Override the Default Severity of Messages

```
set_lp_msg_severity(input string msgIds, input  
string sev);
```

- `msgids` - string type, space separated message IDs
- `sev` - string type, one of INFO/WARNING/ERROR/FATAL

Example:

```
set_lp_msg_severity(" LP_ISOGEN_INIT", "WARNING");
```

Turn ON/OFF Messages

```
set_lp_msg_onoff(input string msgIds, input string  
value);
```

- `msgids` - string type, space separated message IDs
- `value` - string type, must be either ON or OFF

Example:

```
set_lp_msg_onoff( "LP_SS_INIT", "OFF");
```

Filter Messages Based on Severity

```
set_lp_msg_log_severity(input string sev);
```

sev - string type, must be one of INFO/WARNING/ERROR/FATAL

Example:

```
set_lp_msg_log_severity("WARNING");
```

Wildcard Support

The * and ? wildcard characters are supported with the following control options to list the message IDs:

- `set_lp_msg_onoff`
- `set_lp_msg_severity`

Example

Testbench:

```
set_lp_msg_onoff("LP_ISO*", "OFF");
```

```
set_lp_msg_severity("LP_*ILLEGAL", "FATAL");
```

Configuration file:

```
set_lp_msg_onoff OFF -msg {LP_ISO*}
```

```
set_lp_msg_severity FATAL -msg {LP_*ILLEGAL}
```

User-Defined Messages in LP_MSG Format

You can choose to register the custom or user-defined low power messages in the automated assertions format to utilize the benefits of the assertion control options.

The following APIs are supported to register and print the custom messages. You must call these APIs from the testbench and import the `SNPS_LP_MSG` package.

Once the message is registered and printed using the following APIs, the low power message summary includes these custom messages.

The format of the message is same as automated assertions:

[Time] [Severity] [ID] Message

Register Custom Message

API Prototype:

```
void lp_msg_register(input string msgId, input string
severity, input string onOff, input string msgText);
```

Example:

```
lp_msg_register("LP_MY_MSG", "WARNING", "OFF", "Assertion
failure: Primary ground net \"%s\" for power domain '%s' turned
OFF.");
```

Printing the Registered Messages

API Prototype:

```
void lp_msg_print(input string msgId, input string text);
```

Example:

```
lp_msg_print("LP_MY_MSG",  
$sformatf(lp_msg_get_format("LP_MY_MSG"), "VSS", "PD1"));
```

Checking if the Registered User-Defined Message is ON/OFF

API Prototype:

```
bit lp_msg_is_enabled(input string msgId);
```

Example:

```
if(!lp_msg_is_enabled("LP_MY_MSG"))  
    set_lp_msg_onoff("LP_MY_MSG");
```

Assertions Related Clock/Reset Wiggle During Shutdown/Standby

You can use the `-power=assert_clkrsttoggle` compile-time option, as shown below, to enable assertions related clock/reset wiggle during shutdown or standby.

Use Model

```
% vcs <vcs_options> -upf <upf_file> -  
power=assert_clkrsttoggle
```

The assertion is issued for the below conditions when `-power=assert_clkrsttoggle` is specified on VCS step:

- Clock is wiggling during CORRUPT/CORRUPT_ON_ACTIVITY simstate (LP_CLOCK_WIGGLE, LP_COA_CLOCK_WIGGLE)
- Reset is wiggling during CORRUPT/CORRUPT_ON_ACTIVITY simstate (LP_RESET_WIGGLE, LP_COA_RESET_WIGGLE)

Note:

- Enabling these assertions might have simulation performance impact.
- Reset wiggle assertions are for asynchronous resets only.

Limitations

- This feature is supported only for Verilog portion of the design.

Checking for Assertion of Isolation Enable When Source Supply is OFF

You can use the `-power=src_off_clamp_check` compile-time option to enable automated check at runtime for the assertion of an isolation enable when the source supply is OFF.

The `-power=src_off_clamp_check` option generates the `LP_SRC_OFF_CLAMP` message at runtime, if the isolation enable is not active when the source supply for the clamp cell is turned off.

Below is the sample error message:

```
[ERROR] [LP_SRC_OFF_CLAMP] Isolation enable is not asserted
when input to isolation cell
'testbench.top_wr.fflop_inst.result1' is corrupted due to
source domain 'testbench/top_wr/PD_FF' being turned off.
Isolation Strategy: 'testbench/top_wr/PD_FF/isolation/
V1_ISO', Isolation Enable: 'testbench/top_wr/iso_ctrl',
```


Reference Domain: 'testbench/top_wr/PD_FF', Port:
'testbench.top_wr.fflop_inst.result1[7]'.

Key Points to Note

- This feature is supported only for the virtual isolation cells
- This feature is supported only for the isolation strategies specified with the `-source/-sink/-diff_supply_only` options

Limitations

Following are the limitations of this feature:

- Partition compile flow is not supported
- No assertion is reported when the source supply of an isolated port is a supply net specified with `set_related_supply_net`

Power-on Reset Assertions Support

You can use the `-power=assert_reset_sequence` compile-time option to enable the power-on reset assertions in VCS NLP. To enable reset assertions, you must specify `Tmax` and `Twidth` in the `set_design_attributes` command, as shown below:

```
set_design_attributes -attribute  
SNPS_active_reset_duration Twidth
```

```
set_design_attributes -attribute  
SNPS_inactive_reset_duration Tmax
```

Where,

T_{width}

Minimum time duration for which the reset must be asserted after power-on.

T_{max}

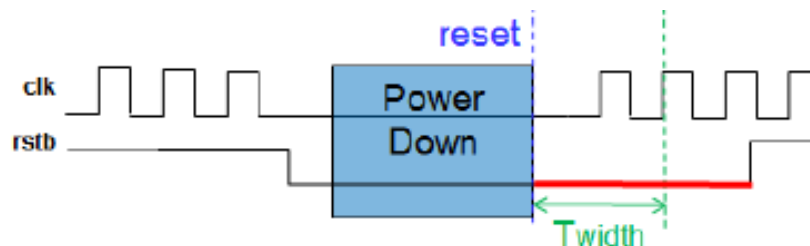
Maximum time limit after power-on within which reset must be asserted.

Example

Consider the following snippet of code:

```
always @(posedge clk or negedge rstb)
    if (!rstb)
        q <= 1'b0;
    else
        q <= dint;
```

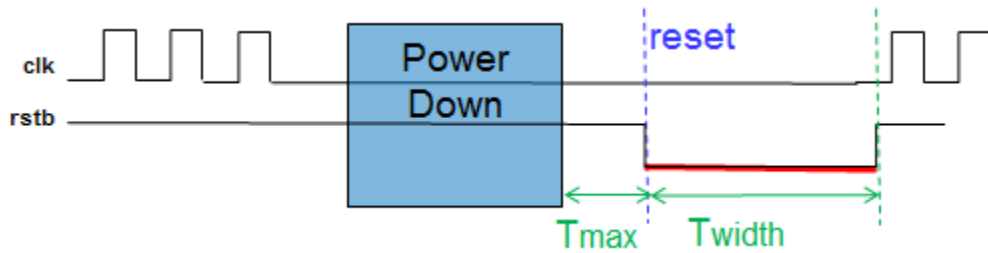
Reset Case-1



In the above figure, reset is asserted before Power Down. After power up, reset is asserted until T_{width} .

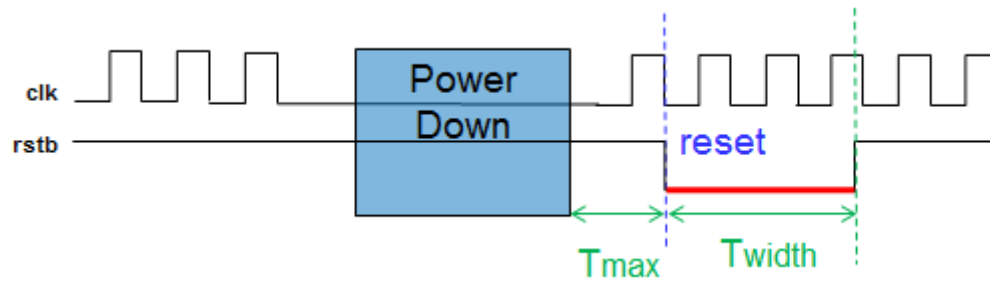
Since reset is already asserted during power up, T_{max} check is not required. However, VCS NLP checks that the reset remains asserted at least for T_{width} time.

Reset Case-2



In the above figure, reset is asserted after Power Down, and the clock is gated.

Reset Case-3



In the above figure, reset is asserted after Power Down, and the clock is not gated.

For **Reset Case-2** and **Reset Case-3**, VCS NLP checks if reset gets asserted within T_{max} time after power up or not. Once asserted, VCS NLP checks that the reset remains asserted for at least T_{width} time.

Use Model

Use the `-power=assert_reset_sequence` option at compile time, as shown below:

```
% vcs -upf <upf_file> -sverilog file_name.v -  
debug_all -power=assert_reset_sequence
```

Following is the sample content in the UPF file to set T_{max} and T_{width} :

```
//Attributes defined to set Tmax, Twidth

set_design_attributes -attribute
SNPS_inactive_reset_duration 5

set_design_attributes -attribute
SNPS_active_reset_duration 3
```

Based on the above sample UPF file:

- Reset must be asserted within 5 time units (T_{max}) after power-up
- Once reset is asserted, it must stay asserted for at least 3 time units (T_{width}) after power-up

VCS NLP generates the following assertion messages, if T_{max} and T_{width} are not specified:

- LP_RESET_ASSERT_FAIL
- LP_RESET_DEASSERT_FAIL

Following are the sample warning messages:

```
[430000 ps] [WARNING] [LP_RESET_ASSERT_FAIL] Reset has not
been asserted within '15' timeunit after power up. Instance:
'tb.u_top.FF_sub', Domain : 'tb/u_top/PD_FF', Clock : 'clk',
Reset : 'rstb'.
```

```
[295000 ps] [WARNING] [LP_RESET_DEASSERT_FAIL] Reset is de-
asserted within '20' timeunit after power up. Instance:
'tb.u_top.FF_sub', Domain : 'tb/u_top/PD_FF', Clock : 'clk',
Reset : 'rstb'.
```

Limitations

Following are the limitations of this feature:

- Only asynchronous resets are supported
- Multiple asynchronous loads are not supported:

```
always @(posedge clk, posedge set, negedge reset)
```
- `Tmax/Twidth` is applicable for the entire design
- This feature does not support waiver mechanism for assertions on instance basis

Power Aware Verification Environment

Power Aware Verification Environment (PAVE) is an infrastructure that enables accessing the UPF objects, monitor low power events, and write power-aware assertions.

PAVE uses the powerful UPF query commands to query the power intent and UPF `bind_checker` command to bind the checker modules to the UPF objects like power domains, power switch, isolation strategy, and retention strategy.

You can use the PAVE infrastructure to write custom assertions.

Note:

PAVE is not supported for pure VHDL designs.

Custom Assertions

PAVE enables writing power aware assertions combining the UPF and design variables. For the custom assertions, UPF variables can be populated with the help of UPF query commands. The `bind_checker` command helps bind these custom assertions to the appropriate target.

Custom checker module is written in SystemVerilog and passed as a regular design file for compilation. The UPF query commands and `bind_checker` are written in a custom bind file (Tcl syntax), and passed to VCS with the `-lpa_bind` compile-time option. VCS considers the last occurrence if you specify multiple `-lpa_bind` options on the same command line.

Note:

The query commands are supported only in the Tcl file passed with `-lpa_bind`. VCS NLP generates an error message, if you use query commands in the UPF file.

Use Model

The following steps describe the use model for custom assertions in detail:

Two-step Flow

Compile the user-written custom SV checker modules along with the `-sverilog` option.

Pass the custom bind file using the `-lpa_bind <custom_bind.tcl>` compile-time option.

Where, `custom_bind.tcl` is the user-written custom bind file.

Note:

- The `query_*` and `bind_checker` commands must be used only in the custom bind file and not in the UPF.
- The `query_*` and `bind_checker` commands do not fully comply with the IEEE 1801 standard.

For information on creating custom bind file, see [“Writing Custom Low Power Assertions”](#) .

Example:

```
% vcs <options> -upf <upf_file> -lpa_bind  
<custom_bind_file> -sverilog
```

Specifying File List with Multiple Bind Checker Tcl Files During Compilation

The `-lpa_bind` option accepts only one Tcl file as input. VCS NLP allows you to specify a file list listing multiple custom bind checker Tcl files using the `-lpa_bind_filelist` compile-time option.

Use Model

Following is the use model of the `-lpa_bind_filelist` compile-time option:

```
% vcs <options> -sverilog -upf <upf_file> file_name.sv  
-lpa_bind_filelist file_list
```

Where,

`file_list`

User-defined text file which contains the path to the Tcl files. The path can be absolute or relative.

Lines starting with # are treated as comments. Blank space or new line is treated as a file separator.

Environment variable usage is supported.

```
# This is a comment
file1.tcl
file2.tcl
file3.tcl file4.tcl
$PROJ_BIND_FILES/lp/file5.tcl
```

Note:

- Only one `-lpa_bind_filelist <file_list>` option must be specified at compile/elaboration. VCS considers the last occurrence in case multiple `-lpa_bind_filelist` options are specified.
- Duplicate Tcl files in the `filelist` results in the Tcl procedures in the files getting executed multiple times.

Writing Custom Low Power Assertions

Perform the following steps to write custom low power assertions in VCS NLP:

1. Create Custom Bind File

This file is written in Tcl. Do the following in the custom bind file:

- Use UPF query commands to collate the necessary low power data.

Note:

Output from the `query_upf` commands may need post-processing in Tcl to generate an appropriate set of ports and parameters that are bound to the checker module.

- Use `bind_checker` command to bind the low power objects and to pass the appropriate low power data to the checker module.
- Bind the dynamically changing data as ports (for example, save signal, power switch control signal).
- Bind the static constant data as parameters (for example, domain name, supply net name).

2. Create SV Checker Module and Register Custom Assertion

The SV checker module is written in SystemVerilog. You must code the necessary assertion based on the data passed from the custom bind file. The ports monitor the low power events, and parameters provide the static information. See [“User-Defined Messages in LP_MSG Format”](#) section for information to register custom assertion.

3. Compile the Custom Bind File and the Checker Module

Use the `-lpa_bind` compile-time option to pass the custom bind file. Specify the custom checker file as any other regular SV file for compilation.

The following section describes the above three steps in detail with examples.

Example

Scenario: It is assumed that you have implemented power gating through footer cells, and want an assertion to ensure that primary power net of any power domain is never OFF.

1. Create Custom Bind File

Collate the necessary low power data using relevant `query_upf` commands. For the requirement mentioned above, you must have the power domain name and its primary power net. You can use `query_power_domain *` and `query_power_domain <domain> -detailed` to get the required data.

Do the following in the custom bind file (`lpa_custom.tcl`):

- Use the following command to get all the power domains defined:
`query_power_domain *`
- Use the following command to iterate over each power domain and to get the domain name and primary power net:
`query_power_domain <domain> -detailed`
- Bind the dynamically changing object (primary power net) as a port.
- Bind the static constants (domain name and primary power net name) as parameters.

Consider the following example:

```
proc primary_power_off {} {  
    foreach PD [query_power_domain *] {  
        array set pd_detail [query_power_domain $PD
```

```

-detailed]
    set PD_NAME [list PD_NAME $pd_detail(domain_name)]
    set PPN_NAME [list PPN_NAME
$pd_detail(primary_power_net)]
    set PPN [list PPN $pd_detail(primary_power_net)]
    set ports_list {}
    lappend ports_list $PPN
    set params_list {}
    lappend params_list $PD_NAME $PPN_NAME
    bind_checker primary_power_off_msg \
        -module primary_power_off \
        -elements [list @$PD ] \
        -parameters $params_list \
        -ports $ports_list
    array unset pd_detail
}
}
primary_power_off

```

In the above example, checker is binded to a power domain (extension of bind_checker), therefore you must use UPF name alias @ with -elements.

Note:

- The names of power domain and the primary power net are constants. Therefore, PD_NAME and PPN_NAME are passed as parameters while binding to the checker module.
- Since the PPN (primary power net) dynamically varies during simulation, it must be bound as a port.

2. Creating SV Checker Module and Registering Custom Assertion

Following is the example code to register custom assertion:

testbench.sv

```
module testbench;
    import UPF::*;
    import SNPS_LP_MSG::*;
    ....
    initial
        lp_msg_register("LP_PPN_OFF", "ERROR", "ON",
            "Assertion failure: Primary power net '%s' for power domain
            '%s' turned OFF.");
    .....
endmodule
```

The checker module defined below checks the state of primary power net of each power domain.

custom_checker.sv

```
module primary_power_off (PPN);
    input PPN;
    import UPF::*;
    import SNPS_LP_MSG::*;
    supply_net_type PPN;
    string PPN_STATE = "";
    parameter string PD_NAME = "";
    parameter string PPN_NAME = "";

    always @(PPN.state)
        begin : my_lp_checker
            PPN_STATE = PPN.state.name;
            if (lp_msg_is_enabled("LP_PPN_OFF"))
                my_assert : assert (PPN_STATE != "OFF") else
                    lp_msg_print("LP_PPN_OFF",
```

```

$ssformatf(lp_msg_get_format("LP_PPN_OFF"), "PPN_NAME",
"PD_NAME"));
        end : my_lp_checker
endmodule

```

3. Compile the Custom Bind File and the Checker Module

Use the following command to compile the custom bind file `lpa_custom.tcl` and the checker module `custom_checker.sv`, along with other design and testbench files.

```

% vcs <options> <design_files> -upf <upf_file>
custom_checker.sv testbench.sv -lpa_bind
lpa_custom.tcl -sverilog

```

Following is the simulation output when primary power net of PD_TOP power domain goes to OFF state:

```

"custom_checker.sv", 18:
testbench.top.PD_TOP.primary_power_off_msg.my_lp_checker.check.my_assert: started at 5s failed at 5s

Offending '(PPN_STATE !== "OFF")'

[5] [ERROR] [LP_PPN_OFF] Primary power net 'testbench/top/VDD' of power domain 'testbench/top/PD_TOP' turned OFF.

```

Compile-Time UPF Parser Checks

The following sections describe the compile-time UPF parser checks:

- [Lint Check for Explicit Primary Supplies for a Domain](#)

- [Error Checks for Non-Existing Design Objects Referred in UPF](#)
- [Lint Check for Mismatch Between Verilog and Liberty Definitions of a Cell](#)

Lint Check for Explicit Primary Supplies for a Domain

The `set_domain_supply_net` and `create_power_domain -supply` commands specify the primary power and ground nets of a power domain explicitly.

As the designs get more and more complex with larger number of power domains, it is useful to identify power domains that do not have explicit specification of primary power or ground nets in the UPF.

You can use the `-power=upf_lint` compile-time option to generate a warning message for every power domain without explicit specification of power or ground nets in the UPF.

Following is the sample warning message:

```
Warning-[UPF_PDWS] Power Domain without supply
top.upf, 7
Power Domain 'testbench/TOP/PD_top' does not have any
'primary' supply specified.
Use 'create_power_domain -supply {primary <sset>}' or
'set_domain_supply_net' to add a supply to the Power Domain.
```

Error Checks for Non-Existing Design Objects Referred in UPF

VCS NLP generates the UPF_ONF and UPF_SPNF error messages at compile time when the design objects referred in the UPF are not found. Multiple scenarios are listed below with corresponding messages:

- **Case-1:** Error message for non-existing design object referred in the UPF

For example, consider the following UPF syntax:

```
create_power_domain Island_V1 -elements
{INST_COUNT_VH1 INST_COUNT_VL}
```

VCS NLP generates the following error message, if INST_COUNT_VH1 does not exist in the design:

```
Error-[UPF_ONF] Object not found
top.upf, 7
'testbench/top_u/INST_COUNT_VH1' specified with '-
elements' option of
'create_power_domain' command could not be resolved to
any valid design element.
No instance found with name 'INST_COUNT_VH1' in scope
'testbench/top_u'
(Entity: 'TOP', top.vhd, 17).
Please specify a valid design object.
```

- **Case-2:** Error message for non-existing object specified with connect_supply_net

For example, consider the following UPF syntax:

```
connect_supply_net VDD1 -ports {VDD1 CORE2/VDD1  
CORE3/CORE22/VDD11}
```

VCS NLP generates the following error message, if CORE3 does not exist in the design:

```
Error-[UPF_ONF] Object not found  
top.upf, 39  
'tb/TOP/CORE3/CORE22/VDD11' specified with '-ports'  
option of 'connect_supply_net' command could not be  
resolved to any valid design element.  
No element found with name 'CORE3' in scope 'tb/TOP'  
(Module: 'top', top.v, 1).  
Unresolved Path: 'CORE3/CORE22/VDD11'  
Please specify a valid design object.
```

- **Case-3:** Error message for non-existing supply port

For example, consider the following UPF syntax:

```
connect_supply_net VDD1 -ports {VDD1 CORE2/VDD1  
CORE2/VCC11}
```

VCS NLP generates the following error message, if VCC11 does not exist in the design:

```
Error-[UPF_SPNF] Supply Port not found  
top.upf, 39  
'tb/TOP/CORE2/VCC11' specified with '-ports' option of  
'connect_supply_net'  
command could not be resolved to any valid design element.  
No Supply Port found with name 'VCC11' in scope 'tb/TOP/  
CORE2' (Module: 'core2', module2.v, 1).  
Please specify a valid design object.
```

Lint Check for Mismatch Between Verilog and Liberty Definitions of a Cell

You can use the `+lint=MV_DB_CELL_MISMATCH` compile-time option to generate a warning message when there is a mismatch between the Verilog definition and liberty definition of a cell.

Following is the sample warning message:

```
Lint- [MV_DB_CELL_MISMATCH] Cell and verilog module mismatch
Library cell for verilog module 'foomem' in library
'MACRO1_LIB' does not match.
Reason: Unmatched Port in the Model : temp.
```

You can use the `-error=MV_DB_CELL_MISMATCH` compile-time option to convert the `MV_DB_CELL_MISMATCH` warning message to error message.

Following is the sample error message:

```
Error- [MV_DB_CELL_MISMATCH] Cell and verilog module mismatch
Library cell for verilog module 'foomem' in library
'MACRO1_LIB' does not match.
Reason: Unmatched Port in the Model : temp.
```


13

Debugging Low Power Designs Using DVE

You can use DVE to debug your low power simulations. DVE allows you to visualize, trace, and analyze the source of low power events.

It simplifies low power debug by allowing you to visualize the power intent (UPF), and supports features to determine whether unexpected design behavior is caused by functional logic or a power event. Low power debug in DVE is available for all UPF-based low power simulations.

This chapter describes debugging low power designs using DVE in the following sections:

- [Disabling the Dumping of Low Power Data to VPD](#)
- [Low Power Debug Features](#)
- [Limitations](#)

Disabling the Dumping of Low Power Data to VPD

For the low power designs, when the VPD dumping is enabled, VCS dumps the low power data into the VPD file by default. You can use the following methods to disable the dumping of the low power data into the VPD file:

- Call the `$vcdpluspowerenableoff` system task in the design
- From UCLI using `dump -power off`
- From UCLI using `call {$vcdpluspowerenableoff}`

Note:

You must call `dump -power off` **before any** `dump -add` command. Similarly, `$vcdpluspowerenableoff` **must be** called before any other `$vcdpluson` or `$vcdplusoff` calls.

Low Power Debug Features

This section describes the following features supported in DVE to debug low power designs:

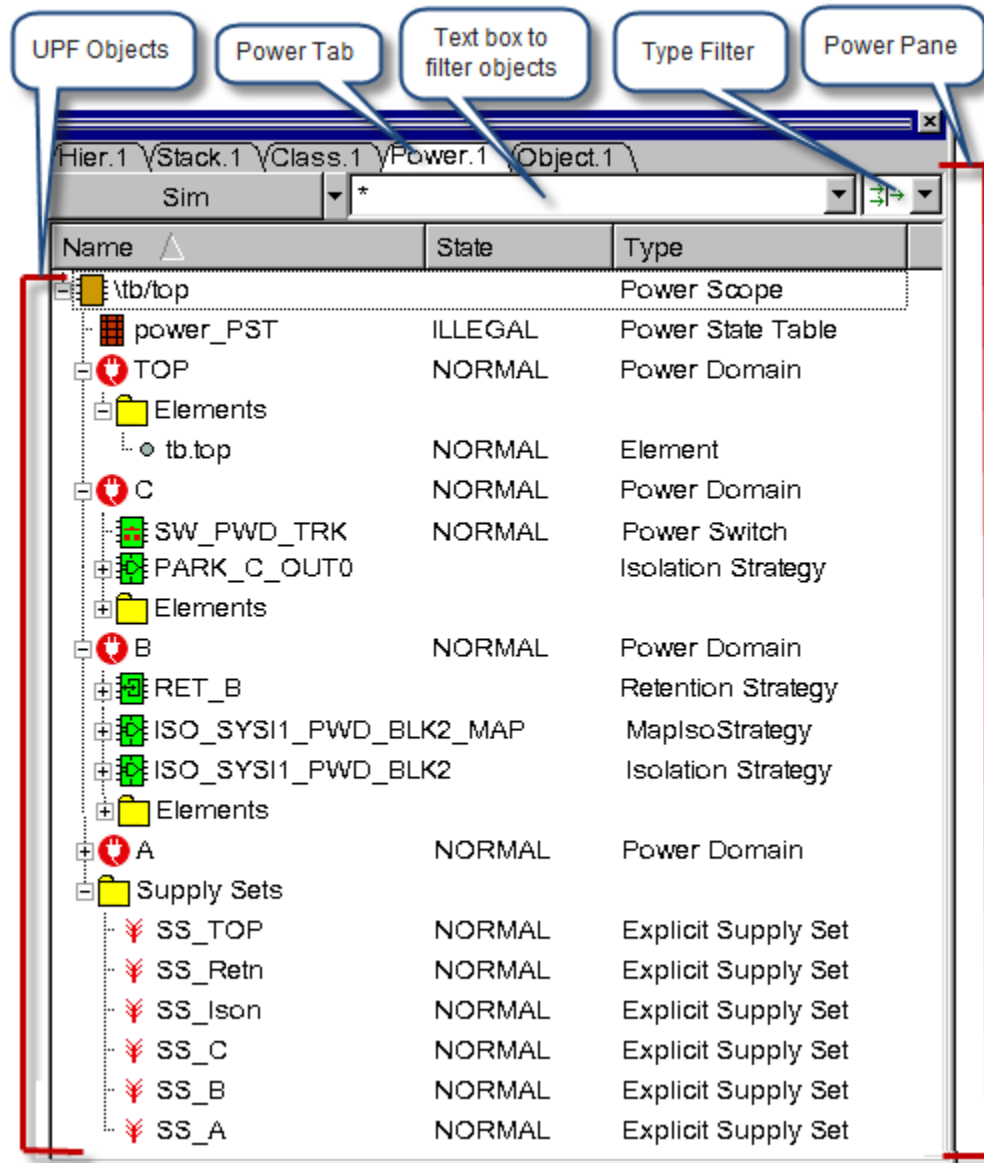
- [Power Pane](#)
- [Low Power Objects in Hierarchy Pane](#)
- [Marking for Low Power Signals in the Data Pane](#)
- [UPF Source View](#)
- [Waveform Shading](#)
- [Power State Table](#)

- [Shading in the Schematic View](#)
- [Drivers/Loads Pane](#)
- [Support for Active Drivers on Low Power Design Elements](#)
- [Viewing the DB Information of Cells](#)
- [Low Power Query Commands](#)

Power Pane

The Power Pane displays the objects defined in the UPF file. Select the Power tab to display the Power Pane and view the low power objects. [Figure 13-1](#) shows the Power Pane.

Figure 13-1 Power Pane



The Power Pane, shown in [Figure 13-1](#), is a tree view composed of the following columns:

- **Name:** Displays the name of a low power object
- **State:** Displays the state of the power domain. The value is updated during the simulation
- **Type:** Displays the type of a low power object

You can detach and reattach the Power tab at any time during your session. To detach the Power tab, toggle the **Power** off by selecting **Window > Panes > Power**. [Table 13-1](#) describes the low power objects in the Power Pane:

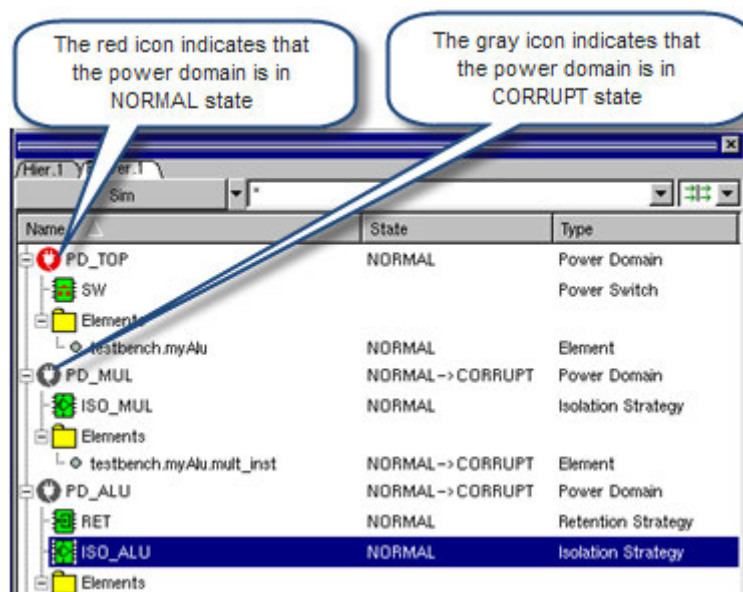
Table 13-1 Low Power Objects in the Power Pane

Object Name	Icon	Type
Power Domain		Design Instances
Power Switch		Power Switch
Isolation Strategy, Map Isolation Strategy		Design Signals/Instances
Retention Strategy		Design Signals/Instances

Level Shifter Strategy		Design Signals/Instances
Supply Set		Explicit Supply Set, Implicit Supply Set
Power State Table		Power State Table
Supply Sets		Explicit Supply Set
Elements List under Power Domain, Elements List under Strategy		Module/Entity

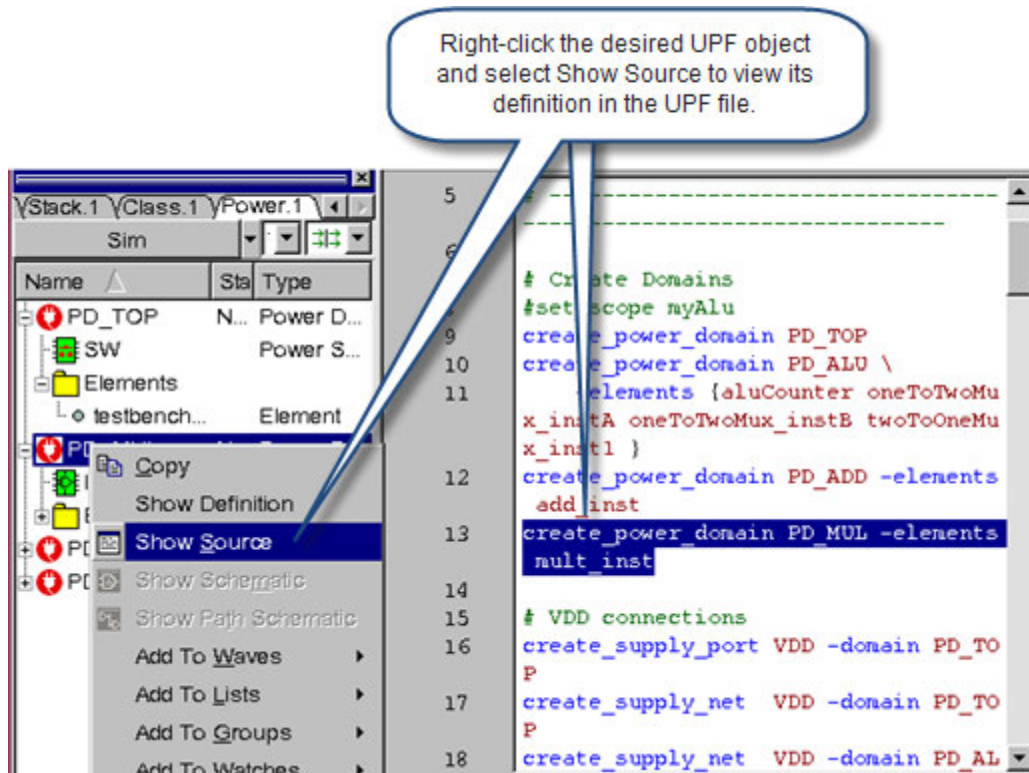
The CORRUPT power domain state is represented as a gray icon.
 Figure 13-2 shows the CORRUPT and NORMAL power domain states.

Figure 13-2 Power Domain State



To view the source of an UPF object in the Source View, right-click the desired UPF object in the Power Pane and select **Show Source**, as shown in [Figure 13-3](#). This option points to the definition in the UPF file.

Figure 13-3 Viewing the Source of an UPF object



Similarly, to switch to scope of an UPF object, right-click the desired UPF object in the Power Pane and select **Show Definition**. The scope where the UPF object is defined is shown in the Hierarchy Pane. For more information on Hierarchy Pane, see [“Low Power Objects in Hierarchy Pane”](#).

Filtering the Objects in the Power Pane

You can filter object types such as Isolation Strategy, Retention Strategy, Level Shifter Strategy, Power Switch, PST, Explicit Supply Set, and Implicit Supply Set in the Power Pane, based on the object types mentioned in [Table 13-1](#).

To filter the data, based on object types, click Type Filter and select or clear the desired object types, as shown in [Figure 13-4](#). By default, all object types except Implicit Supply Set, are selected.

Figure 13-4 Type Filter list in the Power Pane

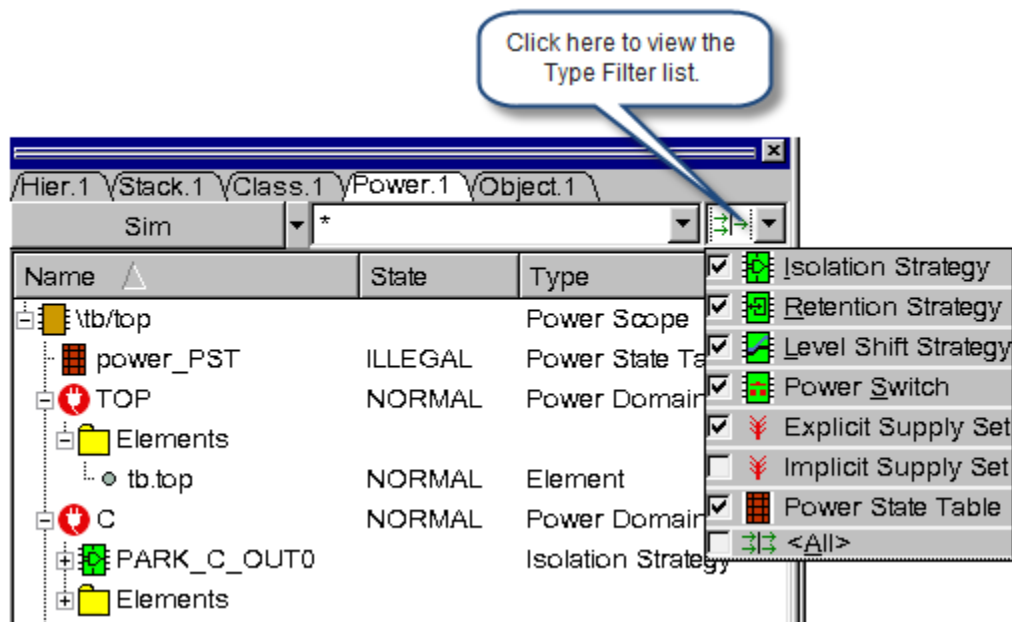


Table 13-2 Mouse Actions in Power Pane

Mouse Action	Command Operations
Hover the mouse cursor on a low power object	Displays tooltip with relevant power domain information. Tooltip shows the name of the object, power domain, and power domain state.

Viewing Supply Sets in the Power Pane

DVE allows you to view supply sets in the Power Pane. A supply set relates multiple supply nets as a complete power source for one or more design elements, as shown below:

```
create_supply_set aohigh \  
  -function {power VDD} \  
  -function {ground VSS}
```

DVE allows you to perform the following:

1. View implicit and explicit supply sets in the Power Pane, as shown in [Figure 13-5](#).
 - View implicit supply sets under UPF objects like power domain, isolation strategy, retention strategy, and so on, as shown in [Figure 13-5](#). These supply sets are not displayed by default. You must select the **Implicit Supply Set** filter option, as shown in [Figure 13-6](#), to view them in the Power Pane.
 - View explicit supply sets under the **Supply Sets** folder, as shown in [Figure 13-5](#). These supply sets are displayed by default in the Power Pane. You can use the **Explicit Supply Set** filter option, as shown in [Figure 13-6](#), to view or hide explicit supply sets in the Power Pane.
2. Select a supply set in the Power Pane to view its functions in the Data Pane, as shown in [Figure 13-7](#). The Data Pane displays the function name in the **Type** column, as shown in [Table 13-3](#).

Table 13-3 Viewing Function Name of a Supply Net in the Data Pane

Function Name	Type
power	Power Function
ground	Ground Function
nwell	NWell Function
pwell	PWell Function
deepnwell	Deep NWell Function
deeppwell	Deep PWell Function

Figure 13-5 Implicit and Explicit Supply Sets in the Power Pane

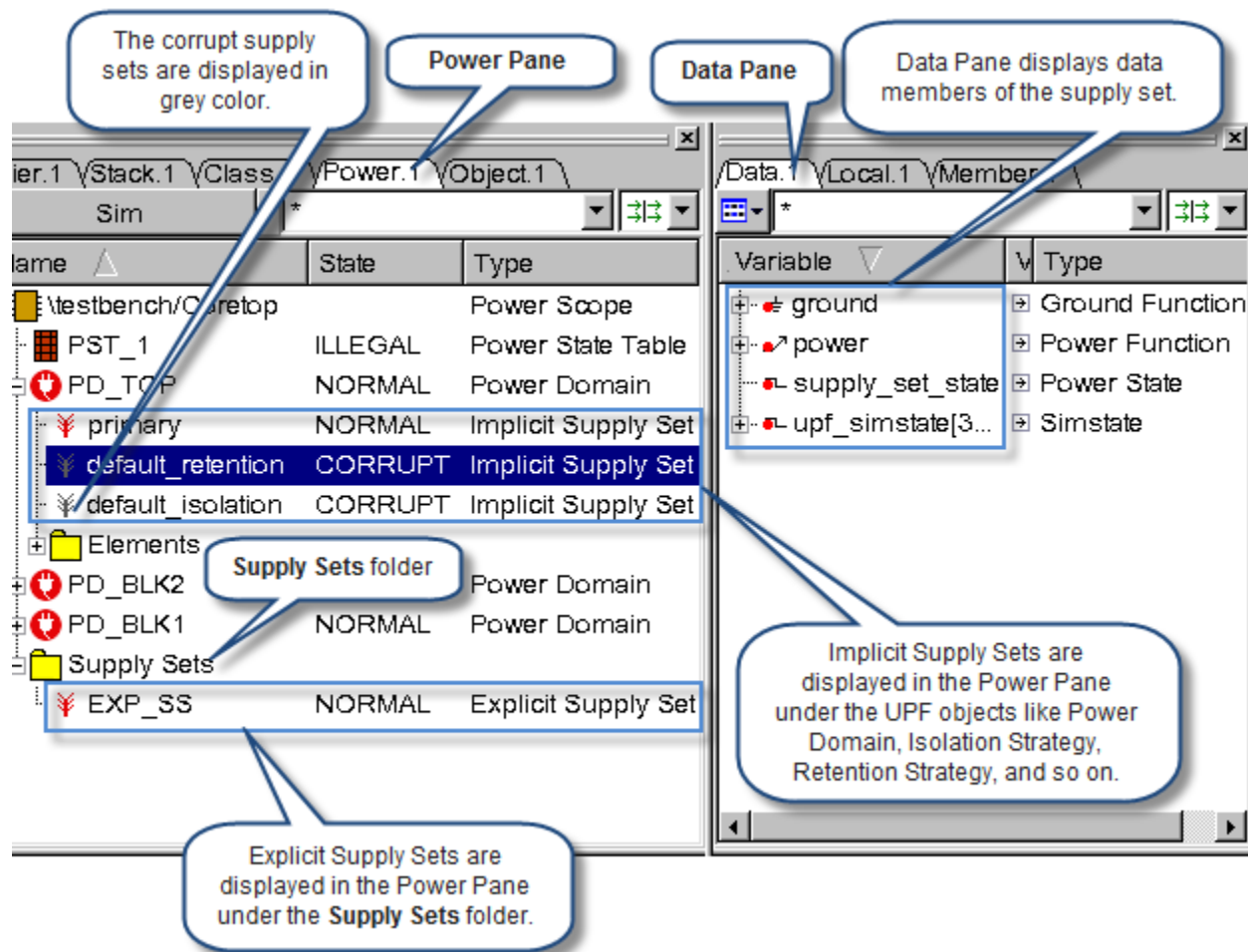


Figure 13-6 Implicit and Explicit Supply Set Filter Options in Power Pane

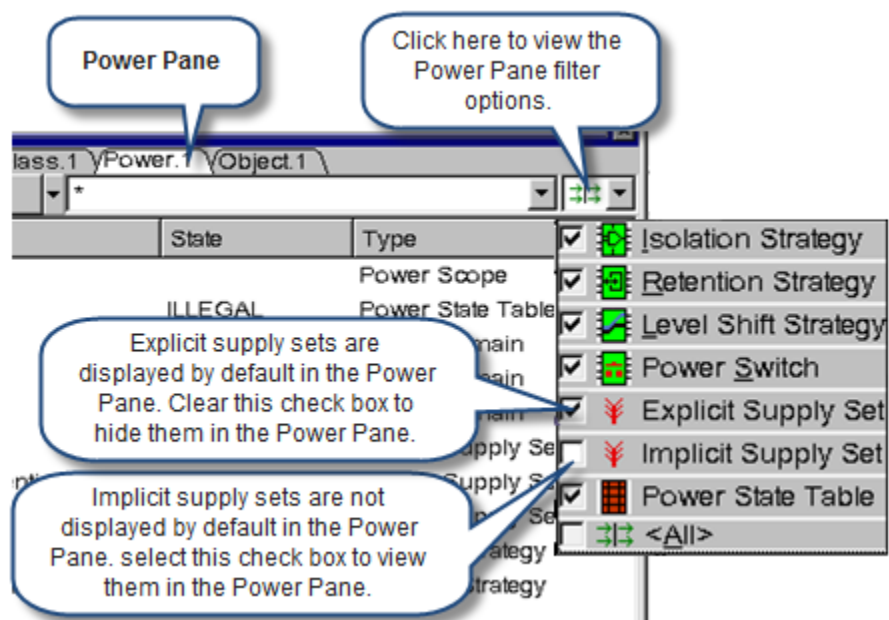
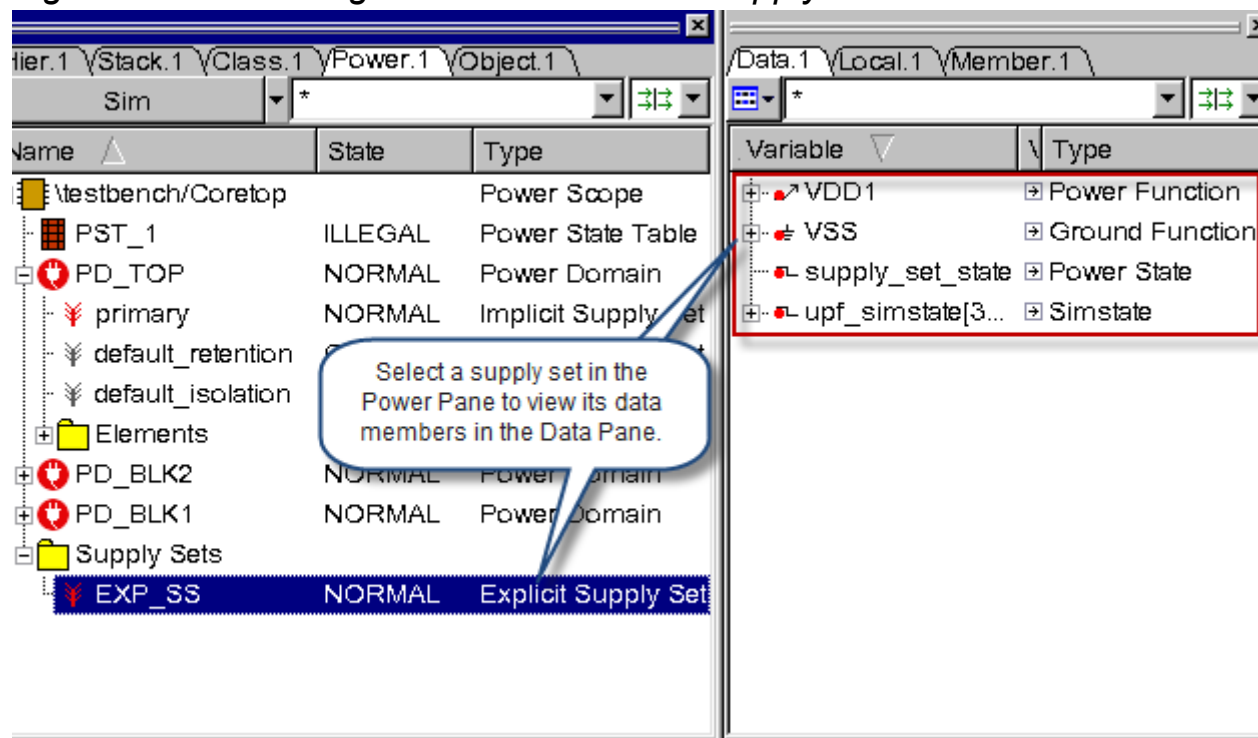


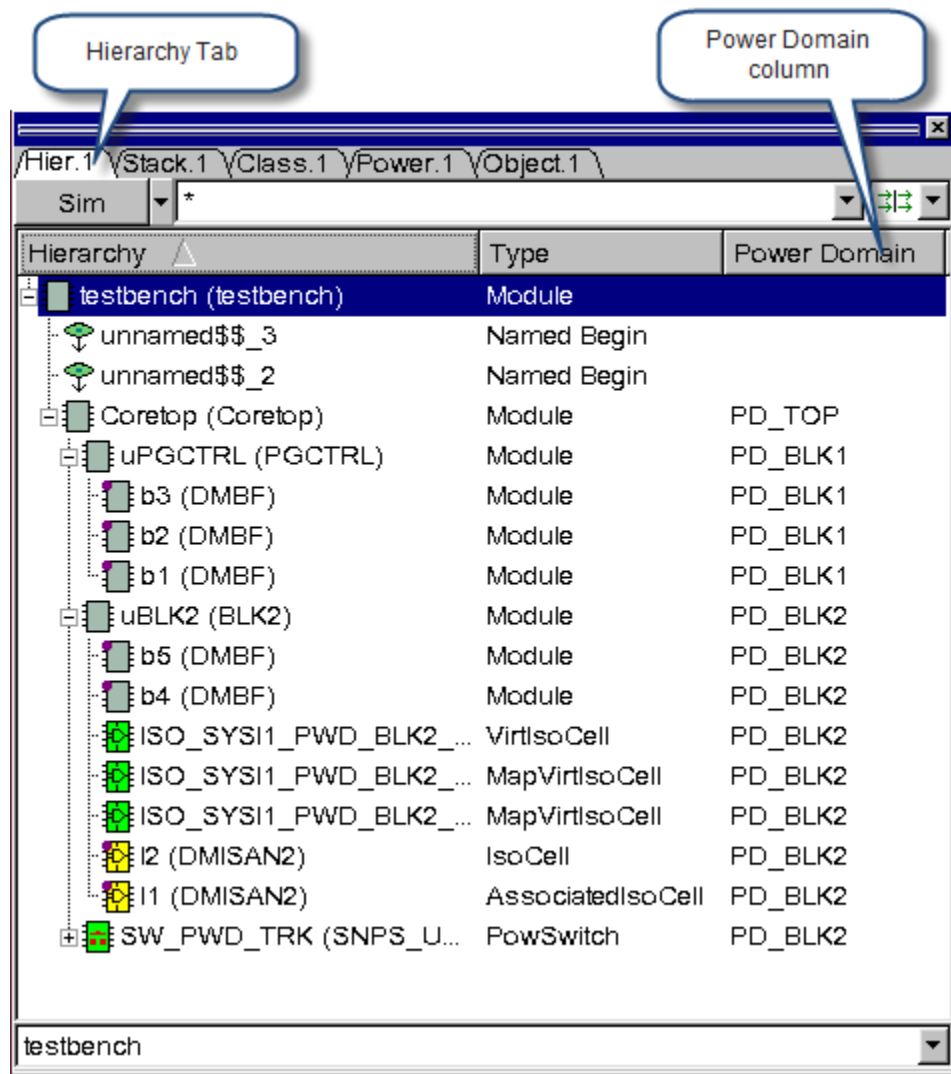
Figure 13-7 Viewing Data Members of a Supply Set in the Data Pane



Low Power Objects in Hierarchy Pane

The **Power Domain** column in the Hierarchy Pane displays the corresponding power domain of the design instance. [Figure 13-8](#) shows the **Power Domain** column.

Figure 13-8 Power Domain Column



The Hierarchy Pane contains the following low power object types:

Table 13-4 Low Power Object Types in the Hierarchy Pane

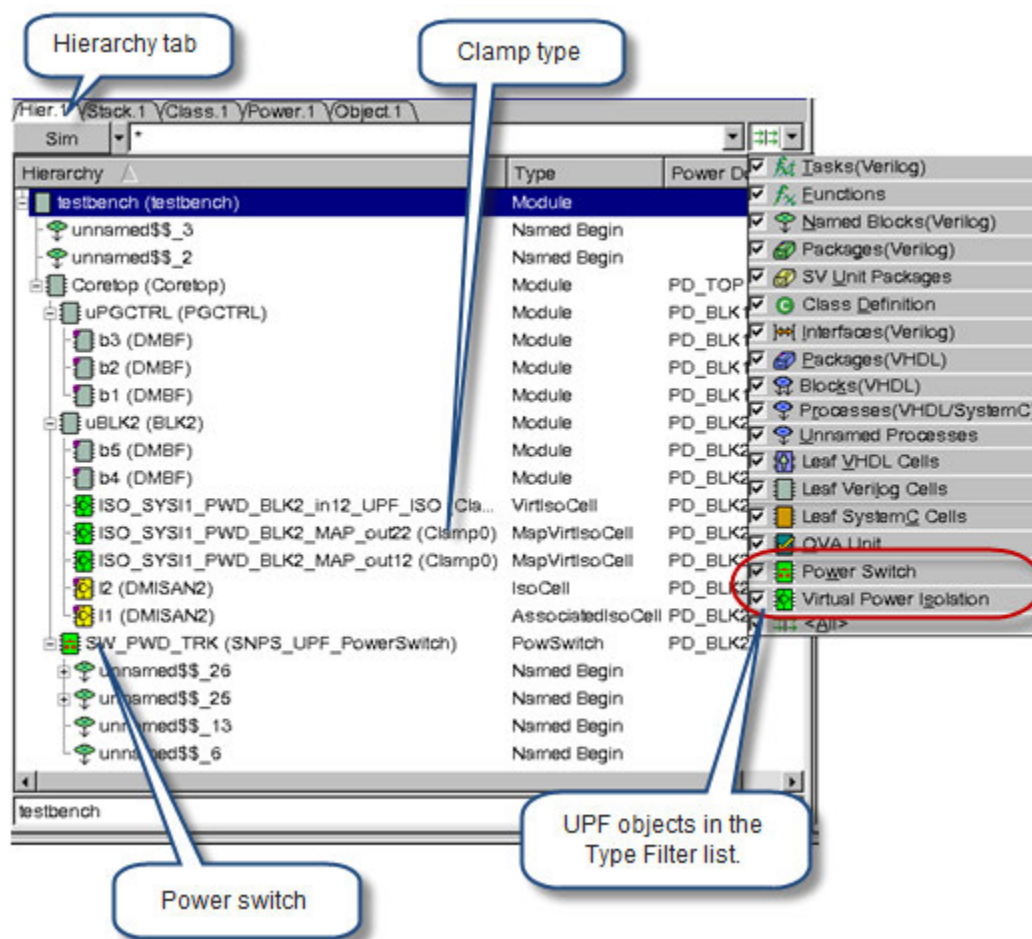
Object Name	Icon	Type
Power Switch		PowSwitch
Isolation Cell		IsoCell
Associated Isolation Cell		AssociatedIsoCell
Virtual Isolation Cell		VirtIsoCell
Map Virtual Isolation Cell		MapVirtIsoCell

The Power Switch and Virtual Isolation Cell object types mentioned in the above table are also available in the Type Filter list, as shown in [Figure 13-9](#).

Note:

The Power Switch and Virtual Power Isolation object types are OFF by default.

Figure 13-9 Low Power Objects in the Hierarchy Pane



DVE allows you to view the corresponding power domain of a design instance in the Hierarchy Pane.

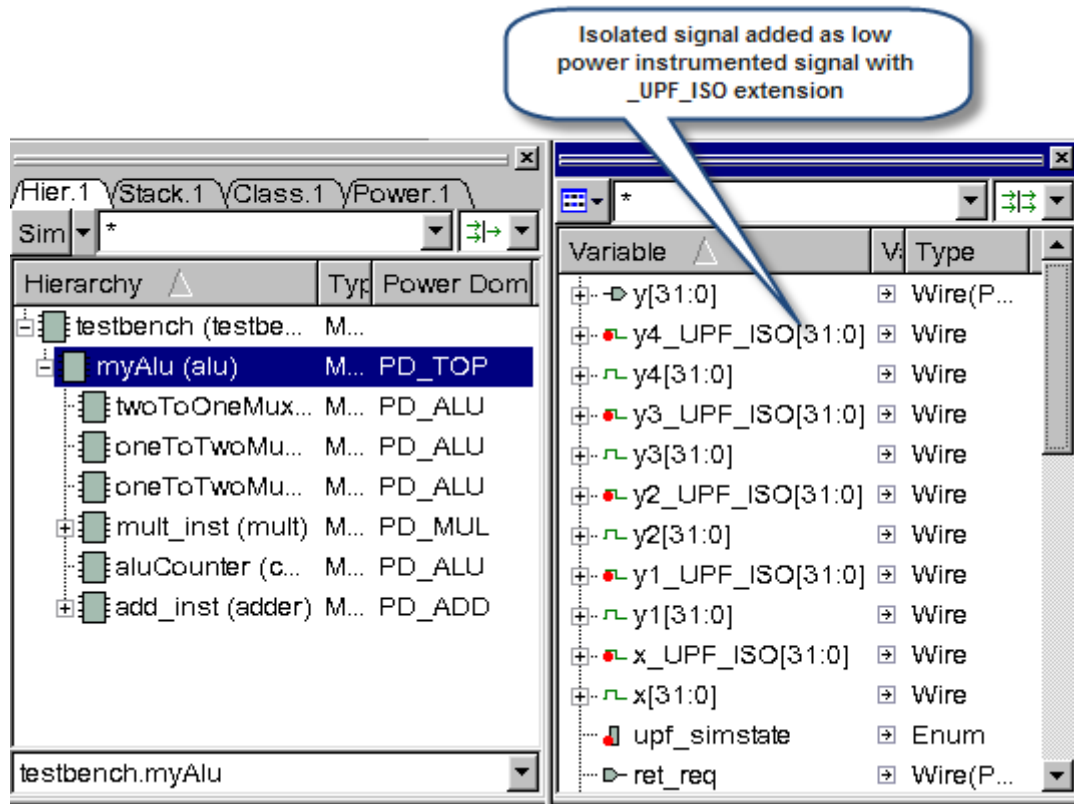
To view the corresponding power domain of a design instance, right-click the desired design instance in the Hierarchy Pane and select **Show Power Domain**. The control shifts to the corresponding power domain in the Power Pane.

DVE allows you to view the source of an UPF object in the Source View. To view the source of an UPF object in the Source View, right-click the desired UPF object in the Hierarchy Pane and select **Show Source**. This option points to the UPF object definition in the UPF file.

Isolated Signals

You can view the isolated signals in an instance in the Data Pane with `_UPF_ISO` extension, as shown in [Figure 13-10](#).

Figure 13-10 Isolated Signals



Shadow Registers

You can view the shadow registers of an instance in the Data Pane with `save_____` or `save__` prefix, as shown in [Figure 13-11](#).

Figure 13-11 Shadow Registers

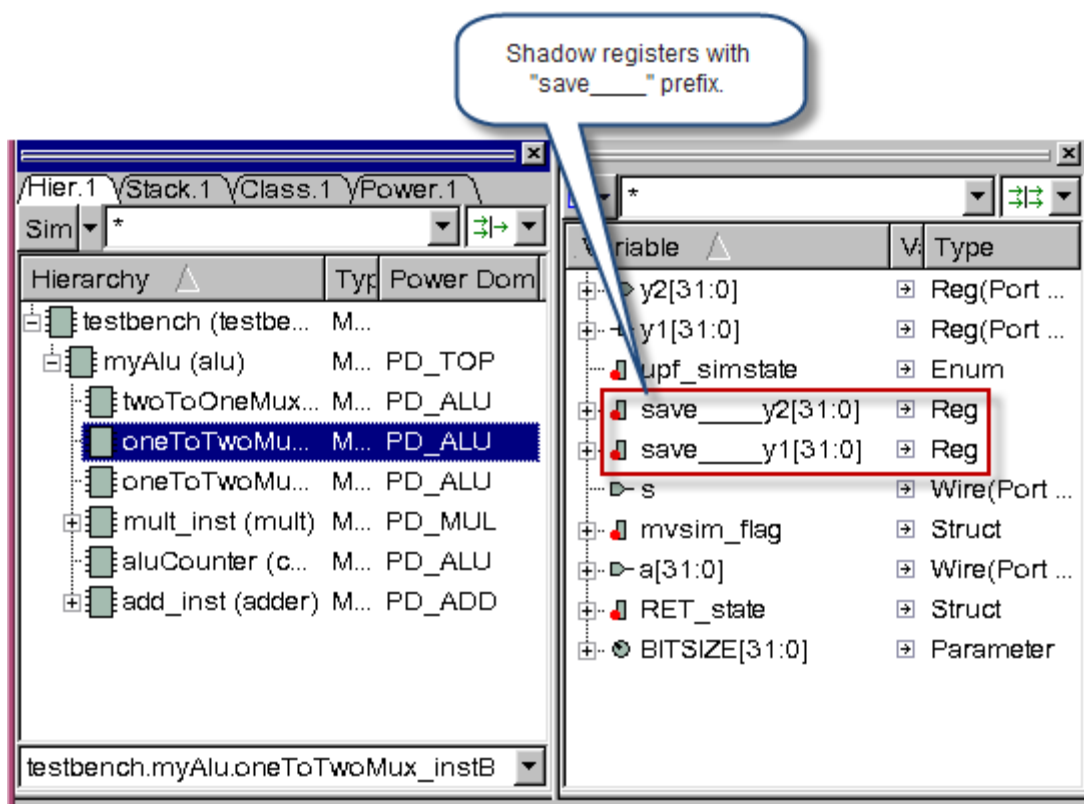


Table 13-5 Mouse Actions in Hierarchy Pane

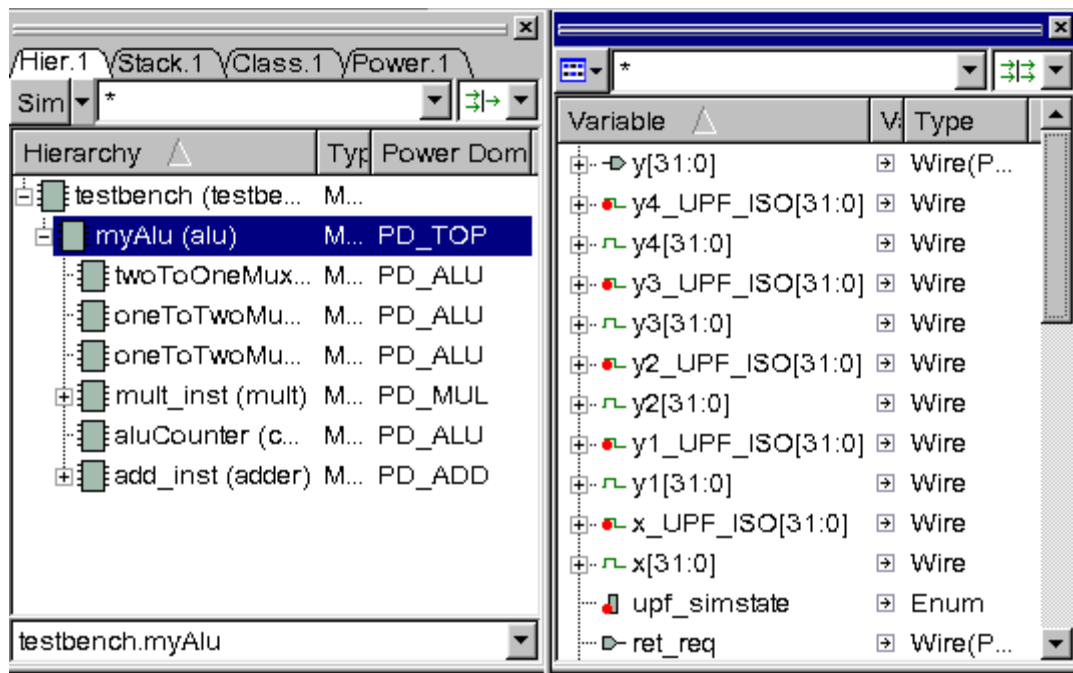
Mouse Action	Command Operations
Hover the mouse cursor on a low power object	Displays tooltip with relevant power domain information. Tooltip shows the name of the object, power domain, and power domain state.

Marking for Low Power Signals in the Data Pane

The Data Pane displays the values of signals and variables of the corresponding instance selected in the Hierarchy Pane. If you select an instance in the Hierarchy Pane, the Data Pane displays the variables of the selected instance. Similarly, Data Pane is populated with the necessary data for the objects in the Power Pane.

All the low power signals (supply nets, upf_simstate, isolated signals, shadow registers) are marked with a red dot, as shown in the following figure, for clear identification.

Figure 13-12 Marking for Low Power Signals in the Data Pane



Viewing mvsim_flag Variable

The `mvsim_flag` variable is an internal variable populated in each power-managed instance, as shown in [Figure 13-13](#). This variable is not populated for the DB cells.

Figure 13-13 Viewing *mvsim_flag* Variable

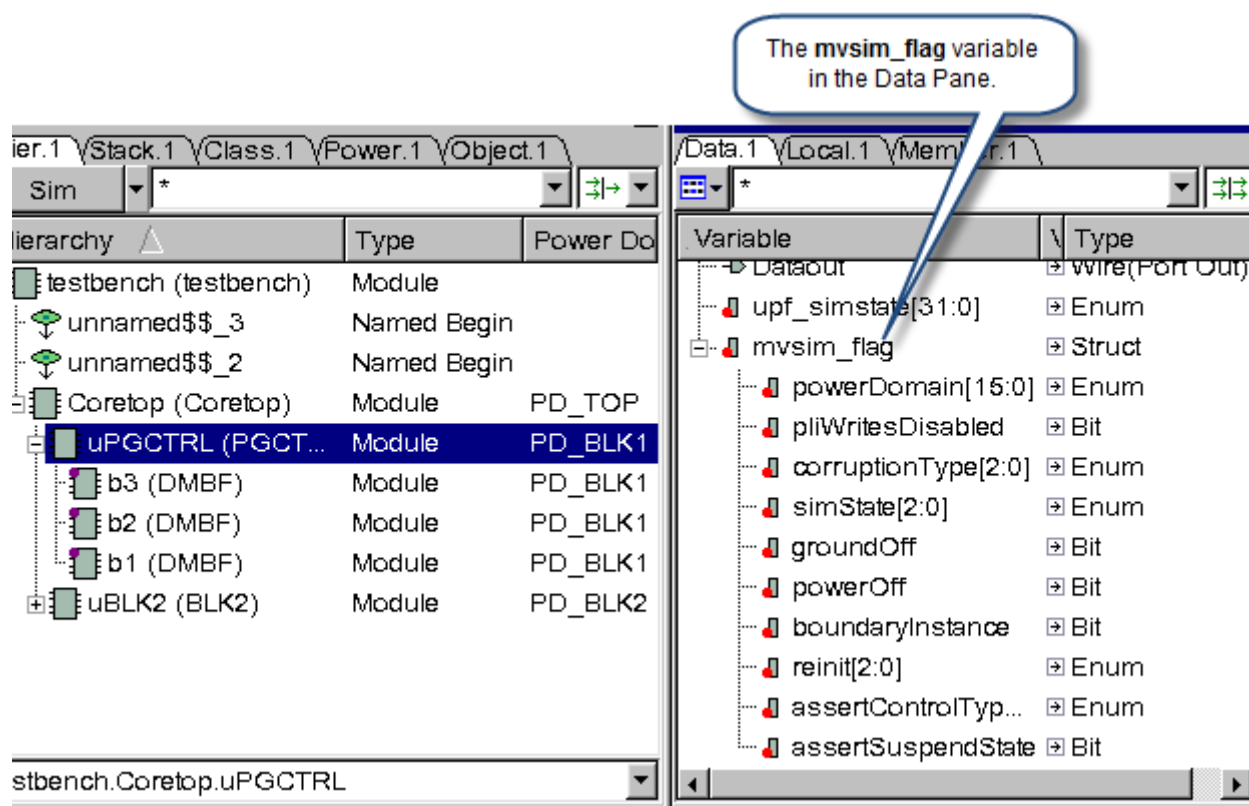


Table 13-6 lists the low power objects of the *mvsim_flag* variable.

Table 13-6 Low Power Objects of the *mvsim_flag* Variable

Variable	Description
powerDomain	Name of the power domain.
pliWritesDisabled	1'b0 if PLI writes are enabled on this instance at a given time. VCS NLP disables them for the regions that are powered OFF.
corruptionType	Corruption type (0, 1, X, Z).
simState	Same as <i>upf_simstate</i> , reflects the simstate of the domain.

Variable	Description
groundOff	State of the primary ground net.
powerOff	State of the primary power net.
boundaryInstance	1'b1 if the instance is a domain boundary instance.
reinit	1'b1 if initial blocks have been marked to be re-initialized on wakeup for this instance.
assert*	Reflects the status of various assertion control functions.

Low Power Objects in the Data Pane

Table 13-7 describes the low power objects in the Data Pane:

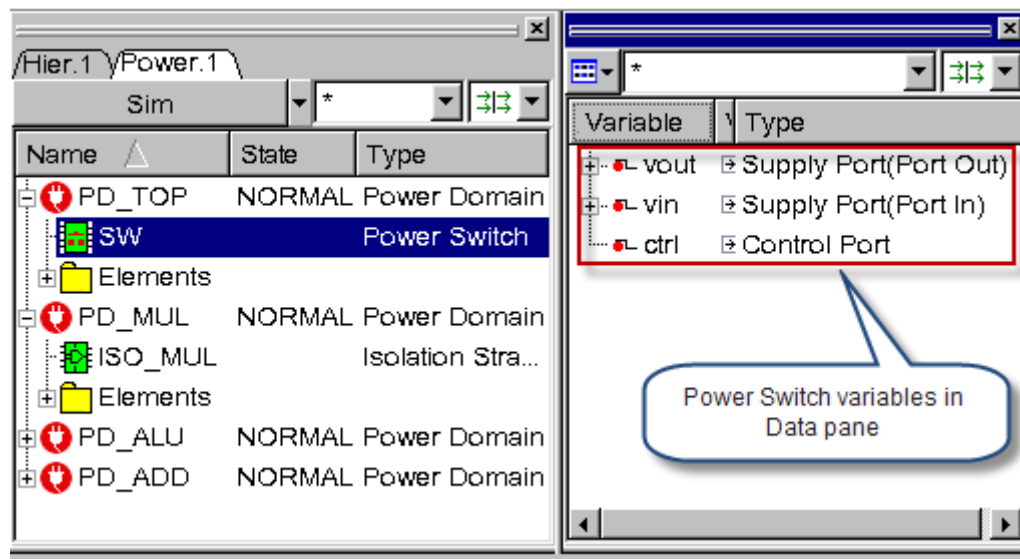
Table 13-7 Low Power Objects in the Data Pane

Icon	Used by these objects	Type
	Primary Net, Isolation Power Net, Primary Power Net, Retention Power Net	Power Net
	Ground Net, Isolation Ground Net, Primary Ground Net, Retention Ground Net	Ground Net
	Isolation Enable Signal	Logic
	Save Signal	Save Signal

	Restore Signal	Restore Signal
	Control Signal	Control Port
	All Input Supply Nets	Input Supply Port
	All Output Supply Nets	Output Supply Port
	Low Power Instrumented Variable	Design or Instrumented Variable

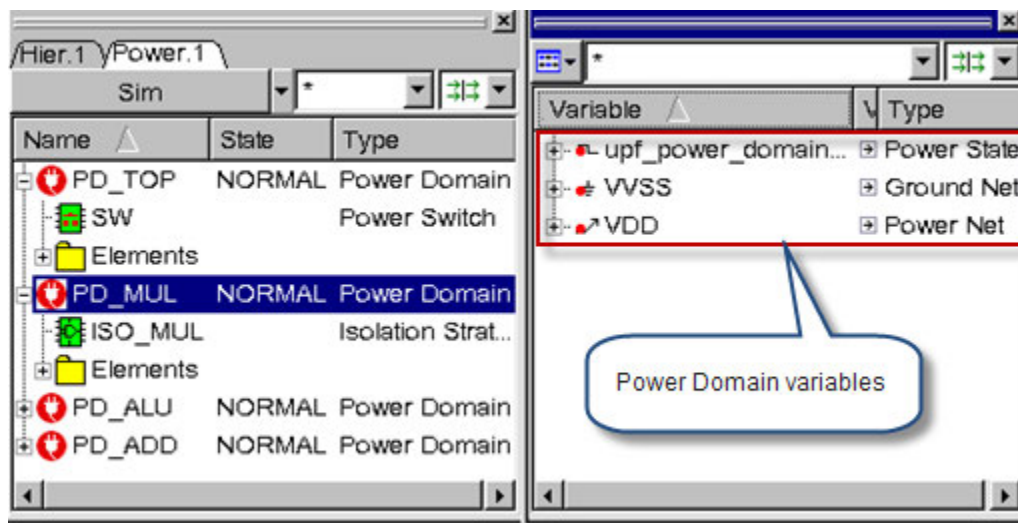
If you select a Power Switch in the Power tab, the Data Pane displays Input Supply Port, Output Supply Port, and Control Port of the Power Switch, as shown in [Figure 13-14](#).

Figure 13-14 Power Switch



If you select a Power Domain in the Power tab, then the Data Pane displays Power Net, Ground Net, and the simstate of the Power Domain, as shown in [Figure 13-15](#).

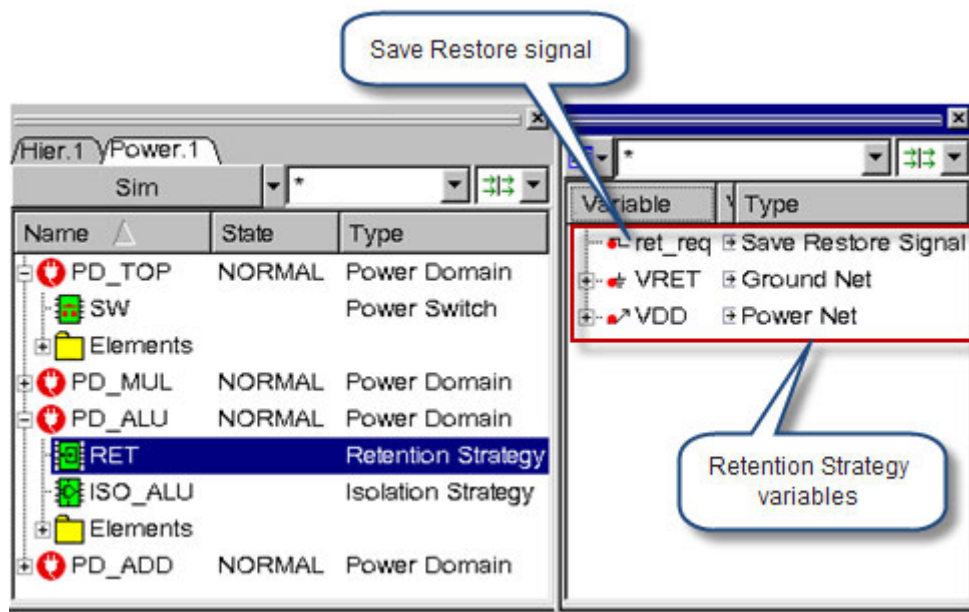
Figure 13-15 Power Domain



If you select Retention Strategy in the Power tab, the Data Pane displays the Retention Power Net, Retention Ground Net, and Save and Restore signals of the Retention Strategy, as shown in [Figure 13-16](#).

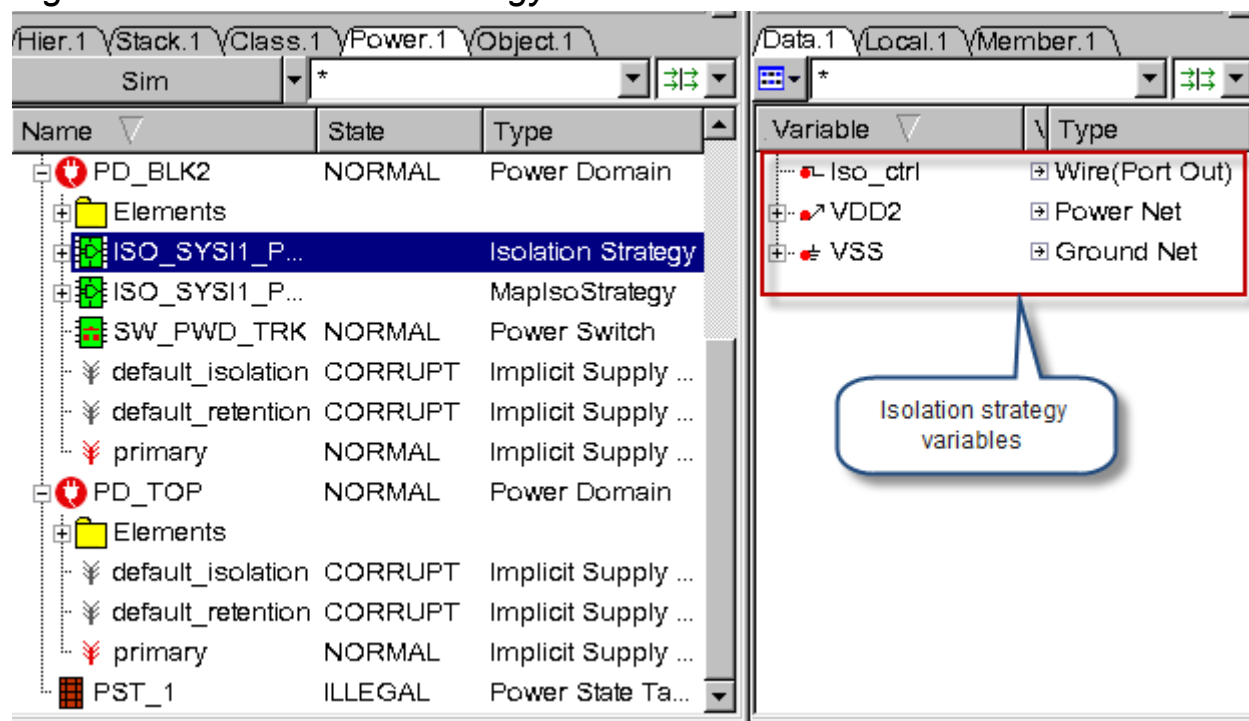
If you use same signal as both Save and Restore signals for the retention policy, the type of that signal is displayed as **Save Restore Signal** in Data Pane, as shown in [Figure 13-16](#).

Figure 13-16 Retention Strategy



Similarly, if you select Isolation Strategy in the Power tab, the Data Pane displays its variables as shown in [Figure 13-17](#).

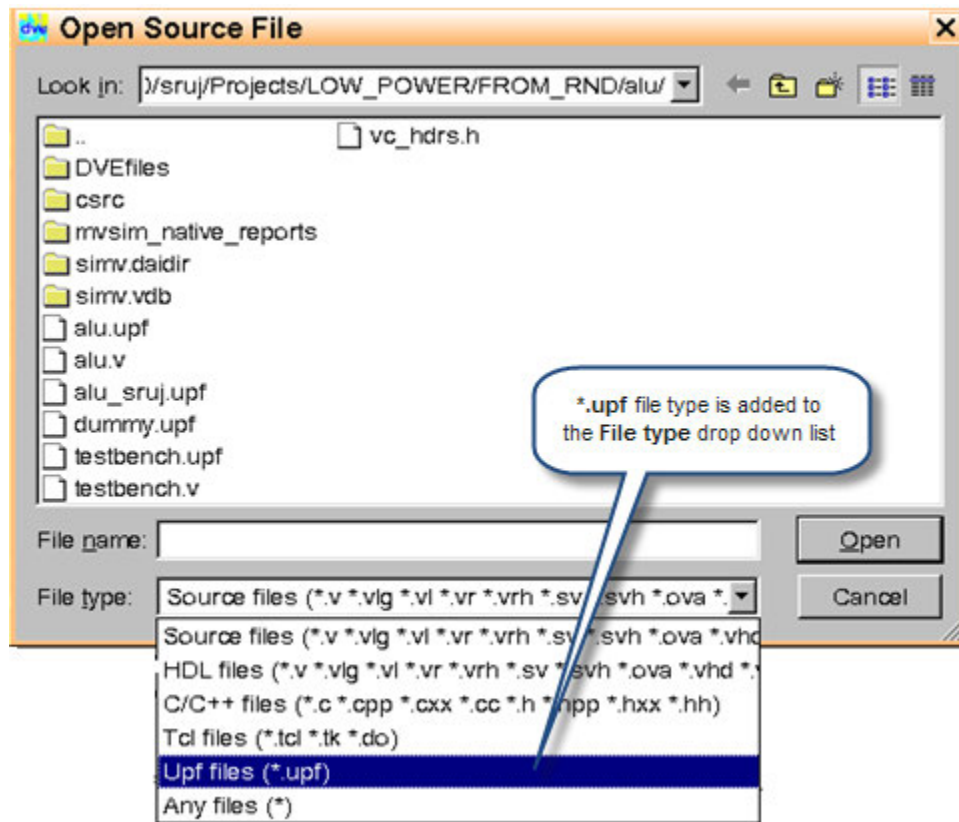
Figure 13-17 Isolation Strategy



UPF Source View

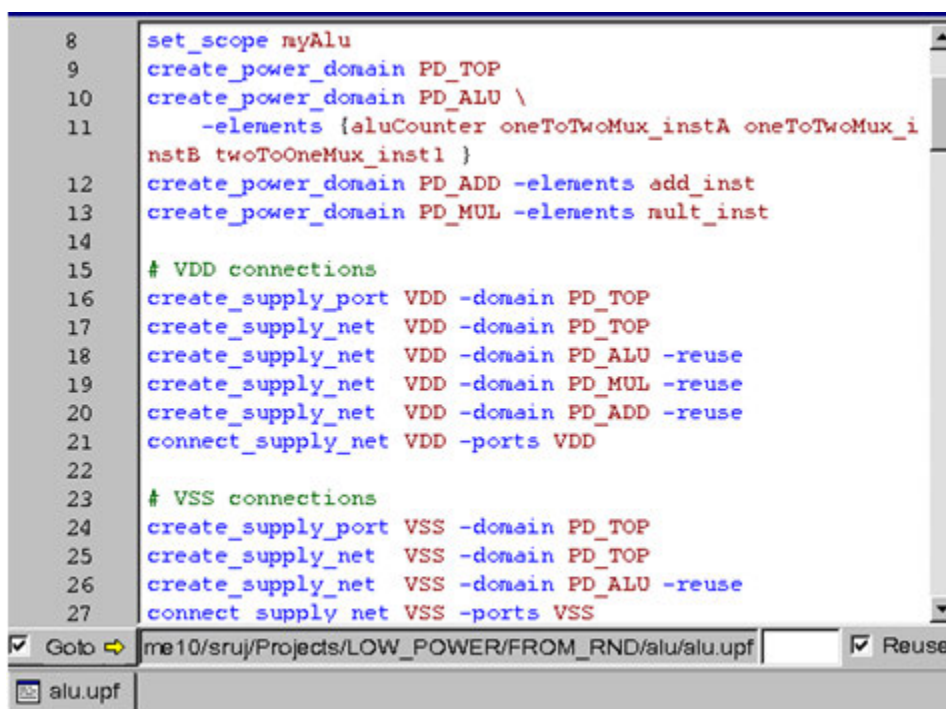
The Upf files (*.upf) file type in the **Open Source File** dialog box, as shown in [Figure 13-18](#), allows you to open UPF files in the Source View.

Figure 13-18 Opening UPF Files in Source Window



DVE supports syntax highlighting/coloring of UPF keywords in *.upf files, as shown in [Figure 13-19](#).

Figure 13-19 Syntax Highlighting of UPF Keywords



```
8  set_scope myAlu
9  create_power_domain PD_TOP
10 create_power_domain PD_ALU \
11     -elements {aluCounter oneToTwoMux_instA oneToTwoMux_i
12     nstB twoToOneMux_inst1 }
13 create_power_domain PD_ADD -elements add_inst
14 create_power_domain PD_MUL -elements mult_inst
15
16 # VDD connections
17 create_supply_port VDD -domain PD_TOP
18 create_supply_net VDD -domain PD_TOP
19 create_supply_net VDD -domain PD_ALU -reuse
20 create_supply_net VDD -domain PD_MUL -reuse
21 create_supply_net VDD -domain PD_ADD -reuse
22 connect_supply_net VDD -ports VDD
23
24 # VSS connections
25 create_supply_port VSS -domain PD_TOP
26 create_supply_net VSS -domain PD_TOP
27 create_supply_net VSS -domain PD_ALU -reuse
28 connect_supply_net VSS -ports VSS
```

me10/sruj/Projects/LOW_POWER/FROM_RND/alu/alu.upf

alu.upf

Using UPF Source Code in the Source View

DVE supports the following operations on the UPF source code in the Source View:

- View the load_upf files by clicking the file name (see [Figure 13-20](#)) or by expanding it in the Source View, as shown in [Figure 13-21](#).

Note:

To view the `load_upf` files in the expandable format, you must enable the **Enable `include file expansion in source code** option in the **Source View** category of the DVE Application Preferences dialog box.

- View value annotation on UPF/HDL objects, as shown in [Figure 13-23](#).
- Add to waveform from the UPF source
- Trace drivers/loads from the UPF source
- View Path Schematic/Schematic from the UPF source
- Cross-link between the Hierarchy Pane/Data Pane and the UPF Source View
- Drag-and-drop functionality

The above operations are supported for the following UPF/HDL objects:

- HDL design scopes like Verilog instances and VHDL entities
- HDL variables (including Verilog and VHDL variables)
- UPF objects that include Power domain, PST, Supply net, Isolation, Retention, Level Shifter and Power Switch strategy

Figure 13-20 Opening the load_upf Files



Figure 13-21 Viewing a load_upf file in the Source View

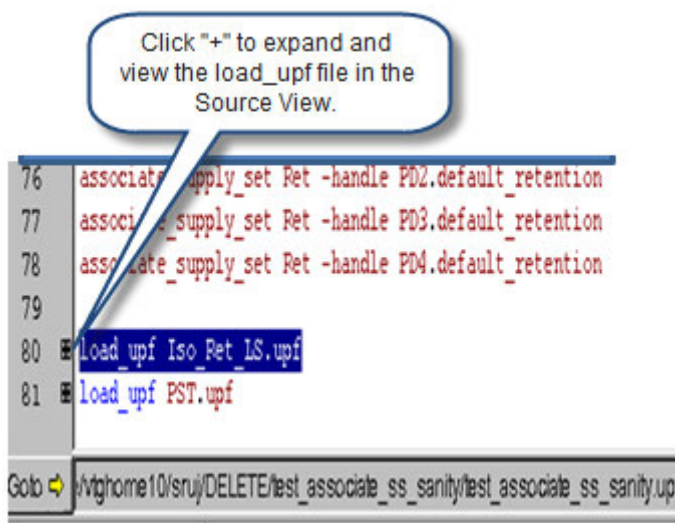


Figure 13-22 Return to the Parent UPF File

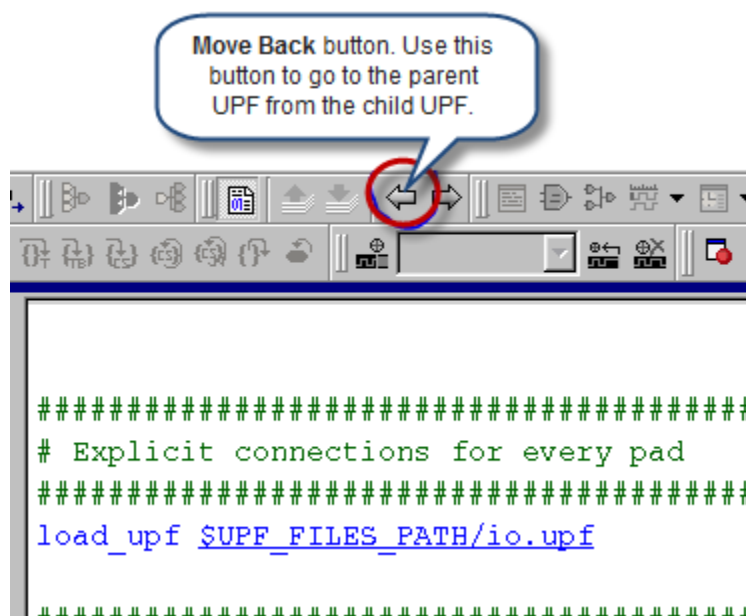
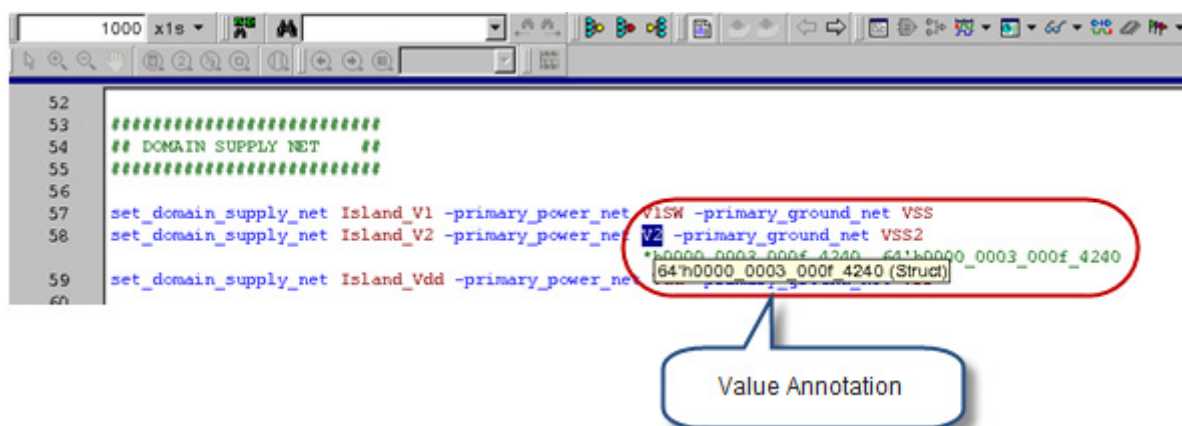


Figure 13-23 Viewing Value Annotation on the UPF/HDL Objects



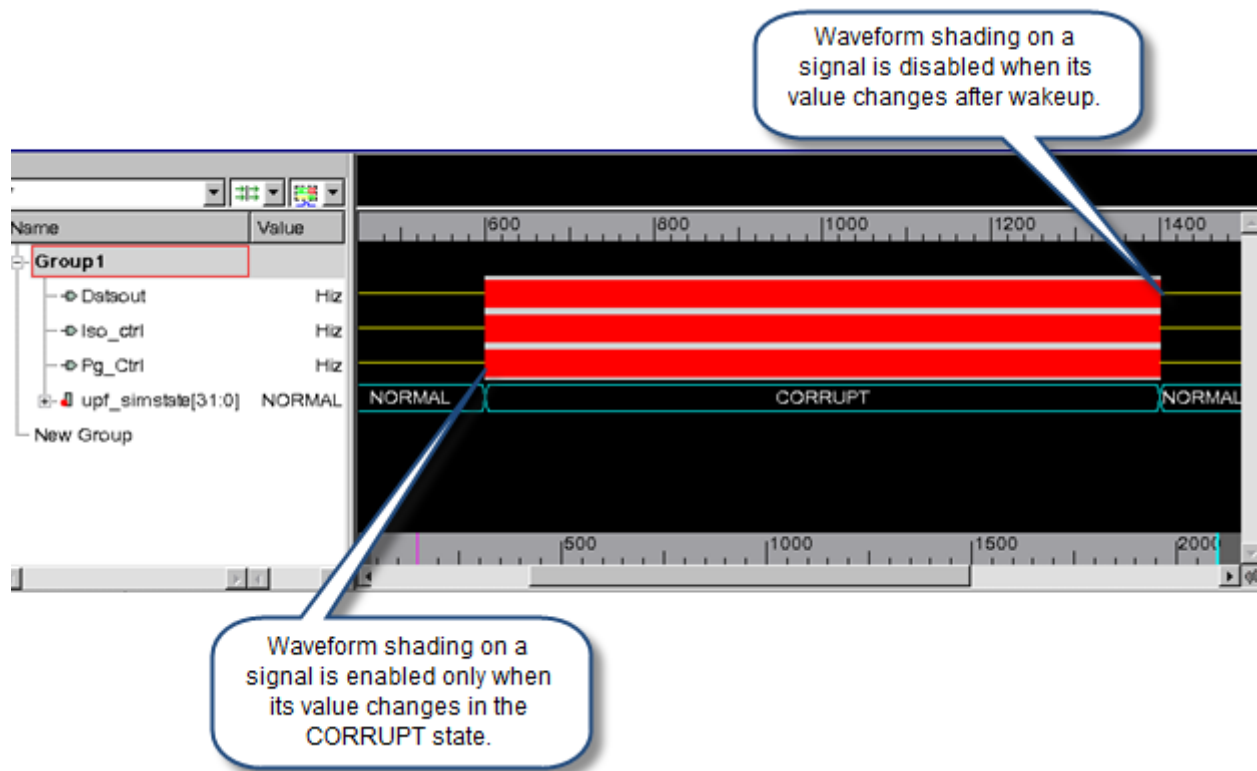
Waveform Shading

Waveform shading is used to distinguish if the value on a signal is the result of its corresponding power domain going to the CORRUPT state.

- Waveform shading is turned ON if a value change is detected on a signal during the `CORRUPT` state, indicating that the current value on the signal is the effect of the power being turned OFF
- Waveform shading remains ON till a value change is detected after the power domain goes to `NORMAL` state. It gives an indication that the signal has not yet been initialized upon wake up
- No waveform shading is seen if there is no value change detected during the `CORRUPT` state, thus indicating that there has been no effect of power OFF on this signal

The following figure describes waveform shading in DVE:

Figure 13-24 Waveform Shading



DVE allows you to view the following in the Wave View:

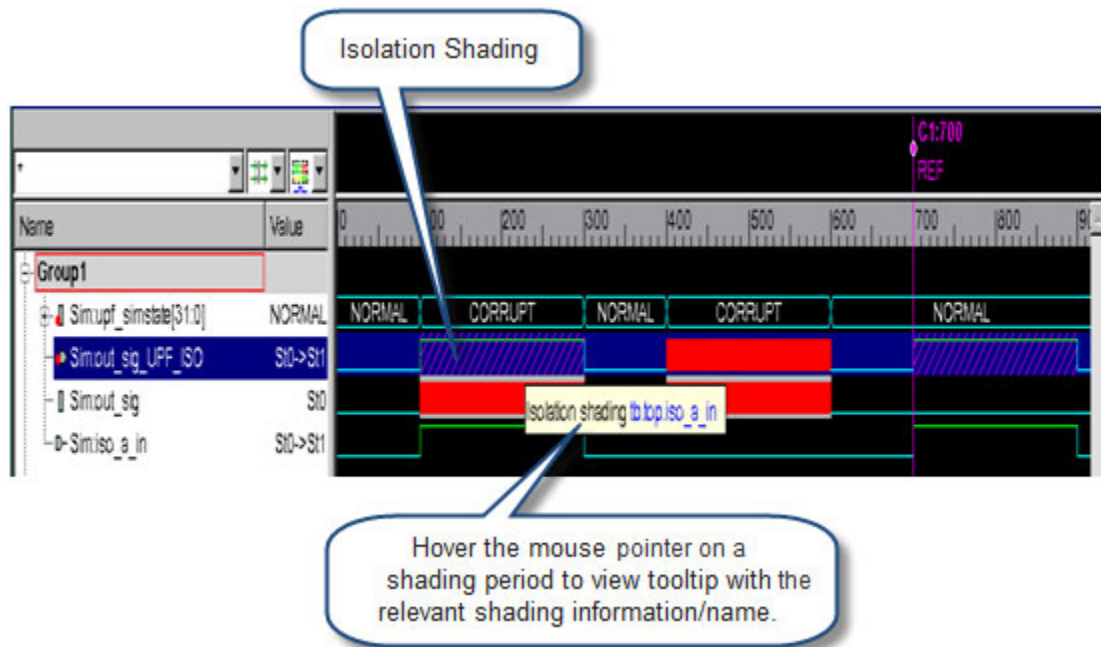
- Isolation shading
- Isolation power off shading
- Input toggle shading
- `set_related_supply_net` (SRSN) shading

Isolation Shading Support

DVE provides the waveform shading for isolated nets/ports in Wave View, as shown in [Figure 13-25](#). This allows you to distinguish whether the value changes on the net/port is due to the applied isolation.

Hover the mouse pointer on a shading period to view tooltip with the relevant shading information/name, as shown in [Figure 13-25](#).

Figure 13-25 Isolation Shading According to Isolation Control Signals

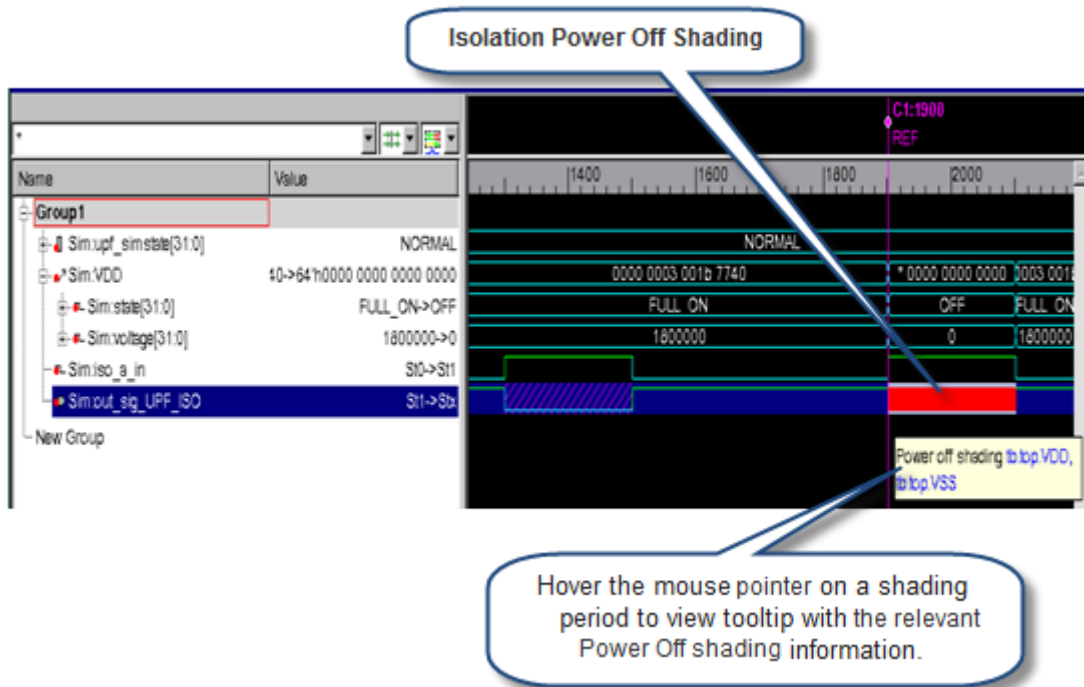


In Figure 13-25, `iso_a_in` is the isolation enabled with sense high. `out_sig` is isolated, therefore isolation shading is on `out_sig_UPF_ISO`.

Shading Support for the `isolation_power_off` Shading

DVE supports the shading of isolated nets/ports when the isolation power or ground nets are turned off, as shown in Figure 13-26. This allows you to distinguish whether the value changes on an isolated net/port is due to the turning off of the isolation supplies.

Figure 13-26 Isolation Power Off Shading

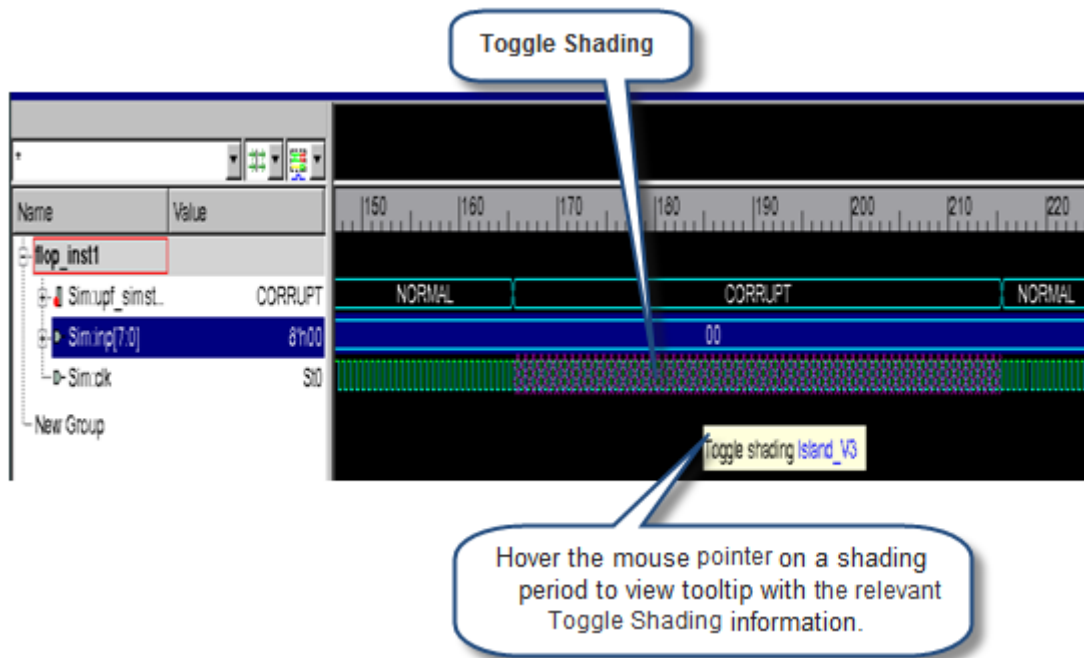


In Figure 13-26, VDD is the isolation power net for out_sig_UPF_ISO.

Support for Toggle Shading on Input Ports

DVE supports the shading of input ports when there is a toggle during the power off period, as shown in Figure 13-27.

Figure 13-27 Toggle Shading for Input Ports



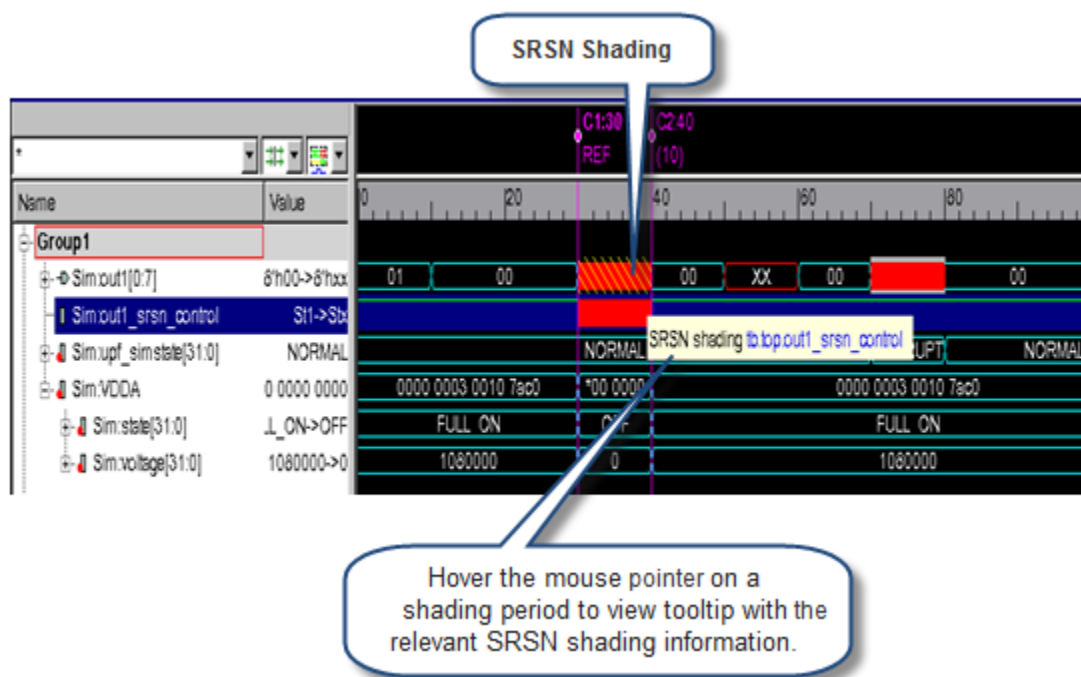
In Figure 13-27, `clk` is the input, which is toggling when the domain is in the `CORRUPT` state. No toggle on `inp`, therefore no shading.

Shading Support for `set_related_supply_net (SRSN)` on Ports

As part of the shading support, the output of the inserted SRSN buffer is shaded as shown in Figure 13-28 when the SRSN supplies go off.

The `*_srsn_control` signal indicates the state of SRSN supplies.

Figure 13-28 SRSN Shading



In Figure 13-28, VDDA is the related supply for out1. When VDDA is OFF, the SRSN shading is applied.

Note:

Waveform shading is enabled/disabled only if there is a value change on ports/nets.

Primary Power Off Shading

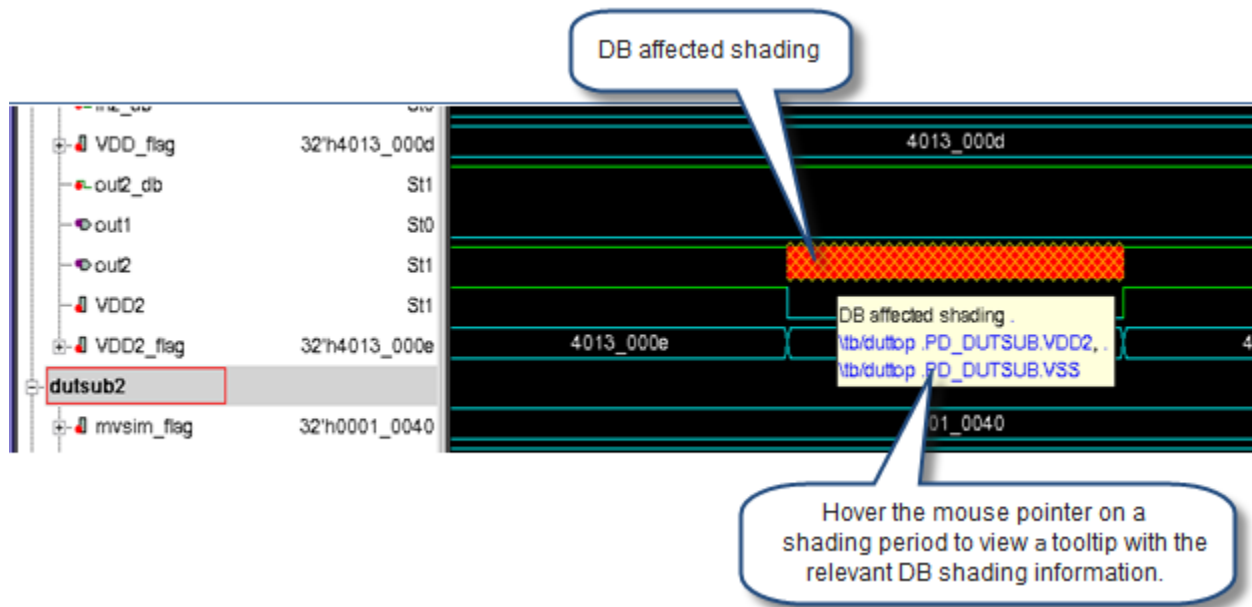
DVE does not support the corruption shading of input ports and elements marked as `always_on`.

Since VCS NLP does not corrupt input ports and `always_on` elements explicitly, the corruption shading is disabled for them.

Power-Off Shading for DB pins

For DB matched cells, the corruption on the ports (logic pins) is based on the related supplies (`related_power_pin`, `related_ground_pin`) for the input ports and the `power_down_function` for the output ports. DVE provides the power off waveform shading, as shown in [Figure 13-29](#), for the ports of a cell.

Figure 13-29 Shading Support for Logic Pins

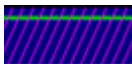
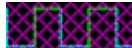


Hover the mouse pointer on a shading period to view a tooltip with the relevant DB shading information, as shown in [Figure 13-29](#).

Configuring the Waveform Shading Pattern Color

You can set different color for a waveform shading pattern. [Table 13-8](#) lists default shading pattern colors of different types of waveform shading.

Table 13-8 Default Shading Pattern Color

Shading type	Default pattern color	Shading pattern
Power off shading	Gray (in background)	
SRSN shading	Yellow	
Isolation shading	Pink	
Toggle shading	Pink	
DB affected shading	Yellow	

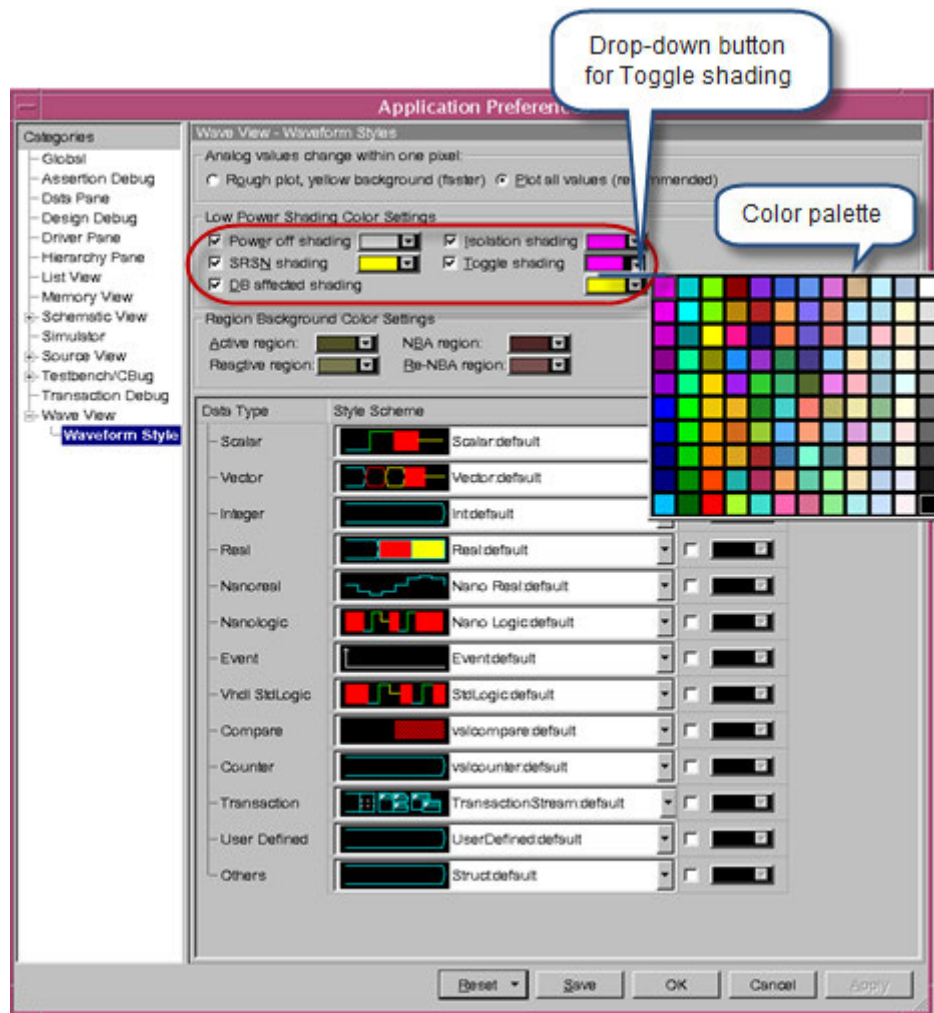
Perform the following steps to change the shading pattern color of a waveform shading:

1. Select **Edit > Preferences**.

The Applications Preferences dialog box appears.

2. In the **Wave View > Waveform Styles** category, **Low Power Shading Color Settings** region, click the drop-down button of a low power shading to open the color palette, as shown in [Figure 13-30](#).

Figure 13-30 Configuring the Waveform Shading Pattern Color



3. Select a color from the color palette and click **Apply**.
4. Click **OK**.

Limitations

DVE does not support the following features:

- The signal highlight feature when an isolation cell is part of the Schematic View

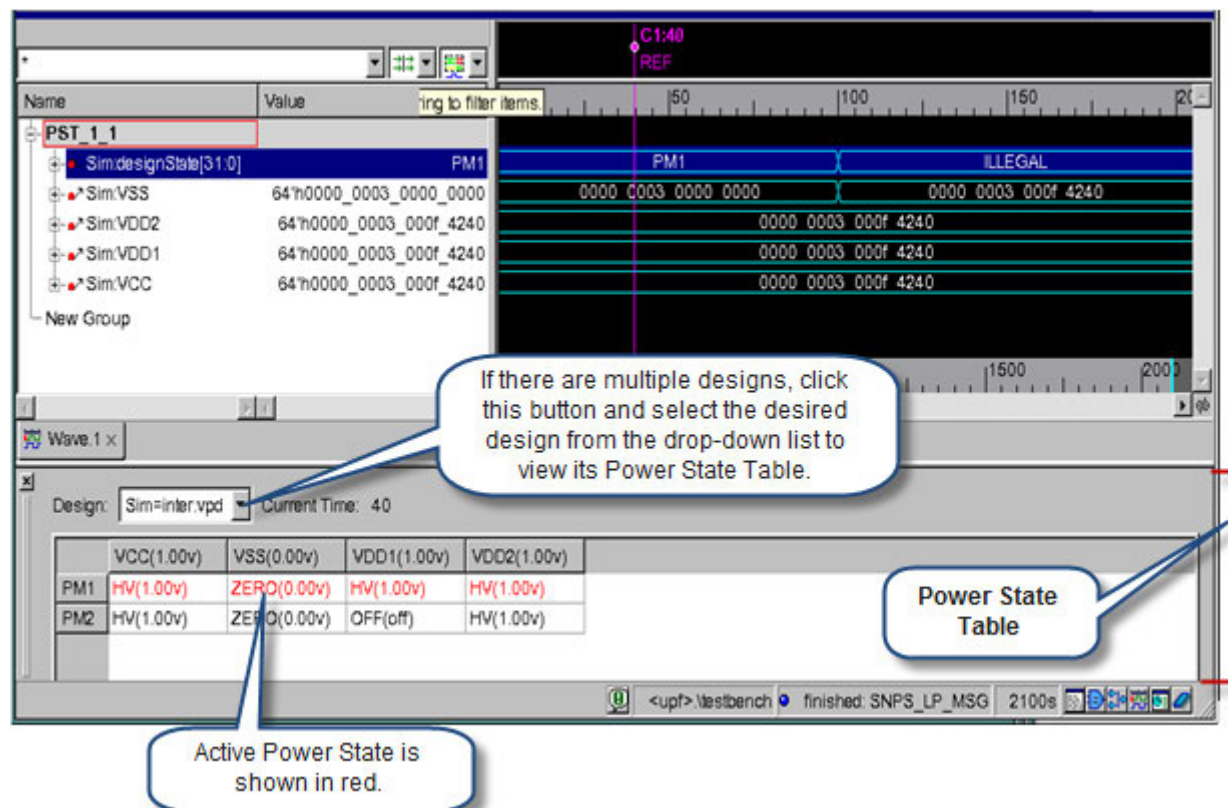
- Waveform shading for isolation with bit-select ports
- SRSN shading for bit/part select of an array in `object_list`
- Isolation shading when clamp type is latch
- Power Off shading and Input Toggle shading on the ports of a cell (model with `.db`) is observed

Power State Table

Use the Power State Table dialog box to view the power state tables of a design loaded in DVE. To open the Power State Table, click **View > Power State Table**. DVE displays Power State Table in the Wave View, as shown in [Figure 13-31](#).

If there are multiple designs, you can use the **Design** drop-down menu and select the desired design to view its power state table. In case of multiple PSTs, each PST is shown in a different tab.

Figure 13-31 Power State Table in the Wave View

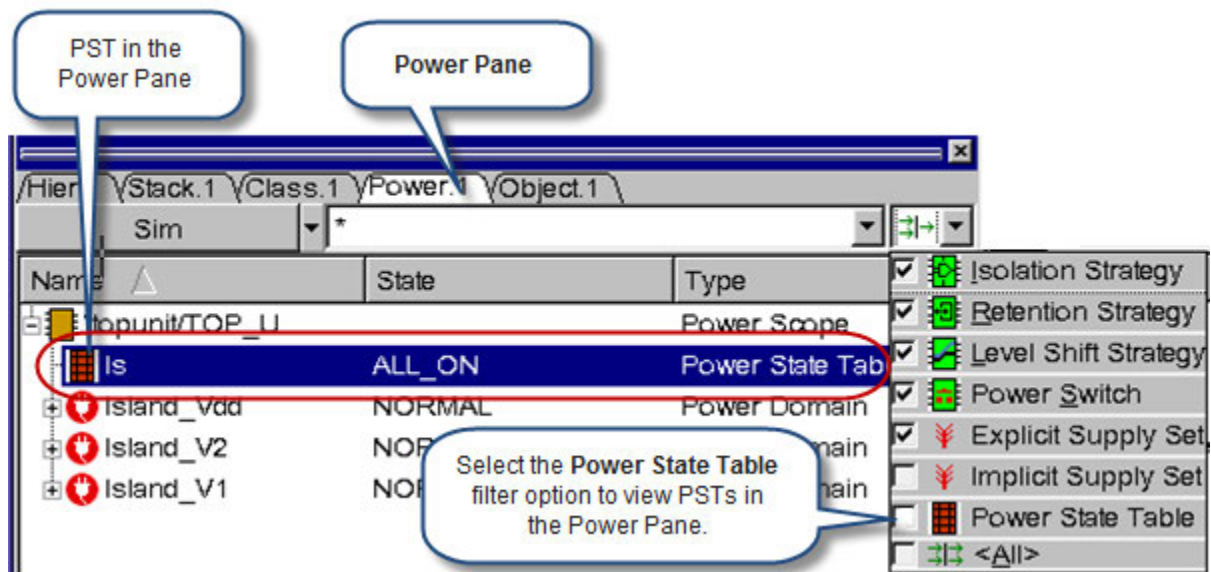


Viewing a Power State Table (PST) in the Power Pane and Wave View

In the Power Pane

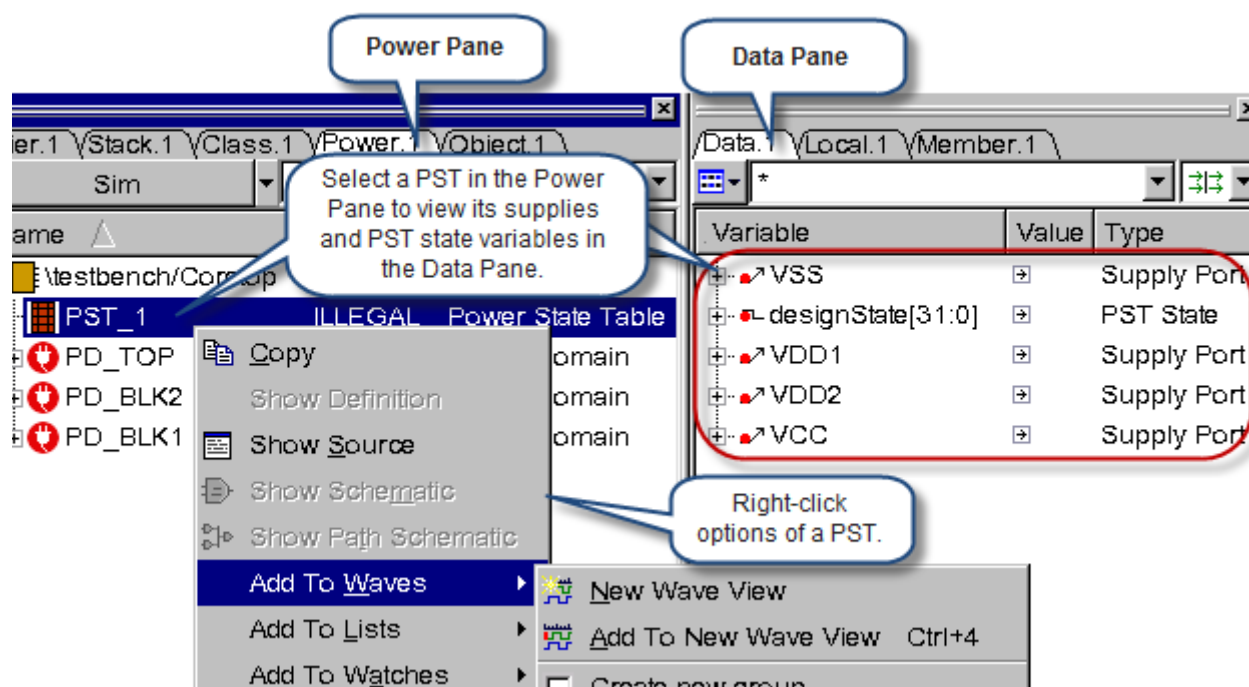
DVE displays a PST under the defined scope in the Power Pane, as shown in [Figure 13-32](#). The PST is displayed under the specific power scope. Select a PST in the Power Pane to view its supplies and PST state variables in the Data Pane, as shown in [Figure 13-33](#). PSTs are displayed by default in the Power Pane. You can use the **Power State Table** filter option, as shown in [Figure 13-32](#), to view or hide PSTs in the Power Pane.

Figure 13-32 Viewing PSTs in the Power Pane



Right-click a PST in the Power Pane to view its right-click options in a menu, as shown in [Figure 13-33](#).

Figure 13-33 Viewing PST Information in the Data Pane



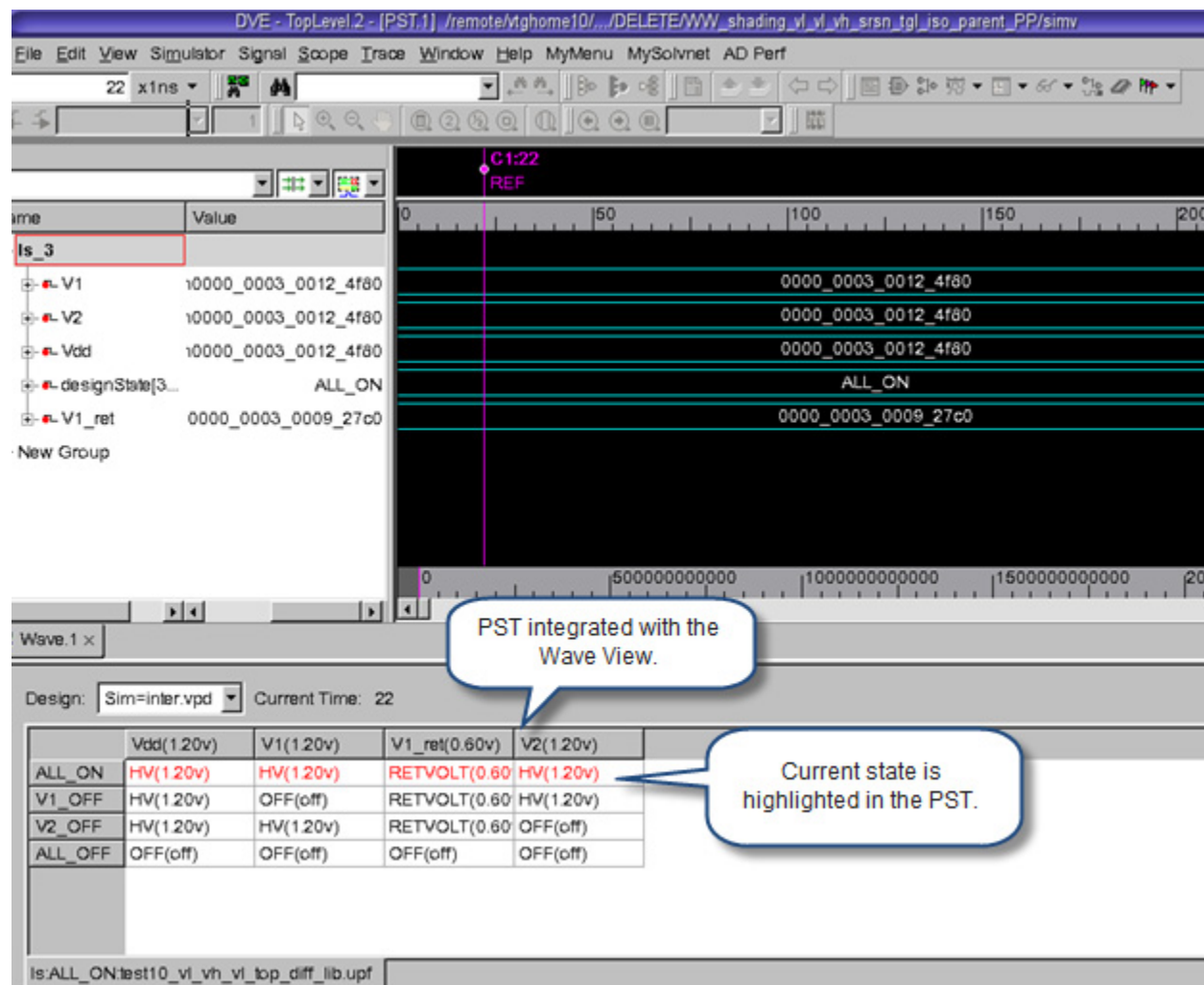
In the Wave View

DVE displays a PST in the Wave View. To view the PST in the Wave View, perform one of the following:

- Right-click a PST in the Power Pane and select **Add To Waves > New Wave View**, as shown in Figure 13-33, to add it to the Wave View.

DVE displays a TopLevel PST window with Wave View and PST pane, as shown in Figure 13-34. If the current TopLevel window already contains a Wave View, then a PST pane is opened in this window.

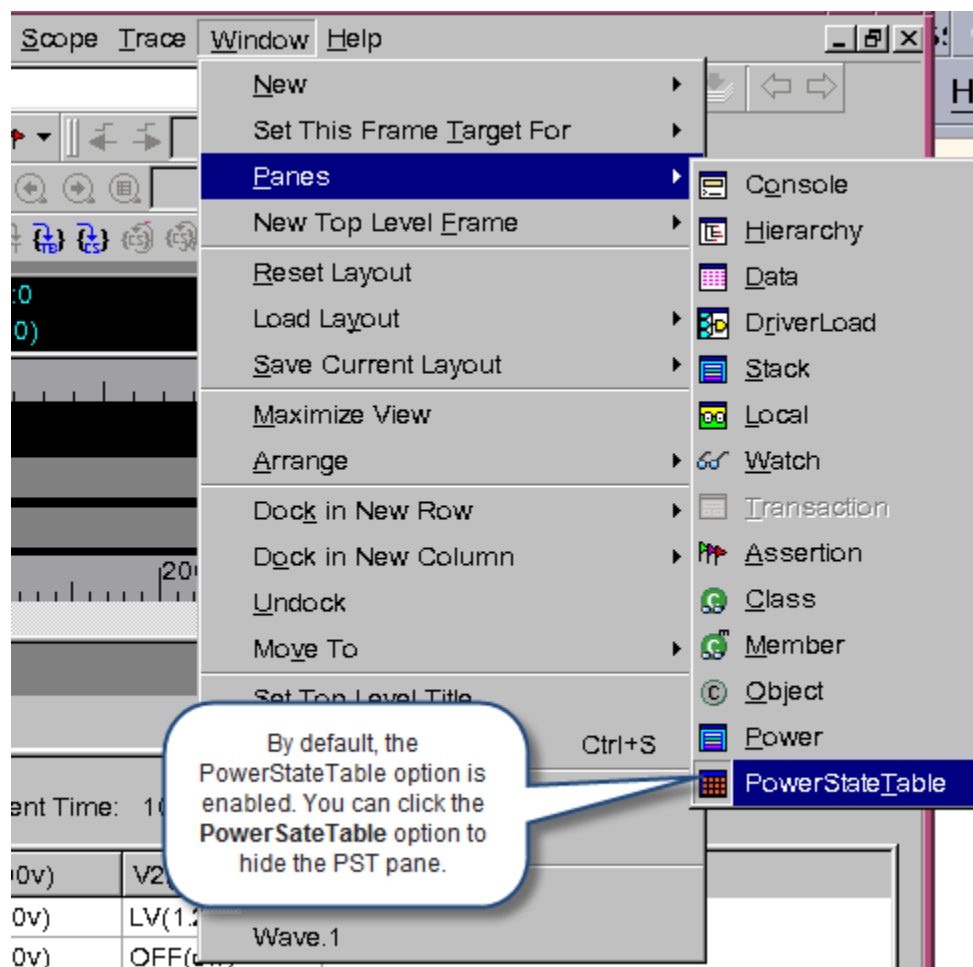
Figure 13-34 PST Integrated with the Wave View



DVE adds the PST supplies and state variables of the selected PST to the Wave View. If cursor C1 changes, then the current state is synced up.

By default, DVE displays the TopLevel window with a Wave View and a PST pane. To hide the PST pane in the TopLevel window, click **Window > Panes > PowerStateTable**, as shown in [Figure 13-35](#). You can use the **PowerStateTable** option to view or hide the PST pane in the TopLevel window.

Figure 13-35 Hiding PST Pane

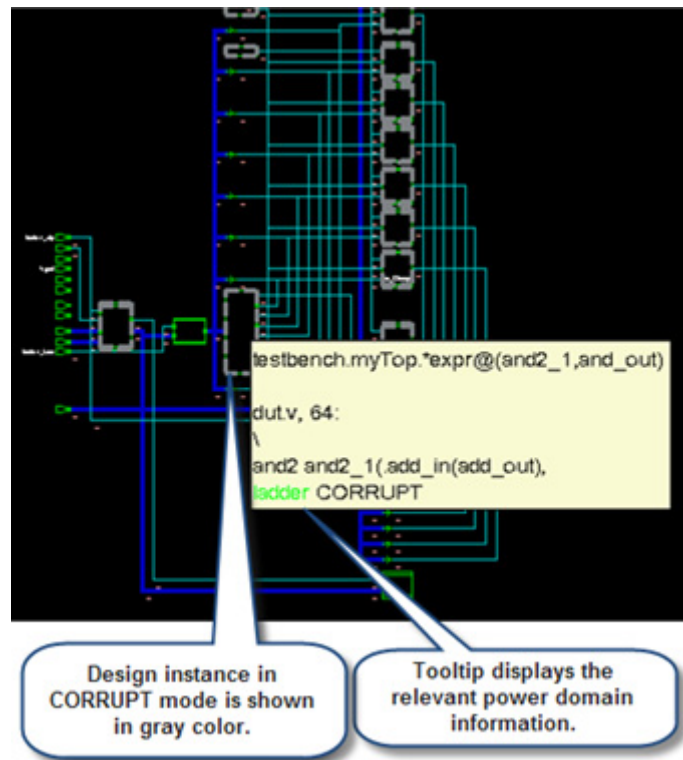


Shading in the Schematic View

The Schematic View automatically shades the design instances in gray when the corresponding power domain is in the CORRUPT state. It helps you to view the states of source and sink power domains of a crossover.

Hovering the mouse cursor on a design instance displays tooltip with relevant power domain information, as shown in the following figure:

Figure 13-36 Schematic Window



Symbol for Isolation Cell in Schematic

DVE provides the schematic symbol shown in [Table 13-9](#) to represent isolation cells.

Table 13-9 Isolation Symbols in the Schematic View

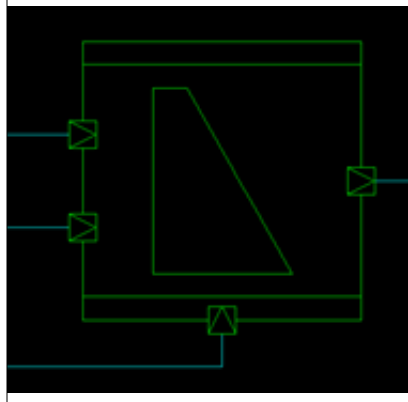
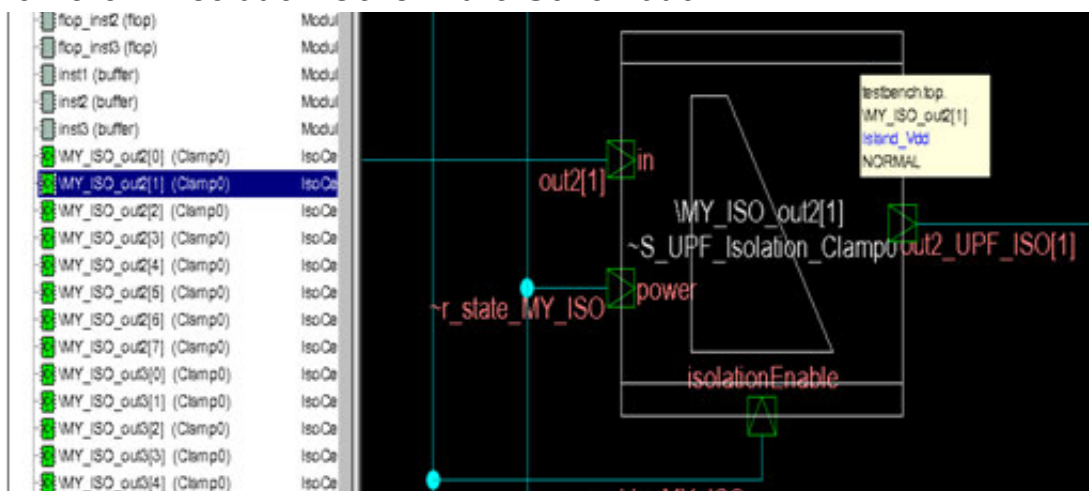
Symbol	Symbol Name
	Isolation Cell

Figure 13-37 shows the isolation cell in the Schematic View.

Figure 13-37 Isolation Cells in the Schematic

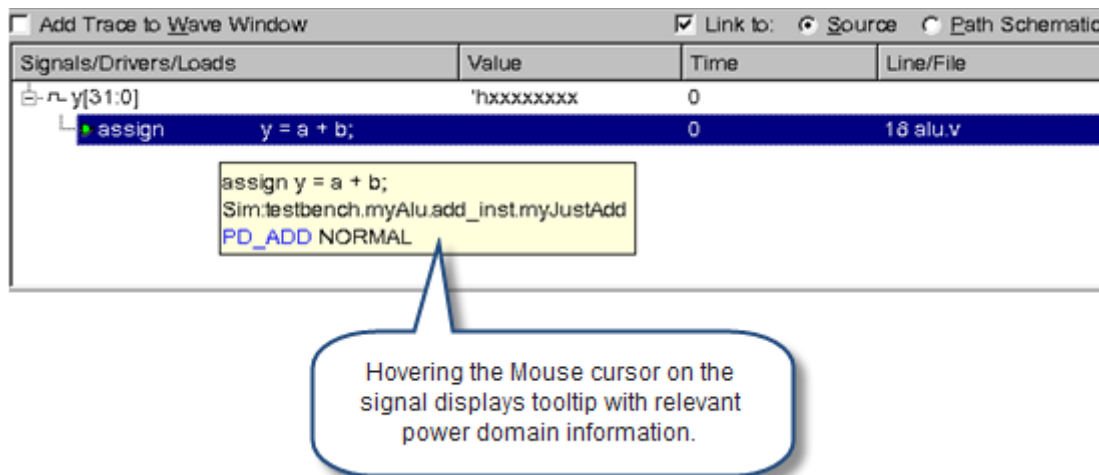


Drivers/Loads Pane

Power Domain Information for Drivers/Loads

The Drivers/Loads Pane displays all drivers/loads that possibly contributed to a signal value. Hovering the mouse cursor on the signal displays its power domain information in a tooltip, as shown in the following figure:

Figure 13-38 Drivers/Loads Pane



Driver/load Tracing for the Supply Network and Power Pane Data Elements

DVE allows you to trace Drivers/Loads for the following in the Hierarchy Pane and Power Pane:

- UPF supply nets, as shown in [Figure 13-39](#), [Figure 13-40](#), [Figure 13-42](#), and [Figure 13-41](#)
- HDL supply nets/ports referred in the UPF

Figure 13-39 Driver/Load from connect_supply_net

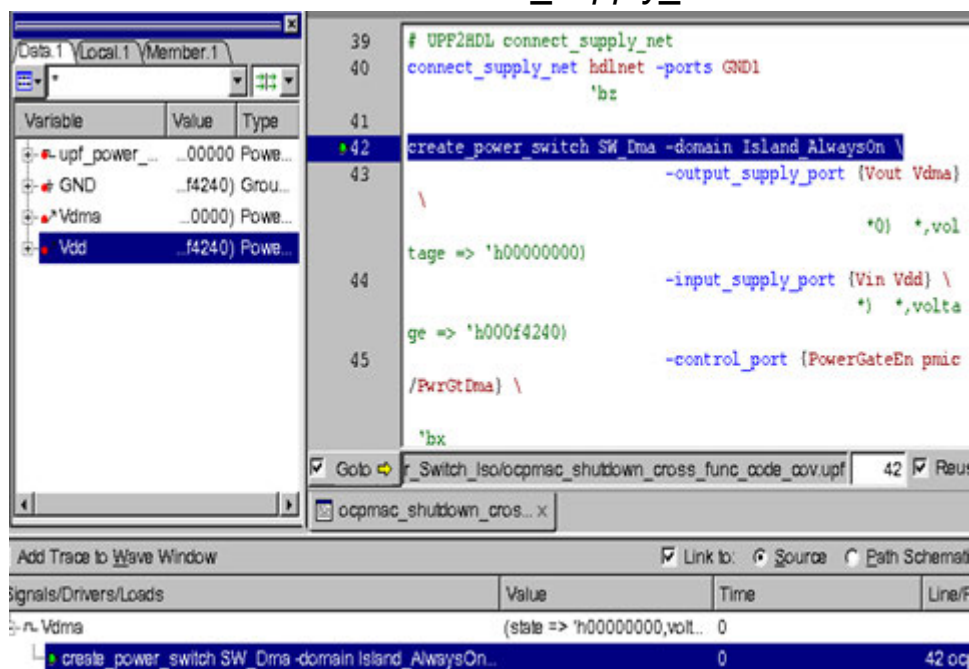


Figure 13-40 Driver from supply_on/off Calls

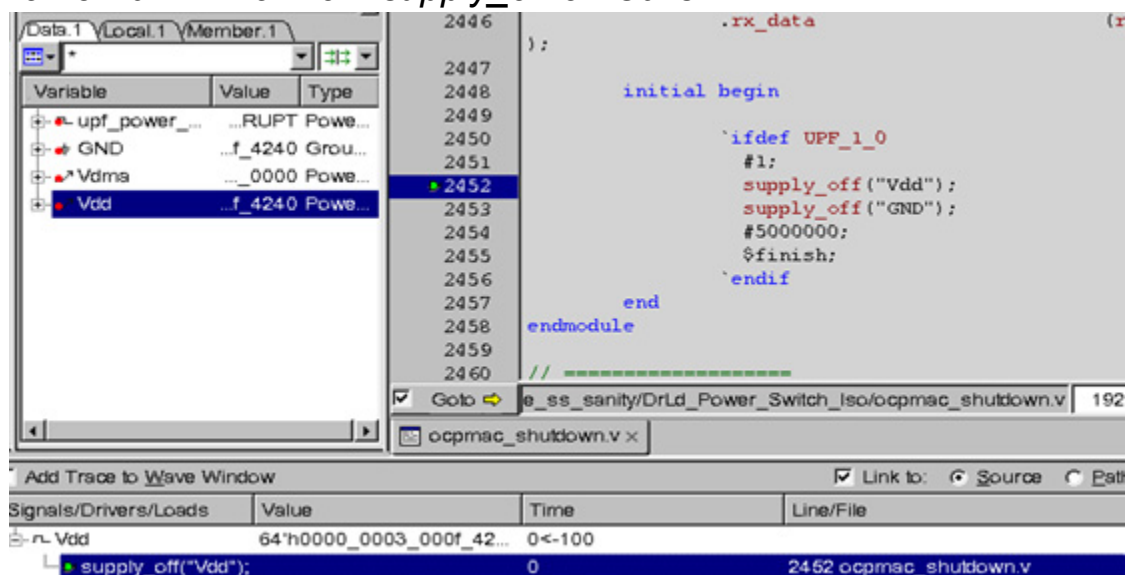


Figure 13-41 Driver from supply_on Calls

The screenshot displays a logic analyzer interface. On the left, a variable table shows:

Variable	Value	Type
upf_power_...	...00000	Powe...
GN	...f4240)	Grou...
ifflop_n	...f4240)	Powe...

The main window shows source code for `testbench.v` with the following relevant lines:

```

36  rst <= 1'b0;
37  end
38
39  initial
40  begin
41
42  // `ifdef UPF 1 0
43  supply_on ("VDD_Iflop", 1.2);
44  supply_on ("VSS", 0.0);
45  supply_on ("Vl_ret", 1.0);
46  #100 supply_off("VDD_Iflop");
47  #200 supply_off("VSS");
48  // `endif
49
50  ctrl_sig <= 1'b0;
51  restore = 1'b0;

```

The bottom trace window shows the following data:

gnals/Drivers/Loads	Value	Time	Line/File
-n ifflop_n	(state => 'h00000003,volt...	0	
supply_on ("VDD_Iflop", 1.2);		0	43 testbench.v
#100 supply_off("VDD_Iflop");		0	46 testbench.v

Figure 13-42 Driver/Load for supply nets

The screenshot displays a logic analyzer interface. On the left, a variable table shows:

Variable	Value	Type
upf_power_...	...00000	Powe...
VDD	...0000)	Powe...
VSS	...0000)	Grou...
Vtop	...0000)	Suppl...

The main window shows source code for `top.upf` with the following relevant lines:

```

28  connect_supply_net VSStop -ports i_core1/VSS
    *tag => 'h000f4240) *,voltage => 'h00
    000000)
29
30  connect_supply_net VDDcore2 -ports i_core2/VDD
    *oltage => 'h000f4240) *,voltage => 'h
    00000000)
31  connect_supply_net VDDtop -ports i_core2/Vtop
    *tag => 'h000f4240) *,voltage => 'h00
    000000)
32  connect_supply_net VSStop -ports i_core2/VSS
    *tag => 'h000f4240) *,voltage => 'h00
    000000)
33
34  set_isolation top_iso_out \
35  -domain TOP \

```

The bottom trace window shows the following data:

gnals/Drivers/Loads	Value	Time	Line/File
-n Vtop	(state => 'h00000000,volt...	0	
connect_supply_net VDDtop -ports i_core2/Vtop		0	31 top.upf

Viewing Logic Nets in DVE

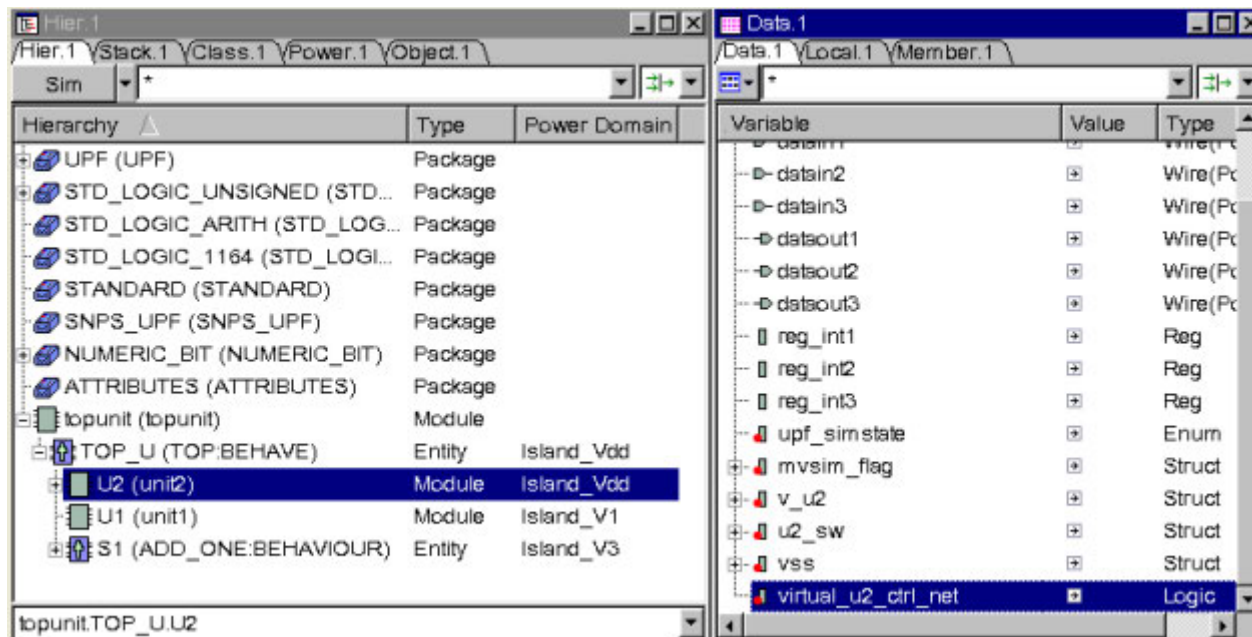
The logic nets created using the `create_logic_net` command in the UPF are displayed in the Data Pane of the corresponding design instance.

Consider the following UPF command at scope `topunit.TOP_U.U2`:

```
create_logic_net virtual_u2_ctrl_net
```

You can click an HDL instance in the Hierarchy Pane to view its logic nets in the Data Pane, as shown in [Figure 13-43](#).

Figure 13-43 Viewing Logic Nets Created by the `create_logic_net` Command



The net's type is determined by the HDL language of the scope where it is created. If the scope is:

- SystemVerilog, the type is `logic`
- VHDL, the type is `std_ulogic`

Limitations

Following are the limitations of this feature:

- Logic nets at SystemC scopes are not supported
- Logic nets are not supported in the Schematic View

Support for Active Drivers on Low Power Design Elements

DVE supports active drivers functionality for the following low power design elements:

- Power Domain — DVE displays the `upf_power_domain_state` signal as a contributor for the corresponding power domain driver in the Driver Load Pane. For more information, see [“Power Domain”](#) section.
- Power Switch — DVE displays the contributor signals for the `create_power_switch` driver in the Driver Load Pane. For more information, see [“Power Switch”](#) section.
- Isolation Strategy — DVE displays the isolation strategy driver for the isolation instrumented signal in the Driver Load Pane. For more information, see [“Isolation Strategy”](#) section.
- Retention Strategy — DVE displays the retention strategy driver for the retained and shadow registers in the Driver Load Pane. For more information, see [“Retention Strategy”](#) section.

- **UPF Supply Functions** — DVE displays the `supply_on` and `supply_off` functions as drivers in the Driver Load Pane. For more information, see [“UPF Supply Functions”](#) section.
- **set_related_supply_net (SRSN) command** — DVE displays the SRSN on Output Port, SRSN on Input Port, or SRSN on Inout Port active driver for the SRSN instrumented signal at a port. For more information, see [“set_related_supply_net \(SRSN\)”](#) section.
- **set_port_attributes (SPA) command** — DVE displays the SPA on Output Port (for `-receiver_supply`) or SPA on Input Port (for `-driver_supply`) active driver for the SPA instrumented signal at a port. For more information, see [“set_port_attributes \(SPA\)”](#) section.

These enhancements help you to view, trace, and analyze the source of power-related errors.

Power Domain

DVE displays the `upf_power_domain_state` signal as a contributor for the corresponding power domain driver in the Driver Load Pane, as shown in [Figure 13-44](#).

Figure 13-44 Viewing Contributor Signal of the Power Domain Driver

Add Trace to Wave Window		Link to:  	
Signals/Drivers/Loads	Value	Time	Line/File
  a_out[7:0]	8'h00->8'hxx	170<-185	
  <power domain 'ladder' is off>		170	7 demo.upf
  upf_power_domain_state[31:0] NORMAL->CORRUPT		170	7 demo.upf
  assign out = in;		170	13 dut.v

Figure 13-45 Tooltip for Power Domain Driver

Signals/Drivers/Loads	Value	Time
  a_out[7:0]	8'h00->8'hxx	170<-185
  <power domain 'ladder' is off>		170
  upf_power_d	CORRUPT	170
  assign out = in;		170

Power domain is switched off and signal value is corrupted. (testbench/myTop/ladder)
[Press 'Shift' to save in Tooltip Viewer](#)

You can hover the mouse pointer on the upf_power_domain_state signal to view its information in a tooltip, as shown in Figure 13-46.

Figure 13-46 Tooltip for upf_power_domain_state Signal

Add Trace to Wave Window		Link to:  	
Signals/Drivers/Loads	Value	Time	Line/File
  a_out[7:0]	8'h00->8'hxx	170<-185	
  <power domain 'ladder' is off>		170	7 demo
  upf_power_domain_state[31:0] NORMAL->CORRUPT		170	7 demo
  assign out = in;			13 dut

Sim: <upf>.
 \testbench\myTop.ladder.upf_power_domain_state[31:0]
 (power domain state)
[Press 'Shift' to save in Tooltip Viewer](#)

Power Switch

DVE displays the contributor signals (both active and inactive) for the `create_power_switch` driver in the Driver Load Pane, as shown in [Figure 13-47](#).

The contributor signal is displayed as an active contributor, if its value changes at the trace time, otherwise it is displayed as an inactive contributor. For example, in [Figure 13-47](#), the contributor `sleepS1` is active and `VDD_SUB` is inactive.

Figure 13-47 Viewing Contributor Signals of the Power Switch Driver

Add Trace to Wave Window		Link to:	
Signals/Drivers/Loads	Value	Time	Line/File
out	(FULL_ON,1000000)->(OFF,...	100<-130	
Power switch 'subsystem_switch'		100	37 test.u
sleepS1	St0->St1	100	25 test.v
VDD_SUB	(FULL_ON,1000000)	100	13 test.u

Acknowledge Port of the Power Switch

DVE displays the contributor signals for the `create_power_switch` driver when tracing the acknowledge port (`-ack_port`) for a power switch. For example, consider the following UPF code:

```
create_power_switch Iff1_switch \  
    -domain SUBSYSTEM \  
    -input_supply_port {in VDD_SUB} \  
    -output_supply_port {out VDD_SUB_S} \  
    -control_port {sleep sleepS1} \  
    -on_state {on in {!sleep}} \  
    -ack_port {my_ack_port my_ack_net}
```

In the above UPF example, when tracing the `my_ack_net` signal specified in the `-ack_port`, DVE displays contributor signals for the `create_power_switch` driver, as shown in Figure 13-48.

Figure 13-48 Contributor Signals for the `create_power_switch` Driver

Add Trace to Wave Window Link to: 			
Signals/Drivers/Loads	Value	Time	Line/File
my_ack_net	St1->St0	135<-157	
Power switch 'lff1_switch'		135	25 test_no_retention_mu
sleeps1	St0->St1	135	3 top.v
VDD_SUB	(FULL_ON,1200000)	135	7 test_no_retention_mult

Figure 13-49 shows the tooltip for the power switch contributor (`sleeps1`). The tooltip displays information on the role of the contributor. In this case, control port for `sleeps1`.

Figure 13-49 Tooltip for Power Switch Contributor

Add Trace to Wave Window		
Signals/Drivers/Loads	Value	Time
my_ack_net	St1->St0	135<-157
Power switch 'lff1_switch'		135
sleeps1	St0->St1	135
VDD_SUB	(FULL_ON,1200000)	135

Sim:testbench.top.sleeps1
 (control port)
 PD_TOP NORMAL
 Press 'Shift' to save in Tooltip Viewer

Isolation Strategy

DVE displays the isolation strategy driver, as shown below, for the isolation instrumented signal (`*_UPF_ISO`) in the Driver Load Pane.

Isolation strategy '<isolation_name>'

For example, consider the following RTL code and the corresponding UPF:

Example 13-1 RTL Code

```
module dut(input logic clock, input logic init, input int
in, output int out);
    wire[31:0] temp1, temp2;
    mul2 s1(clock, in, temp1);
    always @(posedge clock) begin
        out <= !init ? temp1+1 : temp2-1;
    endmodule
```

Example 13-2 UPF

```
create_power_domain SUBSYSTEM -elements s1
set_isolation ISO_out \
    -domain SUBSYSTEM \
    -isolation_power_net VDD \
    -isolation_ground_net VSS \
    -clamp_value 0 \
    -applies_to outputs
set_isolation_control ISO_out \
    -domain SUBSYSTEM \
    -isolation_signal sleepS1 \
    -isolation_sense high \
    -location parent
```

The value of the isolation instrumented signal `temp1_UPF_ISO` is used in the assignment to `out` at the posedge of clock, as shown in [Figure 13-50](#).

Figure 13-50 Isolation Instrumented Signal

Add Trace to Wave Window		Link to: Source	
Signals/Drivers/Loads	Value	Time	Line/File
out[31:0]	0->1	10<-15	
out <= limit ? temp1+1 : temp2-1;		10	40 power_switch_two_st
init	1'b0	10	35 power_switch_two_st
temp1_UPF_ISO[31:0]	32'h0000_0000	10	30 power_switch_two_st
temp2[31:0]	32'h0000_0000	10	30 power_switch_two_st

You can expand the `temp1_UPF_ISO` contributor signal to view the isolation strategy driver, as shown in [Figure 13-51](#).

Figure 13-51 Viewing Isolation Strategy Driver

Add Trace to Wave Window		Link to: Source	
Signals/Drivers/Loads	Value	Time	Line/File
out[31:0]	0->1	10<-15	
out <= limit ? temp1+1 : temp2-1;		10	40 power_switch_two_st
init	1'b0	10	35 power_switch_two_st
temp1_UPF_ISO[31:0]	32'h0000_0000	10	30 power_switch_two_st
Isolation strategy 'ISO_out'	0		50 power_switch_two_st
upf_simstate	NORMAL	0	50 power_switch_two_st
sleepS1	1'b0	0	36 power_switch_two_st
temp1[31:0]	32'h0000_0000	0	30 power_switch_two_st
temp2[31:0]	32'h0000_0000	10	30 power_switch_two_st

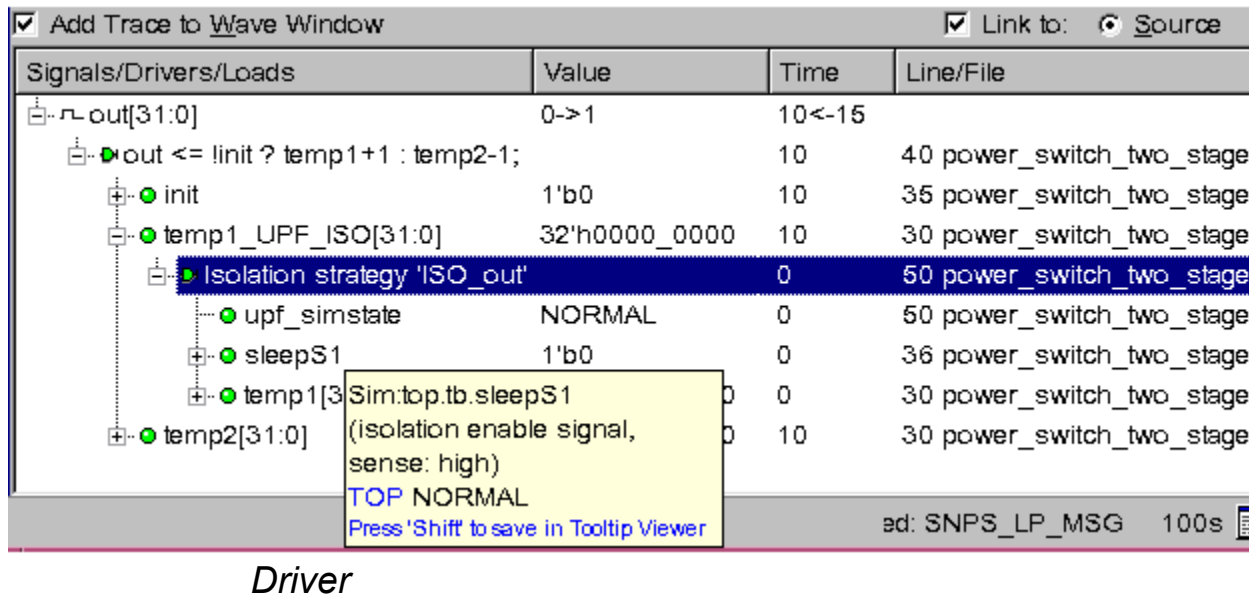
The isolation strategy driver (`ISO_out` driver in [Figure 13-51](#)) displays the following signals as contributors:

- Original signal (`temp1`)
- Isolation signal (`sleepS1`)
- `upf_simstate` (for implicit or explicit isolation supply set)

The contributor signal is displayed as an active contributor, if its value changes at the trace time.

Tooltip for a contributor signal displays its purpose and full hierarchical name, as shown in [Figure 13-52](#).

Figure 13-52 *Tooltip for the Contributor Signal of the Isolation Strategy*



Retention Strategy

Consider the following RTL code ([Example 13-3](#)) and the corresponding UPF ([Example 13-4](#)) with retention policy for `s1` instance.

Example 13-3 RTL Code

```
module fibMul(input logic clock, input logic[31:0] in, output
logic[31:0] out);
    int fib1 = 1;
    int fib2 = 1;
    always @(posedge clock) begin
        fib1 <= fib2;
        fib2 <= fib1 + fib2;
    end
endmodule
```

```

        out <= fib1 * in;
    end
endmodule
module dut(input logic clock, input logic use1,
    input logic saveFib, input logic restoreFib,
    input int in, output int out);
    wire[31:0] temp1, temp2;
    fibMul s1(clock, in, temp1);
    . . .
endmodule

```

Example 13-4 UPF

```

create_power_domain SUBSYSTEM -elements s1
set_retention RET \
    -domain SUBSYSTEM \
    -retention_power_net VDD_SUB \
    -retention_ground_net VSS \
    -save_signal {saveFib high} \
    -restore_signal {restoreFib high}

```

DVE displays the retention strategy driver, as shown below, for the retained and shadow registers in the Driver Load Pane.

Retention strategy '<retention_name>'

For example, [Figure 13-53](#) shows the retention strategy driver, RET, for the signal fib1.

Figure 13-53 Retention Strategy Driver

Add Trace to Wave Window		Link to: Source	
Signals/Drivers/Loads	Value	Time	Line/File
fib1[31:0]	0->13	280<-295	
int fib1 = 1;		280	2 retention_two_pins.v
fib1 <= fib2;		280	6 retention_two_pins.v
Retention strategy 'RET'		280	61 retention_two_pins.upf
restoreFib	1'b0->1'b1	280	23 retention_two_pins.v
save__fib1[31:0]	13	280	2 retention_two_pins.v
upf_simstate	NORMAL	280	61 retention_two_pins.upf

The retention strategy driver displays the following signals as contributors:

- Restore signal (`restoreFib`)
- Shadow register (`save__fib1`). You can expand the shadow register to view its drivers
- `upf_simstate` (for retention supply set)

DVE displays the restore signal and shadow register as active contributors at restore time, if the simstate is not corrupted.

Tooltip for a contributor signal displays its name and information, as shown in [Figure 13-54](#).

Figure 13-54 Tooltip for the Contributor Signal of Retention Strategy Driver

Signals/Drivers/Loads		Value	Time	Line/File
fib1[31:0]		0->13	280<-295	
int fib1 = 1;			280	2 retention_two_pins.v
fib1 <= fib2;			280	6 retention_two_pins.v
Retention strategy 'RET'			280	61 retention_two_pins.upf
restoreFib		1'b0->1'b1	280	23 retention_two_pins.v
save_fib			280	2 retention_two_pins.v
upf_simst			280	61 retention_two_pins.upf

Sim.tb.d1.restoreFib (restore signal) TOP NORMAL Press 'Shift' to save in Tooltip Viewer

UPF Supply Functions

DVE displays the `supply_on` and `supply_off` functions as drivers in the Driver Load Pane.

For example, consider the following testbench code:

Example 13-5 Testbench Code

```
import UPF::*;

initial begin
    #10;
    supply_on("VDD_SUB", 0.8);
    #10;
    supply_off("VDD_SUB");
    #10;
    supply_on("VDD_SUB", 0.5);
end
```

If you trace `VDD_SUB` at a certain time, then DVE analyzes the `supply_on` and `supply_off` drivers and finds an active driver, as shown in [Figure 13-55](#).

Figure 13-55 Viewing `supply_on` and `supply_off` Drivers

Add Trace to Wave Window			
Signals/Drivers/Loads	Value	Time	Line/File
VDD_SUB	(FULL_ON,1000000)->(FULL_O...	10<-15	
supply_on("VDD_SUB", 0.8);		10	9 supply
supply_off("VDD_SUB");		10	12 supply
supply_on("VDD_SUB", 0.5);		10	15 supply

set_related_supply_net (SRSN)

DVE displays the SRSN on Output Port, SRSN on Input Port, or SRSN on Inout Port active driver for the SRSN instrumented signal (*_UPF_SRSN) at a port.

For example, consider the following:

- Module

```
module modA (in, out);
    input [0:7] in;
    output [0:7] out;
    assign out = !in;
endmodule
```

- SRSN is applied in UPF as follows:

```
set_related_supply_net -object_list { instA2/out
} -power VDDA -ground VSS
```

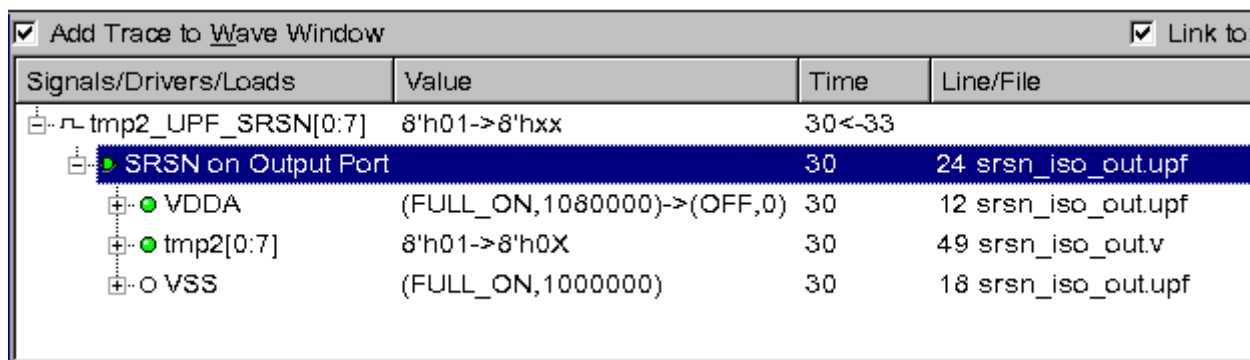
- Module `modA` is instantiated in parent module, and its output is used in some expression, as shown in the following Verilog code:

Example 13-6 Verilog Code

```
module top (in, out1, out2, iso);
  input [0:7] in;
  input iso;
  output [0:7] out1, out2;
  wire [0:7] tmp, tmp2;
  assign out1 = tmp;
  assign out2 = tmp2;
  modA instA1 (.in(in), .out(tmp));
  modA instA2 (.in(tmp), .out(tmp2));
endmodule
```

DVE displays the SRSN on Output Port active driver, as shown in [Figure 13-56](#).

Figure 13-56 Viewing SRSN Active Driver



The screenshot shows the 'Signals/Drivers/Loads' window in DVE. The window has a title bar with 'Add Trace to Wave Window' and 'Link to:'. The table below lists the signals and their values at time 30.

Signals/Drivers/Loads	Value	Time	Line/File
tmp2_UPF_SRSN[0:7]	8'h01->8'hxx	30<-33	
SRSN on Output Port		30	24 srsn_iso_out.upf
VDDA	(FULL_ON,1080000)->(OFF,0)	30	12 srsn_iso_out.upf
tmp2[0:7]	8'h01->8'h0X	30	49 srsn_iso_out.v
VSS	(FULL_ON,1000000)	30	18 srsn_iso_out.upf

The SRSN on Output Port driver displays the following signals as contributors:

- Supply Nets (VDDA and VSS)
- Uninstrumented signal (highconn of the output port, tmp2)

You can further expand the uninstrumented signal tmp2, to view the drivers for output port.

Tooltip for a contributor signal displays its information, as shown in [Figure 13-57](#).

Figure 13-57 Tooltip for the Contributor of “SRSN on Output Port” Driver

Add Trace to Wave Window		Link to	
Signals/Drivers/Loads	Value	Time	Line/File
tmp2_UPF_SRSN[0:7]	8'h01->8'hxx	30<-33	
SRSN on Output Port VDDA tmp2[0:7] VSS	(FULL_ON,1080000)->(OFF,0) 8'h01->8'h0X 0000)	30	24 srsn_iso_out.upf
		30	12 srsn_iso_out.upf
		30	49 srsn_iso_out.v
		30	18 srsn_iso_out.upf

Sim.tb.top.tmp2[0:7]
 (connected signal)
 TOP NORMAL
 Press 'Shift' to save in Tooltip Viewer

.top finished: SNPS_LF

set_port_attributes (SPA)

DVE displays the following active drivers for the SPA instrumented signal at a port:

- SPA on Output Port (for -receiver_supply), as shown in [Figure 13-58](#).
- SPA on Input Port (for -driver_supply)

Figure 13-58 Viewing SPA Active Drivers

Add Trace to Wave Window		Link to	
Signals/Drivers/Loads	Value	Time	Line/File
dataout3	S1->Stx	405<-436	
SPA on Output Port upf_simstate[31:0] dataout3_UPF_SPA_PRE	NORMAL->CORRUPT S11	405	34 test_srcsink.upf
		405	5 test_srcsink.upf
		405	19 top.v

DVE displays the `upf_simstate` signal as a contributor for the SPA on Input Port/SPA on Output Port driver.

Figure 13-59 Tooltip for upf_simstate signal

Signals/Drivers/Loads		Value	Time	Line/File
dataout3		St1->Stx	405<-436	
SPA on Output Port			405	34 test_srcsink.upf
upf_simstate[31:0]		NORMAL->CORRUPT	405	5 test_srcsink.upf
dataout3			405	19 top.v

Sim:<upf>. \topunit\TOP_U.SSA.upf_simstate[31:0] (state of related supply set) Press 'Shift' to save in Tooltip Viewer

Limitations

Following are the limitations of this feature:

- The Power off drivers are displayed only if the simstate is CORRUPT. No special power off analysis is done if the simstate is one of the following: CORRUPT_ON_ACTIVITY, CORRUPT_ON_CHANGE, CORRUPT_STATE_ON_CHANGE, CORRUPT_STATE_ON_ACTIVITY
- UPF supply_on/supply_off calls from UCLI are not listed as drivers

Viewing the DB Information of Cells

DVE displays a magenta dot on the DB cell instances in the Hierarchy Pane, as shown in [Figure 13-60](#). Also, DVE displays the DB information of a scope or variable in a tooltip. Hover the mouse pointer on a scope or variable to view the tooltip with the relevant DB information, as shown in [Figure 13-61](#). [Figure 13-63](#) and [Figure 13-64](#) shows the format in which the DB information is displayed (in the tooltip) for the below cells/pins.

- Isolation cell
- Retention cell
- Level-Shifter cell
- Always_on cell
- Logic pin
- PG pin

Figure 13-60 Magenta Dot on the DB Cells and Pins

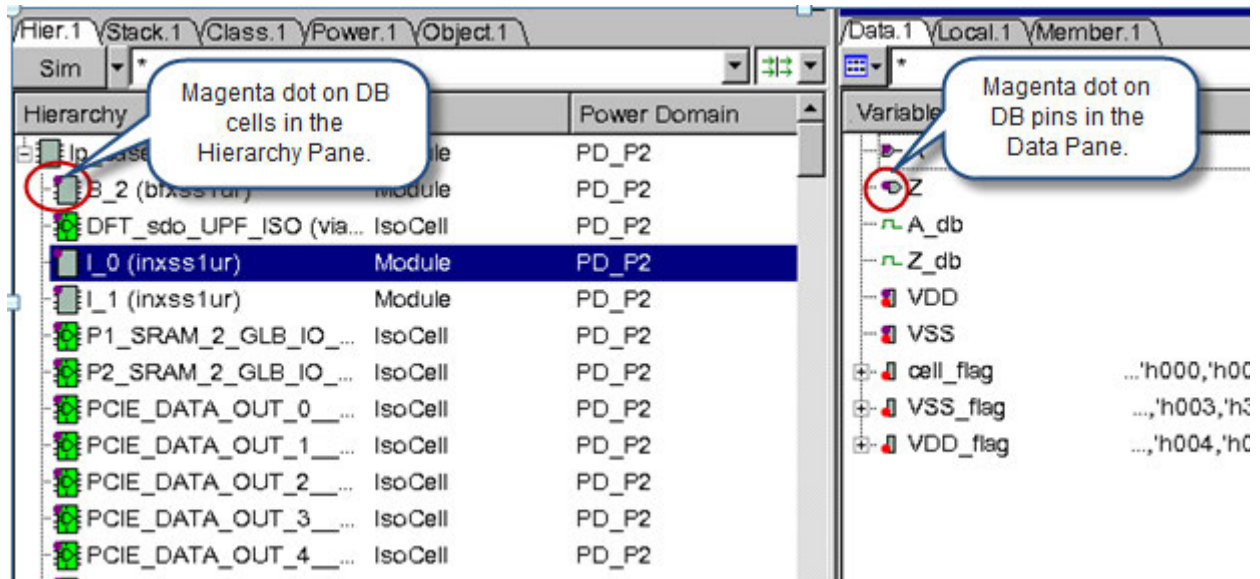
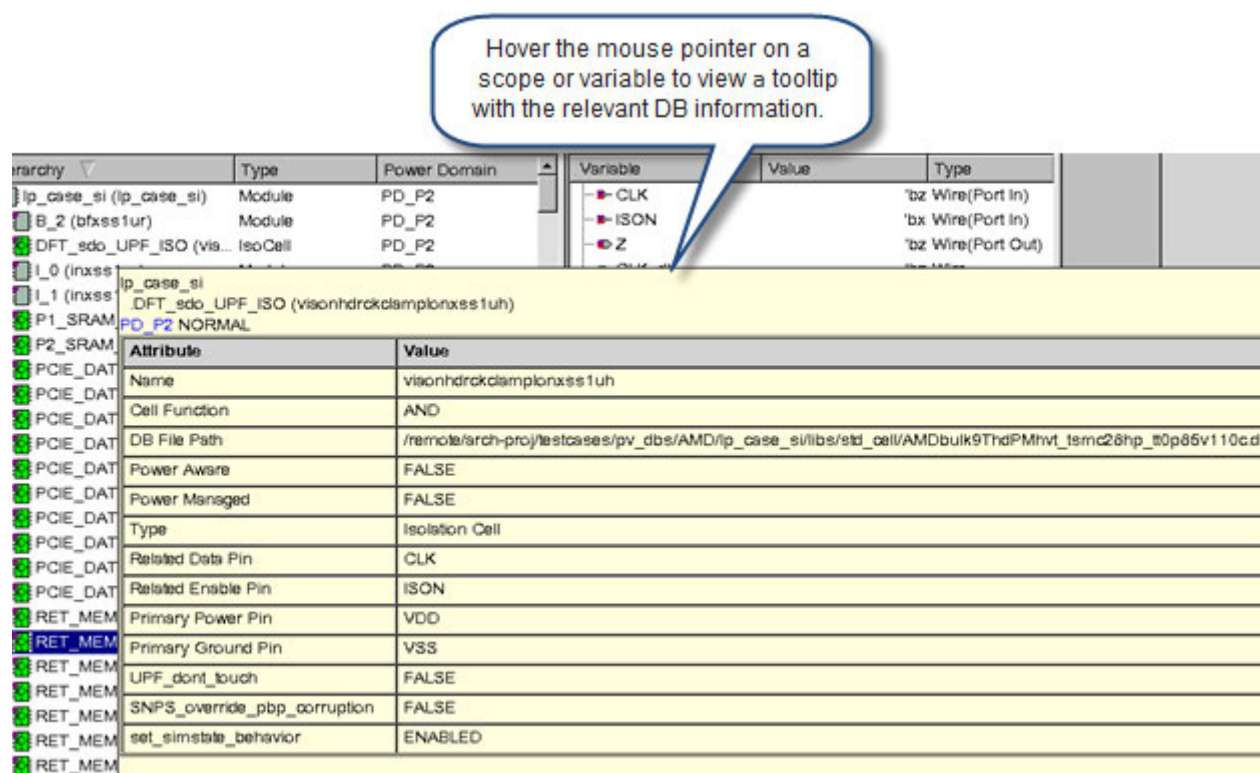


Figure 13-61 Viewing DB Information in a Tooltip



For the normal cells with DB information, DVE displays the tooltip with the relevant DB information, as shown in the [Figure 13-62](#).

Figure 13-62 Viewing DB Information in a Tooltip

Attribute	Value
Name	MACRO_MR
Cell Function	UNKNOWN
DB File Path	macro1.db
Power Aware	TRUE
Power Managed	FALSE
Type	Normal Cell
Primary Power Pin	VDDP, VDDA
Primary Ground Pin	GND
UPF_dont_touch	TRUE
SNPS_override_pbp_corruption	FALSE
set_simstate_behavior	DISABLED

Figure 13-63 Tooltip Information for Variables

Logic Pin		PG Pin	
Attribute	Value	Attribute	Value
Name	Pin Name	Name	Pin Name
Type	<scalar/vector>	Type	Input/Output/reg/wire/supply_net_type/supply0/supply1
Direction	Input/Output	Function	Power/Ground/Back-up
Width	<width>	Power Down Function	Expression, Boolean
Related Power Pin	Pin_Name, Value	Connect Supply Net	<Path_to_Supply_Net>
Related Ground Pin	Pin_Name, Value		
Power Down Function	Expression, Boolean		
Related Supply net	<Path_to_Supply_Net>		

Figure 13-64 Tooltip Information for DB Aware Cells

Isolation Cell		Retention Cell	
Attribute	Value	Attribute	Value
Name	Cell Name	Name	Cell Name
Cell Function	<Function>	Cell Function	<Function>
DB File Path	<path_to_db_file>	DB File Path	<path_to_db_file>
Power Aware	TRUE/FALSE	Power Aware	TRUE/FALSE
Power Managed	TRUE/FALSE	Power Managed	TRUE/FALSE
Type	Isolation Cell	Type	Retention Cell
Primary Power Pin	<Pin_Name>, Value	Related Data Pin	<Pin_Name>, Value
Primary Ground Pin	<Pin_Name>, Value	Retention Save Pin	<Pin_Name>, Value
Related Data Pin	<Pin_Name>, Value	Retention Restore Pin	<Pin_Name>, Value
Related Enable Pin	<Pin_Name>, Value	Primary Power Pin	<Pin_Name>, Value
UPF_dont_touch	TRUE/FALSE	Primary Ground Pin	<Pin_Name>, Value
SNPS_override_pbp_corruption	TRUE/FALSE	Backup Power Pin	<Pin_Name>, Value
set_simstate_behavior	ENABLED/DISABLED	UPF_dont_touch	TRUE/FALSE
		SNPS_override_pbp_corruption	TRUE/FALSE
		set_simstate_behavior	ENABLED/DISABLED
Level-Shifter Cell		Always-On	
Attribute	Value	Attribute	Value
Name	Cell Name	Name	Cell Name
Cell Function	<Function>	Cell Function	<Function>
DB File Path	<path_to_db_file>	DB File Path	<path_to_db_file>
Power Aware	TRUE/FALSE	Power Aware	TRUE/FALSE
Power Managed	TRUE/FALSE	Power Managed	TRUE/FALSE
Type	Level-Shifter Cell (LH/HL/LHL)	Type	Always-On Cell
Related Data Pin	<Pin_Name>, Value	Primary Power Pin	<Pin_Name>, Value
Primary Power Pin	<Pin_Name>, Value	Primary Ground Pin	<Pin_Name>, Value
Primary Ground Pin	<Pin_Name>, Value	Backup Power Pin	<Pin_Name>, Value
Primary Supply Pin	<Pin_Name>, Value	UPF_dont_touch	TRUE/FALSE
UPF_dont_touch	TRUE/FALSE	SNPS_override_pbp_corruption	TRUE/FALSE
SNPS_override_pbp_corruption	TRUE/FALSE	set_simstate_behavior	ENABLED/DISABLED

Low Power Query Commands

You can use low power query commands to build scripts and control low power control signals at runtime.

You can use `lp_show` in the default or Tcl list mode, as shown below:


```
lp_show -mode <TCL_LIST || DEFAULT>
```

By default, the DEFAULT mode is set. You can set the Tcl list mode by using the following option:

```
lp_show -mode TCL_LIST
```

Following is the sample usage for the Tcl list mode:

```
Chronologic VCS simulator copyright 1991-2013
Contains Synopsys proprietary information.
Compiler version H-2013.06; Runtime version H-2013.06; Mar
15 06:30 2013
ucli% lp_show -mode TCL_LIST
Mode set to TCL List

ucli% array set PD1 [lp_show -power_domain -element
testbench/top]
ucli% parray PD1
PD1(Power_Domain) = testbench/top/Island_AlwaysOn
ucli% array set PSW1 [lp_show -info -psw testbench/top/
SW_Dma]
ucli% parray PSW1
PSW1(Ack_Ports) = testbench/top/SW_Dma/PowerGood
PSW1(Control_Ports) = testbench/top/SW_Dma/
PowerGateEn
PSW1(Current_Power_Switch_State) = INVALID
PSW1(Input_Supply_Ports) = testbench/top/SW_Dma/Vin
PSW1(Output_Supply_Port) = testbench/top/SW_Dma/Vout
PSW1(Switch_States) = ON
ucli% q
```

VCS NLP supports the following low power query commands:

Table 13-10 Supported Low Power Query Commands

Command	Function
<code>lp_show -power_top</code>	Returns the scope set as power top.
<code>lp_show -all_power_domains [-scope <ScopeName>]</code>	Lists all Power Domains. If <code>-scope</code> is provided, then list all Power Domains under that scope.
<code>lp_show -power_domain -element <Instance Name></code>	Gets the corresponding power domain for instance.
<code>lp_show -power_domain -element <Net Name></code>	Gets the corresponding power domain for the given supply net.
<code>lp_show -isolation -domain <DomainName></code>	Lists all the isolation strategies for that domain.
<code>lp_show -retention -domain <DomainName></code>	Lists all the retention strategies for that domain.
<code>lp_show -power_switch -domain <DomainName></code>	Lists all power switches for that domain.
<code>lp_show -level_shifter -domain <DomainName></code>	Lists all the level shifter strategies for that domain.
<code>lp_show -all_psts [-scope <ScopeName>]</code>	Lists all PSTs. If <code>-scope</code> is provided, then list all PSTs under that scope.
<code>lp_show -all_supply_sets [-implicit -explicit] -scope <ScopeName></code>	Lists implicit/explicit supply sets based on the option. If no option is provided, then list both.
<code>lp_show -all_port_states -port <PortName></code>	Lists all states of the port.
<code>lp_show -power_state -supply_set <SupplySetName></code>	Lists the power state for the given Supply Set.
<code>lp_show -psw_state -psw <PowerSwitchName></code>	Prints the current state of the power switch.
<code>lp_show -simstate -domain <DomainName></code>	Lists the simstate for the given Power Domain.
<code>lp_show -simstate -supply_set <SupplySetName></code>	Lists the simstate for the given Supply Set.

Command	Function
<code>lp_show -cur_pst_state -pst <PSTName></code>	Lists the current state of the PST.
<code>lp_show -info -domain <DomainName></code>	Displays domain information (primary power/ground net, simstate, and other relevant information).
<code>lp_show -info -iso_strategy <IsoStrategyName></code>	Displays isolation Strategy information (Isolation power/ground net, isolation enable, clamp value, and other relevant information).
<code>lp_show -info -ret_strategy <RetStrategyName></code>	Displays retention Strategy information (Retention power/ground net, save/restore signal/condition/sense, and other relevant information).
<code>lp_show -info -psw <PowerSwitchName></code>	Displays Power Switch information (Input/Output Supply port, Enable, States, Ack, and other relevant information).
<code>lp_show -info -supply_set <SupplySetName></code>	Displays Supply Set information (Power, Ground, supply_set_state, and other relevant information).
<code>lp_show -info -port_state <PortStateName> -port <PortName></code>	Displays information of the port state.
<code>lp_show -info -pst_state <PSTStateName> -pst <PSTName></code>	Displays information of the PST state.
<code>lp_show -info -cell <CellName></code>	Displays low power cell information (Information on DB cell PG pins, power down function, type of cell, and so on).
<code>lp_show -power_ground -element <NetName></code>	Gets the corresponding power and ground for the given supply net.
<code>lp_show -supply_on <SupplyPadName> <voltage></code>	Sets the given supply pad to the mentioned voltage.
<code>lp_show -supply_off <SupplyPadName></code>	Resets the supply for the given supply pad.

Command	Function
<code>lp_show -all_power_scopes</code>	Lists all power scopes present in the given low power design.
<code>lp_show -all_isolations</code>	Lists all isolation strategies present in the given low power design.
<code>lp_show -all_retentions</code>	Lists all retention strategies present in the given low power design.
<code>lp_show -all_level_shifters</code>	Lists all level shifter strategies present in the given low power design.
<code>lp_show -all_power_switches</code>	Lists all power switches present in the given low power design.
<code>lp_show -mode <TCL_LIST DEFAULT></code>	Sets the mode to TCL LIST mode or DEFAULT mode. By default, it is DEFAULT mode.

Limitations

The following features are not supported for the VHDL portion of the design.

- Listing Isolation cells and the Power Switch Instances in the Hierarchy Pane
- Marking of low power signals with red dot in the Data Pane
- Listing isolated signals and shadow registers in the Data Pane

14

Coverage Support for Low Power Objects

The following sections describe coverage support for low power objects:

- [Creating Covergroups for the Low Power Objects](#)
- [Enabling Coverage on Low Power Objects](#)
- [Runtime Control for Low Power Coverage](#)
- [Excluding UPF Objects from Low Power Coverage](#)
- [Constraining Low Power Coverage](#)
- [Post-Processing Options for Low Power Functional Coverage](#)
- [Coverage Reports](#)
- [Dumping Coverage Exclusion Files for Low Power Objects](#)

Creating Covergroups for the Low Power Objects

You can use the `-power=coverage` compile-time option to automatically create covergroups for the low power objects based on the power intent. The covergroups are created for the following low power objects:

- PST states
- Power switch states/transitions
- Supply net/port states/transitions (Root Supply)
 - States defined through `add_port_state` of connected port
 - States inferred from `supply_expr` of `add_power_state` for the connected supply set
- Supply set power states/transitions
 - `primary`
 - `isolation_supply_set`
 - `ret_supply`
- Supply set simstate states/transitions
 - `primary` (domain simstate)
 - `isolation_supply_set`
 - `ret_supply`
 - `default_isolation`
 - `default_retention`
- Isolation enable signal

- Retention SAVE/RESTORE signals
- Power switch control and ACK ports

Note:

For the PST state coverage, final PST state change is sampled if there are multiple PST state changes in the same simulation time stamp.

Enabling Coverage on Low Power Objects

You can use the compile-time options listed in [Table 14-1](#) to selectively enable coverage on the low power objects in a design. This allows you to enable low power coverage from the `vcs` command line without the need to change the UPF frequently.

Table 14-1 *Compile-time Options to Enable Coverage*

Option	Description
<code>-power=cov_pd_simstate</code>	Enable the power domain simstate coverage.
<code>-power=cov_port_state</code>	Enables port state coverage.
<code>-power=cov_implicit_supply_set</code>	Enables coverage on the implicit supply sets only (simstate and power state). Simstate coverage is enabled only for a power domain. Power state coverage is enabled only for the supply sets if the power states are defined using <code>add_power_state</code>. Coverage is disabled for implicit supply sets <code>default_isolation</code> and <code>default_retention</code>.
<code>-power=cov_explicit_supply_set</code>	Enables power state coverage for the explicit supply sets. Power state coverage is enabled only for the supply sets if the power states are defined using <code>add_power_state</code>. Simstate coverage is disabled for the explicit supply sets.
<code>-power=cov_supply_set</code>	Enables all types of supply set coverage.
<code>-power=cov_psw</code>	Enables power switch coverage on the following low power objects: • Power switch state (on/off/partial_on) • ACK port • Control port

Option	Description
-power=cov_pst_state	Enables only the state coverage of PST.
-power=cov_pst_transition	Enables only transition coverage of PST.
-power=cov_pst	Enables Power State Table (PST) coverage. This option enables both the state and transition coverage for PST.
-power=cov_ret	Enables coverage on the retention save and restore signals.
-power=cov_iso	Enables coverage on the isolation enable signal.

You can specify more than one option on the command line using a plus (+) as a delimiter. For example, as shown below:

```
% vcs -power=cov_psw+cov_implicit_supply_set -upf
<upf_file> -sverilog file_name.v
```

The above command enables coverage on the power switch and implicit supply sets.

Note:

The `deselect_coverage_object` design attribute is given higher priority than the command line option. Therefore, if a specific type of low power coverage is enabled using a command line option, and if `deselect_coverage_object` is specified in the UPF file on the same low power object, then VCS NLP do not report coverage on that low power object. For example, consider the following command:

```
% vcs -power=cov_supply_set -upf <upf_file>
-sverilog file_name.v
```

If the UPF file contains the following:

```
set_design_attributes -attribute  
deselect_coverage_object default_supply_set
```

then VCS NLP reports coverage only on the explicit supply sets.

Runtime Control for Low Power Coverage

The following options allow you to disable low power coverage at runtime:

Table 14-2 Runtime Options to Disable Low Power Coverage

Option	Description
-covg_disable_cg[=ALL]	Default behavior. Same as using -covg_disable_cg. This option disables all covergroups (low power and user-defined).
-covg_disable_cg=LP	Disables only low power covergroups.
-covg_disable_cg=USER	Disables only user-defined covergroups.

Excluding UPF Objects from Low Power Coverage

You can use the `deselect_coverage_object` design attribute to specify the low power objects that you want to exclude from the coverage collection.

Use Model

Use the following command in the UPF file to disable the coverage collections for various UPF objects:

```
set_design_attributes -attribute
deselect_coverage_object <Attribute value>
```

Table 14-3 lists the supported attribute values:

Table 14-3 Supported Attribute Values

Attribute Value	Description
power_switch	Turn off power switch coverage
power_state_table	Turn off both PST and port state coverage
supply_port_state	Turn off port state coverage
supply_set	Turn off default and explicit supply set coverage for power state and simstate
default_supply_set	Turn off default supply set coverage for power state and simstate
supply_set_power_state	Turn off coverage for both default and explicit supply set power state
default_supply_set_power_state	Turn off default supply set power state
supply_set_simstate	Turn off coverage for both default and explicit supply set simstate
default_supply_set_simstate	Turn off coverage for default supply set simstate
save_signal	Turn off coverage for save signal
restore_signal	Turn off coverage for restore signal
isolation_enable	Turn off coverage for isolation enable
control_port	Turn off coverage for control port switch
ack_port	Turn off coverage for ack_port switch

Example

The following example code disables the coverage of the default supply set, but not the explicit supply set:

```
set_design_attributes -attribute deselect_coverage_object
default_supply_set
```

Following is the syntax to deselect single coverage object:

```
set_design_attributes -attribute deselect_coverage_object
{value}
```

Example:

```
set_design_attributes -attribute deselect_coverage_object
{control_port}
```

Following is the syntax to deselect multiple coverage objects:

```
set_design_attributes -attribute deselect_coverage_object
{value1} -attribute deselect_coverage_object {value2}
```

Example:

```
set_design_attributes -attribute deselect_coverage_object
{isolation_enable} -attribute deselect_coverage_object
{save_signal}
```

Limitations

Following is the limitation of this feature:

- Instance-based disabling of the low power coverage objects is not supported.

Constraining Low Power Coverage

You can use the `describe_state_transition` UPF command to describe the desired state transitions or sequences.

By default, VCS NLP assumes that all state transitions between legal and user-defined states are valid transitions. You can also use this command to constraint the set of valid transitions and to define illegal transitions.

Note:

If `describe_state_transition` is specified on an object, then the transitions are not generated automatically.

Following is the syntax of the `describe_state_transition` command:

```
describe_state_transition transition_name
-object object_name
{-from {from_list} -to {to_list} | -paired {{from_state to_state}*} |
-from {from_list} -to {to_list} -paired {{from_state to_state}*}}
-through {through_list}
[-legal | -illegal]
```

Options

`transition_name`

Name of the set of transitions for the object defined using this command.

`Object`

VCS NLP supports following objects:

- PST
- Power Switch — on/on_partial/error/off states are considered.

- Supply Set — For supply sets, VCS NLP allows transitions to be captured between Named Power States defined using `add_power_state` and `simstates`. A transition can be from one named power state to another named power state or from one `simstate` to another `simstate`.
- Supply Net — These are states defined using `add_port_state` of a connected port or inferred from `supply_expr` of the `add_power_state` command issued for the supply set to which this supply net is connected.
- Supply Port

`-from, -to`

These options take a list of states. The final transitions are cross of the states in `from_list` and `to_list`. If `from_list` is `{}` and `to_list` has valid states, then the command is expanded to all possible transitions to the states in `to_list`.

Similarly, if `from_list` contains valid states and `to_list` is `{}`, then the command is expanded to all possible transitions from the states in `from_list`.

`-through`

If `-from` and `-to` is used, then `-through` can be used to define intermediate states between `-from` and `-to` states to create a sequence. This is a VCS NLP specific option to capture a state sequence.

`-paired`

Takes a paired list of transitions.

`-illegal | -legal`

This option is used to mark the transitions defined in the command as illegal/legal. Illegal transitions are marked in illegal bins.

Key points to note

- Transitions/Sequences can be generated using `-from`, `-to`, `-through`, or `-paired`
- For `-from/-to`, transitions are generated from each `from_state` to `to_state`
- `-from {} -to {to_states}` or `-from {from_states} -to {}` is supported
- Use `-paired` to specify pair of states for transition
- Use `-through` to specify sequence (from `->` through `->` to)
- A transition involving `ILLEGAL` cannot be tagged legal using `-legal`
- All objects with legal/illegal states/transitions being covered are captured in the [mvsim_native_reports/upf_state_machine.rpt](#) file. You can use this file to verify the output of the `describe_state_transition` command after compile
- The functional coverage on the control ports/signals of a power switch, isolation strategy, and retention strategy is captured in the `upf_control_port_coverage.rpt` file generated in the `mvsim_native_reports` directory. This file contains the details on all the cover group instances for low power functional coverage of the control ports. For more information, see [“Low Power Functional Coverage Report Example”](#) section.

Examples

Consider MYPST has states s1, s2, s3, s4, s5

```
describe_state_transition pst_tran1-object MYPST
-from {s1 s2} -to {s4 s5}
```

Expanded Transitions: {{s1, s4} {s1 s5} {s2 s4} {s2 s5}}

```
describe_state_transition pst_tran2 -object MYPST
-from {} -to {s4}
```

Expanded Transitions: {{s1, s4} {s2 s4} {s3 s4} {s5 s4}}

Usage Example of describe_state_transition

Consider the following PST description:

Supplies:

```
{VDD VDD_LOW VDDS_p0 VDDS_p1 VDD_LOWS_p2
 VDD_LOWS_p3 VDDS_misc VSS}
```

PST States:

```
INIT, P0_BOOT, P0_P1_BOOT, P0_P1_P3_BOOT, ALL_ON,
HIGH_PERF, LOW_PERF, P3_DROWSY, P2_DROWSY,
P1_DROWSY, P0_DROWSY, ULTRA_DROWSY
```

PST states hit during simulation:

```
INIT, ALL_ON, P0_BOOT
```

PST state transitions hit during simulation:

```
INIT->P0_BOOT
```

[Figure 14-1](#) shows PST states for the above PST description.

Figure 14-1 PST States

The screenshot shows a power simulation tool interface. The top panel contains code for creating and adding PST states. The bottom panel shows a tree view of PST states and a table of bin results.

```

4 create_pst LEON3_MP_PST -supplies {VDD VDD_LOW VDD_P0 VDD_P1 VDD_L0WS_P2
  D_L0WS_P3 VDD_misc VSS}
5 add_pst_state INIT -pst LEON3_MP_PST -state {TOP_HV TOP_LV P0_OFF P1_OFF P2_OFF
  /remote/scratch06/vamsid/BOOTCAMP/lab3/leon3mp_2_SAED.retention.fx.upf
  
```

The bottom panel shows a tree view of PST states and a table of bin results.

over Group Item	Score	Bin Name	Status	At Least	Size	Hit Count
PST	12.86%	-illegal_state	Illegal Bin	1	1	
top/wrap63/LEON3_MP_PST.pst_state	12.86%	-INIT	✓	1	1	
top/wrap63/LEON3_MP_PST.pst_state	12.86%	-ALL_ON	✓	1	1	
states	25.00%	-P0_BOOT	✓	1	1	
transitions	0.76%	-ULTRA_DROWSY	✗	1	1	
		-P3_DROWSY	✗	1	1	
		-P2_DROWSY	✗	1	1	
		-P1_DROWSY	✗	1	1	
		-P0_P1_P3_BOOT	✗	1	1	
		-P0_P1_BOOT	✗	1	1	
		-P0_DROWSY	✗	1	1	
		-LOW_PERF	✗	1	1	
		-HIGH_PERF	✗	1	1	

Marking a Transition Illegal

You can use `describe_state_transition` to mark a legal state transition as illegal. For example, you can use the following commands to mark the transition `INIT->P0_BOOT`(transition highlighted in [Figure 14-2](#)) as illegal.

```

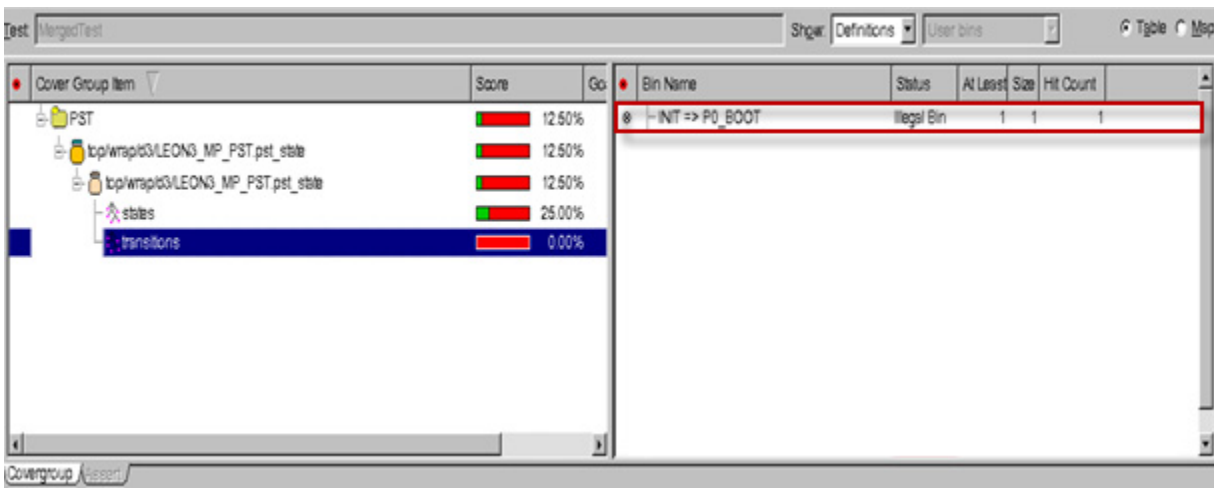
describe_state_transition pst_trans_ill \
-object LEON3_MP_PST \
-paired {{INIT P0_BOOT}} \
-illegal
  
```

or

```

describe_state_transition pst_trans_ill \
-object LEON3_MP_PST \
-from {INIT} \
-to {P0_BOOT} \
-illegal
  
```

Figure 14-2 *describe_state_transition – Marking a Transition Illegal*



Generating a Sequence

You can use `describe_state_transition` to cover a sequence. For example, you can use the following commands to cover the INIT->P0_BOOT->ALL_ON sequence, as shown in [Figure 14-3](#).

```
describe_state_transition pst_seq \
  -object LEON3_MP_PST \
  -from {INIT} -to {ALL_ON} -through {P0_BOOT} \
  -legal
```

Figure 14-3 *describe_state_transition* – Generating a Sequence

The screenshot shows the VCS NLP low power coverage tool interface. The left pane displays a tree view of cover groups under the 'Test' tab. The right pane shows a table of bin results.

Cover Group Item	Score	Goal	Weight
PST	12.50%		
top/wrap/d3/LEON3_MP_PST.pst_state	12.50%		
top/wrap/d3/LEON3_MP_PST.pst_state	12.50%	100%	1
states	25.00%	100%	1
transitions	0.00%	100%	1

Bin Name	Status	At Least	Size	Hit Count
INIT => P0_BOOT => ALL_ON	X	1	1	0

Post-Processing Options for Low Power Functional Coverage

VCS NLP low power coverage feature has resulted in a new classification for the covergroups as follows:

- Covergroups that represent testbench coverage
- Covergroups instrumented for low power coverage

The testbench coverage groups (user groups) are user written while the low power groups (LP groups) are tool generated. Currently, VCS saves both user groups and LP groups to a single coverage test database during simulation.

None of the post-processing tools (UCAPI, URG and DVE) recognize categorization, and displays all the covergroups as a single unit under the testbench coverage metric, with the metric score computed from all of the covergroups.

The following sections describe the filtering mechanism for processing covergroups in the post-processing tools (UCAPI, URG and DVE). Using the proposed options, coverage reports can be customized for low power coverage or testbench coverage.

Filtering Covergroups in URG and DVE

You can use the following URG options to filter covergroups:

- `-group show_lpgroups_only`
- `-group show_usergroups_only`

You can use these options to restrict the set of covergroups to one of the two categories: LP groups and user groups. By default, URG and DVE display both LP and user groups present in the loaded set of databases.

The `-group show_lpgroups_only` option displays only the LP groups and disables all user groups. Similarly, the `-group show_usergroups_only` option displays only the user groups and disables all LP groups.

The disabled covergroups are removed from the universe, that is, the testbench metric score shown in the URG reports and DVE is computed only from the enabled covergroups. Similarly, when grading is applied along with any of the two options, only the enabled set of covergroups are graded and reported.

Use Model

Following are the use model scenarios:

Enable all covergroups:

```
urg -dir <list_of_databases>
```

Enable only low power covergroups:

```
urg -dir <list_of_databases> -group show_lpgroups_only
```

Enable only user written covergroups:

```
urg -dir <list_of_databases> -group show_usergroups_only
```

Complete customization of low power coverage, where only LP groups are considered for coverage and reporting with the rest of the metrics disabled, can be achieved by using the `-metric group` and `-group show_usergroups_only` options in conjunction, as follows:

```
urg -dir <list_of_databases> -metric group -group  
show_lpgroups_only
```

If URG or DVE encounters both options in the command line, an error indicating the incompatibility is issued. Following is the sample error message:

```
Error-[URG-ICF] Incompatible options  
The '-group show_lpgroups_only' option is not compatible  
with the '-group show_usergroups_only' option.  
You may use only one of them. Please refer to the URG user  
guide for information.
```

If the loaded database contains covergroup data, but has no group matching the specified filter option, a warning message such as the following is issued:

```
Warning-[URG-GF-NCGF] No covergroup found for the filter  
None of the specified coverage directories contain  
covergroups matching the filter 'show_lpgroups_only'.
```

If this is not expected, please check if the correct set of coverage directories are specified or change the filter option.

Filtering Covergroups in UCAPI

To support covergroup filtering in UCAPI, the existing API `covdb_configure` is enhanced by adding a new configuration item, `covdbGroupFilter`, to the enum `covdbConfigItemT` located in the UCAPI header file `covdb_user.h`.

This configuration accepts three values: `show_lpgroups_only`, `show_usergroups_only`, and `show_allgroups`. By default, all covergroups are enabled, which makes the last of the three values the default configuration; the first two values are already described in the previous section.

You can reconfigure the value any number of times. The configuration is overwritten and the subsequent iteration on the test handle for covergroups reflects the latest value. Following are the semantics of the API for the three configuration values:

```
covdb_configure(covdbGroupFilter, "show_lpgroups_only")
covdb_configure(covdbGroupFilter, "show_usergroups_only")
covdb_configure(covdbGroupFilter, "show_allgroups")
```

Alternatively, covergroups can be filtered using the `covdb_get_integer_annotation` API. This API can be invoked with the newly added key `is_lp_group` on any covergroup definition, variant, or instance handle. The API returns 1 if the handle is an LP covergroup, 0 otherwise.

```
covdb_get_integer_annotation(GrpHandle, "is_lp_group")
```

Coverage Reports

The following sections describe the low power coverage reports:

- [DVE Coverage Report](#)
- [Low Power Coverage Report View in URG](#)
- [Low Power Functional Coverage Report Example](#)

DVE Coverage Report

DVE coverage allows you to group all the low power coverage items together in DVE, so that it does not interfere with the user-defined functional covergroups. Perform the following steps to view the DVE Coverage GUI, as shown in [Figure 14-4](#).

1. Execute the following command:

```
% dve -cov -dir simv.vdb &
```

2. Select **File > Import User Defined Groups**
3. Double-click the **mvsim_native_reports** folder in the **Import User Defined Groups** dialog box, select **lp_covergroups.tcl**, and then click **Load** to view the DVE Coverage GUI.

Figure 14-4 DVE Coverage GUI with Low Power Cover Groups

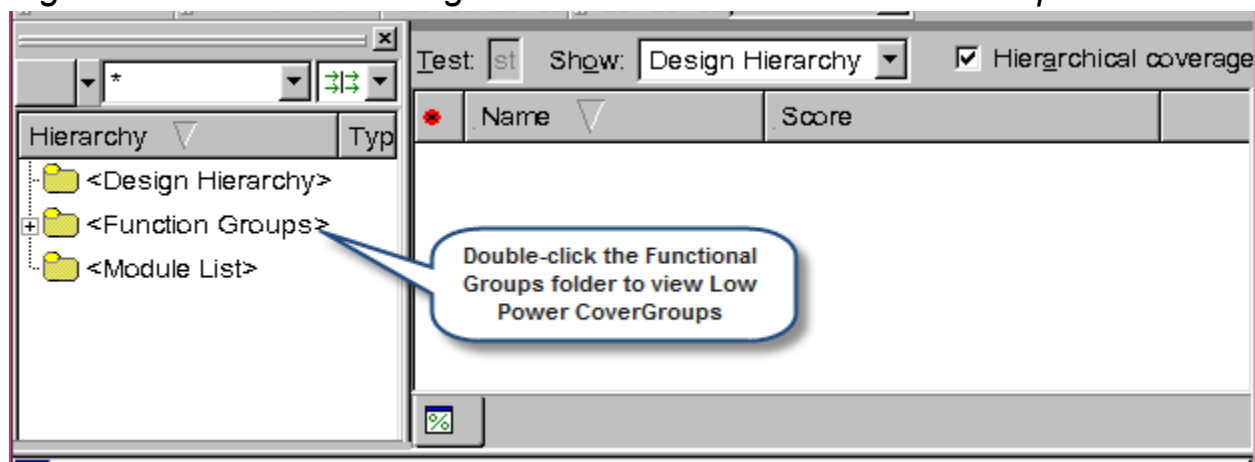
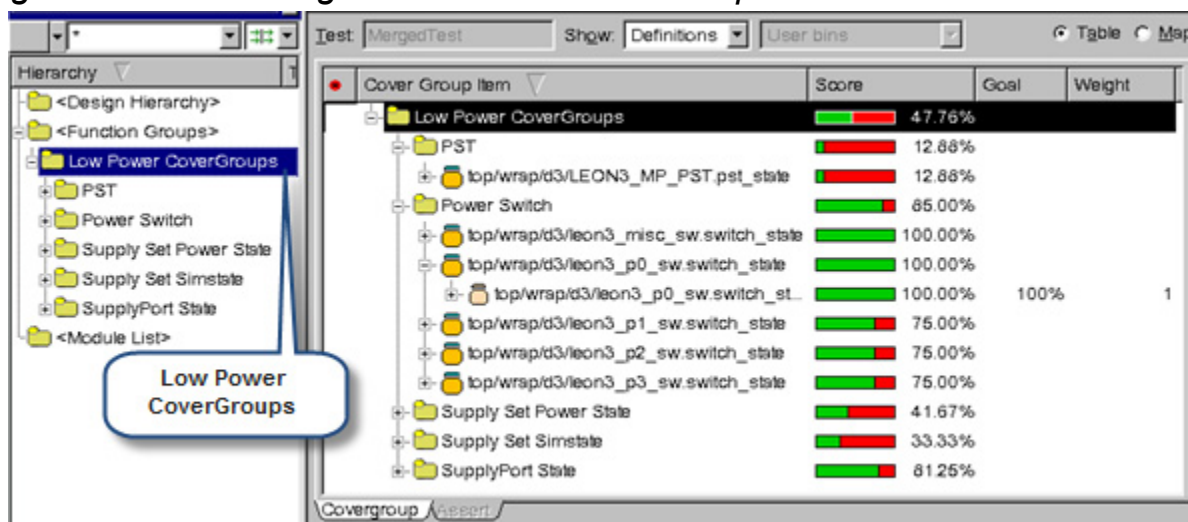


Figure 14-5 Viewing Low Power Cover Groups



Click a transition in the Hierarchy Pane to view its definition in the Source View, as shown in [Figure 14-6](#).

Figure 14-6 Cross-linked with Definition in the UPF File

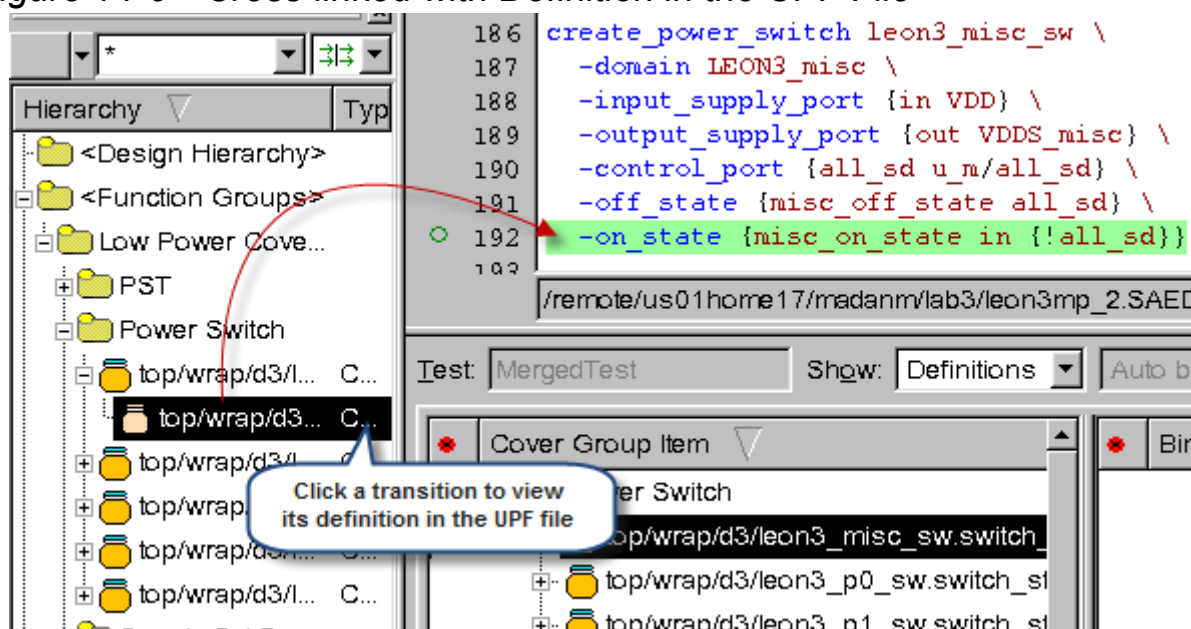
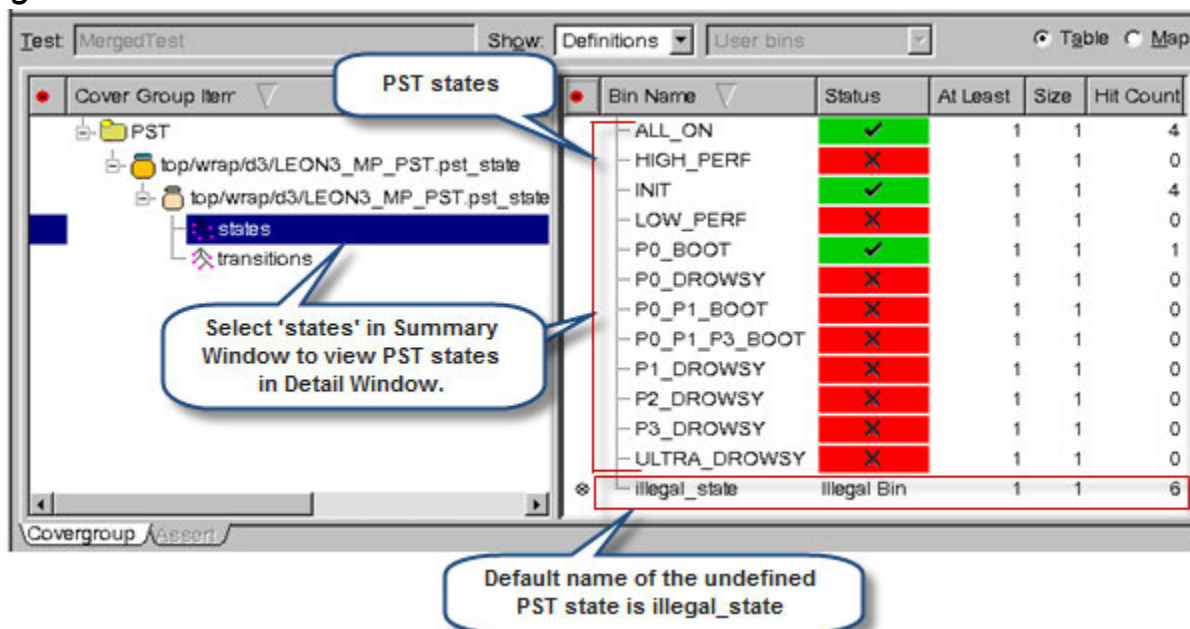


Figure 14-7 shows the PST states in DVE. One bin per PST state is created by default. Illegal bin is created for the Undefined/Illegal PST State.

Figure 14-7 Default States/Transitions – PST States



Low Power Coverage Report View in URG

You can use the `-power=dump_hvp` compile-time option along with `-power=coverage` to dump the coverage database. You can then use the `-lpcov` option to generate low power coverage reports in URG.

Use Model

To dump the necessary information into the coverage database, include the `-power=dump_hvp` compile-time option along with `-power=coverage` on the `vcs` command line, as shown below:

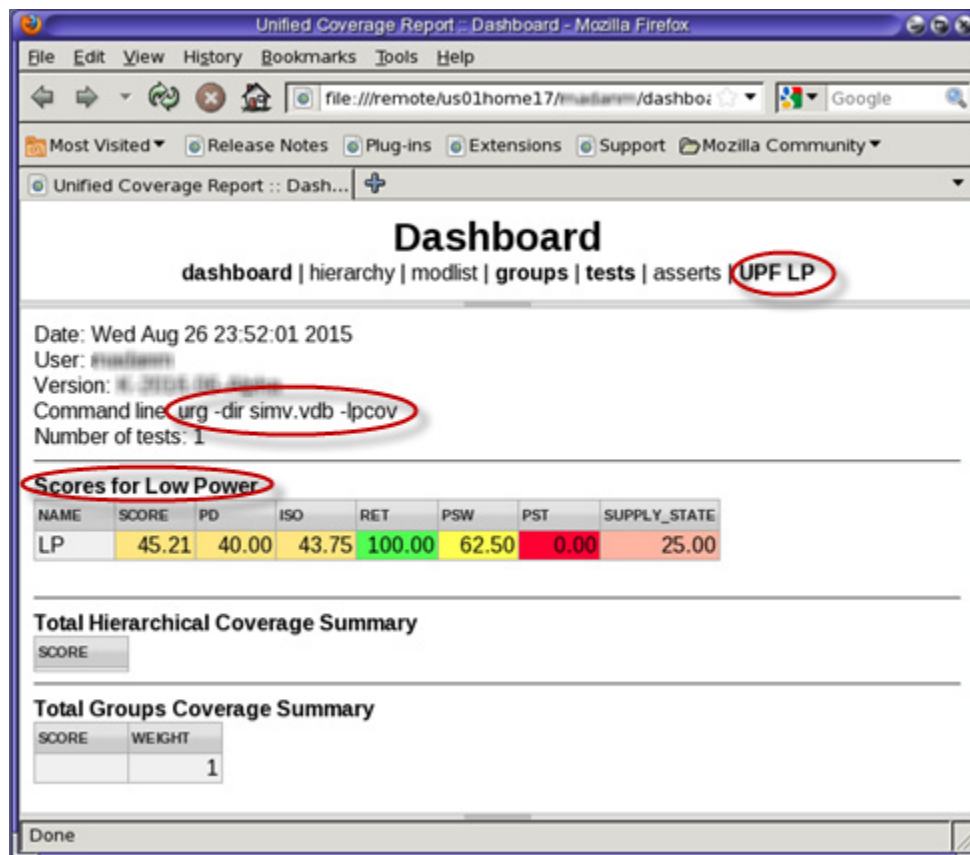
```
% vcs -sverilog -upf ./upf/top.upf \
-power_top top dut.v -power=coverage -power=dump_hvp \
-R
```

To view the low power coverage report in URG, include the `-lpcov` option in the `urg` command line, as shown below:

```
% urg -dir simv.vdb -lpcov
```

The revised dashboard is displayed as shown in [Figure 14-8](#):

Figure 14-8 URG Dashboard With Low Power Coverage



When the `-lpcov` option is used, URG generates the `urgReport/lphvp.LP.html` file. This file displays the low power objects in the Power Domain :: Power Scope format and in a hierarchical manner, as shown in [Figure 14-9](#). Click an object name to go to the Power Scope Detail page or the Power Domain Detail page. Each node can be expanded or collapsed to view the sub-node in the scope.

Figure 14-9 LP Hierarchy Page

NAME	SCORE	PD	ISO	RET	PSW	PST	SUPPLY_STATE
TOP::/	45.21	40.00	43.75	100.00	62.50	0.00	25.00
GENPP::/	25.00	25.00	25.00				
GPRS::/	100.00	100.00	100.00	100.00			
INST::/	25.00	25.00	25.00				
MULT::/	25.00	25.00	25.00				

The Power Scope Detail page displays the coverage scores for all the objects in the respective power scope, as shown in [Figure 14-10](#).

Use the following command to generate the coverage reports in the text format:

```
% urg -dir simv.vdb -format text
```

The `grpinfo.txt` file is generated in the `urgReport` directory. This file contains the coverage information.

Examples for Different Scenarios

Following are the examples for different scenarios:

- [Functional Coverage Report on Isolation Enable](#)
- [Functional Coverage Report on Retention SAVE/RESTORE Signals](#)
- [Functional Coverage Report on Power Switch Control Port](#)

Functional Coverage Report on Isolation Enable

The coverage report displays the following:

- Hierarchical name of the isolation enable signal with a suffix `__ISO_SIG`.
- State information like `ACTIVE_LEVEL`, `INACTIVE_LEVEL`, `ACTIVE_X`, `ACTIVE_Z`

Transitions like `INACTIVE_TO_ACTIVE` and `ACTIVE_TO_INACTIVE`

Following is the sample coverage report of the isolation enable:

```
=====  
Group : tb/dut/GPRS/isolation/gprs_iso_in/  
gprs_iso_in__ISO_SIG
```

```

=====
SCORE  WEIGHT  GOAL
100.00 1      100

-----

Summary for Group  tb/dut/GPRS/isolation/gprs_iso_in/
gprs_iso_in__ISO_SIG

CATEGORY  EXPECTED  UNCOVERED  COVERED  PERCENT
Variables 4          0          4      100.00

Variables for Group  tb/dut/GPRS/isolation/gprs_iso_in/
gprs_iso_in__ISO_SIG

VARIABLE      EXPECTED  UNCOVERED  COVERED  PERCENT  GOAL  WEIGHT
states         2          0          2      100.00  100   1
transitions    2          0          2      100.00  100   1

-----

Summary for Variable states

CATEGORY      EXPECTED  UNCOVERED  COVERED  PERCENT
User Defined Bins 2          0          2      100.00

User Defined Bins for states

Excluded/Illegal bins

NAME      COUNT
ACTIVE_Z  0      Excluded
ACTIVE_X  0      Excluded

Covered bins

NAME      COUNT AT LEAST
INACTIVE_LEVEL 2      1
ACTIVE_LEVEL   1      1

-----

```


Summary for Variable transitions

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	0	2	100.00

User Defined Bins for transitions

Bins

NAME	COUNT	AT LEAST
INACTIVE_TO_ACTIVE	1	1
ACTIVE_TO_INACTIVE	1	1

Functional Coverage Report on Retention SAVE/RESTORE Signals

The coverage report displays the hierarchical name of the retention SAVE/RESTORE signal with a suffix `__RET_SAVE` and `__RET_RESTORE` respectively.

The coverage report does not display information of the `-save_condition` and `-retention_condition`, but the coverage is computed considering these conditions. Other state information and transition information remains same as the isolation enable.

Following is the sample coverage report of the retention SAVE (`__RET_SAVE`) signal:

```
=====
Group : tb/dut/inst_ret/inst_save__RET_SAVE
=====
SCORE  WEIGHT  GOAL
25.00  1       100
```

Summary for Group tb/dut/inst_ret/inst_save__RET_SAVE

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	4	3	1	25.00

Variables for Group tb/dut/inst_ret/inst_save__RET_SAVE

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT
states	2	1	1	50.00	100	1
transitions	2	2	0	0.00	100	1

Summary for Variable states

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	1	1	50.00

User Defined Bins for states

Uncovered bins

NAME	COUNT	AT LEAST	NUMBER
ACTIVE_LEVEL	0	1	1

Excluded/Illegal bins

NAME	COUNT
ACTIVE_Z	0 Excluded
ACTIVE_X	0 Excluded

Covered bins

NAME	COUNT	AT LEAST
INACTIVE_LEVEL	1	1

Summary for Variable transitions

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	2	0	0.00

User Defined Bins for transitions

Uncovered bins

NAME	COUNT	AT LEAST	NUMBER
INACTIVE_TO_ACTIVE	0	1	1
ACTIVE_TO_INACTIVE	0	1	1

Following is the sample coverage report of the retention RESTORE
(__RET_RESTORE) signal:

```
=====
Group : tb/dut/gprs_ret/gprs_nrestore__RET_RESTORE
=====
SCORE  WEIGHT  GOAL
100.00  1      100
```

Summary for Group tb/dut/gprs_ret/
gprs_nrestore__RET_RESTORE

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	4	0	4	100.00

Variables for Group tb/dut/gprs_ret/
gprs_nrestore__RET_RESTORE

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT
states	2	0	2	100.00	100	1
transitions	2	0	2	100.00	100	1

Summary for Variable states

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	0	2	100.00

User Defined Bins for states

Excluded/Illegal bins

NAME	COUNT	
ACTIVE_Z	0	Excluded
ACTIVE_X	0	Excluded

Covered bins

NAME	COUNT	AT LEAST
INACTIVE_LEVEL	2	1
ACTIVE_LEVEL	1	1

Summary for Variable transitions

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	0	2	100.00

User Defined Bins for transitions

Bins

NAME	COUNT	AT LEAST
INACTIVE_TO_ACTIVE	1	1
ACTIVE_TO_INACTIVE	1	1

Functional Coverage Report on Power Switch Control Port

The coverage report displays the following:

- State information like SIGNAL_1, SIGNAL_0, SIGNAL_X, SIGNAL_Z

- Transitions like LOW_TO_HIGH and HIGH_TO_LOW

Following is the sample coverage report of the power switch control port:

```
=====
Group : tb/dut/gprs_sw/gprs_on__PSW_CTRL
=====
SCORE  WEIGHT  GOAL
100.00 1      100
```

```
-----
```

Summary for Group tb/dut/gprs_sw/gprs_on__PSW_CTRL

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	4	0	4	100.00

Variables for Group tb/dut/gprs_sw/gprs_on__PSW_CTRL

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT
states	2	0	2	100.00	100	1
transitions	2	0	2	100.00	100	1

```
-----
```

Summary for Variable states

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
User Defined Bins	2	0	2	100.00

User Defined Bins for states

Excluded/Illegal bins

NAME	COUNT
SIGNAL_Z	0 Excluded
SIGNAL_X	0 Excluded

Covered bins

```

NAME          COUNT AT LEAST
SIGNAL_1 1      1
SIGNAL_0 2      1

```

Summary for Variable transitions

```

CATEGORY          EXPECTED UNCOVERED COVERED PERCENT
User Defined Bins 2          0          2      100.00

```

User Defined Bins for transitions

Bins

```

NAME          COUNT AT LEAST
HIGH_TO_LOW 1      1
LOW_TO_HIGH 1      1

```

Dumping Coverage Exclusion Files for Low Power Objects

You can use the `-dump full_exclusions group URG` option to dump the exclusion files with user-defined covergroups and automated low power covergroups. [Table 14-4](#) describes the exclusion files generated by `-dump full_exclusions group`.

Table 14-4 *Exclusion Files Generated by -dump full_exclusions group*

Exclusion File	Description
fullexclude.tb_def	Contains user-defined covergroups
fullexclude.tb_inst	Contains instances of the user-defined covergroups
fullexclude_lp.tb_def	Contains automated low power covergroups

For more information on the `-dump full_exclusions` option, see *Coverage Technology User Guide*.

15

Gate-level Netlist Simulations (Simulating with Liberty (.db) Files)

In VCS NLP, gate-level netlist simulations are not the same as RTL simulations. In addition to the design (gates and models) and the UPF, VCS NLP can also read some special attributes about the cells from the .db representations. The following sections describe gate-level netlist simulations (GLS) with VCS NLP:

- [Using Connect Supply Net and Library Mapping](#)
- [DB Matching Criteria](#)
- [Supported .db Attributes](#)
- [Power Aware and Non-Power Aware Models](#)
- [Simulation Behavior for Power Aware Models](#)
- [Simulation Behavior for Non-Power Aware Models](#)

- [Implicit Supply Net Connections](#)
- [UPF2SV/SV2UPF Connections](#)
- [Value Conversion Tables \(VCTs\)](#)
- [Multi-Rail Macro Simulation](#)
- [Association of Pre-Inserted Isolation Cells](#)
- [Support for Timing Simulation With SDF in Presence of UPF](#)

Using Connect Supply Net and Library Mapping

Library mapping is used to determine a matching library cell for a Verilog or VHDL module-based architecture. A module is considered as a cell, if a matching library cell for the module exists in a specified database.

The `connect_supply_net` command is used to connect the UPF supply nets to the HDL supply nets in the Verilog or PG cell models.

Use Model

Use the following search path and link library command in the configuration file. You can specify this file as an argument to the `-power_config <file_name>` compile-time option.

```
db_search_path = {path}
```

```
db_link_library = {.db files}
```

Use the following command to run the testcase:

```
vcs -upf out.upf -power_config lp_config
```

Example

The following example shows the usage of the explicit `connect_supply_net` command:

```
create_power_domain Island_Vdd -include_scope
create_power_domain Island_V1 -elements inst1

create_supply_port V1 -domain Island_V1
create_supply_port Vdd -domain Island_Vdd
create_supply_port VSS -domain Island_Vdd

create_supply_net V1 -domain Island_V1

connect_supply_net Vdd -ports { Vdd out_0__UPF_ISO/VDD
out_1__UPF_ISO/VDD out_5__UPF_ISO/VDD out_6__UPF_ISO/VDD
out_7__UPF_ISO/VDD }
connect_supply_net VSS -ports { VSS out_0__UPF_ISO/VSS
out_1__UPF_ISO/VSS out_5__UPF_ISO/VSS out_6__UPF_ISO/VSS
out_7__UPF_ISO/VSS }

create_power_switch V1_header_switch -domain Island_V1
```

In this example UPF, the `connect_supply_net` command has the explicit connection from the UPF supply port `Vdd` to the HDL cell supplies `out_0__UPF_ISO/VDD`..`out_1__UPF_ISO/VDD`, and so on. VCS NLP propagates the state and voltage values of the UPF supply net to the HDL supplies based on the connection.

DB Matching Criteria

DB matching in VCS NLP is stricter than MVSIM-PLI. The following conditions must be satisfied for the cell definition in the `.db` to be termed as matched to the RTL model.

- Name of the Verilog model and db cell must match
- All RTL ports must match the cell logic pins in name and width
- Any RTL port matching db PG pin must not have width greater than 1

Simulation Semantics for Cells

- No corruption semantics are applied on internals of the model
- Corruption semantics of pins is based on the liberty attributes read
- A cell is treated as a black-box for LP instrumentation
- Internal PG pins must be driven by the model in Verilog

Cell information is captured in the `mvsim_native_reports/library_mapping` report.

Matching/non-matching cells are listed in the `mvsim_native_reports/library_match` report.

Supported .db Attributes

VCS NLP supports the following .db attributes. For more information on these attributes, see the *Library Compiler LRM* (Liberty Format). These attributes help VCS NLP to accurately simulate the power-aware behavior of the netlist and UPF.

- **Logic Pins** — These are the input and output ports of the cells, same as the ports listed in the model of the cell. VCS NLP expects the logic pins of .db and the model to match exactly. This is how VCS NLP maps a cell model with the appropriate .db. If the module names match, but the logic pins do not, VCS NLP ignores the .db representation of the cell.
- **PG Pins** — These are the power and ground pins of the cell. These may be primary, backup, or internal PG pins. They may or may not present in the model.
- **Power Down Function** — This is a Boolean function of PG pins, one for each output pin of the cell.
- **Related PG Pin** — The `related_pg_pin` attribute links a logic pin with a PG pin. In effect, it indicates which PG pin controls the logic pin's shutdown behavior.
- **Cell Type** — Following are the special types of cells that VCS NLP supports:
 - `always_on` cells
 - isolation cells
 - level shifter cells
 - retention cells

- power switch cells
- Nor-Style Isolation cells and Nor-Style Enabled Level Shifter (ELS) cells (see the following section).

Support for Nor-Style Cells

In multi-voltage designs, Enabled Level Shifter (ELS) cells are used to achieve the functionality of both level-shifting and isolation. ELS cells with NOR-style isolation are referred as NOR-style ELS cells.

NOR-style isolation is applicable to the isolation strategies with clamp value 0. For more information on NOR-style isolation, see [“NOR-Style Isolation Support”](#).

The following .db attribute supports both NOR-style isolation cells and NOR-style ELS cells. For more information on this attribute, refer to the *Library Compiler LRM* (Liberty Format).

```
alive_during_partial_power_down : true
```

This attribute can be present on the enable pin and output pin of an ELS cell.

If you use this attribute, VCS NLP does not perform input corruption based on `related_power_pin` and `related_ground_pin` of the enable pin.

Example

```
cell (ELS) {  
  
  pin (EN) {  
    related_power_pin : "VDDOUT";  
    related_ground_pin : "VSS";  
  }  
}
```

```

    isolation_cell_enable_pin : true;
    alive_during_partial_power_down : true;
    direction : "input";
}
pin (Z) {
    related_power_pin : "VDDOUT";
    related_ground_pin : "VSS";
    direction : "output";
    alive_during_partial_power_down : true;
    power_down_function : "!VDDIN * (!VDDOUT*!EN) + VSS";
    function : "!(!A + EN)";
} /* end cell */

```

Power Aware and Non-Power Aware Models

The model for a cell may or may not include PG pins. A model which does not contain PG pins is referred to as a “non-power aware” model. If the PG pins are present in a model, they must exactly match what is present in the .db representation. Otherwise, VCS NLP treats it as a non-power aware model.

In a power aware model, the PG pins can be:

- input/output ports of the model
- reg/wire
- supply_net_type
- supply0/supply1 type

VCS NLP uses standard VCTs, as defined in the LRM, for conversion between various types. The default VCS NLP behavior seeks to match the current MVSIM behavior. However, you can use other different VCTs. For more information on VCTs, see [“Value Conversion Tables \(VCTs\)”](#) section.

Simulation Behavior for Power Aware Models

If a model is recognized as power aware, it means that the model itself completely describes the power behavior. That is, VCS NLP does not corrupt anything inside the model. It only drives the appropriate supply values to the PG pins of the cell. The model takes care of corruption. For example, in the following power aware model, the output pin Z is corrupted to x when the PG pin VDD is turned off.

```
module BUFFD0HVT (I, Z, VDD, VSS);  
  
    input I, VDD, VSS;  
    output Z;  
  
    assign Z = (VDD) ? I : 1'bx;  
  
endmodule
```

Simulation Behavior for Non-Power Aware Models

If a model is recognized as non-power aware, VCS NLP corrupts the outputs of the cell based on the power-down function. If there is no power-down function, VCS NLP uses the `related_pg_pin` attribute to corrupt the ports. You can override the above default behavior by using the following `set_design_attributes` commands:

- Use the `SNPS_override_pbp_corruption` design attribute, as shown below, to force VCS NLP to corrupt the ports for all the cells even when the models are power aware.

```
set_design_attributes -attribute \  
SNPS_override_pbp_corruption TRUE
```


- Use the `SNPS_override_pbp_corruption` design attribute, as shown below, to force VCS NLP not to corrupt the ports for all cells even when the models are non-power aware.

```
set_design_attributes -attribute \  
SNPS_override_pbp_corruption FALSE
```

Implicit Supply Net Connections

The UPF may have `connect_supply_net` (CSN) commands to the PG pins of various cells. If there is no CSN in the UPF, the PG pin is assumed to be connected to the primary power or primary ground of the domain that the cell is in.

The isolation cells (ISO/ELS) and retention cells are exceptions to this rule. If the UPF contains isolation strategies, VCS NLP tries to match isolation strategies with the implemented ISO and ELS cells in the netlist. Once an ISO cell is matched with a strategy, the `isolation_power_net` and the `isolation_ground_net` specified in the strategy are implicitly connected to the PG pins of the ISO cell.

In the Synopsys low power flow, explicit CSN is required for the level shifter cells, else the level shifter cells are treated as always-on cells, and no implicit connections are made to the PG pins.

If there is an explicit CSN, that connectivity is honored. Similarly, if there is a retention strategy, VCS NLP tries to associate the retention cell with a strategy. If there is no explicit CSN, the retention power/ground nets in the strategy implicitly drive the backup PG pins of the retention cell.

For the blocks that are not bias enabled using the `enable_bias` design attribute (see [“Enabling Bias Feature”](#)), unconnected bias PG pins (pwell/nwell) are tied to the domain supplies implicitly. pwell is tied to the primary ground, and nwell is tied to the primary power of the domain. To keep the unconnected bias PG pins ON, use the `-power=unconnected_bias_pins_on` compile-time option.

For FDSOI (Fully Depleted Silicon on insulator), both pwell and nwell are tied to the ground supply. To enable FDSOI, set the `is_soi` design attribute to `TRUE` in UPF, as follows:

```
set_design_attributes -models <model_name> -  
attribute is_soi TRUE
```

UPF2SV/SV2UPF Connections

In presence of the appropriate `.db` files, the information in `.db` is considered golden for determining the connection (`sv2upf/`
`upf2sv`). The `pg_type` and/or `direction` attributes decide the drive of the supply connection.

SV2UPF Connection

- In case of DB matched model:
 - `pg_type` attribute is `internal_power/internal_ground` or
 - `direction` attribute is `output`
- In case of Verilog model with no corresponding `.db`:

- For the ports in the same scope, the direction of the Verilog port is input
- For the ports in the lower scope, the direction of the Verilog port is output
- Connection to a non-port type in the same scope

UPF2SV Connection

- In case of DB matched model:
 - `pg_type` in `.db` is not `internal_power/`
`internal_ground`
- In case of Verilog model with no corresponding `.db`:
 - For the ports in the same scope, the direction of the Verilog port is output
 - For the ports in the lower scope, the direction of the Verilog port is input
 - Connection to a non-port type in the lower scope

Value Conversion Tables (VCTs)

A value conversion table (VCT) defines how values are mapped for the connections from the UPF to the HDL model or from the HDL model to the UPF.

You can use the `-vct` option of the `connect_supply_net` command to specify VCT.

UPF supply net can be connected to:

- `wire/reg/logic/UPF::supply_net_type` in Verilog
- `std_logic/std_ulogic/bit/UPF::supply_net_type` in VHDL
- VCT is not required for connection to `UPF::supply_net_type` port/net in HDL.

Supported VCTs

The following table describes the supported VCTs.

Table 15-1 Supported VCTs

Type	VCT	Table
HDL2UPF	SV_LOGIC2UPF *	1 - FULL_ON 0 - OFF z - UNDETERMINED x - UNDETERMINED
	SV_LOGIC2UPF_GNDZERO **	1 - OFF 0 - FULL_ON z - UNDETERMINED x - UNDETERMINED
	SV_LOGIC2UPF_MD	1 - FULL_ON 0 - OFF z - OFF x - UNDETERMINED
	SV_LOGIC2UPF_GNDZERO_MD	1 - OFF 0 - FULL_ON z - OFF x - UNDETERMINED
	SV_LOGIC2UPF_2S	1 - FULL_ON 0 - OFF z - OFF x - OFF

Table 15-1 Supported VCTs

	SV_LOGIC2UPF_GNDZERO_2S	1 - OFF 0 - FULL_ON z - OFF x - OFF
UPF2HDL	SV_TIED_HI	FULL_ON - 1 OFF - $\bar{X}$ PARTIAL_ON - X UNDETERMINED - X
	SV_TIED_LO	FULL_ON - 0 OFF - $\bar{X}$ PARTIAL_ON - X UNDETERMINED - X
	UPF2SV_LOGIC ^	FULL_ON - 1 OFF - $\bar{0}$ PARTIAL_ON - X UNDETERMINED - X
	UPF_GNDZERO2SV_LOGIC ^^	FULL_ON - 0 OFF - $\bar{1}$ PARTIAL_ON - X UNDETERMINED - X
	VHDL_TIED_HI	FULL_ON - 1 OFF - $\bar{X}$ PARTIAL_ON - X UNDETERMINED - X
	VHDL_TIED_LO	FULL_ON - 0 OFF - $\bar{X}$ PARTIAL_ON - 0 UNDETERMINED - X
	UPF2VHDL_SL	FULL_ON - 1 OFF - $\bar{0}$ PARTIAL_ON - X UNDETERMINED - X
	UPF_GNDZERO2VHDL_SL	FULL_ON - 0 OFF - $\bar{1}$ PARTIAL_ON - X UNDETERMINED - X

* **SV_LOGIC2UPF** is the default HDL2UPF VCT used for the power ports.

**** SV_LOGIC2UPF_GNDZERO** is the default HDL2UPF VCT used for the ground ports.

^ UPF2SV_LOGIC is the default UPF2HDL VCT used for the power ports.

^^ UPF_GNDZERO2SV_LOGIC is the default UPF2HDL VCT used for the ground ports.

Attributes Supported for Overriding the Default VCTs

Table 15-2 Attributes supported for overriding the default VCTs

Attribute	Type
SNPS_default_power_sv2upf_vct	SV2UPF for power ports
SNPS_default_ground_sv2upf_vct	SV2UPF for ground ports
SNPS_default_power_upf2sv_vct	UPF2SV for power ports
SNPS_default_ground_upf2sv_vct	UPF2SV for ground ports
SNPS_default_power_upf2vh_vct	UPF2VH for power ports
SNPS_default_ground_upf2vh_vct	UPF2VH for ground ports

Changing default VCT for the whole design:

```
set_design_attributes -attribute <attribute>  
<vct_name>
```

Changing default VCT for a DB Cell:

```
set_design_attributes -models <model_name> -  
attribute <attribute> <vct_name>
```

Changing VCT for a particular supply net connection:

```
connect_supply_net <net_name> -ports {<port_list>}  
-vct {<vct_name>}
```

Precedence of VCT in High to Low Order

1. VCT in CSN
2. VCT of the cell, if defined
3. VCT of the design, if defined
4. Default VCT

Limitations

Following are the limitations of this feature:

- Defining custom VCTs is not supported
- For HDL2UPF, `-vct` with `connect_supply_net` cannot be used for the supply net connections to a DB matched cell port

Multi-Rail Macro Simulation

Multi-Rail Macro (MRM) simulation associates different HDL supply rail connections to different PG ports. The PG cell liberty contains the related power pin and ground pin for input and output ports. The `connect_supply_net` command establishes connection from the UPF supply ports to the HDL supplies.

Use Model

The following are the inputs that you require to setup the Multi-Rail Macro simulation:

- The `-power_config` file.

Note:

For MRM, you must specify the link library and database (DB) path of the MRM cells in the `-power_config` file, which is similar to `connect_supply_net` and library mapping feature.

- UPF compliant liberty DB's for all Multi-Rail Macros.

Note:

Only liberty DB's are supported. Library files (`.lib`) are not supported.

- UPF with `connect_supply_net` statements for the power pins of all MRM instances.
- If required, use the `set_related_supply_net` command for the boundary port of the design.

You require these inputs only when the primary input/output ports are not associated with the same supply rail as the power domain, in which the design top is partitioned.

Usage Example

The following is the usage example of the MRM cell:

```
cell (mad3amux2x1) {
    area : 1316.044800;
    dont_use : true;
    dont_touch : true;
    interface_timing : true;
    timing_model_type : "abstracted";
    pg_pin (VDDIO) {
        pg_type : "primary_power";
        voltage_name : "VDDIO";
    }
    pg_pin (VDDR) {
        pg_type : "backup_power";
        voltage_name : "VDDR";
    }
    pg_pin (VSS) {
        pg_type : "primary_ground";
        voltage_name : "VSS";
    }
    pin ("In0_VIO_Q9") {
        direction : "input";
        capacitance : 238.046500;
        max_transition : 56.456710;
        related_power_pin : VDDR;
        related_ground_pin : VSS;
    }
}
```

```

    pin ("In1_VIO_Q9") {
        direction : "input";
        capacitance : 203.497300;
        max_transition : 56.545580;
        related_power_pin : VDDR;
        related_ground_pin : VSS;
    }
    pin ("LowPower_VIO") {
        direction : "input";
        capacitance : 10.103840;
        max_transition : 56.483670;
        related_power_pin : VDDR;
        related_ground_pin : VSS;
    }
    pin (Sel) {
        direction : "input";
        capacitance : 3.405169;
        max_transition : 56.473290;
        related_power_pin : VDDR;
        related_ground_pin : VSS;
    }
    pin ("Out0_VIO_Q9") {
        direction : "output";
        related_power_pin : VDDR;
        related_ground_pin : VSS;
    }
    pin ("Out1_VIO_Q9") {
        direction : "output";
        related_power_pin : VDDIO;
        related_ground_pin : VSS;
    }
}
default_operating_conditions : "BALANCED";
/* Wire length (mm) for 8 fanouts */
default_wire_load : "DEFAULT";
}

```

In the above example, all pins are connected to power pin VDDR, except for Out1_VIO_Q9, which is connected to VDDIO.

The following is the example UPF:

```
## CREATE POWER DOMAINS ##
create_power_domain Island_V1 -elements inst1
create_power_domain Island_V2 -elements inst2
create_power_domain Island_V3 -elements inst3
create_power_domain Island_multi_rail -elements
MULTI_RAIL_MACRO
create_power_domain Island_VTOP

# CREATE SUPPLY PORTS  ##
create_supply_port VTOP_VDD
create_supply_port VTOP_VSS
create_supply_port V1_VDD
create_supply_port V2_VDD
create_supply_port V3_VDD
create_supply_port MULTI_RAIL_PRIMARY_POWER

## CREATE SUPPLY NETS  ##
create_supply_net V1 -domain Island_V1
create_supply_net V2 -domain Island_V2
create_supply_net V3 -domain Island_V3
create_supply_net MULTI_RAIL_PRIMARY_POWER -domain
Island_multi_rail

create_supply_net VDD -domain Island_VTOP
create_supply_net VSS -domain Island_VTOP
create_supply_net VSS -domain Island_V1 -reuse
create_supply_net VDD -domain Island_V1 -reuse
create_supply_net VSS -domain Island_V2 -reuse
create_supply_net VSS -domain Island_V3 -reuse
create_supply_net VSS -domain Island_multi_rail -reuse

## CONNECT SUPPLY NETS  ##
connect_supply_net VDD -ports {VTOP_VDD MULTI_RAIL_MACRO/
VDDR}
connect_supply_net VSS -ports {VTOP_VSS MULTI_RAIL_MACRO/
VSS}
connect_supply_net V1 -ports V1_VDD
connect_supply_net V2 -ports V2_VDD
connect_supply_net V3 -ports V3_VDD
connect_supply_net MULTI_RAIL_PRIMARY_POWER -ports
```

```
{MULTI_RAIL_PRIMARY_POWER MULTI_RAIL_MACRO/VDDIO}

## DOMAIN SUPPLY NET      ##
set_domain_supply_net Island_V1 -primary_power_net V1 -
primary_ground_net VSS
set_domain_supply_net Island_V2 -primary_power_net V2 -
primary_ground_net VSS
set_domain_supply_net Island_V3 -primary_power_net V3 -
primary_ground_net VSS
set_domain_supply_net Island_VTOP -primary_power_net VDD -
primary_ground_net VSS
set_domain_supply_net Island_multi_rail -primary_power_net
MULTI_RAIL_PRIMARY_POWER -primary_ground_net VSS
```

In the above UPF, the MRM pin Out1_VIO_Q9's power pin VDDIO is connected to a different UPF supply port, and pins VDDR and VSS are connected to a different UPF supply port.

For the Multi-Rail Macro simulation to work, you must define the DB search path and link library, as you do for `connect_supply_net` and library mapping.

Note:

DB matching is done for Verilog portion of the design only.

Configuring Port-Based Corruption on Macros

You can use the `SNPS_macro_port_based_corruption` design attribute at compile time to force/prevent port-based corruption (by setting it to `ENABLE` or `DISABLE`) for a particular macro or for the full design.

By default, port-based corruption is enabled for the cells identified as non-power-aware, and disabled for the power aware cells.

Use Model

Use the following command to enable port-based corruption for all the cells in the design:

```
set_design_attributes -attribute  
SNPS_macro_port_based_corruption ENABLE
```

Use the following command to disable port-based corruption for all the cells in the design:

```
set_design_attributes -attribute  
SNPS_macro_port_based_corruption DISABLE
```

Use the following command to enable port-based corruption for specific macro(s):

```
set_design_attributes -models <list_of_models> -  
attribute SNPS_macro_port_based_corruption ENABLE
```

Use the following command to disable port-based corruption for specific macro(s):

```
set_design_attributes -models <list_of_models> -  
attribute SNPS_macro_port_based_corruption DISABLE
```

Note:

Any usage of the `set_design_attributes`
`SNPS_override_pbp_corruption` and/or
`set_simstate_behavior` command in UPF is ignored if the
usage of `set_design_attributes`
`SNPS_macro_port_based_corruption` is found.

Example

Example 15-1 Verilog Design Example

```
module MACRO_MR (A, CLK, SLEEP, RST, Q, VDDP, VDDA, GND);
    output Q;
    input A;
    input CLK;
    input SLEEP;
    input RST;
    input VDDP, VDDA, GND; // all PG pins defined in Verilog

    reg Q;
    reg Qint, A_wire;
    .....
    .....
```

UPF example (enabling port-based corruption on a power aware macro):

```
set_design_attributes -models {MACRO_MR} -attribute
SNPS_macro_port_based_corruption ENABLE
```

Refer to the PortBasedCorruption field of the cell in the mvsim_native_reports/library_mapping_after_upf.rpt report, to identify if the port-based corruption is ENABLED/DISABLED.

```
mvsim_native_reports/library_mapping_after_upf.rpt
```

```
Cell: MACRO_MR
Library = MACRO1_LIB: Library Path = macro1.db
```

```
Cell Id = 1   Cell Type: Normal Cell
Cell Function: UNKNOWN
IsPowerAwareModuleCell: true
IsNameBasedMatchingDone: No
PortBasedCorruption: Enabled
```

Association of Pre-Inserted Isolation Cells

VCS NLP performs the following:

- Associates an existing isolation cell in the RTL/netlist with the corresponding isolation strategy.
- Generates an association summary report (`mvsim_native_reports/isolation_association_summary.rpt`) capturing the details of the associated isolation cell instances and their associated isolation strategies. This report also captures the list of isolation cells that could not be associated with any isolation strategy in the UPF.
- Inserts the virtual isolation cells for cases where pre-inserted isolation cell is not found or could not be associated.
- Skips the insertion of the virtual isolation cells when the `-power=accurate` compile-time option is specified.

Note:

- For association, liberty definition (.db) must be provided for the pre-inserted Isolation/ELS cells.
- For each boundary port in the report (`mvsim_native_reports/isolation_association.rpt`), `-instance` points to the associated isolation cell instance name.
- Unassociated cell PG pins are tied to the domain supplies.

- To treat the unassociated Isolation/ELS cells as always-on, you must pass the below design attribute at runtime through the power configuration file:

```
set_design_attributes -attribute
SNPS_keep_unassociated_iso_on TRUE
```

```
% simv -power config.tcl
```

- The following factors are considered as hard constraints for association:

```
-location
```

```
-applies_to
```

These factors should match between the isolation cell in the design and the isolation strategy in the UPF.

Examples

The following example design code contains isolation cells and isolation cells inferred by VCS NLP based on the UPF:

Example 15-2 Isolation Cells in the Low Power Design

```
top.v
//isolation cell already present in the rtl

DMISAB2 cnt1_0_iso(
    .A(cnt_out1_iso_out[0]),
    .B(iso_en),
    .Z(cnt_out1[0])
);

DMISAB2 cnt1_1_iso(
    .A(cnt_out1_iso_out[1]),
    .B(iso_en),
```



```

        .Z(cnt_out1[1])
    );

iso.upf

//isolation strategy specified in the upf

set_isolation iso_pd3 \
    -domain PD3 \
    -clamp_value 0 \
    -applies_to both

set_isolation_control iso_pd3 \
    -domain PD3 \
    -isolation_signal iso_en \
    -isolation_sense high \
    -location self

```

Following is the sample output of the
isolation_association.rpt report file:

```

// information on isolation cells present in the rtl

# Number of Locations for Instrumentation: 1
set ISO_SIGNET_PORT(3) { { SEQ/cnt_out1[1] -policy PD1/
isolation/iso_pd1 -domain PD1 }
{ -isolation_signal iso_en -isolation_sense high -
clamp_value 0 -location parent }
{ -instrumented_node cnt_out1_iso_out[1] -instance
testbench.DUT.cnt1_1_iso -driver_supply PD1_prim -
receiver_supply TOP_prim } }

# Number of Locations for Instrumentation: 1
set ISO_SIGNET_PORT(4) { { SEQ/cnt_out1[0] -policy PD1/
isolation/iso_pd1 -domain PD1 }
{ -isolation_signal iso_en -isolation_sense high -
clamp_value 0 -location parent }
{ -instrumented_node cnt_out1_iso_out[0] -instance
testbench.DUT.cnt1_0_iso -driver_supply PD1_prim -
receiver_supply TOP_prim } }

```

```
// information on the isolation cells inferred by VCS NLP
based on isolation strategy
```

```
# Number of Locations for Instrumentation: 1
set ISO_SIGNET_PORT(13) { { COMB/or2bit[0] -policy PD3/
isolation/iso_pd3 -domain PD3 }
{ -isolation_signal iso_en -isolation_sense high -
clamp_value 0 -location self }
{ -instrumented_node COMB/or2bit[0] -driver_supply PD3_prim
-receiver_supply TOP_prim } }
```

```
# Number of Locations for Instrumentation: 1
set ISO_SIGNET_PORT(14) { { COMB/and2bit[1] -policy PD3/
isolation/iso_pd3 -domain PD3 }
{ -isolation_signal iso_en -isolation_sense high -
clamp_value 0 -location self }
{ -instrumented_node COMB/and2bit[1] -driver_supply
PD3_prim -receiver_supply TOP_prim } }
```

Support for Timing Simulation With SDF in Presence of UPF

VCS NLP supports the following simulation semantics in presence of the UPF:

- Corruption is applied immediately on an element without any delay when the power domain changes its state from `NORMAL` to `CORRUPT`
- Timing checks on elements of a power domain are turned off when a domain simstate changes from `NORMAL` to `CORRUPT`.
- For the non-power-aware DB cells, the timing checks on the logic ports are turned off if the corresponding related power or ground pins are turned off

- For power-aware cells, timing checks are not turned off automatically. User must ensure that the PG pins are part of the timing check definition
- When a simstate transitions from `CORRUPT` to `NORMAL`, timing checks are turned on and all delays are re-evaluated, similar to the time 0 behavior
- All child instances within a DB cell are treated as non-power-managed, hence no specific action related to the timing checks is taken

You can use the `-power=dump_pd_tcheck_rpt` compile-time option to capture the information related to the timing checks on elements of a power domain in the `pd_tcheck_mapping.rpt` file generated in the `mvsim_native_reports` directory.

Following is the message format for the disabled timing checks:

```
[Time] [INFO] [LP_TIMING_CHECKS_OFF] All timing checks on
elements inside power domain '<DOMAIN_NAME>' are turned OFF
as the domain simstate changed from NORMAL to CORRUPT.
```

For example,

```
[28 ns] [INFO] [LP_TIMING_CHECK_OFF] All timing checks on
elements inside power domain 'tb/inst/Island_Vdd' are turned
OFF as the domain simstate changed from NORMAL to CORRUPT.
```

Following is the message format for the enabled timing checks:

```
[Time] [INFO] [LP_TIMING_CHECKS_ON] All timing checks on
elements inside power domain '<DOMAIN_NAME>' are turned ON
as the domain simstate changed from CORRUPT to NORMAL.
```

For example,

```
[48 ns] [INFO] [LP_TIMING_CHECK_ON] All timing checks on
elements inside power domain 'tb/inst/Island_Vdd' are turned
ON as the domain simstate changed from CORRUPT to NORMAL.
```

Following is the report format for the timing checks. This report contains the mapping between instances having timing checks and power domain to which it belongs.

Power domain <Full path of power domain> has the following instances having timing checks.

<Instance name that has timing checks one per line>

For example,

Power domain "tb/pd" has the following instances having timing checks.

"tb/inst"

16

Synopsys Low Power Flow Consistency

The following sections describe Synopsys low power consistency in detail:

- [Synopsys Low Power Flow Consistency Check](#)
- [-simulation_only Support](#)
- [Error Checking for Isolation Strategy with -location parent on Top-level Ports](#)
- [Handling Conflicting Port State Names in the add_port_state Command](#)

Synopsys Low Power Flow Consistency Check

You can use the `-power=snps_lp_flow` compile-time option to maintain consistency of the UPF across the Synopsys Low Power Flow. If you use this option, the UPF commands/options listed in [Table 16-1](#) result in an error as these are not supported across all the tools in the Synopsys Low Power flow.

Use Model

```
%vcs <vcs_options> -upf <upf_file> -  
power=snps_lp_flow
```

Following is the error code for the UPF command/option:

```
Error- [UPF_UMFCO] Unsupported mode for Command/  
option
```

Table 16-1 Options that error out with -power=snps_lp_flow

Commands	Options
add_power_state	-update
set_isolation	-clamp_value Z
set_isolation_control	-location automatic
set_level_shifter	-source, -sink
set_retention_control	-save_signal/-restore_signal <posedge/ negedge>
set_simstate_behaviour	

-simulation_only Support

If you use the `-power=sim_only` compile-time option, the UPF commands, options, and attributes listed in [Table 16-2](#) result in parsing error if they are not part of the UPF file that is loaded with the following command:

```
load_upf <upf_file> -simulation_only
```

Following is the error code for the UPF command/option:

```
Error- [UPF_UMFCO] Unsupported mode for Command/  
option
```

Following is the error code for attribute:

```
Error- [UPF_UMFCA] Unsupported mode for Attribute
```

Table 16-2 Options Supported with -simulation_only

Commands	Options/Attributes
bind_checker	All options and custom options
load_upf	-model
create_power_domain	-simulation_only
set_design_attributes	Simulation-only: UPF_dont_touch SNPS_override_pbp_corruption SNPS_reinit SNPS_dont_reinit Non-simulation-only: (SNPS_default_power_upf2sv_vct, SNPS_default_ground_upf2sv_vct, SNPS_default_power_upf2vh_vct, SNPS_default_ground_upf2vh_vct, SNPS_default_power_sv2upf_vct, SNPS_default_ground_sv2upf_vct, iso_source, iso_sink, related_supply_default_primary)

Error Checking for Isolation Strategy with -location parent on Top-level Ports

When an isolation strategy applies to a top-level port, the isolation strategy can be implemented only in the “self” location that is visible in the design scope.

VCS NLP generates an error message when an isolation strategy with location “parent” is specified on the top-level ports of the design.

Example

```
set_design_top tb/dut
create_power_domain PD_TOP -include_scope
..
```



```
set_isolation PD_ISO -domain PD_TOP
set_isolation_control PD_ISO -domain PD_TOP -
isolation_signal iso_sig -location parent
```

Error Message

```
Error-[UPF_IOPTP] Isolation on the parent of power top
TOP.upf, 48
Isolation policy 'PD_ISO' of power domain 'PD_TOP' is
specified on power top instance 'tb/dut'. Isolation policy
with location 'parent' cannot be specified on the parent of
power top instance.
Please consider specifying the policy with location 'self'.
```

Handling Conflicting Port State Names in the add_port_state Command

You can use the `add_port_state` command to add state information to a supply port.

Syntax

```
add_port_state port_name
    {-state {name <nom | min max | min nom max | off>}}
```

The `name` argument defines the name for a state of the supply port.

Two supply ports which are at two different scopes can be connected using a supply net. The supply net serves as a way to propagate voltage between these two supply ports.

You cannot use the same port state name with different voltage values for multiple supply ports tied together. VCS generates the `Duplicate Port States` error message for such usage. See the following examples for more information.

Example-1: In this example, the name of the supply port state 'ON' is used to refer to two voltage values, therefore VCS generates the following error message:

Error- [UPF_CPSE] Conflicting supply states error

top.upf

```
set_design_top TB/DUT
..
add_port_state V1 -state {ON 2.0}
connect_supply_net VT -ports {V1 child/V2}
..
load_upf child.upf -scope child
```

child.upf

```
create_supply_set ..
..
add_port_state V2 -state {ON 4.0}
..
```

Example-2: This example describes the proper usage.

top.upf

```
set_design_top TB/DUT
..
add_port_state V1 -state {ON 2.0}
connect_supply_net VT -ports {V1 child/V2}
..
load_upf child.upf -scope child
```

child.upf

```
create_supply_set ..
..
add_port_state V2 -state {ON1 4.0}
```

..

Example-3: In this example, different port state names are used to refer the same voltage value. Two state names with same voltage value are aliased to one state name, and the following warning message is generated:

Warning-[UPF_MSSV] Multiple states with same voltage values

top.upf

```
set_design_top TB/DUT
..
add_port_state V1 -state {ON 2.0}
connect_supply_net VT -ports {V1 child/V2}
..
load_upf child.upf -scope child
```

child.upf

```
create_supply_set ..
..
add_port_state V2 -state {ON_1 2.0}
```

Example-4: In this example, same port state names are used to refer to the same voltage value. One of the definitions is considered while the other is ignored, and the following note message is generated:

Note-[UPF_DPSSV] Duplicate supply port state

top.upf

```
set_design_top TB/DUT
..
add_port_state V1 -state {ON 2.0}
connect_supply_net VT -ports {V1 child/V2}
..
load_upf child.upf -scope child
```

child.upf

```
create_supply_set ..
..
add_port_state V2 -state {ON 2.0}
```

17

Connecting UPF Supply Network to Wreal Supply Ports

VCS NLP allows you to connect wreal signals to the UPF supply network using the `connect_supply_net` UPF command.

This feature allows you to connect wreal netlist to the UPF supplies in a mixed-signal design environment.

The following sections describe the use model:

- [Connecting UPF Supply Nets to Wreal Input and Inout Ports](#)
- [Connecting UPF Supply Nets to Wreal Output Ports](#)

Connecting UPF Supply Nets to Wreal Input and Inout Ports

You can use the `connect_supply_net` command to connect UPF supply nets to wreal input and inout ports.

VCS NLP considers `connect_supply_net` to wreal inout connection as `connect_supply_net` to wreal input connection.

As per the UPF resolution semantics, resolved value of the UPF driver is propagated to the wreal port after being converted through CSN-TO-WREAL Value Conversion Table (VCT). Propagated value is resolved with other wreal drivers (if any) according to the wreal resolution semantics.

Figure 17-1 Connecting UPF supply Nets to Wreal Input Ports

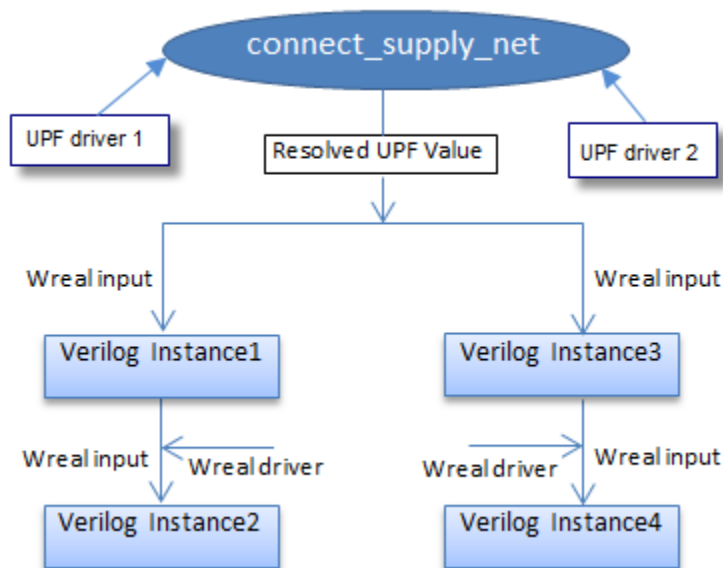
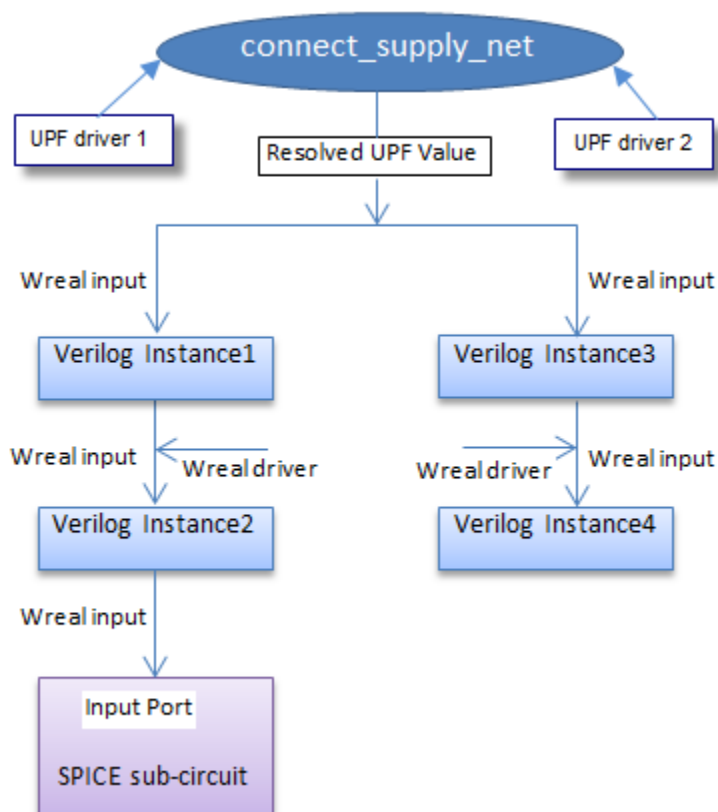


Figure 17-1 shows UPF to wreal supply connection in Verilog hierarchy.

VCS NLP also supports connecting UPF supply nets to the SPICE blocks through wreal network. Figure 17-2 shows UPF to wreal to SPICE connection through the Verilog hierarchy.

Figure 17-2 UPF->Wreal->SPICE Connection Through Verilog Hierarchy



Example

Example 17-1 test_input.v

```

`timescale 10ps/10ps

module tb();
  import UPF::*;
  top top();

  initial begin

```

```

    $display("start simulation");
    supply_on("vdd", 3.00);
    supply_on("vss", 0.00);
    #1 supply_on("vdd", 3.1);
    #1 supply_off("vdd");
    #1 supply_on("vdd", 3.20);
    #1 supply_off("vdd");
    #1 supply_on("vdd", 3.3);
    #1 $finish;
end
endmodule

module top();
    wreal vdd1, vss1;

    real rvdd, rvss;

    assign vdd1 = 2.0;
    assign vss1 = 2.0;

    assign rvdd = vdd1;
    assign rvss = vss1;
    always @(rvdd, rvss) $display ("%g: %m: vdd = %f | vss = %f", $time, rvdd, rvss);

    bot BOT (vdd1, vss1);
endmodule

module bot(vdd2, vss2);
    input wreal vdd2, vss2;
    real rvdd, rvss;

    assign vdd2 = 1.0;

    assign rvdd = vdd2;
    assign rvss = vss2;
    always @(rvdd, rvss) $display ("%g: %m: vdd = %f | vss = %f", $time, rvdd, rvss);

    disp DISP (vdd2, vss2);
endmodule

```



```

module disp(vdd3, vss3);
    input wreal vdd3, vss3;
    real rvdd, rvss;

    assign vdd3 = 1.0;

    assign rvdd = vdd3;
    assign rvss = vss3;

    always @(rvdd, rvss) $display ("%g: %m: vdd = %f | vss
= %f", $time, rvdd, rvss);
endmodule

```

Example 17-2 *test_input.upf*

```

set_design_top tb/top

create_power_domain pd1 -include_scope

create_supply_net vdd -domain pd1 -resolve unresolved
create_supply_net vss -domain pd1 -resolve unresolved

connect_supply_net vdd -ports { BOT/vdd2 }

```

Compile test_input.v as shown below:

```

% vcs test_input.v -upf test_input.upf -wreal
res_sum -full64 -debug_all -sverilog

```

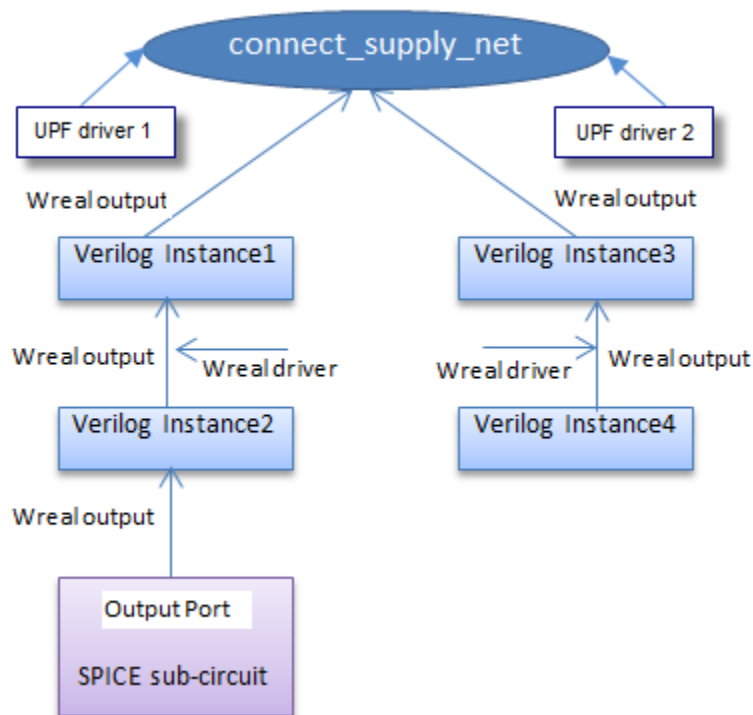
Connecting UPF Supply Nets to Wreal Output Ports

You can use the `connect_supply_net` command to connect the UPF supply net to wreal output ports. Resolved value of wreal ports are propagated to `connect_supply_net` after being converted

through WREAL-TO-CSN Value Conversion Table (VCT).
Propagated value is resolved with other UPF drivers (if any)
according to the UPF resolution semantics.

Figure 17-3 shows SPICE to Wreal to UPF connection through the Verilog hierarchy.

Figure 17-3 Connecting UPF supply Nets to Wreal Output Ports



Example

Example 17-3 test_output.v

```
`timescale 10ps/10ps

module tb();
  import UPF::*;
  top top();
endmodule
```

```

endmodule

module top();
    wreal vdd1, vss1;

    real rvdd, rvss;

    assign vdd1 = 2.0;
    assign vss1 = 2.0;

    assign rvdd = vdd1;
    assign rvss = vss1;
    always @(rvdd, rvss) $display ("%g: %m: vdd = %f | vss = %f", $time, rvdd, rvss);

    bot BOT (vdd1, vss1);
endmodule

module bot(vdd2, vss2);
    output wreal vdd2, vss2;
    real rvdd, rvss;
    real rvdddrv, rvssdrv;

    assign vdd2 = rvdddrv;
    assign vss2 = rvssdrv;

    initial begin
        $display("start simulation");
        rvdddrv = 3.0;
        rvssdrv = 0.0;
        #1 rvdddrv = 3.1;
        #1 rvdddrv = -3.0;
        #1 rvdddrv = 3.20;
        #1 rvdddrv = -3.0;;
        #1 rvdddrv = 3.3;
        #1 $finish;
    end

    assign rvdd = vdd2;
    assign rvss = vss2;
    always @(rvdd, rvss) $display ("%g: %m: vdd = %f | vss =

```

```

%f", $time, rvdd, rvss);

    disp DISP (vdd2, vss2);
endmodule

module disp(vdd3, vss3);
    input wreal vdd3, vss3;
    real rvdd, rvss;

    assign vdd3 = 1.0;

    assign rvdd = vdd3;
    assign rvss = vss3;

    always @(rvdd, rvss) $display ("%g: %m: vdd = %f | vss
= %f", $time, rvdd, rvss);
endmodule

```

Example 17-4 test_output.upf

```

set_design_top tb/top

create_power_domain pd1 -include_scope

create_supply_net vdd -domain pd1 -resolve unresolved
create_supply_net vss -domain pd1 -resolve unresolved

connect_supply_net vdd -ports { BOT/vdd2 }
connect_supply_net vss -ports { BOT/vss2 }

```

Compile test_output.v as shown below:

```

% vcs test_output.v -upf test_output.upf -wreal
res_sum -full64 -debug_all -sverilog

```

18

Appendix

This chapter contains the following sections:

- [Low Power Assertions](#)
- [Warning Messages for Unconnected Supply Ports](#)
- [Root Supply Driver](#)
- [TFIPC-L Lint Check in Presence of UPF Connections](#)

Low Power Assertions

This section describes the predefined data types and UPF query commands that are used to write low power assertions. This information is organized into the following sections:

- [List of Automated Low Power Messages](#)
- [Enabling Runtime Isolation Messages](#)
- [Supported UPF Query Commands](#)

List of Automated Low Power Messages

[Table 18-1](#) lists all the automated low power messages.

Table 18-1 Automated Low Power Messages (LP_MSG)

Context	ID	Severity	Message
HDL2UPF Connections	LP_SNET_HDL_SNET_UPDATE	INFO	HDL Supply Net '%s' updated to value '%s'.
	LP_SNET_HDL_SNET_CONN_UPDATE	INFO	HDL Supply Port '%s' driving UPF Supply Net '%s' updated to value '%s'.
Clock/Reset Wiggle	LP_CLOCK_WIGGLE, LP_RESET_WIGGLE	WARNING	Clock '%s' wiggling in instance '%s' during shutdown for the power domain '%s' at time %0t. Reset '%s' wiggling in instance '%s' during shutdown for the power domain '%s' at time %0t.
	LP_COA_CLOCK_WIGGLE, LP_COA_RESET_WIGGLE	WARNING	Clock '%s' wiggling in instance '%s' during standby for the power domain '%s' at time %0t. Reset '%s' wiggling in instance '%s' during standby for the power domain '%s' at time %0t.
Isolation	LP_ISO_EN	INFO	Isolation enabled for isolation strategy '%s' of power domain '%s'.
	LP_ISO_EN_PWRDN	INFO	Isolation enabled for isolation strategy '%s' when Power Domain '%s' is OFF.
	LP_ISO_DIS	INFO	Isolation disabled for isolation strategy '%s' of power domain '%s'.

Context	ID	Severity	Message
	LP_ISO_DIS_PWRDN	INFO	Isolation disabled for isolation strategy '%s' when Power Domain '%s' is OFF.
	LP_ISOEN_INIT_VALUE	INFO	Isolation enable '%s' of isolation strategy '%s' of power domain '%s' started with value '%s'. Isolation sense specified is '%s'.
	LP_ISO_INIT_OFF	WARNING	Isolation strategy '%s' of power domain '%s' started in CORRUPT state.
	LP_ISOEN_INIT_INVALID	WARNING	Isolation enable '%s' of isolation strategy '%s' of power domain '%s' started with an invalid value '%s'.
	LP_ISOEN_CHANGE	INFO	Isolation enable '%s' of isolation strategy '%s' of power domain '%s' changed from '%s' to '%s'. Isolation sense specified is '%s'.
	LP_ISOEN_INVALID*	ERROR	Isolation enable '%s' of isolation strategy '%s' of power domain '%s' changed to an invalid value '%s'.
	LP_ISOPN_INIT	INFO	Isolation power net '%s' for isolation strategy '%s' of power domain '%s' started in state %s.

Context	ID	Severity	Message
	LP_ISOPN_STATE	INFO	State of isolation power net '%s' for isolation strategy '%s' of power domain '%s' changed from %s to %s.
	LP_ISOGN_INIT	INFO	Isolation ground net '%s' for isolation strategy '%s' of power domain '%s' started in state %s.
	LP_ISOGN_STATE	INFO	State of isolation ground net '%s' for isolation strategy '%s' of power domain '%s' changed from %s to %s.
	LP_ISOPN_OFF*	WARNING	State of isolation power net '%s' for isolation strategy '%s' of power domain '%s' changed to %s.
	LP_ISOGN_OFF*	WARNING	State of isolation ground net '%s' for isolation strategy '%s' of power domain '%s' changed to %s.
	LP_ISO_SEQFAIL1*	WARNING	Isolation enable '%s' for isolation strategy '%s' is not active when the source power domain '%s' changed to %s state.
Low-Vdd Standby/ CORRUPT_ON_ACTIVITY	LP_COA_CORRUPT_ALW	WARNING	"LHS in always block corrupted due to activity on sensitivity list elements in simstate 'CORRUPT_ON_ACTIVITY'. Instance: %s, File: %s, Line: %s."

Context	ID	Severity	Message
	LP_COA_CORRUPT_CA	WARNING	"LHS of cont-assign corrupted due to activity on RHS in simstate 'CORRUPT_ON_ACTIVITY'. Instance: %s, File: %s, Line: %s."
	LP_COA_CORRUPT_GATE	WARNING	"Output of primitive gate corrupted due to activity on inputs in simstate 'CORRUPT_ON_ACTIVITY'. Instance: '%s', File: '%s', Line: '%s'."
	LP_COA_CORRUPT_UDP	WARNING	"Output of UDP corrupted due to activity on inputs in simstate 'CORRUPT_ON_ACTIVITY'. Instance: %s, File: %s, Line: %s."
	LP_COA_CORRUPT_SIG	WARNING	Signal %s corrupted due to activity.
	LP_COA_CORRUPT_VAR	WARNING	Variable %s corrupted due to activity.
Message Control	LP_CFG_FILE_ERR	ERROR	"Cannot open specified configuration file: %s, please check for the file existence and file permissions and rerun the simulation."
	LP_MSG_SUMMARY	INFO	
	LP_MSG_ONOFF	INFO	Message '%s' is now '%s'.
	LP_MSG_NF	WARNING	"'%s', '%s': Message '%s' referenced in function '%s' not found."
	LP_MSG_INVALID_ARG	WARNING	"'%s', '%s': Argument '%s' in function '%s' is invalid. Valid values are '%s'."

Context	ID	Severity	Message
	LP_MSG_SEV	INFO	Severity of message '%s' is now '%s'.
	LP_MSG_REG	INFO	Message '%s' is registered successfully.
	LP_MSG_INT	WARNING	" '%s', '%s': '%s' is an internal message. 'lp_msg_print' can be used only with messages registered through 'lp_msg_register'."
	LP_MSG_AE	WARNING	" '%s', '%s': Message '%s' in function '%s' already Exists. Already registered message cannot be registered again."
Power Domain	LP_PD_INIT_STATE	INFO	Power domain '%s' started in '%s' state.
	LP_PD_STATE_CHANGE	INFO	Power domain '%s' state changed from '%s' to '%s'.
	LP_PPN_INIT_STATE	INFO	Primary power net '%s' of power domain '%s' started in %s state.
	LP_PPN_INIT_VALUE	INFO	Primary power net '%s' of power domain '%s' started with voltage %s V.
	LP_PGN_INIT_STATE	INFO	Primary ground net '%s' of power domain '%s' started in %s state.
	LP_PGN_INIT_VALUE	INFO	Primary ground net '%s' of power domain '%s' started with voltage %s V.
	LP_PPN_STATE_CHANGE	INFO	State of the primary power net '%s' of power domain '%s' changed from %s to %s.

Context	ID	Severity	Message
	LP_PGN_STATE_CHANGE	INFO	State of the primary ground net '%s' of power domain '%s' changed from %s to %s.
	LP_PPN_VALUE_CHANGE	INFO	Voltage of the primary power net '%s' of power domain '%s' changed from %s V to %s V.
	LP_PGN_VALUE_CHANGE	INFO	Voltage of the primary ground net '%s' of power domain '%s' changed from %s V to %s V.
Power Switch	LP_PSW_CTRL_INIT_INVALID	ERROR	Signal '%s' connected to control port '%s' of power switch '%s' started with an invalid value '%s'. Note: This message is not reported when the input supply of the power switch is in OFF state.
	LP_PSW_CTRL_INIT_VALUE	INFO	Signal '%s' connected to control port '%s' of power switch '%s' started with a value '%s'.
	LP_PSW_ISP_INIT_STATE	INFO	Supply net '%s' tied to input supply port '%s' of power switch '%s' started with state %s.
	LP_PSW_ISP_INIT_VALUE	INFO	Supply net '%s' tied to input supply port '%s' of power switch '%s' started with voltage %s V.
	LP_PSW_OSP_INIT_STATE	INFO	Supply net '%s' tied to output supply port '%s' of power switch '%s' started with state %s.

Context	ID	Severity	Message
	LP_PSW_OSP_INIT_VALUE	INFO	Supply net '%s' tied to output supply port '%s' of power switch '%s' started with voltage %s V.
	LP_PSW_INIT_STATE	INFO	Power switch '%s' started in %s (%s) state.
	LP_PSW_STATE_CHANGE	INFO	State of power switch '%s' changed from %s (%s) to %s (%s).
	LP_PSW_CTRL_CHANGE	INFO	Signal '%s' connected to control port '%s' of power switch '%s' changed from '%s' to '%s'.
	LP_PSW_CTRL_INVALID	ERROR	Signal '%s' connected to control port '%s' of power switch '%s' changed from '%s' to an invalid value '%s'. Note: This message is not issued when the input supply of the power switch is in OFF state. This message is issued on the change in the power switch control signal when the input supply of the power switch is in ON state. The message is issued even if the input supply of the power switch switches to OFF state in the same simulation cycle after the power switch control signal is sampled as X or Z.

Context	ID	Severity	Message
	LP_PSW_OSP_STATE	INFO	State of supply net '%s' tied to output supply port '%s' of power switch '%s' changed from %s to %s.
	LP_PSW_OSP_VALUE	INFO	Voltage of supply net '%s' tied to output supply port '%s' of power switch '%s' changed from %s V to %s V.
	LP_PSW_ISP_STATE	INFO	State of supply net '%s' tied to input supply port '%s' of power switch '%s' changed from %s to %s.
	LP_PSW_ISP_VALUE	INFO	Voltage of supply net '%s' tied to input supply port '%s' of power switch '%s' changed from %s V to %s V.
	LP_PSW_SUPPLY_CHANGE	INFO	Output Supply Port of Power Switch '%s' changed to state = '%s' and voltage = %s volt(s) with enable expression '%s'.
	LP_PSW_INIT	INFO	Output Supply Port of Power Switch '%s' initialized to state = '%s' and enable expression '%s'.
PST	LP_PST_INIT_STATE	INFO	Design started with '%s' state in pst '%s'.
	LP_PST_INIT_ILLEGAL	WARNING	Design started with '%s' state (illegal) in pst '%s'.
	LP_PST_STATE_CHANGE	INFO	Design state changed from '%s' to '%s' in pst '%s'.

Context	ID	Severity	Message
	LP_PST_STATE_ILLEGAL	WARNING	Design state changed from '%s' to '%s' (illegal) in pst '%s'.
PST (enabled using the - <code>power=pst_state_msg</code> compile-time option)	LP_PST_STATE_INIT	INFO	Design started with '%s' state in pst '%s' with Supplies '%s=%s, %s=%s'
	LP_PST_STATE_ILLEGAL_INIT	WARNING	Design started with 'ILLEGAL' state in pst '%s' with Supplies '%s=%s, %s=illegal'.
	LP_PST_STATE_ILLEGAL_COMBINATION_INIT	ERROR	Design started with 'ILLEGAL' state in pst 'test/top/blk_pst' with Supplies 'SS_VDD1.power=off, SS_VDD2.power=off, SS_VDD1.ground=GND'.
	LP_PST_STATE_TRANSITION	INFO	Design state changed from '%s' to '%s' or 'ILLEGAL' in pst '%s' with Supplies '%s=%s, %s=%s' and Supply '%s' changed.
	LP_PST_STATE_ILLEGAL_TRANSITION	WARNING	Design started changed from '%s' to ILLEGAL in pst '%s' with Supplies '%s=%s, %s=%s' and Supply '%s' changed.

Context	ID	Severity	Message
	LP_PST_STATE_ILLEGAL_COMBINATION	ERROR	Design state changed from 'state0' to 'ILLEGAL' in pst 'test/top/blk_pst' with Supplies 'SS_VDD1.power=V_MIN,SS_VDD2.power=V_NOM,SS_VDD1.ground=GND' and Supply 'SS_VDD2.power' changed.
Random Corruption	LP_DOMAIN_RAND_SEED	INFO	Random corruption seed specified for domain '%s' is '%s'.
	LP_DESIGN_RAND_SEED	INFO	Random corruption seed is not specified for domain '%s'. Using random corruption seed '%s' specified for design.
	LP_AUTO_RAND_SEED	INFO	Random corruption seed is not specified for domain '%s'. Using automatic generated random corruption seed '%s'.
Retention	LP_RETPN_INIT	INFO	Retention power net '%s' for retention strategy '%s' of power domain '%s' started with %s state.
	LP_RETGN_INIT	INFO	Retention ground net '%s' for retention strategy '%s' of power domain '%s' started with %s state.
	LP_RETPN_STATE	INFO	State of retention power net '%s' for retention strategy '%s' of power domain '%s' changed from %s to %s.

Context	ID	Severity	Message
	LP_RETGN_STATE	INFO	State of retention ground net '%s' for retention strategy '%s' of power domain '%s' changed from %s to %s.
	LP_RET_PWR_OFF	WARNING	Retention power for retention strategy '%s' of power domain '%s' turned OFF. Shadow registers are corrupted.
	LP_SAVE_ON	INFO	Save asserted for retention strategy '%s' of power domain '%s'.
	LP_SAVE_OFF	INFO	Save deasserted for retention strategy '%s' of power domain '%s'.
	LP_SAVE_RETPWRDN	ERROR	Save event triggered for retention strategy '%s' of power domain '%s' when retention power is OFF.
	LP_RESTORE_ON	INFO	Restore asserted for retention strategy '%s' of power domain '%s'.
	LP_RESTORE_OFF	INFO	Restore deasserted for retention strategy '%s' of power domain '%s'.
	LP_RESTORE_RETPWRDN	ERROR	Restore event triggered for retention strategy '%s' of power domain '%s' when retention power is OFF.
	LP_SAVE_INIT	INFO	Save signal '%s' for retention strategy '%s' of power domain '%s' started with a value '%s'. Save sense specified is '%s'.

Context	ID	Severity	Message
	LP_SAVE_INIT_INVALID	WARNING	Save signal '%s' for retention strategy '%s' of power domain '%s' started with an invalid value '%s'. Note: This message is not reported when the retention supplies are in OFF state.
	LP_SAVE_CHANGE	INFO	Save signal '%s' for retention strategy '%s' of power domain '%s' changed from '%s' to '%s'. Save sense specified is '%s'.
	LP_SAVE_INVALID	WARNING	Save signal '%s' for retention strategy '%s' of power domain '%s' changed from '%s' to '%s'. Note: This message is not issued when the retention supplies are in OFF state. This message is issued on the change in the save control signal when the retention supply is in ON state. The message is issued even if the retention supply switches to OFF state in the same simulation cycle after the save signal is sampled as X or Z.

Context	ID	Severity	Message
	LP_SAVE_PRIMPWR_OFF	WARNING	Save operation enabled for retention strategy '%s' of power domain '%s' while the primary power of domain is OFF. Save signal '%s' is at an active level '%s'.
	LP_RESTORE_INIT	INFO	Restore signal '%s' for retention strategy '%s' of power domain '%s' started with a value '%s'. Restore sense specified is '%s'.
	LP_RESTORE_INIT_INVALID	WARNING	Restore signal '%s' for retention strategy '%s' of power domain '%s' started with an invalid value '%s'. Note: This message is not reported when the retention supplies are in OFF state.
	LP_RESTORE_CHANGE	INFO	Restore signal '%s' for retention strategy '%s' of power domain '%s' changed from '%s' to '%s'. Restore sense specified is '%s'.

Context	ID	Severity	Message
	LP_RESTORE_INVALID	WARNING	Restore signal '%s' for retention strategy '%s' of power domain '%s' changed from '%s' to '%s'. Note: This message is not issued when the retention supplies are in OFF state. This message is issued on the change in the restore control signal when the retention supply is in ON state. The message is issued even if the retention supply switches to OFF state in the same simulation cycle after the restore signal is sampled as X or Z.
	LP_RESTORE_PRIMPWR_OFF	WARNING	Restore operation disabled. Primary power of power domain '%s' is turned OFF while restore signal '%s' of retention strategy '%s' is at an active level '%s'.
	LP_RESTORE_NOSAVE	ERROR	Restoring X on the registers of power domain '%s'. Restore operation is enabled without a save event, after retention power is turned ON. Retention Strategy: '%s', Restore Signal: '%s', Save Signal: '%s'.

Context	ID	Severity	Message
Retention assert_mutex	LP_ASSERT_R_MUTEX	ERROR	'assert_r_mutex' defined for retention strategy '%s' of power domain '%s' failed. Restore signal '%s' ('%s') and signal '%s' ('%s') are not mutually exclusive.
	LP_ASSERT_S_MUTEX	ERROR	'assert_s_mutex' defined for retention strategy '%s' of power domain '%s' failed. Save signal '%s' ('%s') and signal '%s' ('%s') are not mutually exclusive.
	LP_ASSERT_RS_MUTEX	ERROR	'assert_rs_mutex' defined for retention strategy '%s' of power domain '%s' failed. Restore signal '%s' ('%s') and save signal '%s' ('%s') are not mutually exclusive.
Supply Sets [*] [*] Issued only for the supply sets that have the power states defined in the UPF.	LP_SS_INIT	INFO	Supply Set '%s' initialized to state '%s' with simstate '%s'.
	LP_SS_ILLEGAL	WARNING	Supply Set '%s' initialized to illegal state '%s' with simstate '%s'.
	LP_SS_STATE_CHANGE	INFO	Supply Set '%s' transitioned to state '%s' with simstate '%s'.

Context	ID	Severity	Message
	LP_SS_STATE_CHANGE_ILLEGAL	WARNING	Supply Set '%s' transitioned to illegal state '%s' with simstate '%s'.
	LP_SS_EXPR_MISMATCH	ERROR	Supply expression's value '%s' does not match with Logic expression value '%s' for state '%s' of Supply Set '%s'.
UPF::supply_on/supply_off	LP_SUPPLY_PAD_NOT_FOUND	WARNING	Supply Pad '%s' not found in 'UPF::%s' function.
	LP_SUPPLY_PAD_CALL	INFO	Supply Pad function 'UPF::%s' called for Supply Pad '%s'. Supply Pad value changed to '%s'.
	LP_SUPPLY_PAD_VOLTAGE	WARNING	Supply Pad function 'UPF::%s' called for Power Supply Pad '%s' with '0' voltage. Supply Pad value changed to '%s'.

Enabling Runtime Isolation Messages

By default, the following runtime isolation messages do not consider the state of the sink supply:

- LP_ISO_SEQFAIL1
- LP_ISOEN_INVALID
- LP_ISOPN_OFF
- LP_ISOGN_OFF

You can use the `-power=sinkbasedisoassert` compile-time option to avoid reporting of the above messages when all sink supplies/domains are OFF. If this option is used, then the above messages are issued only when at least one of the sink supply/domain is ON.

Verbose Illegal PST State Change Messages

During simulation, PST state changes are captured by LP automated messages. In case of a PST state transition to or from `ILLEGAL` state, it is helpful to know the contributing supplies that resulted in an `ILLEGAL` state.

You can use the `-power=pst_state_msg` compile-time option to enable verbose messaging for the following:

- Illegal state transitions (PST state changes to or from `ILLEGAL` state)
- Invalid combinations (Any valid combination of PST state supplies that is not defined in the PST table)

For example, consider following UPF code and PST:

```
# Power State Information:
add_power_state SS_VDD1 -state V_MIN {-supply_expr {power ==
`{FULL_ON,0.7}}}}
add_power_state SS_VDD1 -state V_NOM {-supply_expr {power ==
`{FULL_ON,0.8}}}}
add_power_state SS_VDD1 -state V_MAX {-supply_expr {power ==
`{FULL_ON,0.9}}}}
add_power_state SS_VDD1 -state V_OFF {-supply_expr {power ==
`{OFF}}}}
add_power_state SS_VDD1 -state GND {-supply_expr {ground ==
`{FULL_ON,0.00}}}}
add_power_state SS_VDD2 -state V_MIN {-supply_expr {power ==
`{FULL_ON,0.7}}}};
add_power_state SS_VDD2 -state V_NOM {-supply_expr {power
==`{FULL_ON,0.8}}}};
add_power_state SS_VDD2 -state V_MAX {-supply_expr {power
```

```

==`{FULL_ON,0.9}}};
add_power_state SS_VDD2 -state V_OFF {-supply_expr {power ==
`{OFF}}};

#PST Details:
create_pst blk_pst -supplies {SS_VDD1.power SS_VDD2.power
SS_VDD1.ground};
add_pst_state state0 -pst blk_pst -state {V_MIN V_MIN GND}
add_pst_state state1 -pst blk_pst -state {V_MIN V_OFF GND}
add_pst_state state2 -pst blk_pst -state {V_MAX V_OFF GND}
add_pst_state state3 -pst blk_pst -state {V_OFF V_OFF GND}

```

Various possible cases and example LPA messages generated using the above PST are described below:

Case-1: For a given PST, if any of the supplies go into an ILLEGAL/UNDEFINED state, then that transition causes the PST to enter into ILLEGAL state.

In this case, VCS NLP flags LP_PST_STATE_ILLEGAL_INIT at time 0 and LP_PST_STATE_ILLEGAL_TRANSITION for later part of the simulation (non-time 0).

Example message at time 0:

```

[0] [WARNING] [LP_PST_STATE_ILLEGAL_INIT] Design started
with 'ILLEGAL' state in pst 'test/top/blk_pst' with Supplies
'SS_VDD1.power=off, SS_VDD2.power=illegal,
SS_VDD1.ground=GND'.

```

Example message at non-time 0:

```

[55] [WARNING] [LP_PST_STATE_ILLEGAL_TRANSITION] Design
state changed from 'ILLEGAL' to 'ILLEGAL' in pst 'test/top/
blk_pst' with Supplies
'SS_VDD1.power=illegal, SS_VDD2.power=V_NOM, SS_VDD1.ground=
GND' and Supply 'SS_VDD1.power' changed.

```


Case-2: For a given PST, if all the supplies have legal values, but the combination of the supply state does not match any PST entry in UPF.

In this case, VCS NLP flags

LP_PST_STATE_ILLEGAL_COMBINATION_INIT at time 0 and
LP_PST_STATE_ILLEGAL_COMBINATION during later part of the
simulation (non-time 0).

Example message at time 0:

```
[0] [ERROR] [LP_PST_STATE_ILLEGAL_COMBINATION_INIT] Design
started with 'ILLEGAL' state in pst 'test/top/blk_pst' with
Supplies 'SS_VDD1.power=off, SS_VDD2.power=off,
SS_VDD1.ground=GND'.
```

Example message at non-time 0:

```
[35] [ERROR] [LP_PST_STATE_ILLEGAL_COMBINATION] Design
state changed from 'state0' to 'ILLEGAL' in pst 'test/top/
blk_pst' with Supplies
'SS_VDD1.power=V_MIN,SS_VDD2.power=V_NOM,SS_VDD1.ground=GN
D' and Supply 'SS_VDD2.power' changed.
```

Case-3: For a given PST, all the supplies have legal values, and the combination of supply state also matches an entry in the PST.

In this case, VCS NLP flags LP_PST_STATE_INIT at time 0 and
LP_PST_STATE_TRANSITION during later part of the simulation
(non-time 0).

Example message at time 0:

```
[0] [INFO] [LP_PST_STATE_INIT] Design started with 'state3'
state in pst 'test/top/blk_pst' with Supplies
'SS_VDD1.power=off, SS_VDD2.power=off,
SS_VDD1.ground=GND'.
```

Example message at non-time 0:

```
[15] [INFO] [LP_PST_STATE_TRANSITION] Design state changed
from 'state2' to 'state1' in pst 'test/top/blk_pst' with
Supplies
'SS_VDD1.power=V_MIN,SS_VDD2.power=off,SS_VDD1.ground=GND'
and Supply 'SS_VDD1.power' changed.
```

Note:

The severity of the LPA messages

`LP_PST_STATE_ILLEGAL_COMBINATION_INIT` and
`LP_PST_STATE_ILLEGAL_COMBINATION` is ERROR. These two
LPA messages imply that the simulation environment generated
a valid combination of supplies and this combination is not defined
in the PST. You can downgrade the severity to `WARNING` or `INFO`.
For more details on how to control severity of LPA messages,
refer to [“Override the Default Severity of Messages”](#).

Supported UPF Query Commands

You can use the following UPF query commands to write custom low power assertions.

Note:

- The query commands are supported only in the Tcl file passed with `-lpa_bind`. VCS NLP generates an error message, if you use query commands in the UPF file.
- The names of the UPF/design objects passed to query commands must be specified using absolute paths.

query_power_domain

The following commands describe the usage of this command:

- `query_power_domain *`

Return Type: Tcl List

Return Data: Returns a list of all the power domains defined (full hierarchical path).

- `query_power_domain`
`<full_hierarchical_path_of_domain> -detailed`

Return Type: Tcl array, list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
domain_name	Full hierarchical name of the power domain.	SV string
elements	NA	NA
primary_ground_net	Full hierarchical path of the primary ground net.	UPF::supply_net_type/ SV string

Key	Value	Bind Type
primary_power_net	Full hierarchical path of the primary power net.	UPF::supply_net_type/ SV string
supply	NA	NA
sim_state	sim_state of the power domain.	UPF::power_state_simstate type/ SV string

NA - Unsupported Keys

query_supply_net

The following commands describe the usage of this command:

- `query_supply_net*`

Return Type: Tcl List

Return Data: Returns a list of all the supply nets defined in the UPF (full hierarchical path).

- `query_supply_net <net_name> -detailed`

Return Type: Tcl array, list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
domain	Full hierarchical name of the power domain.	SV string
is_supply	Returns 1 if the specified net_name is a supply port, else returns 0.	SV bit
net_name	Name of the supply net.	SV string
resolve	NA	NA

query_pst

The following commands describe the usage of this command:

- `query_pst *`

Return Type: Tcl List

Return Data: Returns a list of all the power state tables defined (full hierarchical path).

- `query_pst <pst_name> -detailed`

Return Type: Tcl array, list of {key value} pairs

Following is the output of the above command:

Key	Value	Bind Type
supplies	List of supply nets or ports of specified pst table.	UPF::supply_net_type/ SV string
table_name	Full hierarchical pst name.	SV string
design_state	Index of current state of the design. Integer index to the list of states returned by the <code>query_pst_state * -pst <pst_name></code> command. Illegal/undefined state is represented by index 0, so while populating the list of pst states in Tcl, the first state has to be manually added as “illegal” or any other user-defined name for illegal/undefined state.	SV int

query_pst_state

The following commands describe the usage of this command:

- `query_pst_state * -pst <pst_name>`

Return Type: Tcl List

Return Data: Returns a list of all states defined for the specified power state table.

- `query_pst_state <state> -pst <pst_name> -detailed`

Return Type: Tcl list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
pst	Full hierarchical name of the pst	SV string
state	List of power states of the supplies for the specified pst_state	SV string
state_name	Name of the pst_state	SV string

query_power_switch

The following commands describe the usage of this command:

- `query_power_switch *`

Return Type: Tcl List

Return Data: Returns a list of all the power switches defined (full hierarchical path).

- `query_power_switch <switch_name> -detailed`

Return Type: Tcl list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
ack_delay	NA	NA
ack_port	NA	NA
control_port	{port_name net_name} net_name is returned in the name alias format. Use the 'query_original_name' tcl proc to get its simple name	{'SV string' 'SV wire/SV string'}
domain	NA	NA
error_state	List of all error states	SV string
input_supply_port	{port_name supply_net_name}	{SV string UPF::supply_net_type}
off_state	List of all off states	SV string
on_partial_state	NA	NA
on_state	List of all on states	SV string

Key	Value	Bind Type
output_supply_port	{port_name supply_net_name}	{SV string UPF::supply_net_type}
states	{state_name state_type*} {state_name state_type*} {...}	{SV string SV string}
supply_set	NA	NA
switch_name	Full hierarchical name of the power switch	SV string
switch_state	Index that reflects the current state of the power switch. array_of_all_state_names[index]. Index 0 represents undetermined state. Array of all state_names must be generated from 'state_name' value of 'states' key.	SV int

* state_type is UNDETERMINED, ON, PARTIAL_ON, ERROR, OFF (in the same order) based on which of these are defined.

query_isolation

The following commands describe the usage of this command:

- `query_isolation * -domain <ref_domain_name>`

Return Type: Tcl List

Return Data: Returns a list of all the isolation strategies defined for the specified domain.

- `query_isolation <isolation strategy> -domain <full_hierarchical_path_of_ref_domain > -detailed`

Return Type: Tcl list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
clamp_value	Specified clamp value (0/1/Z/any/latch)	SV string
domain	Full hierarchical name of the power domain	
elements	NA	NA
force_isolation	NA	NA
isolation_ground_net	Name alias of isolation ground net. Use 'query_original_name' tcl proc to get the simple name of the isolation ground net.	UPF::supply_net_type/SV string
isolation_name	Name of the isolation strategy	SV string
isolation_power_net	Name alias of isolation power net. Use 'query_original_name' tcl proc to get the simple name of the isolation power net.	UPF::supply_net_type/SV string
isolation_signal	Name alias of isolation signal specified for the isolation strategy. Use 'query_original_name' tcl proc to get the simple name of the isolation signal.	SV wire/SV string
isolation_supply_set	NA	NA

Key	Value	Bind Type
location	Location specified for the isolation strategy (automatic self fanout parent)	SV string
no_isolation	Returns 1 if -no_isolation specified, else returns 0.	SV bit
sink_off_clamp	NA	NA
source_off_clamp	NA	NA
isolation_sense	Number representing the isolation sense specified in isolation strategy (0 - low, 1 - high)	SV bit
primary_power_net	Name alias of primary power net of the power domain. Use 'query_original_name' tcl proc to get the simple name of the primary power net	UPF::supply_net_type/SV string
primary_ground_net	Name alias of primary ground net of the power domain. Use 'query_original_name' tcl proc to get the simple name of the primary ground net.	UPF::supply_net_type/SV string

query_retention

The following commands describe the usage of this command:

- `query_retention * -domain <full_hierarchical_path_of_domain>`

Return Type: Tcl List

Return Data: Returns a list of all the retention strategies defined for the specified power domain.

- `query_retention <retention strategy> -domain <full_hierarchical_path_of_domain> -detailed`

Return Type: Tcl list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
domain	Full hierarchical name of the power domain.	SV string
elements	NA	NA
exclude_elements	NA	NA
no_retention	NA	NA
output_related_supply_set	NA	NA
parameters	NA	NA
primary_power_net	Name alias of primary power net of the power domain. Use 'query_original_name' tcl proc to get the simple name of the primary power net.	UPF::supply_net_type/SV string
primary_ground_net	Name alias of primary ground net of the power domain. Use 'query_original_name' tcl proc to get the simple name of the primary ground net.	UPF::supply_net_type/SV string
restore_signal	{restore_signal sense} Restore_signal is returned in the name alias format. Use 'query_original_name' tcl proc to get its simple name. Sense is either high, low, posedge, or negedge.	{SV wire/string SV string}

Key	Value	Bind Type
restore_condition	Name alias of specified restore_condition. Returns 1 if restore_condition evaluates to true, else returns 0.	SV logic
retention_condition	NA	NA
retention_power_net	Name alias of retention power net. Use 'query_original_name' tcl proc to get the simple name of the retention power net.	UPF::supply_net_type/SV string
retention_ground_net	Name alias of retention ground net. Use 'query_original_name' tcl proc to get the simple name of the retention ground net.	UPF::supply_net_type/SV string
retention_name	Name of the retention strategy.	SV string
retention_supply_set	NA	NA
save_signal	{save_signal sense} save_signal is returned in the name alias format. Use 'query_original_name' tcl proc to get its simple name. Sense can be high, low, posedge, or negedge.	{SV wire/string SV string}
save_condition	Name alias of save_condition specified. Returns 1 if restore_condition evaluates to true, else returns 0.	SV logic
use_retention_as_primary	NA	NA

query_retention_control

The following commands describe the usage of this command:

- `query_retention_control * -domain
<full_hierarchical_path_of_domain>`

Return Type: Tcl List

Return Data: Returns a list of all the retention strategies defined for the specified power domain.

- `query_retention_control <retention strategy> -
domain <domain> -detailed`

Return Type: Tcl list of {key value} pairs

Following is the output of the above command.

Key	Value	Bind Type
retention_name	Name of the retention strategy.	SV string
assert_r_mutex	{net_name sense} Sense is either high, low, posedge, or negedge.	{SV wire/SV string SV string}

Key	Value	Bind Type
assert_rs_mutex	1. Key not available if not defined. 2. If rs_mutex is default, then key is available with no value. 3. If logic net is specified, then {net_name sense} Sense is either high, low, posedge, or negedge.	1. NA 2. "" 3. {SV wire/SV string SV string}
assert_s_mutex	{net_name sense} Sense is either high, low, posedge, or negedge.	{SV wire/SV string SV string}
domain	Full hierarchical name of the power domain.	SV string
restore_signal	{restore_signal sense} restore_signal is returned in the name alias format, use 'query_original_name' tcl proc to get its simple name. Sense is either high, low, posedge, or negedge.	{SV wire/string SV string}
save_signal	{save_signal sense} save_signal is returned in the name alias format, use 'query_original_name' tcl proc to get its simple name. Sense is either high, low, posedge, or negedge.	{SV wire/string SV string}

Warning Messages for Unconnected Supply Ports

VCS NLP generates the following five different warning message tags for unconnected supply ports based on the direction and the connectivity of the port.

Case-1: Supply port is neither an input port nor an output port

```
Warning-[UPF_USP] Unconnected supply port
save_restore_high.upf, 6
Supply Port 'TOP/VDD' does not have a supply net connected
either to its lowconn or highconn.
```

Case-2: Supply port is an input port

- **No Lowconn connection**

```
Warning-[UPF_ISPNL] Input Supply Port with no load
save_restore_high.upf, 6
Input Supply Port 'TOP/VDD' does not have a supply net
load.
Add a load by using 'connect_supply_net' to a net at the
current/lower scope.
```

- **No Highconn connection**

```
Warning-[UPF_ISPND] Input Supply Port with no driver
save_restore_high.upf, 6
Input Supply Port 'TOP/VDD' does not have a supply net
driver.
Add a driver by using 'connect_supply_net' to a net at a
higher scope.
```

Case-3: Supply port is an output port

- **No Lowconn connection**

```
Warning-[UPF_OSPND] Output Supply Port with no driver
save_restore_high.upf, 6
Output Supply Port 'TOP/VDD' does not have a supply net
driver.
Add a driver by using 'connect_supply_net' to a net at
the current/lower scope.
```

- **No Highconn connection**

```
Warning-[UPF_OSPNL] Output Supply Port with no load
```

```
save_restore_high.upf, 6
Output Supply Port 'TOP/VDD' does not have a supply net
load.
Add a load by using 'connect_supply_net' to a net at a
higher scope.
```

Root Supply Driver

A root supply driver is of the following type:

- Undriven supply net
- Undriven supply port
- Output supply port of the power switch
- HDL/Spice driver to the UPF using `connect_supply_net`
- HDL Net - A net that is brought into the UPF using `create_supply_net -reuse`, the first time it is created in the UPF

Root supply drivers are those supply ports/nets which can inject supply state from either HDL/Spice drivers, power switch, or `supply_on/supply_off` calls.

A supply source can have multiple root supply drivers if at least one of the connected nets is resolved `parallel` or resolved `one_hot`. A supply source cannot have both resolve `one_hot` and resolve `parallel` nets.

TFIPC-L Lint Check in Presence of UPF Connections

To detect unconnected ports in the design, you can use the `+lint=TFIPC-L` compile-time option. This option flags the following lint message for the unconnected ports:

```
Lint-[TFIPC-L] Too few instance port connections
```

The above message is issued only for the ports that are unconnected in either design or UPF.

19

VCS NLP Quick Reference

This chapter provides information on all the low power command line options and UPF design attributes supported by VCS NLP. This chapter includes the following sections:

- [Command Line Options](#)
- [Design Attributes to be Specified in UPF](#)
- [HDL Attributes](#)

Command Line Options

- [Compile-Time Options](#)
- [Simulation Option](#)

Compile-Time Options

This section describes all the compile-time command line options supported by VCS NLP.

Table 19-1 Compile-time Options

Compile-Time Option	Description
<code>-error=UPF_MDOAPS</code>	Allows you to convert the UPF_MDOAPS warning message to error message. For more information, see “Driver Checks on Power Switch Acknowledge Port Signal” section.
<code>+lint=MV_DB_CELL_MISMATCH</code>	Generates a warning message when there is a mismatch between the Verilog definition and liberty definition of a cell. For more information, see “Lint Check for Mismatch Between Verilog and Liberty Definitions of a Cell” section.
<code>+lint=TFIPC-L</code>	Allows you to detect unconnected ports in the design. For more information, see “TFIPC-L Lint Check in Presence of UPF Connections” section.
<code>-lpa_bind <filename></code>	Allows you to specify the custom bind Tcl file. For more information, see “Custom Assertions” section.

Compile-Time Option	Description
<code>-lpa_bind_filelist <file_list></code>	Allows you to specify a file list listing multiple custom bind checker Tcl files. For more information, see “Custom Assertions” section.
<code>-mvrpt</code>	Allows you to redirect or rename the compile static reports directory. For more information, see “Redirecting/Renaming mvsim_native_reports Directory” section.
<code>-pa_random_corrupt</code>	Enables the random corruption feature. For more information, see “Random Corruption” section.
<code>-power_config <config file></code>	Allows you to specify the compile-time configuration file that specifies the <code>search_path</code> and <code>link_library</code> for the liberty (.db) files. For more information, see “Using Connect Supply Net and Library Mapping” section.
<code>-power_top <module name></code>	In VCS NLP, you can specify power top either in UPF using the <code>set_design_top</code> command with the full path, or on VCS command-line using the <code>-power_top</code> option.

List of “-power” Compile-Time Options

This topic describes all the “-power” compile-time command-line options supported by VCS NLP. Following is the syntax of “-power” compile-time option:

```
%vcs -power=<option1> -power=<option2>... [other_options]
```

Or,

```
%vcs -power=<option1>+<option2>... [other_options]
```

[Table 19-2](#) describes the supported options.

Table 19-2 List of “-power” Compilation Options

Option	Description
accurate	Skips the insertion of the virtual isolation cells. This option is useful when the isolation cells are pre-inserted in the design, and isolation strategies are defined in the UPF. For more information, see “Association of Pre-Inserted Isolation Cells” section.
attributes_on	Enables the following attributes which are used to control the corruption and wake-up semantics of the initial and always blocks: vcs_reinit, vcs_dont_reinit, vcs_always_on, vcs_dont_trigger_on_wakeup For more information, see “Controlling the Corruption and Wake-up Semantics of the Initial and Always Blocks” section.
apfcompat	Skips the corruption of reals, constants, and the signals assigned inside the initial blocks. For more information, see “Skipping Corruption” section.
auto_complete	Enables the auto-completion for the UPF commands and command options. For more information, see “Support for Auto-Completion of UPF Commands” section.
assert_clkrsttoggle	Generates assertion for the below conditions: • Clock is wiggling during CORRUPT simstate (LP_CLOCK_WIGGLE). • Reset is wiggling during CORRUPT simstate (LP_RESET_WIGGLE). For more information, see “Assertions Related Clock/Reset Wiggle During Shutdown/Standby” section.

Option	Description
<code>ackport_driver_check</code>	Allows you to check for the drivers of the <code>ack_port</code> signal in the entire design hierarchy and reports the <code>UPF_MDOAPS</code> warning message (with driver information), if there is a driver that can contribute to a value on the <code>ack_port</code> signal present in the design. For more information, see “Driver Checks on Power Switch Acknowledge Port Signal” section.
<code>assert_reset_sequence</code>	Enables the power-on reset assertions in VCS NLP. For more information, see “Power-on Reset Assertions Support” section.
<code>const_corruption_old</code>	Enables supply based corruption of constants. For more information, see “Supply Based Corruption of Constants” section.
<code>clk_gated_zero_pin_retention</code>	Enables clock-gated zero-pin retention. This option is applicable for the retention strategies that are specified at domain or instance level. Below semantic is applied when you use this option: During power-up (restore), if <code>-retention_condition</code> is true, then the saved value will be restored to the register, but stops the execution of always block (actual process), so reset/clock is not honored. If the <code>-retention_condition</code> is false, then the execution of always block is allowed. For more information, see “Clock-Gated Zero-Pin Retention” .
<code>check_power_down_write</code>	Reports write attempt on OFF domain element during the simulation. For more information, see “Reporting Write Attempt on OFF Domain Element” section.

Option	Description
<code>corrupt_binds</code>	Allows corruption of modules bound to an RTL design using the bind statement. By default, VCS NLP avoids corruption of modules bound to an RTL design. For more information, see “Corruption of Bound Modules” section.
<code>coverage</code>	Creates covergroups for low power objects based on the power intent (UPF). For more information, see “Creating Covergroups for the Low Power Objects” section.
<code>cov_pd_simstate</code>	Enables the power domain simstate coverage. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_port_state</code>	Enables port state coverage. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_implicit_supply_set</code>	Enables coverage on the implicit supply sets only (implicit supply set simstate and power state). For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_explicit_supply_set</code>	Enables coverage on the explicit supply sets only (explicit supply set simstate and power state). For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_supply_set</code>	Enables all types of supply set coverage. For more information, see “Enabling Coverage on Low Power Objects” section.

Option	Description
<code>cov_psw</code>	Enables power switch coverage for the following low power objects: • Power switch state (on/off/partial_on) • ACK port • Control port For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_pst_state</code>	Enables only the state coverage of PST. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_pst_transition</code>	Enables only transition coverage of PST. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_pst</code>	Enables Power State Table (PST) coverage. This option enables both the state and transition coverage for PST. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_ret</code>	Enables coverage on the retention save and restore signals. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>cov_iso</code>	Enables coverage on the isolation enable signal. For more information, see “Enabling Coverage on Low Power Objects” section.
<code>dont_touch_mem</code>	Prevents the corruption of memories that are initialized using <code>\$readmem*</code> and have no other structural drivers. For more information, see “Preventing Corruption of Memories During Power Shutdown” section.

Option	Description
<code>dccompat_gen_inst_name_match</code>	Treats the generate instance name (any depth) as a single token for matching wildcard character “*” specified in the UPF. For more information, see “Treating Generate Instance as a Single Token for Wildcard Matching” section.
<code>dont_isolate_floating_nets</code>	Skips the isolation on the undriven/ floating boundary ports. For more information, see “Skipping Isolation on the Undriven/Floating Ports” section.
<code>discreteportstates</code>	Allows you to change the interpretation of the voltages defined for port/power states to discrete values, rather than a range of voltages. For more information, see “Support for Voltage Triplets in the add_port_state and add_power_state Commands” section.
<code>dump_hvp</code>	Allows you to dump the coverage database to be used with the <code>-lpcov</code> URG option to generate low power coverage reports. For more information, see “Low Power Coverage Report View in URG” section.
<code>dump_pd_tcheck_rpt</code>	Allows you to capture the information related to the timing checks on elements of a power domain in the <code>pd_tcheck_mapping.rpt</code> file. For more information, see “Support for Timing Simulation With SDF in Presence of UPF” section.
<code>force_priority</code>	Overrides corruption with the HDL force. For more information, see “Overriding Corruption with HDL Forces” section.
<code>ignore_latch_retention</code>	Ignores latches from retention. For more information, see “Ignoring Latches From Retention” section.

Option	Description
<code>nofeedthru</code>	Allows corruption on the feedthrough paths. For more information, see “Treating Continuous Assignment Statements as Feedthrough” section.
<code>pass_thru_const</code>	Skips the corruption on constants. For more information, see “Handling of Constants for Corruption and Isolation” section.
<code>partial_single_pin_retention</code>	Enables both “Save Signal Only” and “Restore Signal Only” retention schemes. For more information, see “Supported Retention Schemes” section.
<code>portconnect_exprs</code>	Enables isolation/corruption on the logical expressions in the port map. By default, isolation/corruption is not enabled on the logical expressions in the port map. For more information, see “Isolation/Corruption on Ports with Logical Expressions in Port Map” section.
<code>pst_state_msg</code>	Use this option to enable verbose messaging for the following: • Illegal state transitions (PST state changes to or from <code>ILLEGAL</code> state) • Invalid combinations (Any valid combination of PST state supplies that is not defined in the PST table) For more information, see “Verbose Illegal PST State Change Messages” section.

Option	Description
resolve_ar_sched	Allows you to avoid possible glitches on the resolved supply nets when the multiple drivers change in different deltas of the same simulation time. For more information, see “Support for Voltage Triplets in the add_port_state and add_power_state Commands” section.
sim_only	Generates parsing error for the simulation only UPF commands/options/attributes which are not part of the UPF file that is loaded with <code>load_upf <upf_file> -simulation_only</code> . For more information, see “-simulation_only Support” section.
snps_lp_flow	Generates an error message for the below list of UPF commands/options as these are not supported across all the tools in Synopsys Low Power Flow: • <code>add_power_state <-update></code> • <code>set_isolation <-clamp_value Z></code> • <code>set_isolation_control <-location automatic></code> • <code>set_level_shifter <-source> <-sink></code> • <code>set_retention_control -save_signal/-restore_signal <posedge/negedge></code> • <code>set_simstate_behaviour</code> For more information, see “Synopsys Low Power Flow Consistency Check” section.

Option	Description
<code>src_off_clamp_check</code>	Enables automated check at runtime for the assertion of an isolation enable when the source supply is OFF. For more information, see “Checking for Assertion of Isolation Enable When Source Supply is OFF” section.
<code>ssdefaulton</code>	Allows you to set the default power state of the supply set to the ON power state when only one ON power state is defined. For more information, see “Setting Default Power State for a Supply Set” section.
<code>UPF_dont_touch_BC</code>	Enables the backwards compatibility when the <code>UPF_dont_touch</code> attribute is used in UPF. For more information, see “Excluding Parts of Design from Being Power Managed” section.
<code>unconnected_bias_pins_on</code>	Keeps the unconnected bias PG pins of cells ON. For more information, see “Implicit Supply Net Connections” section.
<code>upf_lint</code>	Generates a warning message for every power domain without explicit specification of power or ground nets in the UPF. For more information, see “Lint Check for Explicit Primary Supplies for a Domain” section.
<code>undriven_supplies_off</code>	Allows you to initialize all the supply ports (except supply pads) and supply set functions with no explicit nets to OFF state. For more information, see “Supply Network Initialization” section.

Option	Description
<code>verbose1</code>	Allows the tool to display the boundary_port_partitioning.rpt file. This file shows the repeaters implemented and its associated supplies. For more information, see “Using the - repeater_supply Option” section.
<code>write_mvinfo</code>	Allows you to generate the virtual isolation insertion report (isolation_insertion_info.txt) / virtual isolation inference report (new_isolation_association.rpt). For more information, see “Generating Virtual Isolation Reports” section.
<code>xmr_corrupt</code>	Allows corruption of XMR based on its domain simstate. For more information, see “Corruption of XMR Based on its Domain Simstate” section.

Simulation Option

[Table 19-3](#) describes the simulation option supported by VCS NLP.

Table 19-3 Simulation Option Quick Reference

Runtime Option	Description
<code>-power <config_file_name></code>	Allows you to specify the runtime configuration file. You can specify the runtime design attributes and Unified Message Control options in the runtime configuration file. Example: % <code>simv -power power_config.tcl</code> Example of <code>power_config.tcl</code> file: <pre>set_design_attributes -elements \ {power_pa_tb/noRandomCorruption} \ -attribute SNPS_random_corruption 01X</pre>

[Table 19-4](#) lists the runtime design attributes supported by VCS NLP. You must specify these attributes in the configuration file you specify with the `-power <config_file_name>` runtime option.

Table 19-4 Runtime Design Attributes Quick Reference

Attribute	Value	Description
SNPS_apf_compat	TRUE	Skips the corruption of VHDL variables, real and integer types. For more information, see “Skipping Corruption” section.
SNPS_keep_unassociated_iso_on	TRUE	Treats the unassociated ISO/ELS cells as always-on. For more information, see “Association of Pre-Inserted Isolation Cells” section.
SNPS_power_switch_on_delay	value	Allows you to specify the ON delay value of a power switch. For more information, see “Modeling Delay Specification for a Power Switch” section.
SNPS_random_seed	value	Allows you to specify a seed using the <code>set_design_attributes</code> command. A seed is a number that initializes a pseudo-random generator used for corruption during shutdown. For more information, see “Random Corruption” section.
SNPS_random_corruption	value	Runtime attribute. Allows you to set the corruption value to any of 0/1/X/Z during shutdown. For more information, see “Random Corruption” section.
SNPS_treat_undriven_as_off		Initializes all the supply ports (except supply pads) and supply set functions with no explicit nets to OFF state. For more information, see “Supply Network Initialization” section.

Attribute	Value	Description
SNPS_treat_undriven_as_undetermined		Initializes all the undriven supply ports/nets and supply set functions with no explicit nets to UNDETERMINED state. For more information, see “Supply Network Initialization” section.
SNPS_treat_undriven_supply_nets_as_undetermined		Initializes all the undriven supply net to UNDETERMINED state. For more information, see “Supply Network Initialization” section.
SNPS_voltage_based_supply_on	TRUE	Turn OFF the supply when <code>supply_on</code> is called with 0 volts. For example: <code>supply_on (“VDD”, 0.0)</code> . For more information, see “Voltage-Based supply_on” section.

Design Attributes to be Specified in UPF

[Table 19-5](#) lists the compile-time design attributes supported by VCS NLP. You must specify these compile-time design attributes in the UPF.

Table 19-5 Compile-Time Design Attributes Quick Reference

Attribute	Value	Description
<code>conservative_diff_supply_only_isolation</code>	TRUE	Allows you to insert an isolation cell for the <code>-diff_supply_only</code> isolation strategies in the cases where one or more of the loads/sinks might have the same supply as the driver/source, but at least one of them uses a different supply. Specify the following in the UPF. For more information, see “Support for Conservative diff_supply_only Isolation” section.
<code>deselect_coverage_object</code>	value	Allows you to specify the low power objects that you want to exclude from the coverage collection. For more information, see “Excluding UPF Objects from Low Power Coverage” section.
<code>enable_bias</code>	TRUE / FALSE	When set to TRUE, enables the bias feature. Transitive effect is applied on child blocks under top bias block. For more information, see “Enabling Bias Feature” section.
<code>hetero_fanout_isolation</code>	true	Allows you to insert isolation cell in the extent of the domain of the self or parent hierarchy/instance. For more information, see “Path-Based Isolation Support for Heterogeneous Loads” section.

Attribute	Value	Description
iso_nor	TRUE	Indicates that a certain isolation strategy is defining a NOR style isolation. NOR style isolation is applicable to isolation strategies with clamp value 0. In general, these are single rail isolation cells with switched supply. Output of the isolation cell is 0 when the power is OFF and isolation enable is active. For more information, see “NOR-Style Isolation Support” section.
is_soi	TRUE	Enables FDSOI. For FDSOI, both pwell and nwell are tied to the ground supply. To enable FDSOI, set the is_soi design attribute to TRUE in UPF, as shown below: <pre>set_design_attributes - models <model_name> - attribute is_soi TRUE</pre> For more information, see “Implicit Supply Net Connections” section.
lower_domain_boundary	TRUE/ FALSE	When set to TRUE, extends the definition of a power domain boundary to include the lower domain boundary ports/pins for isolation. For more information, see “Supporting Lower Domain Boundary for Isolation” section.
repeater_ground_net	supply_net_ground	Allows you to specify ground net. For more information, see “Using the -repeater_supply Option” section.
repeater_power_net	supply_net_power	Allows you to specify power net. For more information, see “Using the -repeater_supply Option” section.

Attribute	Value	Description
SNPS_active_reset_duration	Twidth	Allows you to enable the power-on reset assertions. For more information, see “Power-on Reset Assertions Support” section.
SNPS_dont_reinit	TRUE/ FALSE	When set to TRUE, enables dont_reinit property on the initial blocks targeted by the command. For more information, see “Initial Block Retriggering” section.
SNPS_default_power_upf2sv_vct	vct_name	Allows you to change the default VCT for power port connection.
SNPS_default_power_upf2vh_vct	vct_name	
SNPS_default_power_sv2upf_vct	vct_name	
SNPS_default_ground_upf2sv_vct	vct_name	Allows you to change the default VCT for ground port connection.
SNPS_default_ground_upf2vh_vct	vct_name	
SNPS_default_ground_sv2upf_vct	vct_name	
SNPS_default_supply_net_state	STATE_NAME	Allows you to set default state on the supply port. For more information, see “Setting Supply Port Default Value” section.
SNPS_force_semantics	CONFIG NAME	Allows you to configure semantics for runtime forces during power shutdown and wake-up. For more information, see “Configuring Semantics for Runtime Forces During Power Shutdown and Wake-up” section.
SNPS_inactive_reset_duration	Tmax	Allows you to enable power-on reset assertions. For more information, see “Power-on Reset Assertions Support” section.

Attribute	Value	Description
SNPS_macro_port_based_corruption	ENABLE/ DISABLE	Enable/disable the port-based corruption on a particular macro or a full design. For more information, see “Configuring Port-Based Corruption on Macros” section.
SNPS_override_pbp_corruption	TRUE / FALSE	When set to TRUE, forces VCS NLP to corrupt the ports for all cells even when the models are power aware. When set to FALSE, forces VCS NLP not to corrupt the ports for all cells even when the models are non-power aware. For more information, see “Simulation Behavior for Non-Power Aware Models” section.
SNPS_reinit	TRUE/ FALSE	When set to TRUE, enables re-triggering of the initial blocks on wake-up. For more information, see “Initial Block Retriggering” section.
terminal_boundary		Use the <code>terminal_boundary</code> design attribute to mark hierarchical boundaries as terminal points for isolation strategies. For more information, see “Support for Isolation Terminal Points at the Hierarchical Boundaries” section.
UPF_dont_touch	TRUE	Excludes certain portions of the design from being power managed. For more information, see “Excluding Parts of Design from Being Power Managed” section.

HDL Attributes

Table 19-6 lists the HDL attributes used to control the corruption and wake-up semantics of the initial and always blocks in Verilog. For more information, see [“Controlling the Corruption and Wake-up Semantics of the Initial and Always Blocks”](#) section. You must specify these attributes in the design code.

Table 19-6 HDL Attributes to Control Corruption and Wake-up Semantics

Attribute	Description
<code>(* vcs_reinit *)</code>	Allows you to retrigger the selected initial blocks on power-up without using the UPF commands.
<code>(* vcs_dont_reinit *)</code>	Disables the retriggering of the selected initial block after power-up.
<code>(* vcs_always_on *)</code>	Marks the selected always block as always-on.
<code>(* vcs_dont_trigger_on_wakeup *)</code>	Disables the automatic retriggering of the selected always block on power-up.

Index

A

add_domain_elements [3-34](#)
add_port_state [3-19](#)
add_power_state [3-19](#)
associate_supply_set [3-4](#)
-attribute name value [6-16](#)

B

bind_checker [3-35](#)

C

connect_logic_net [3-22](#)
connect_supply_net [3-7](#), [17-1](#)
connect_supply_set [3-8](#)
-covg_disable_cg=ALL [14-6](#)
-covg_disable_cg=LP [14-6](#)
-covg_disable_cg=USER [14-6](#)
create_hdl2upf_vct [3-36](#)
create_logic_net [3-21](#)
create_logic_port [3-21](#)
create_power_domain [3-5](#)
create_power_state_group [3-20](#)
create_power_switch [3-5](#)
create_pst [3-18](#)
create_supply_net [3-7](#), [3-65](#)

create_supply_port [3-6](#)
CSN [15-9](#)

D

describe_state_transition [3-37](#)
-diff_supply_only [9-20](#)
DISABLE_INSTANCE_WITH_ELEM [9-34](#)
don't reinit [6-17](#)
-dump full_exclusions group [14-34](#), [14-35](#)
dump_pd_tcheck_rpt [15-27](#)

E

-elements [6-16](#)
ELS [15-9](#)
enable_state_propagation_in_add_power_state [5-9](#)
-exclude_elements [9-29](#)

F

find_objects [3-23](#)

I

ISO [15-9](#)
iso_nor [9-5](#)

isolation_ground_net [15-9](#)
isolation_power_net [15-9](#)

L

load_simstate_behavior [3-37](#)
load_upf [3-30](#)
load_upf_protected [3-38](#)
-location fanout [9-45](#)
-lpa_bind_filelist [12-17](#)
-lpcov [14-23](#)
lphvp.LP.html [14-23](#)

M

map_isolation_cell [3-14](#)
map_level_shifter_cell [3-11](#)
map_power_switch [3-9](#)
map_retention_cell [3-17](#)
merge_power_domains [3-33](#)
-models model_list [6-16](#)
mv_upf_strategy_in_inst_name [9-9](#)

N

name_format [3-11](#)
NLP [8-5](#)

P

-pa_random_corrupt [8-5](#)
package_name

 resolution_name [3-65](#)
Power Domain Detail page [14-23](#)
Power Domain detail page [14-25](#)
Power Scope Detail page [14-23](#), [14-24](#)
-power=write_mvinfo [9-34](#), [9-36](#)

Q

query_cell_instances [3-24](#)
query_cell_mapped [3-24](#)
query_design_attributes [3-26](#)
query_hdl2upf_vct [3-27](#)
query_net_ports [3-24](#)
query_port_net [3-25](#)
query_power_domain [3-27](#)
query_power_switch [3-26](#)
query_pst [3-25](#)
query_pst_state [3-25](#)
query_retention [3-28](#)
query_retention_control [3-28](#)
query_supply_net [3-28](#)
query_upf [3-26](#)

R

reate_composite_domain [3-36](#)
reinit [6-15](#)
retention_element_list [11-37](#)
retention_list_name [11-34](#)

S

set_design_attributes [3-31](#)
set_design_top [3-29](#), [3-38](#)
set_domain_supply_net [3-9](#)
set_equivalent [3-38](#)
set_isolation [3-12](#)
set_isolation_control [3-13](#)
set_level_shifter [3-10](#)
set_partial_on_translation [3-29](#)
set_port_attributes [3-31](#), [3-90](#)
set_power_switch [3-32](#)
set_retention [3-15](#), [11-34](#)
set_retention_control [3-16](#)
set_retention_elements [3-17](#), [11-33](#)
set_scope [3-30](#)
set_simstate_behavior [3-39](#)

- sink [9-20](#)
- SNPS_dont_reinit [6-17](#)
- SNPS_random_seed [8-7](#)
- SNPS_reinit [6-17](#)
- source [9-20](#)
- Source Pane
 - Toolbar icon [13-14](#), [13-21](#)

T

- transitive [6-16](#)

U

- unconnected [3-90](#)
- update [9-29](#)
- UPF [15-9](#)
- upf_version [3-39](#), [3-96](#)

W

- wreal netlist [17-1](#)

